Arhitecturi si Prelucrari Paralele
APP

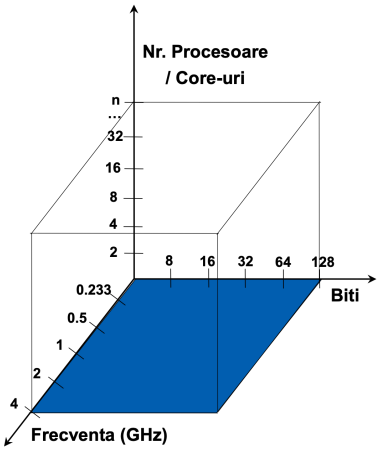
Nicolae Tapus nicolae.tapus@cs.pub.ro
Emil Slusanschi emil.slusanschi@cs.pub.ro

University Politehnica of Bucharest

Computer Science & Engineering Department

1

Structuri de Calcul cu Prelucrare Paralela



- *Aria → puterea de calcul*
- *Marimea puterii de calcul: volumul cubului - "n" e limitat "doar" de cost →* ***Arhitecturi paralele de calcul***
- *Orice problema are un grad de paralelism intrinsec ce depinde de natura ei*
- *Trebuie tinut seama de:*
  - *Algoritmi paraleli specifici*
  - *Limbaje adecvate de programare*
  - *Limitarile SO*
  - *Arhitectura intrinseca a SC utilizat*

Gradul general de paralelism al codurilor este ~ 10%

2

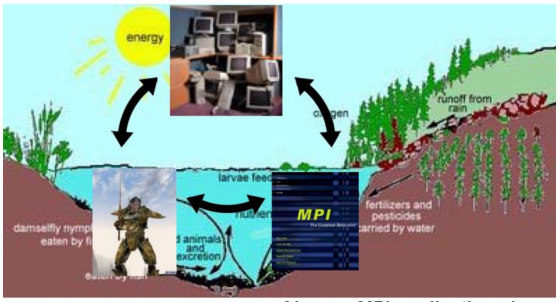
Trends in Parallel Computing in 2010 – 202x+

3

- Computing is changing more rapidly than ever before, and scientists have the unprecedented opportunity to change computing directions

Commercial Off The Shelf technology (COTS)

Supercomputing Ecosystem 20xx



"Clusters" years of legacy MPI applications base

©Horst Simon
LBNL@UC Berkeley

3

Traditional Sources of Performance Improvement are Flat-Lining

4

- **New Constraints**
  - 15 years of *exponential* clock rate growth has ended
- **Moore's Law reinterpreted:**
  - How do we use all of those transistors to keep performance increasing at historical rates?
  - Industry Response: #cores per chip doubles every 18 months *instead* of clock frequency!

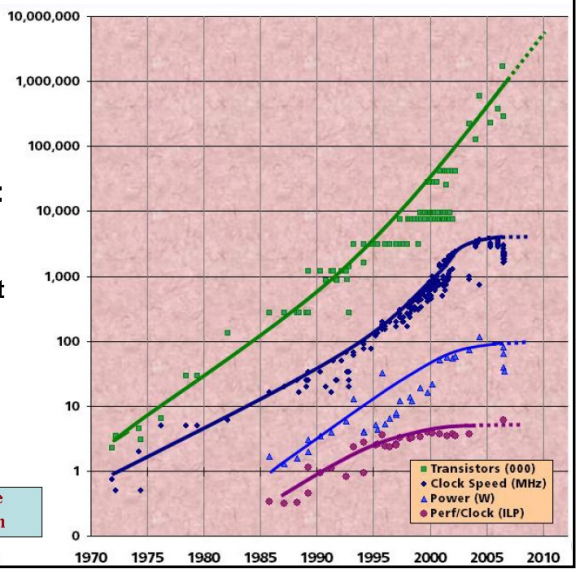


Figure courtesy of Kunle Olukotun, Lance Hammond, Herb Sutter, and Burton Smith

4

5

Supercomputing Ecosystem 20xx

Commercial Off The Shelf technology (COTS)



PCs and desktop systems are no longer the economic driver.





Architecture and programming model are about to change

"Clusters"

years of legacy MPI applications base

5

6

Breaking the 1PFlop Barrier

- 1,026 Tflop/s on LINPACK reported on June 9, 2008
- 6,948 dual core Opteron + 12,960 cell BE
- 80 TByte of memory
- IBM built, installed at LANL



Roadrunner

6

7

Cray XT5@ORNL



Jaguar	Total	XT5	XT4
Peak Performance	1,645	1,382	263
AMD Opteron Cores	**181,504**	**150,176**	**31,328**
System Memory (TB)	**362**	**300**	**62**
Disk Bandwidth (GB/s)	**284**	**240**	**44**
Disk Space (TB)	**10,750**	**10,000**	**750**
Interconnect Bandwidth (TB/s)	**532**	**374**	**157**

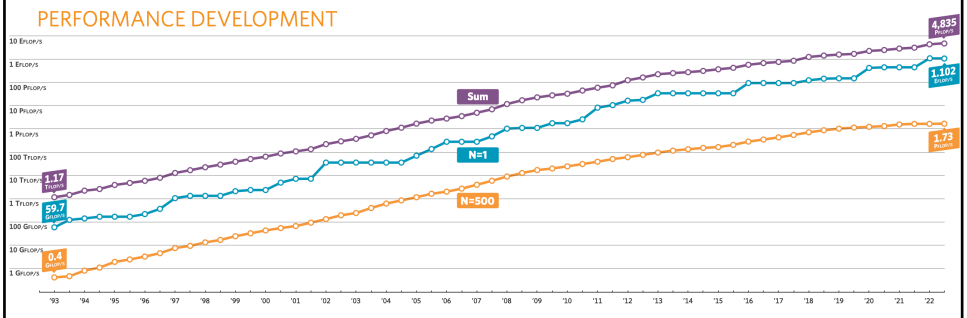
The systems will be combined after acceptance of the new XT5 upgrade. Each system will be linked to the file system through 4x-DDR Infiniband

U.S. DEPARTMENT OF ENERGY
Office of Science

7

8

Top 500 – 1993-2023



8

Moore's Law "Reloaded"

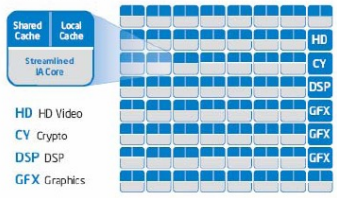
- Number of cores per chip will double every two years
- Clock speed will not increase (possibly decrease)
- Need to deal with systems with millions of concurrent threads
- Need to deal with inter-chip parallelism as well as intra-chip parallelism



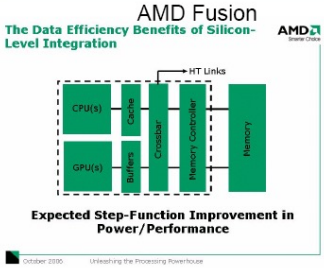
9

Industry presentations show changing trends in processors

Intel's Microprocessor Research Lab



AMD Fusion



Intel's Visual Computing Group - Larabee



nVidia G80 - 2006



Taken from publicly available information

Los Alamos NATIONAL LABORATORY EST. 1943

Operated by Los Alamos National Security, LLC for NNSA

UNCLASSIFIED

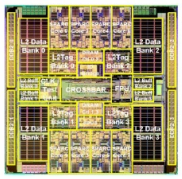
WEAPONS SCIENCE & ENGINEERING CAPABILITY REVIEW

NNSA

10

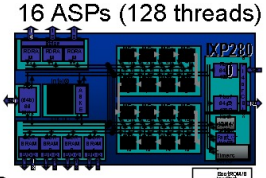
Multicore comes in a wide variety

- Multiple parallel general-purpose processors (GPPs)
- Multiple application-specific processors (ASPs)

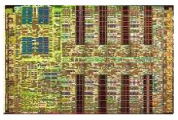


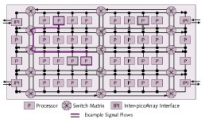
Sun Niagara
8 GPP cores (32 threads)

Intel Network Processor
1 GPP Core
16 ASPs (128 threads)



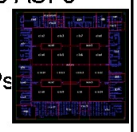
IBM Cell
1 GPP (2 threads)
8 ASPs





Picochip DSP
1 GPP core
248 ASPs

Cisco CRS-1
188 Tensilica GPPs



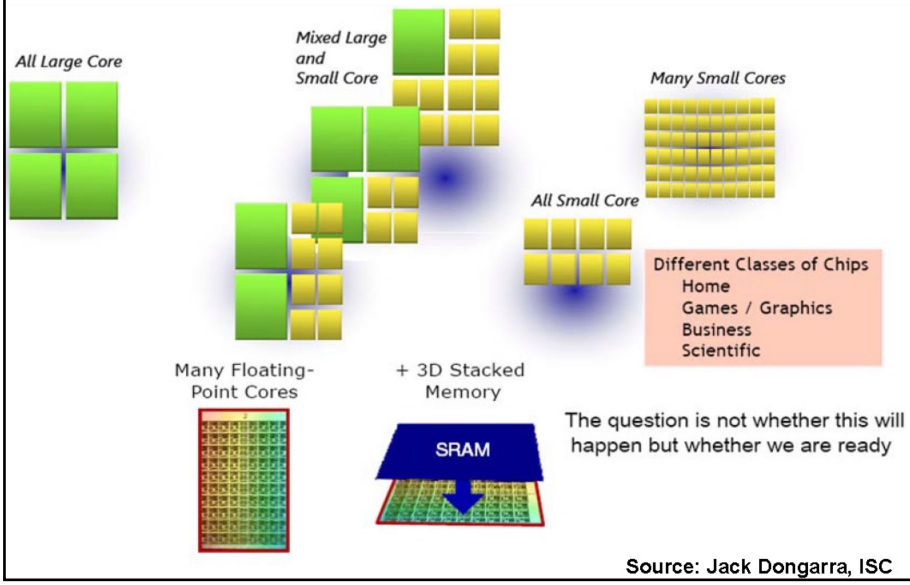


Intel 4004 (1971):
4-bit processor,
2312 transistors,
~100 KIPS,
10 micron PMOS,
11 mm² chip

1000s of processor cores per die

"The Processor is the new Transistor" [Rowen]

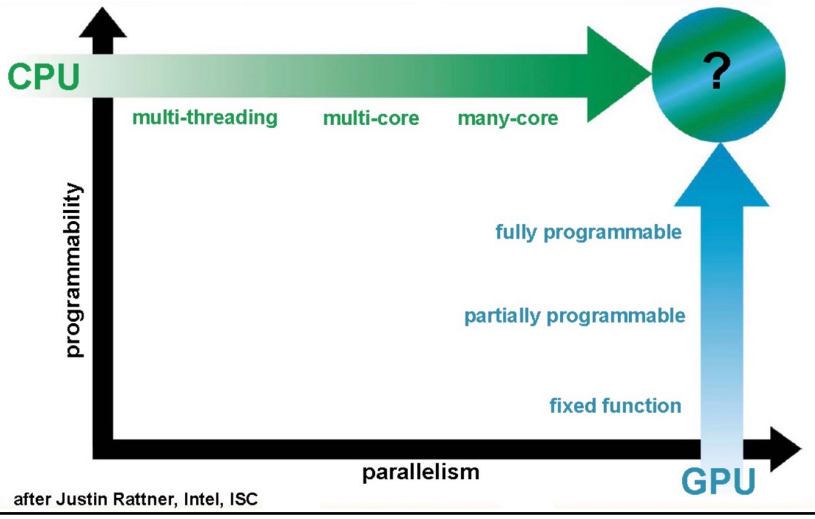
What's next?



Source: Jack Dongarra, ISC

13

A Likely Trajectory: Collision or Convergence?



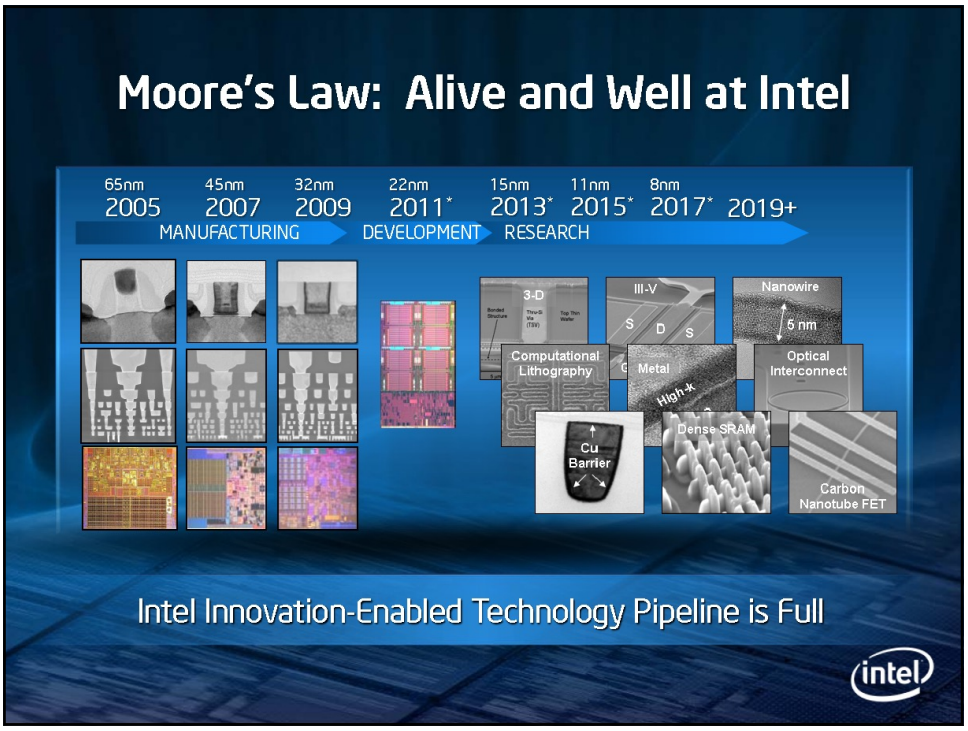
13

14

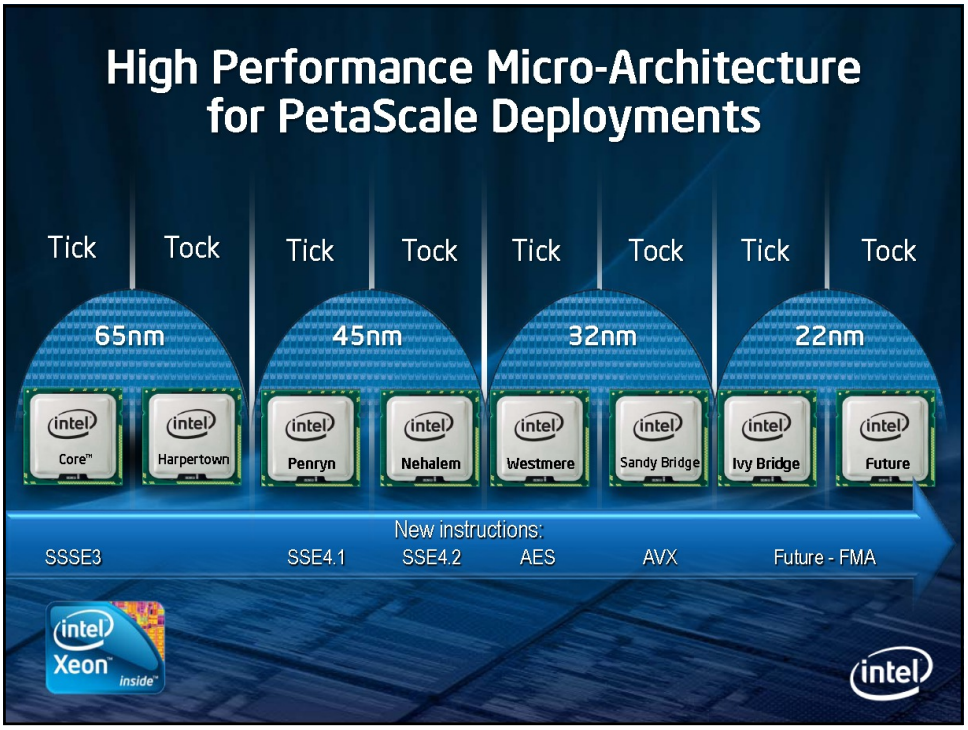
Trends for the next 5-10 years

- After a period of rapid architectural change we will likely settle on a future standard processor architecture
- A good bet:
  - Intel will continue to be an important actor
  - AMD will be the challenger in the CPU space
  - NVidia will lead in the accelerator space
- The impact of this disruptive change on software and systems architecture is not clear yet

14


Moore's Law: Alive and Well at Intel
65nm 2005
45nm 2007
32nm 2009
22nm 2011*
15nm 2013*
11nm 2015*
8nm 2017*
2019+
MANUFACTURING
DEVELOPMENT
RESEARCH
3-D
III-V
Nanowire
5 nm
Computational Lithography
Metal
High-k
Optical Interconnect
Cu Barrier
Dense SRAM
Carbon Nanotube FET
Intel Innovation-Enabled Technology Pipeline is Full
intel

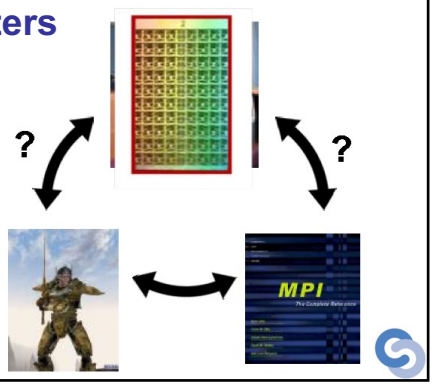
15


High Performance Micro-Architecture for PetaScale Deployments
Tick
Tock
65nm
45nm
32nm
22nm
Core™
Harpertown
Penryn
Nehalem
Westmere
Sandy Bridge
Ivy Bridge
Future
New instructions:
SSSE3
SSE4.1
SSE4.2
AES
AVX
Future - FMA
intel Xeon inside™
intel

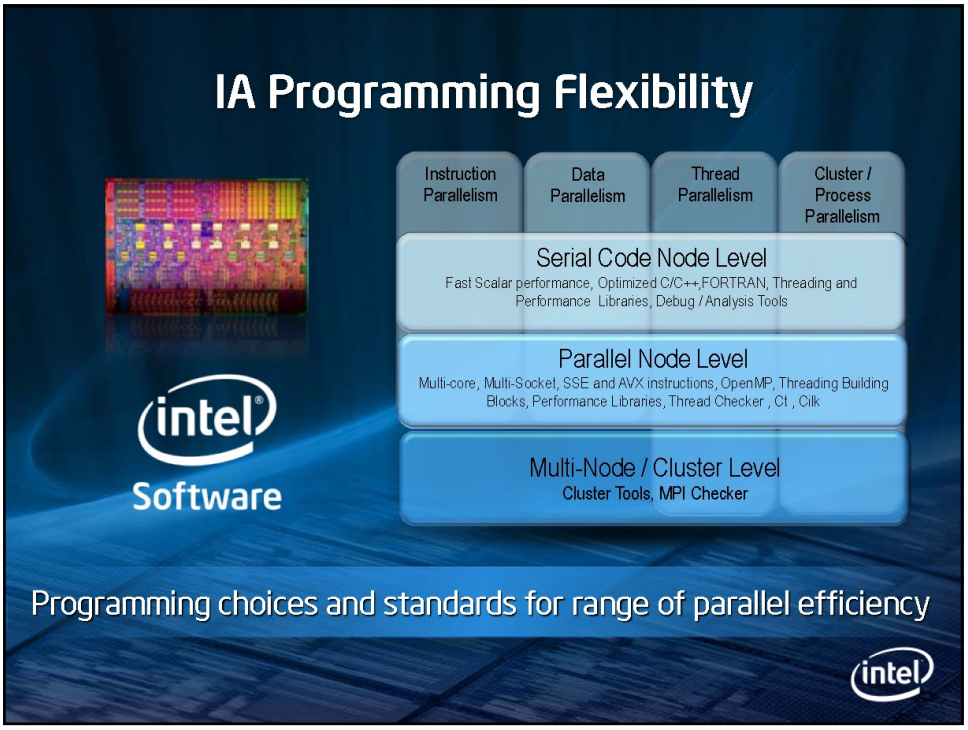
16

Impact on Software

- **We will need to rethink and redesign our software**
  - **Similar challenge as the 90's transition to clusters and MPI**
- What will be the ?

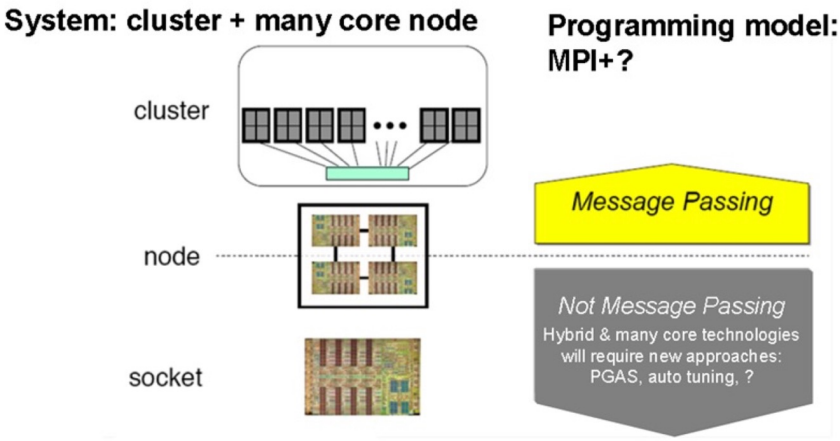


17



18

A Likely Future Scenario



after Don Grice, IBM, Roadrunner Presentation, ISC



Why MPI will persist

- Obviously MPI will not disappear in five years
- Today we have more than 30 years of legacy software in MPI
- New systems are not sufficiently different to lead to a radical new programming model



21

What will be the "?" in MPI+?

- Likely candidates are
  - PGAS languages
  - Autotuning
  - CUDA, OpenCL, OpenACC
  - A wildcard from the commercial space



21

What's Wrong with MPI Everywhere?

22

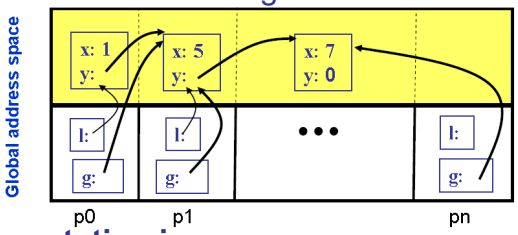
- One MPI process per core is wasteful of intra-chip latency and bandwidth
- **Weak scaling**: success model for the "cluster era" not enough memory per core
- **Heterogeneity**: MPI per CUDA thread-block?



22

PGAS Languages

- **Global address space**: thread may directly read/write remote data
- **Partitioned**: data is designated as local or global

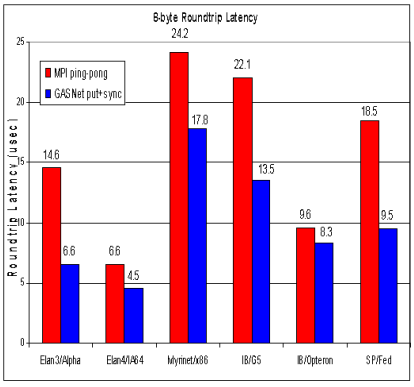


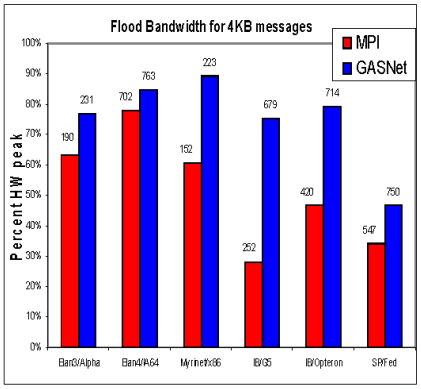
- **Implementation issues:**
  - Distributed memory: Reading a remote array or structure is explicit, not a cache fill
  - Shared memory: Caches are allowed, but not required
- No less scalable than MPI!
- Permits sharing, whereas MPI rules it out!



Performance Advantage of One-Sided Communication

- The put/get operations in PGAS languages (remote read/write) are one-sided (no required interaction from remote proc)
- This is faster for pure data transfers than two-sided send/receive



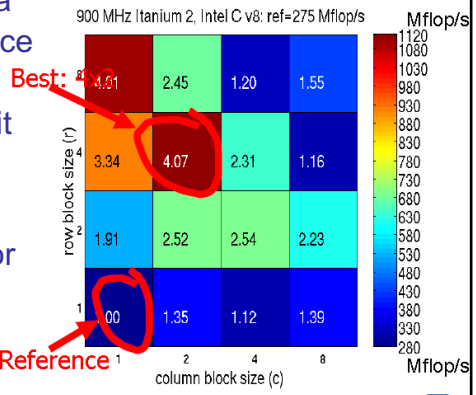




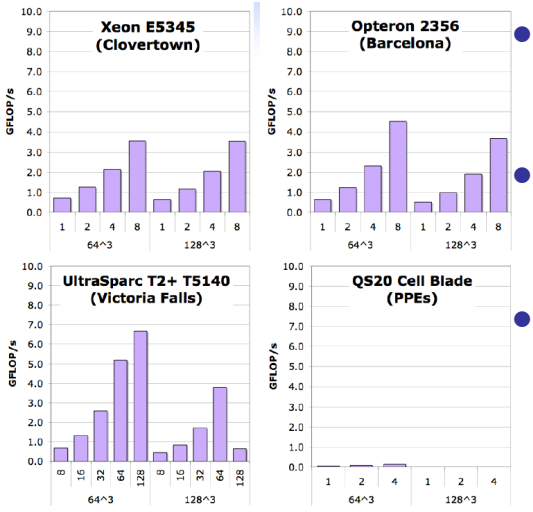
Autotuning

- Write programs that write programs
  - Automate search across a complex optimization space
  - Generate space of implementations, search it
  - Performance far beyond current compilers
  - Performance portability for diverse architectures!
  - Past success stories: PhiPAC, ATLAS, FFTW, Spiral, OSKI

Finite Element Problem
[Im, Yelick, Vuduc]



Performance

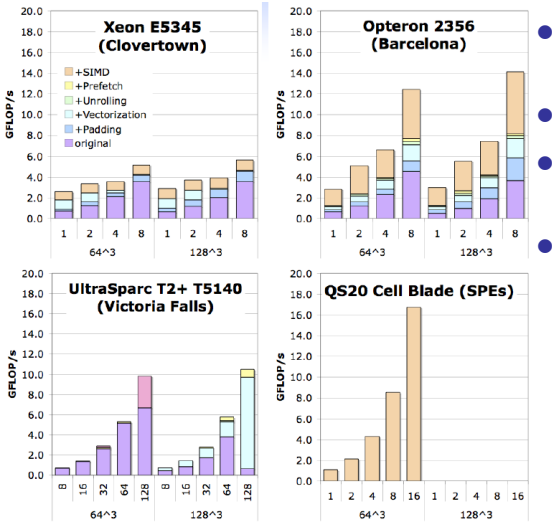


- Reference code was nominated for a Gordon Bell prize
- Used for out-of-box study on multicore performance
- Superficially, scalability looks good, but is performance good?
  - **No**
  - Performance model

POC: Leonid Oliker, Samuel Williams (LBNL)

Reference+NUMA

Autotuning



- Auto-tuning dramatically improved performance
- Clearly no silver bullet
- SIMD partially breaks portability premise
- Note: Cell version was optimized, not auto-tuned

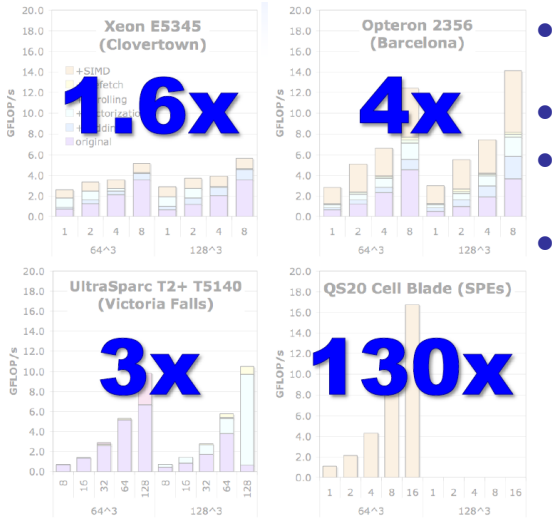


POC: Leonid Oliker, Samuel Williams (LBNL)



27

Autotuning



- Auto-tuning dramatically improved performance
- Clearly no silver bullet
- SIMD partially breaks portability premise
- Note: Cell version was optimized, not auto-tuned



POC: Leonid Oliker, Samuel Williams (LBNL)

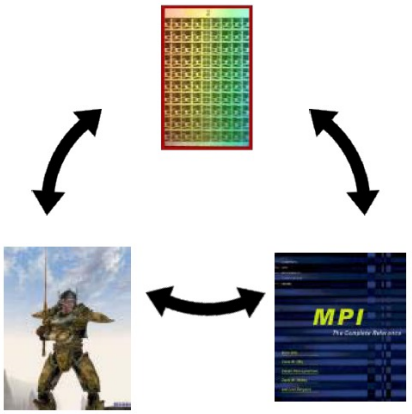


28

29

The Likely HPC Ecosystem from 202x+

CPU + GPU = future many-core driven by commercial applications



MPI+(autotuning, PGAS, ??)

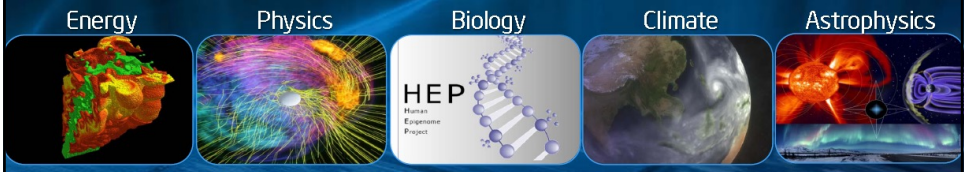
Next generation "clusters" with many-core or hybrid nodes

29

Exascale: *The Next Frontier*

Challenges

- Power – energy / operation of computation, data transport, memory
- Threading software to millions/billions of threads
- Memory/Storage capacity and bandwidth
- Managing high-node count systems in the existence of failures (MTBF)
- Affordability



Intel committed to solving the challenges of Exascale

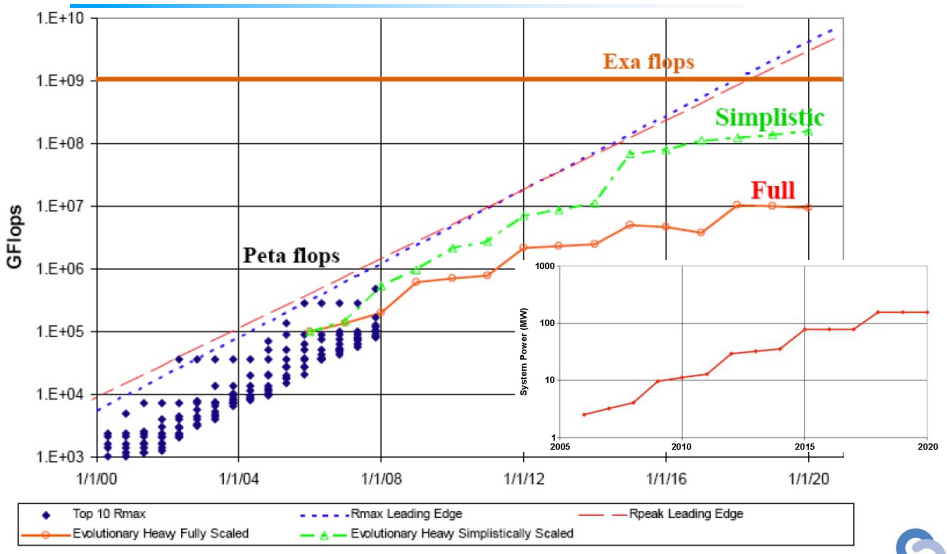
30

DARPA Exascale Study

- Commissioned by DARPA to explore the challenges for Exaflop computing
- Two models for future performance growth
  - **Simplistic**:
    - ITRS (International Technology Roadmap for Semiconductors) roadmap
    - Power for memory grows linear with # of chips
    - Power for interconnect stays constant
  - **Fully scaled**: same as simplistic, but memory and router power grow with peak flops per chip



Won't reach Exascale this way



From Peter Kogge, DARPA ExascaleStudy



33

Extrapolating to Exaflop/s

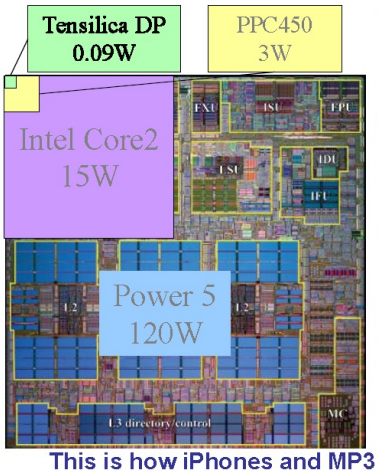
	BlueGene/L (2005)	Exaflop Directly scaled	Exaflop compromise using expected technology	Assumption for "compromise guess"
Node Peak Perf	5.6GF	20TF	20TF	Same node count (64k)
hardware concurrency/node	2	8000	1600	Assume 3.5GHz
System Power in Compute Chip	1 MW	3.5 GW	35 MW	100x improvement (very optimistic)
Link Bandwidth (Each unidirectional 3-D link)	1.4Gbps	5 Tbps	1 Tbps	Not possible to maintain bandwidth ratio.
Wires per unidirectional 3-D link	2	400 wires	80 wires	Large wire count will eliminate high density and drive links onto cables where they are 100x more expensive. Assume 20 Gbps signaling
Pins in network on node	24 pins	5,000 pins	1,000 pins	20 Gbps differential assumed. 20 Gbps over copper will be limited to 12 inches. Will need optics for in rack interconnects. 10Gbps now possible in both copper and optics.
Power in network	100 KW	20 MW	4 MW	10 mW/Gbps assumed. Now: 25 mW/Gbps for long distance (greater than 2 feet on copper) for both ends one direction. 45mW/Gbps optics both ends one direction. + 15mW/Gbps of electrical Electrical power in future: separately optimized links for power.
Memory Bandwidth/node	5.6GB/s	20TB/s	1 TB/s	Not possible to maintain external bandwidth/Flop
L2 cache/node	4 MB	16 GB	500 MB	About 6-7 technology generations
Data pins associated with memory/node	128 data pins	40,000 pins	2000 pins	3.2 Gbps per pin
Power in memory I/O (not DRAM)	12.8 KW	80 MW	4 MW	10 mW/Gbps assumed. Most current power in address bus. Future probably about 15mW/Gbps maybe get to 10mW/Gbps (2.5mW/Gbps is c*v^2*f for random data on data pins) Address power is higher.
QCD CG single iteration time	2.3 msec	11 usec	15 usec	Requires: 1) fast global sum (2 per iteration) 2) hardware offload for messaging (Driverless messaging)

Source: David Turek, IBM

33

34

Design for Low Power: More Concurrency



- Cubic power improvement with lower clock rate due to V^2F
- Slower clock rates enable use of simpler cores
- Simpler cores use less area (lower leakage) and reduce cost
- Tailor design to application to reduce waste

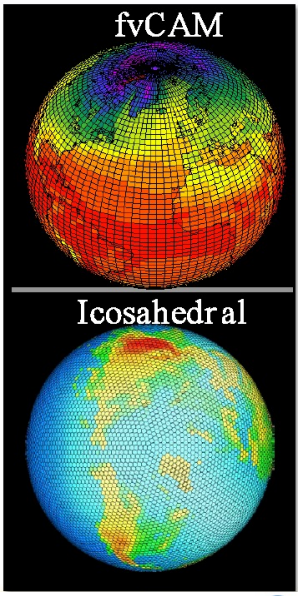
This is how iPhones and MP3 players are designed to maximize battery life and minimize cost

34

Green Flash: Ultra-Efficient Climate Modeling

35

- An alternative route to exascale computing
  - Exascale science questions are identified
  - The idea is to target specific machine designs to each of these questions
    - Possible because of new technologies driven by the consumer market
- We want to turn the process around
  - Ask "What machine do we need to answer a question?"
  - Not "What can we answer with that machine?"
- Goal: influence the HPC industry by evaluating a prototype design





35

36

Green Flash Strawman System Design

We examined three different approaches

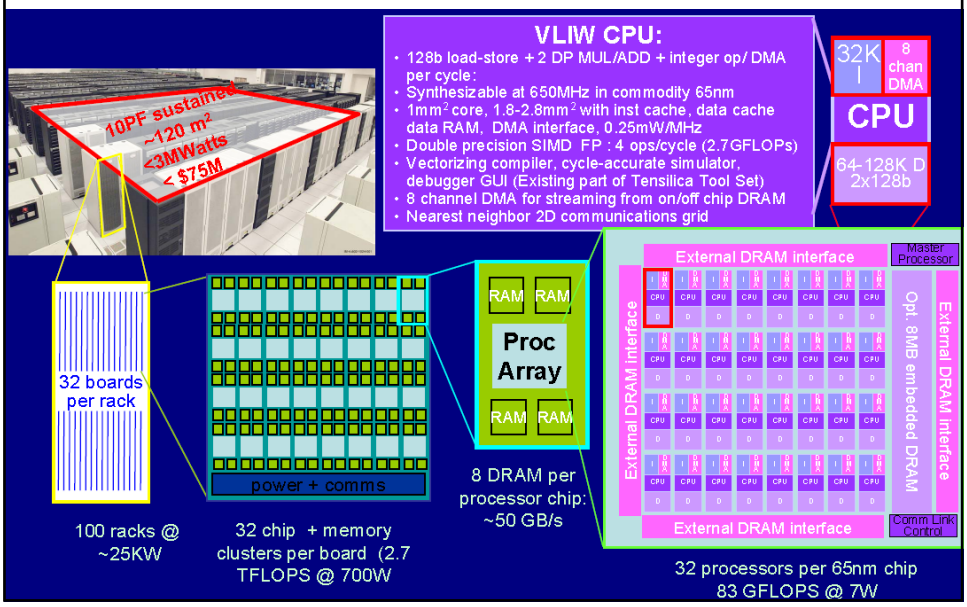
Computation: 10 PFlops sustained, ~200 PFlops peak

- **AMD Opteron**: Commodity approach, lower efficiency for scientific applications offset by cost efficiencies of mass market
- **BlueGene**: Generic embedded processor core and customize system-on-chip (SoC) to improve power efficiency for scientific applications
- **Tensilica XTensa**: Customized embedded CPU w/SoC provides further power efficiency benefits but maintains programmability

Processor	Clock	Peak/ Core (Gflops)	Cores/ Socket	Sockets	Cores	Power	Cost 2008
AMD Opteron	2.8GHz	5.6	2	890K	1.7M	179 MW	$1B+
IBM BG/P	850MHz	3.4	4	740K	3.0M	20 MW	$1B+
Green Flash / Tensilica XTensa	650MHz	2.7	32	120K	4.0M	3 MW	$75M

36

Climate System Design Concept



37

Summary on Green Flash

- Exascale computing is vital for numerous key scientific areas
- We propose a new approach to high-end computing that enables transformational changes for science
- Research effort: study feasibility and share insight w/ community
- This effort will augment high-end general purpose HPC systems
  - Choose the science target first (climate in this case)
  - Design systems for applications (rather than the reverse)
  - Leverage power efficient embedded technology
  - Design hardware, software, scientific algorithms together using hardware emulation and auto-tuning
  - Achieve exascale computing sooner and more efficiently

Applicable to broad range of exascale-class applications



38

39

What about 1 million cores?

- What are applications developers concerned about?
- … but before we answer this question, the more interesting question is …
  - 1000 cores on the laptop ?
- What are commercial applications developers going to do with it?



39

40

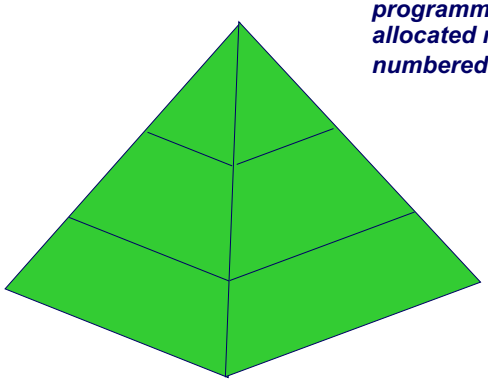
Summary

- Major Challenges are ahead for HPC
  - Power
  - Parallelism
  - …and many others not discussed here
- We will need completely new approaches and technologies to reach the Exascale level
- This opens up a unique opportunity for science applications to lead extreme scale systems development



40

The Pyramid of Parallel Programming



Thousand-node systems with MPI-style programming, >1-10 PFLOPS, $M, allocated machine time (programmers numbered in hundreds)

Hundred-core systems with CUDA-style programming, 1-5 TFLOPS, $K, machines widely availability (programmers numbered in 10s of thousands)

Hundred-core systems with MatLab-style programming, 10-100 GFLOPS, $K, machines widely available (programmers numbered in millions)



Foster's Design Methodology

- From *Designing and Building Parallel Programs* by Ian Foster
- Four Steps:
  - **Partitioning**
    - Dividing computation and data
  - **Communication**
    - Sharing data between computations
  - **Agglomeration**
    - Grouping tasks to improve performance
  - **Mapping**
    - Assigning tasks to processors/threads



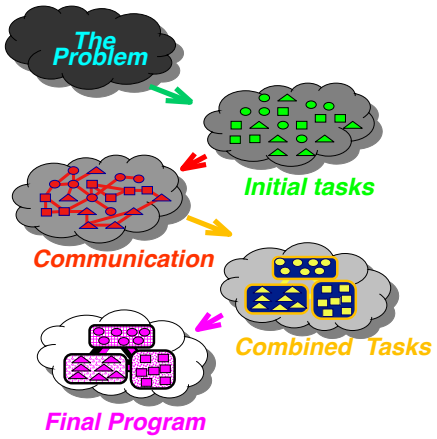
Parallel Algorithm Design: PCAM

- **Partition**: Decompose problem into fine-grained tasks to maximize potential parallelism
- **Communication**: Determine communication pattern among tasks
- **Agglomeration**: Combine into coarser-grained tasks, if necessary, to reduce communication requirements or other costs
- **Mapping**: Assign tasks to processors, subject to tradeoff between communication cost and concurrency



Designing Threaded Programs

- **Partition**
  - Divide problem into tasks
- **Communicate**
  - Determine amount and pattern of communication
- **Agglomerate**
  - Combine tasks
- **Map**
  - Assign agglomerated tasks to created threads





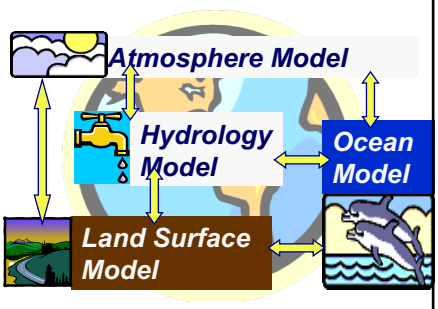
Parallel Programming Models

- Functional Decomposition
  - Task parallelism
  - Divide the computation, then associate the data
  - Independent tasks of the same problem
- Data Decomposition
  - Same operation performed on different data
  - Divide data into pieces, then associate computation

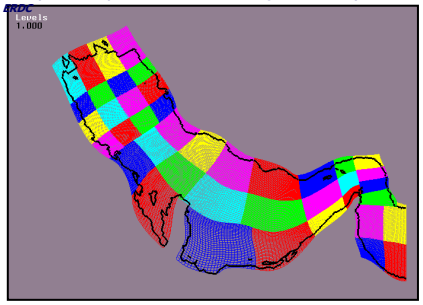


Decomposition Methods

- Functional Decomposition
  - Focusing on computations can reveal structure in a problem



Grid reprinted with permission of Dr. Phu V. Luong, Coastal and Hydraulics Laboratory, ERDC



Domain Decomposition

- *Focus on largest or most frequently accessed data structure*
- *Data Parallelism*
  - *Same operation applied to all data*



Example: Computing Pi



- We want to compute π
- One method: method of darts*
- Ratio of area of square to area of inscribed circle proportional to π



Disclaimer: this is a TERRIBLE way to compute π. Don't even think about doing it this way in real life!



Method of Darts

- Imagine dartboard with circle of radius R inscribed in square
- Area of circle $= \pi R^2$
- Area of square $= (2R)^2 = 4R^2$
- Area of circle / Area of square $= \frac{\pi R^2}{4R^2} = \frac{\pi}{4}$





Method of Darts



- So, ratio of areas proportional to π
- How to find areas?
  - Suppose we threw darts (**completely randomly**) at dartboard
  - Could count number of darts landing in circle and total number of darts landing in square
  - Ratio of these numbers gives approximation to ratio of areas
  - Quality of approximation increases with number of darts
    - $\pi = 4 \times \frac{\text{\# darts inside circle}}{\text{\# darts thrown}}$



Method of Darts

- Okay, but how in the world do we simulate this experiment on a computer?
  - Decide on length R
  - Generate pairs of random numbers (x, y) so that $-R \leq x, y \leq R$
  - If (x, y) within circle (i.e. if $(x^2+y^2) \leq R^2$), add one to tally for inside circle
  - Lastly, find ratio



51

Parallelization Strategies

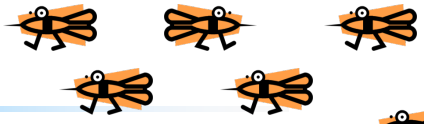
- What tasks independent of each other?
- What tasks must be performed sequentially?
- Using PCAM parallel algorithm design strategy



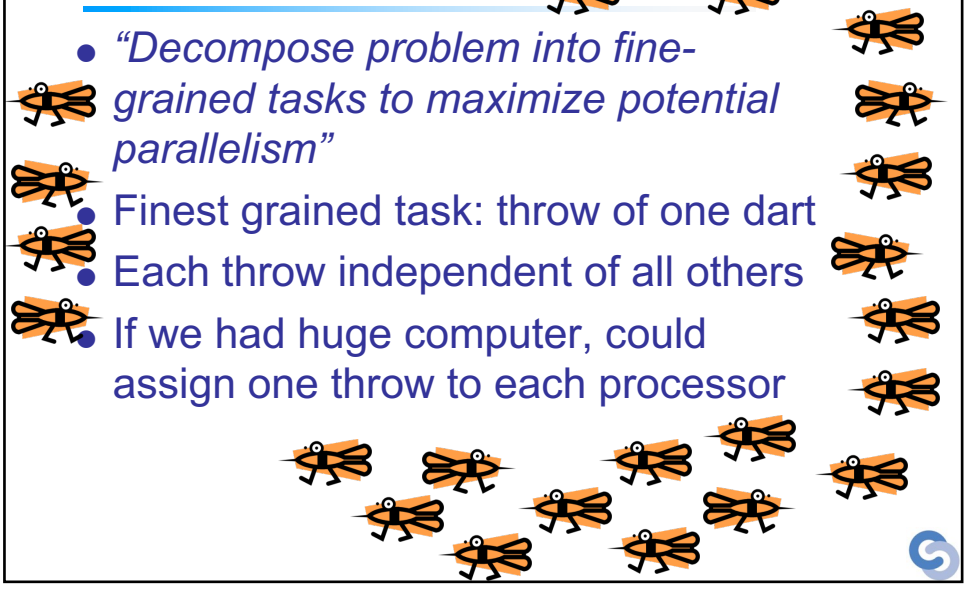
51

52

Partition



- *"Decompose problem into fine-grained tasks to maximize potential parallelism"*
- Finest grained task: throw of one dart
- Each throw independent of all others
- If we had huge computer, could assign one throw to each processor



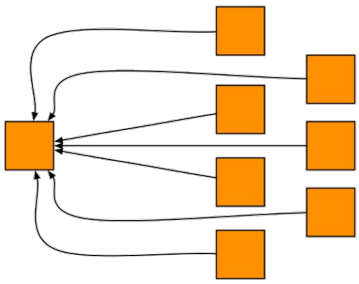


52

Communication

"Determine communication pattern among tasks"

- Each processor throws dart(s) then sends results back to manager process







Agglomeration

"Combine into coarser-grained tasks, if necessary, to reduce communication requirements or other costs"

- To get good value of π, must use millions of darts
- We don't have millions of processors available
- Furthermore, communication between manager and millions of worker processors would be very expensive
- Solution: divide up number of dart throws evenly between processors, so each processor does a share of work



55

Mapping

"Assign tasks to processors, subject to tradeoff between communication cost and concurrency"

- Assign role of "manager" to processor 0
- Processor 0 will receive tallies from all the other processors, and will compute final value of π
- Every processor, including manager, will perform equal share of dart throws

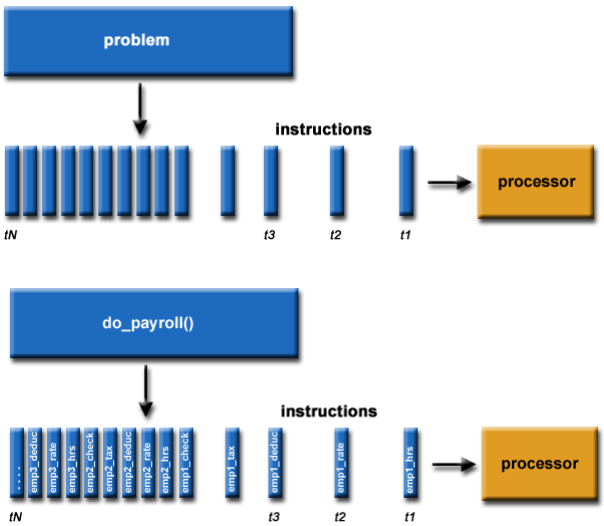




55

56

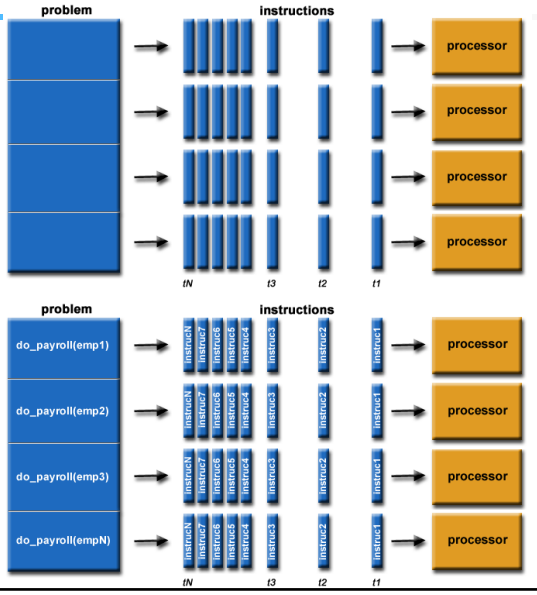
Serial Approach



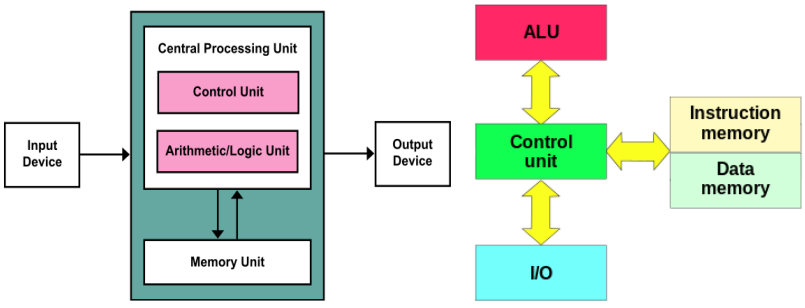


56

Parallel Approach



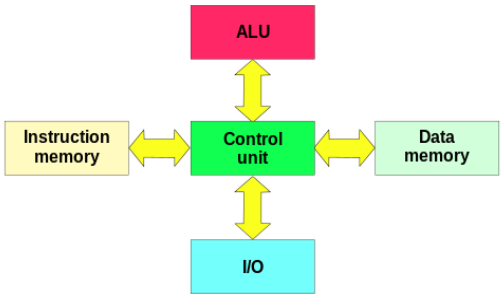
Von Neumann Architectures



- Instruction Flow
- Data Flow
- Single Address Space for Intsr & Data
- Low I/O Capabilities
- General computing use

59

Harvard Architecture



- **Separate** Instruction & Data Flow
- Separate Memory Spaces for Instructions & Data
- Used for DSPs



59

60

Most Current Computing Systems

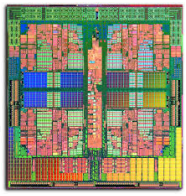
- Are parallel at multiple HW levels
  - L1 / L2
  - Prefetching
  - Pipeline
  - Floating point operations
  - GPU processing
  - Multi-cores
  - Many-cores
  - Multi-threading (HW)

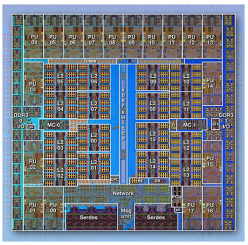


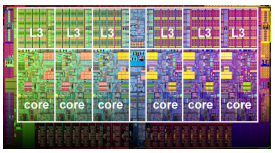
60

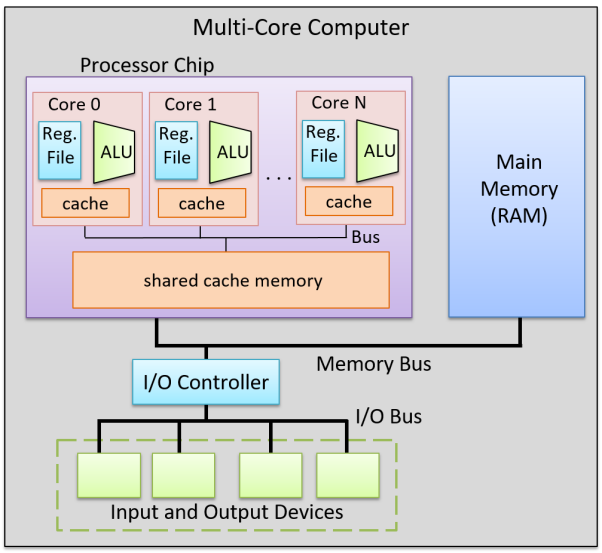
61

Multi-cores Processors





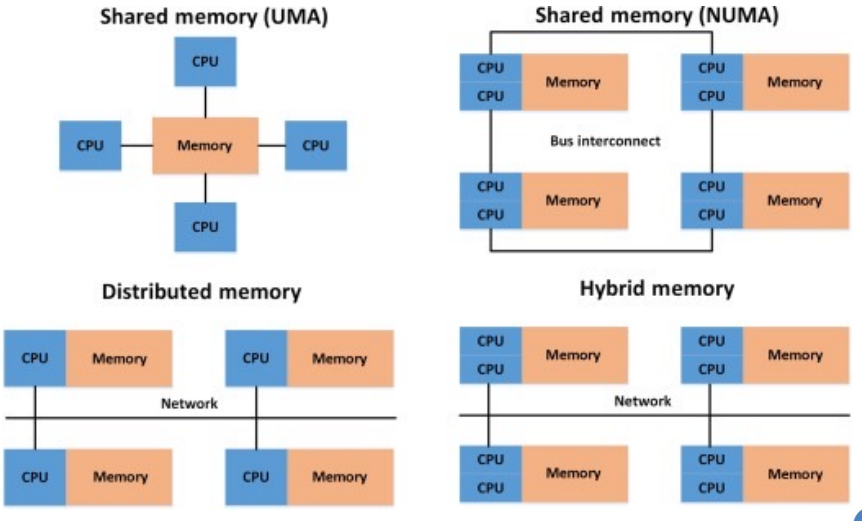




61

62

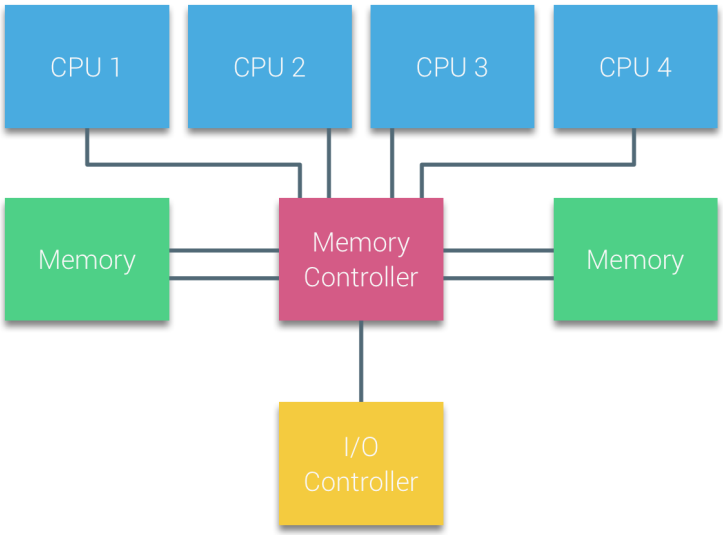
Evolution & Variants of Systems



62

63

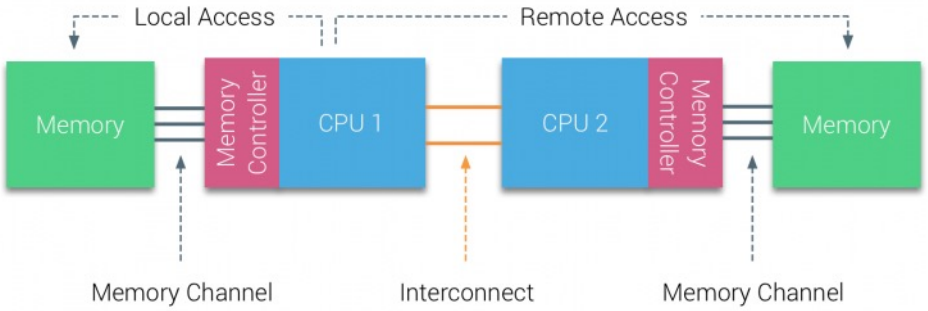
UMA – Uniform Memory Access



63

64

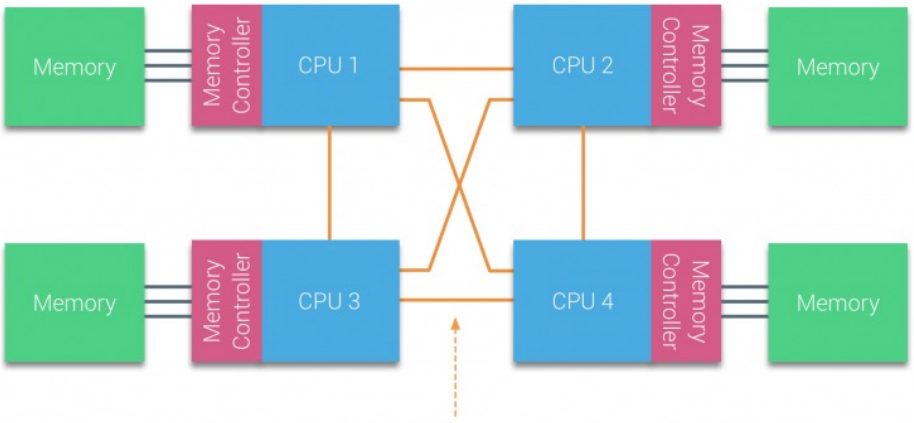
NUMA – Non-Uniform MA



64

65

Inter-processor Communication



HT
NVlink



65

66

UMA vs. NUMA

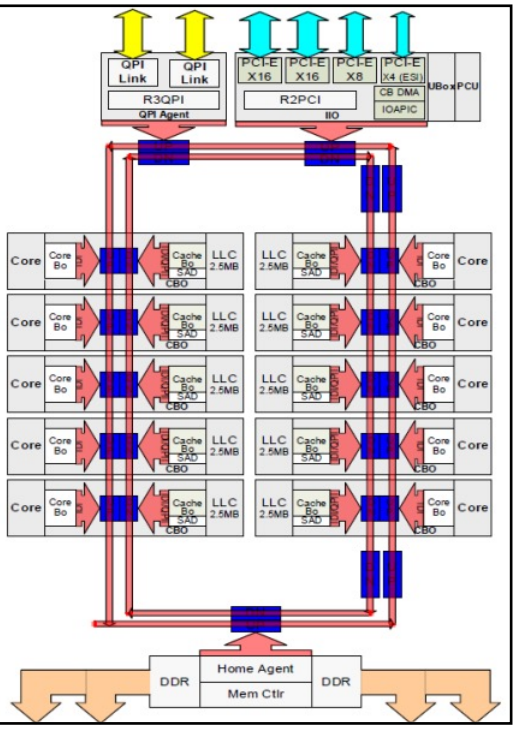
Uniform memory access	Non-uniform memory access
Offers limited bandwidth.	Offers relatively more bandwidth than UMA.
Since it uses a single memory controller, it is slower than NUMA.	It uses multiple memory controls to speed up operations compared to UMA.
Memory access time is equal or balanced.	Memory access time is unequal.
It is mainly used in time-sharing and general-purpose applications.	It offers better performance and speed, making it suitable for real-time and time-critical applications.



66

CC-NUMA

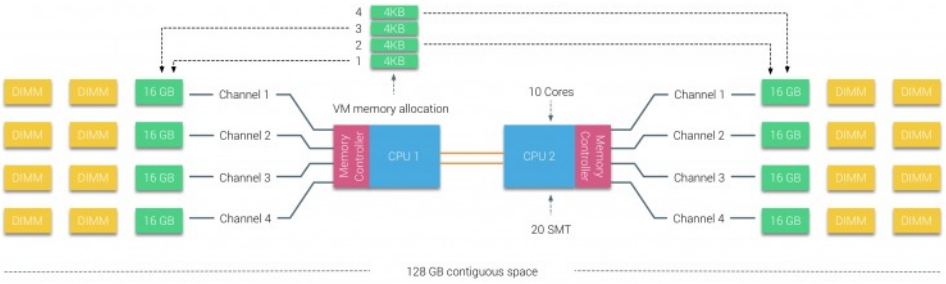
- Coherennce at the Cache Level
- Coherence solved in HW by snooping algorithm



67

68

SUMA – Sufficiently UMA



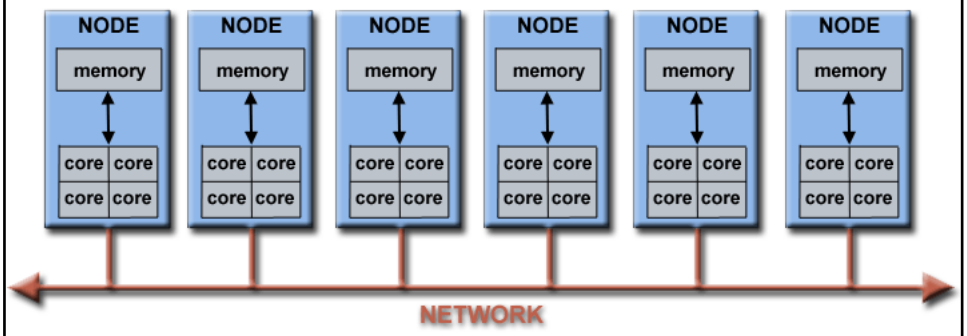
- Splits a NUMA in 4k segments
- Maps segments in round-robin across nodes
- Access time is uniform & depends on QPI/HT contention

NUMA	Memory Node 0	Memory Node 1	–	SUMA	Memory Node 0	Memory Node 1
Socket 0	75.7	132.0	–	Socket 0	105.5	106.4
Socket 1	131.9	75.8	–	Socket 1	106.0	104.6

68

69

Clusters of Computers





69

70

Cluster@CS – 2005-2012



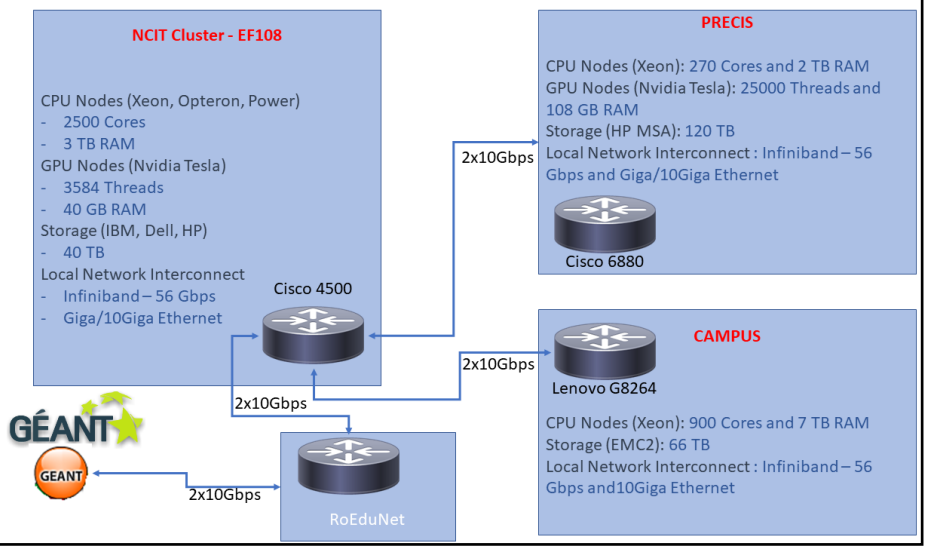




70

71

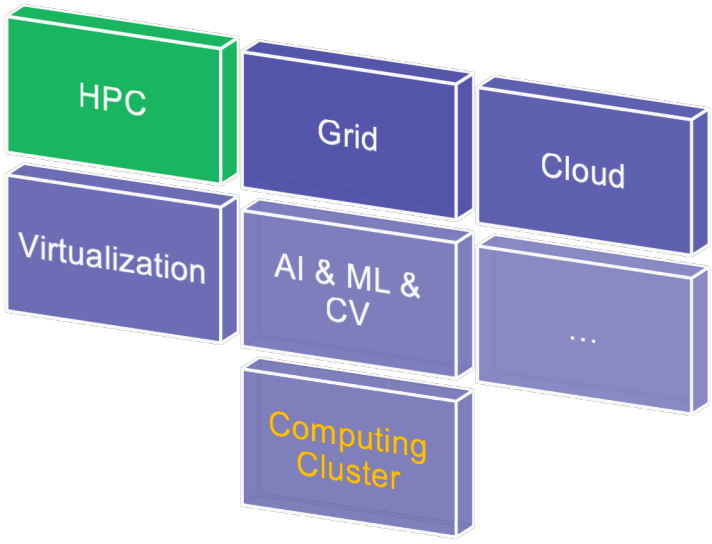
Clusters@CS – 2015-202x



71

72

Cluster Services



72

Our training & user domains

Hardware needs people to function

MPI OpenMP PThreads

CUDA / Multicore Computing

Summer Schools

Profiling / Tuning

Diploma

Master R&D

first sem. *second sem.* Σ *HPC / PP / Cluster / Cloud / Q*

3rd year of study *4th year of study* *2 years*



Projects Research PhDs

3-5 years



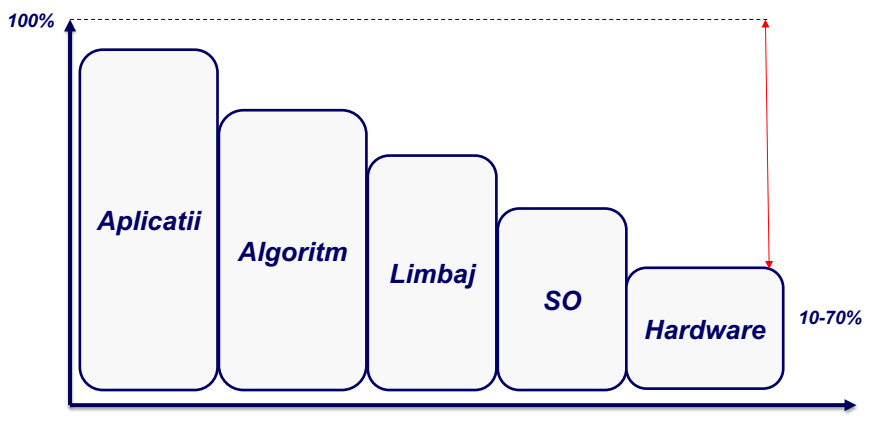
Problematici asociate calculului paralel

Problemele de baza in calculul parallel se refera la urmatoarele aspecte:

- Granularitatea taskurilor
- Elaborarea algoritmilor cu paralelism intrinsec
- Proiectarea limbajelor de programare
- Proiectarea unor arhitecturi hardware adecvate
- Proiectarea unor sisteme de operare adecvate



Gradul de Paralelism





Nivelurile de Paralelism

Paralelismul poate fi examinat la multe niveluri in functie de complexitatea dorita. Frecvent gasim descrierea paralelismului la nivelurile:

- Job — $JOB = \{ JOB_1, \dots, JOB_i, \dots\dots, JOB_m \}$
- Task — $JOB_i = \{ Ti_1, \dots, Ti_j, \dots\dots, Ti_n \}$
- Proces — $Ti_j = \{Pi_{j1}, \dots, Pi_{jk}, \dots, Pi_{jp}\}$
- Thread — $Pi_{jk} = \{THi_{jk1} \dots THi_{jkt}\}$
- Variabila
- Instructiune
- Bit



Nivelurile de Paralelism – Problema

- **Problema**
  - Un robot cu mai multe grade de libertate
  - Pentru fiecare grad de libertate exista un procesor
  - Programul de conducere al robotului este partitionat in task-uri care se ocupa de cite un grad de libertate al robotului
  - Taskurile se executa in paralel insa **interactioneaza** pentru miscarea robotului
- **Nivelul de Job** – Gestionarea robotului
  - $JOB = \{ JOB_1, \dots, JOB_i, \dots\dots, JOB_m \}$
- **Nivelul de Task** – Gestionarea joburilor robotului
  - $JOB_i = \{ Ti_1, \dots, Ti_j, \dots\dots, Ti_n \}$
  - Fiecare Ti se executa pe cate un procesor distinct
  - Exista o relatie intre Ti si Tj pentru transferul de date / completarea functiilor de prelucrare realizate in cadrul job-ului.



Nivelul de Task

- $T_j = \{Pi_{j1}, ...,Pi_{jk}, ..., Pi_{jp}\}$
- Task-urile sunt alcatuite din procese identificate de utilizator sau compilator pentru structuri multiprocesor

Task-ul

```
for i=1 to n
      x(i)=x(i-2)+ y(i-2)
      y(i)=x(i-2)* y(i-1)
      suma(i)=x(i) +y(i)
      if s(i) > a then c=c+1
                  else d=d+1
 end for
```

```
P1(i)
      x(i)=x(i-2) + y(i-2)
      y(i)=x(i-2) * y(i-1)
P2(i)
      suma(i)=x(i) +y(i)
      if s(i) > a then c=c+1
                  else d=d+1
```

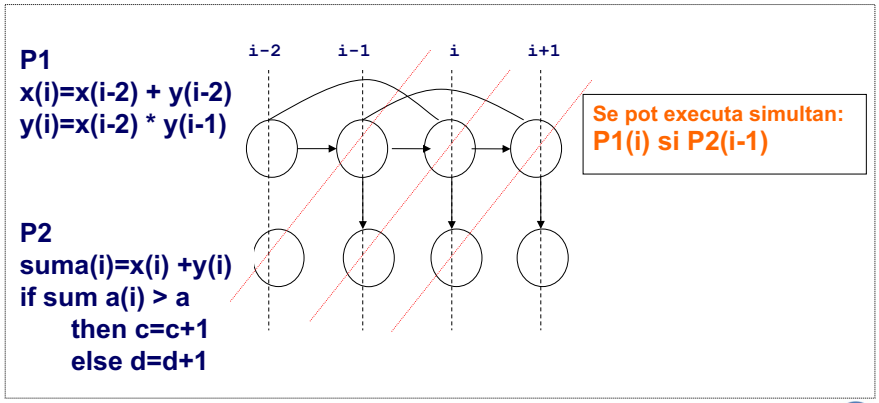
are doua procese P1 si P2

- Intre P1(i) si P2(i) exista o **dependenta de date**, deci nu pot fi efectuate in paralel
- Insa **P1(i)** si **P2(i-1)** pot fi efectuate in paralel



Nivelul de Proces

- Interdependenta intre procese este





Nivelul de Thread

- Pi_{jk} ={THi_{jk1} … THi_{jkt} }
- Procesele se mapeaza peste threadurile de la nivel de SO
- Runtime-ul SO decide modul de executie al threadurilor
- Sistemul de scheduling face maparea threadurilor utilizator peste cele SO



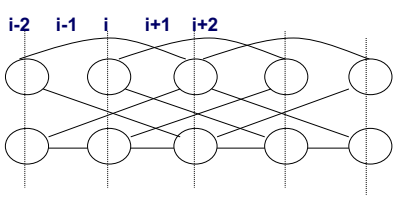
Nivelul de Variabila

- Pi= {I_{i1}, ….I_{ik}}
  - Fiecare proces este format dintr-o multime de instructiuni care calculeaza variabilele de iesire in functie de variabilele de intrare
  - Paralelismul in cadrul unui proces se realizeaza prin calculul simultan a mai multe variabile de iesire
  - Putem considera ca in cadrul procesului P_1 variabilele
    - x(i) si y(i) se pot calcula in paralel
    - x(i) si y(i+1) se pot calcula in paralel

P_1
x(i) = x(i-2) + y(i-2)
y(i) = x(i-2) * y(i-1)

x(i) = x(i-2) + y(i-2)

y(i) = x(i-2) * y(i-1)

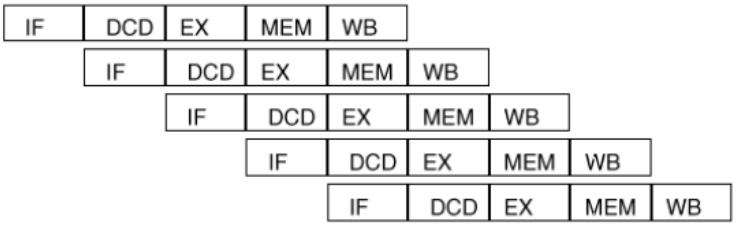




Nivelul de Instructiune

Generarea codului, pentru calculul expresiei z =x + y, in structura pipeline

lw r1, x	IF	ID	EX	MEM	WB				
lw r2, y		IF	ID	EX	MEM	WB			
add r3, r1, r2			IF	ID	stall	EX	MEM	WB	
sw z, r3				IF	stall	ID	EX	MEM	WB



Problema

- Consideram un procesor ce implementează o structura paralela pipeline de citire-interpretare-executie pentru procesare suprascalară
- Arătați îmbunătățirea performanței față de **procesarea scalară cu pipeline** și **procesarea fără pipeline**, presupunând ca intr-un ciclu de instrucțiune:
  - Citirea necesita **o singura** perioada de ceas
  - Decodarea necesita **două perioade** de ceas
  - Executia necesita **trei** perioade de ceas
- In total avem o secvență de **200 de instrucțiuni**

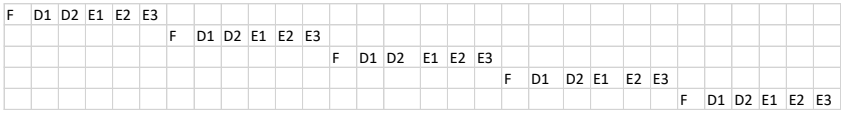
84

Solutie Simpla + Pipeline Scalara

- O structure fara pipeline necesita 1200 de cicli de ceas: (n * 6)

  200 * (1 + 2 + 3) = 1200

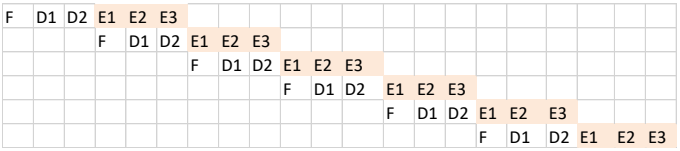
  n * (1 + 2 + 3)



- O structura pipeline scalară necesita 603 cicli de ceas: ((n-1) * 3 + 6)

  1 + 2 + (200 * 3) = 603

  1 + 2 + (n * 3)





84

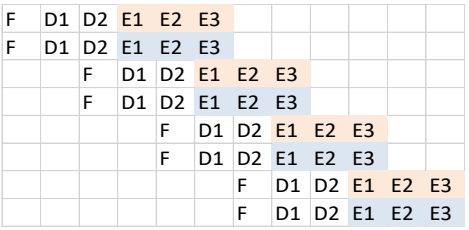
85

Solutie pipeline Superscalara x2

- Pipeline superscalar cu două unități paralele necesita 303 cicli de ceas (n / 2 + 1) * 3)

  1 + 2 + ((200/2) * 3) = 303

  1 + 2 + ((n / 2) * 3)

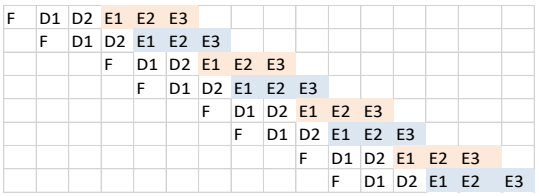


- Daca fetch-ul se realizeaza secvential

  1+2+((200/2)*4= 403

  1+2+((200/2)*(1+3))

  1 + 2 + ((n / 2) * 4)

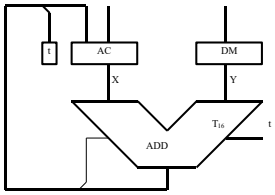


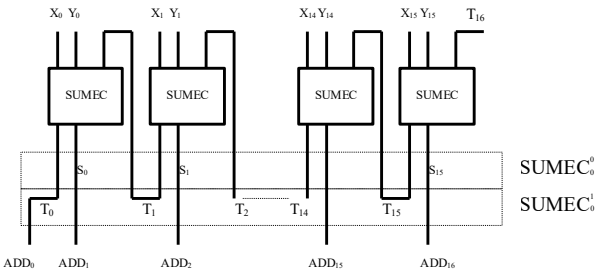


85

Nivelul de Bit

- Toate procesoarele utilizeaza unitati aritmetice paralele
  - Cu/fara anticiparea transportului
  - Cu/fara structura pipeline







Ce este Performanța Paralelă?

- Performanța paralela a unui sistem de calcul implica doi factori
  - Cerințele de calcul (ce trebuie făcut)
  - Resursele de calcul (cat costă să o faci)
- Problemele de calcul se traduc în cerințe de **cod/algoritmi**
- Resursele de calcul: **interacțiune** și **compromisuri** intre componentele sistemului
- **Performanța** este măsura a cât de bine pot fi satisfăcute cerințele de calcul
- **Evaluăm performanța** pentru a **înțelege relațiile** dintre cerințe și resurse
- Decidem cum "**schimbam soluțiile**" pentru a atinge obiectivele
- **Măsurile de performanță** reflectă deciziile despre **cum** și **cât de bine** sunt capabile „soluțiile" să satisfacă cerințe de calcul



88

Probleme de Performanță

- Când folosim un sistem paralel de calcul
  - Performanța este rațiunea de “a fi” pentru paralelism
  - Performanță vs. eficienta sistemului paralel de calcul
- Performanță paralelă vs. secvențială
- Fara **performanță** paralelismul este inutil…
- Prelucrarea în paralel include tehnici și tehnologii necesare pentru calculul în parallel:
  - Hardware, rețele, sisteme de operare, biblioteci paralele
  - Limbajele de programare, compilatoare, algoritmi, profiling, ...
- Paralelismul trebuie să ofere performanță
- Cum? Cât de bine?
  - Trebuie sa introducem / definim metrici si unitati de masura



88

89

Limitele Calcului Paralel

- Dacă fiecare procesor este evaluat la
  - k GFLOPS și există p procesoare,
- Ar trebui să vedem performanța k * p GFLOPS?
  - Dacă durează 100 de secunde la un procesor, ar trebui sa dureze 10 secunde pe 10 procesoare?
- Mai multi factori afectează performanța
  - Fiecare factor trebuie înțeles separat
- Dar ei interacționează între ei în moduri complexe
  - Soluția la o problemă poate crea alta
  - O problemă poate masca alta problema, etc.
- Scalarea (dimensionarea problemei) poate schimba performantele rezolvarii problemei
- Trebuie să înțelegem limitele calculului parallel
  - Performanta vs. eficienta



89

90

Embarassingly Parallel

- Un calcul “trivial” paralel este unul care poate fi împărțit în task-uri complet independete ce pot fi executate simultan
  - Într-un calcul **trivial paralel** nu există interacțiune între procese
  - Într-un calcul **aproape trivial paralel** există calcule ce trebuie distribuite și colectate / combinate într-un fel
- Calculele **trivial paralele** au potențial pentru a atinge performanta maximă pe sisteme de calcul paralele
  - Timpul T secvențial poate fi redus la T / P ruland în paralel cu P procesoare
  - De ce acest lucru nu este întotdeauna adevarat?



90

91

Algoritmi / Arhitecturi Paralele

- Prelucrarea paralela include algoritmi si arhitecturi paralele
- Un algoritm paralel poate fi considerat ca o colectie de procese independente ce se executa simultan, procesele comunicand in timpul executiei
- Un algoritm se executa pe unitati functionale hardware care in general constau din
  - Elemente de procesare
  - Module care transfer date
- Ceea ce intereseaza este cum se transpun eficient procesele pe unitatile functionale hardware



91

92

Algoritmi / Arhitecturi Paralele v2

- H.T.Kung propune corelarea Algoritmi/Arhitectura:

Algoritmi paraleli	Arhitecturi paralele
Granularitate	Complexitate procesor
Control concurrent/sincronizare	Mod de operare
Transferul datelor	Structura memoriei
Geometria comunicatiei	Retele de comutare
Complexitate algoritm	Numar de procesoare
Dependenta taskuri	Dimensiune memorie



92

93

Granularitate / Complexitate

- Granularitatea obiectelor se refera la complexitatea modulului care poate fi job, task, process, thread sau instructiune
  - De obicei exista posibilitati de paralelism la o granularitate mare dar care nu este exploatata deoarece implica o comunicatie intensa si o crestere a complexitatii software-ului dezvoltat
  - La acest nivel se face o analiza intre granularitate si comunicatie pentru a stabili solutia cea mai buna / eficienta



93

94

Control Conturent / Operare

- Control concurent se refera la schema de selectie a modulelor pentru executie
  - Trebuie sa satisfaca dependenta de date si control
  - Asigurarea excluderii mutuale a accesului la resurse
- Schemele de control sunt bazate pe:
  - Disponibilitatea datelor (data flow)
  - Controlul centralizat (sinchronized)
  - Cererile de acces (demand-driven)
- Exemple:
  - Algoritmii ce proceseaza matrice se potrivesc pe procesoare sistolice
  - Algoritmii ce utilizeaza transferuri conditionate / iregularitati se potrivesc pe arhitecturi asincrone / data-flow



94

95

Transfer / Geometrie / Complexitate

- Transferul datelor se refera utilizarea operanzilor
  - Datele furnizate de instructiuni pot fi utilizate ca "date pure" in structurile data-flow sau pot fi depuse in locatii adresabile in masinile Von Neumann
- Geometria comunicatiei se refera la sablonul de interactiune intre module
  - Geometria comunicatiei poate fi regulata sau neregulata
- Complexitatea algoritmilor se refera la numarul de operatii necesare pentru implementarea unui algoritm
  - Influenteaza direct numarul de procesoare si dimensionarea memoriei necesare / utilizate



95

96

Performanță și Scalabilitate

- Evaluare
  - Runtime secvențial (T_{seq}) este o funcție de
    - Dimensiunea problemei și arhitectura secventiala utilizata
  - Runtime paralel (T_{par}) este o funcție de
    - Dimensiunea problemei și arhitectura paralelă utilizata
    - Numarul de procesoare utilizate în execuție + modul de interconectare
- Performanță paralelă este afectată atat de algoritm cat si de arhitectura
- Scalabilitatea este:
  - Abilitatea algoritmului paralel de a **creste performanța proporțional** cu **numărul de procesoare** și cu **dimensiunea problemei**



96

97

Strong vs. Weak Scaling

- Strong scaling
  - Masoara cum timpul de executie **scade** odata cu **cresterea numarului de procesoare** pentru o **dimensiune fixa** a problemei
  - Este o masura a paralelismului intrinsec dintr-o problema
  - Foarte util pentru probleme cu granularitate crescuta
    - Taskuri independente ce pot fi executate simultan



97

98

Exemplu de Strong Scaling

- Presupunem ca un cod parallel rezolva o problema in 100s folosind un processor
- Daca folosim 2 procesoare si programul necesita 50s pentru rezolvare acesta a atins scalabilitate “**puternica**” perfecta
  - Programul a putut fi spart in subprobleme complet independente
  - Fiecare processor si-a putut executa taskul in jumatate din timpul necesar initial



98

99

Strong vs. Weak Scaling

- Weak scaling
  - Masoara cum timpul de executie **creste** odata cu **cresterea dimensiunii problemei** si a **numarului de procesoare**
  - **Incarcarea pe fiecare processor ramane constanta**
  - Este o masura a scalabilitatii unui cod la dimensiuni crescute



99

100

Exemplu de Weak Scaling

- Presupunem ca un cod parallel poate rezolva o problema de dimensiune 1M in 100s folosind un processor
- Daca vom creste dimensiunea problemei la 2M si vom utiliza 2 procesoare iar timpul de procesare ramane 100s programul atinge scalabilitate "**slaba**" perfecta
  - Programul a scalat la dimensiuni mai mari fara nicio pierdere de performanta



100

101

State of Play

- In practica, e rar ca un program paralel sa atinga scalare perfecta "strong" sau "weak"
- Masurand scalabilitatea strong/weak putem identifica zone in care putem imbunatati programele noastre
- Ce tip de scalabilitate conteaza mai mult, depinde de natura problemei/aplicatiei
  - Daca e importanta **scaderea timpului total de rezolvare** – **strong scaling** este mai relevant
  - Daca e importanta **rezolvarea unei probleme mari** ce **nu poate fi rezolvata de un singur processor** – **weak scaling** este mai relevant



101

102

Strong/Weak Scaling Powerups

- Evitati zonele secventiale de cod
- Folositi algoritmi eficienti de comunicare
- Utilizati algoritmi eficienti de load-balancing
- Echilibrati incarcarea wordloadurilor intre procesoare
- Realizati un pas de tunare / optimizare al executiei aplicatiilor paralele pentru hardware-ul specific pe care se realizeaza rularea



102

103

Scalabilitate vs. Eficienta

- Un program poate utiliza mai multe procesoare
  - Cum evaluezi scalabilitatea?
- Cum evaluezi performantele scalabilității?
- Evaluare comparativă
  - Dacă se dublează numărul de procesoare, la ce să ne așteptăm?
  - Scalabilitatea este liniară?
- Se utilizeaza o măsură de **eficiență paralelă**
  - Este menținută eficiența pe măsură ce crește dimensiunea problemei?
- Cum se evalueaza valorile de performanță?



103

Indicatori de Performantele ai Calculului Paralel – v1

104

- **Rata de executie** masoara rata de producere a unor rezultate in unitatea de timp a unei utitati single/multiprocesor
  - MIPS / GIPS / TIPS – milioane/giga/tera de instructiuni pe secunda
    - Inadecvat SIMD
  - MOPS / GOPS / TOPS – milioane/giga/tera de operatii pe secunda care masoara operatiile efectuate de unitatile de prelucrare
    - Ignora lungimea cuvantului si natura operatiilor
  - MFLOPS / GFLOPS / TFLOPS / PFLOPS – milioane /giga/tera/peta de operatii cu virgula mobila pe secunda
    - Util in zona numerica / putin relevant in zona alphanumerica sau ML/AI
  - MLIPS / GLIPS/ TLIPS – milioane /giga/tera de inferente logice pe secunda
    - Utile in aplicatii ML/AI



104

Indicatori de Performantele ai Calculului Paralel – v2

105

- Avem o structura paralela cu **p** procesoare
- **Viteza de prelucrare:** $V_p = \frac{T_1}{T_p}$ unde
  - T_1 – timpul pentru prelucrarea pe un procesor
  - T_p – timpul pentru prelucrarea pe p procesoare
  - V_p = raportul intre prelucrarea secventiala si cea paralela
  - Deoarece exista overhead cu sincronizarea: $1 \leq V_p < p$
- **Eficienta:** $E_p = \frac{V_p}{p} = \frac{T_1}{p * T_p} < 1$ unde
  - Eficienta este ca raportul dintre viteza de prelucrare si numarul de procesoare
  - Este o masura a eficientei costului
  - Cost p* T_p este p* T_p >> T_1



105

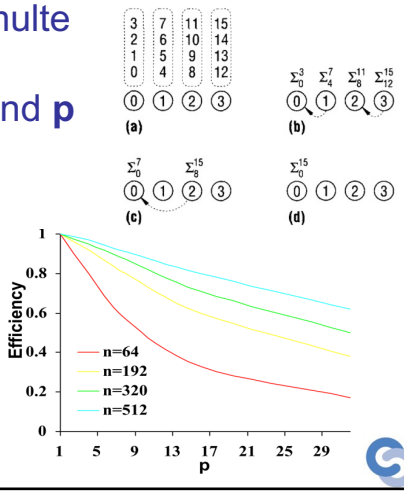
Indicatori de Performantele ai Calculului Paralel – v3

- **Redundanta:** $R_p = \frac{O_p}{O_1} =$ $\frac{\text{nr total de operatii efectuate pe cele p procesoare}}{\text{nr de operatii necesare efectuarii pe 1 procesor}}$ unde
  - **R_p** este raportul intre numarul total al operatiilor **O_p** necesar efectuarii calculului cu p procesoare si numarul de operatii **O_1** necesar efectuarii calculului cu un singur procesor
- **Utilizarea:** $U_p = \frac{O_p}{p * T_p} \leq 1$ unde
  - **U_p** este raportul dintre numarul de operatii **O_p** necesar efectuarii calculului cu p procesoare si numarul de operatii care ar fi putut fi efectuate cu **p** procesoare in timpul **T_p**



Exemplu – Scalabilitatea adunarii a n numere

- Scalabilitatea unui sistem paralel este masura capacitatii de a creste viteza de prelucrare utilizand mai multe procesoare
- Adunarea a **n** numere utilizand **p** procesoare conduce la:
- $T_p = \frac{n}{p} + 2 * \log p$
- $V_p = \frac{T_1}{T_p} = \frac{n}{\frac{n}{p} + 2 * \log p}$
- $E_p = \frac{V_p}{p} = \frac{T_1}{p * Tp} = \frac{n}{n + 2 * p * \log p}$
- $E_p = \frac{1}{1 + 2 * p/n * \log p}$



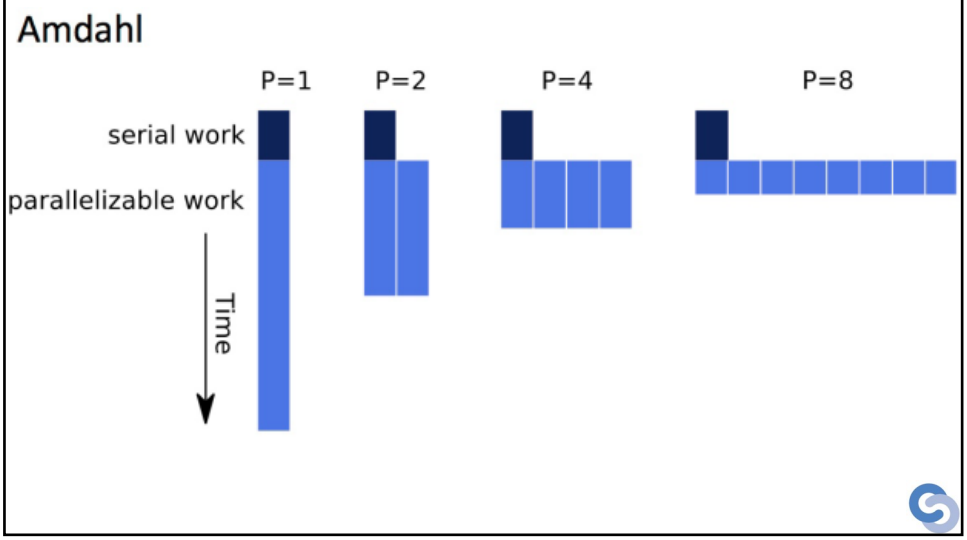


Legea lui Amdahl

- Sa consideram
  - f = f_{seq} procentul de program care se executa secvential
  - 1-f = f_{par} procentul de program care poate fi paralelizat
- Fie T_1 timpul de executie pe o structura cu 1 procesor
- Fie T_p timpul de executie pe o structura cu p procesoare
- Viteza de prelucrare
  - $V_p = \frac{T_1}{T_p} = \frac{T_1}{f*T_1+(1-f)*T_1/p} = \frac{1}{f+(1-f)/p)}$
  - $V_p = \frac{1}{f_{seq}+f_{par}/p} = \frac{1}{f_{seq}+(1-f_{seq})/p}$ cu $p \rightarrow \infty$
  - Astfel $V_p = \frac{1}{f} = \frac{1}{f_{seq}}$



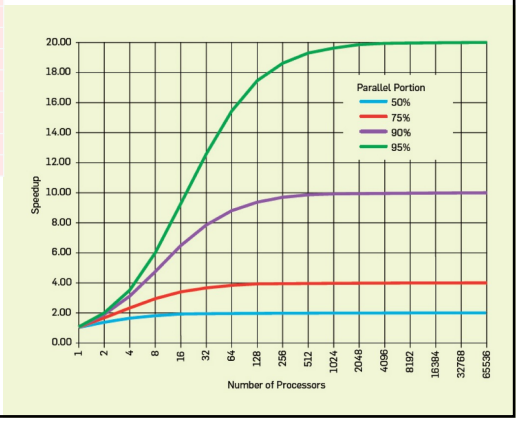
Legea lui Amdahl – v2





Scalabilitatea Legii lui Amdahl – v1

	Crestere viteza			
f este % prelucrare **secventiala**	*f=100%*	*f=50%*	*f=10%*	*f=1%*
1-f este % prelucare **paralela**	*(1-f)=0%*	*(1-f)=50%*	*(1-f)=90%*	*(1-f)=99%*
Numar procesoare				
1	1	1.00	1.00	1.00
2	1	1.33	1.82	1.98
5	1	1.67	3.57	4.81
10	1	1.82	5.26	9.17
100	1	1.98	9.17	50.25
1,000	1	2.00	9.91	90.99
10,000	1	2.00	9.99	99.02
100,000	1	2.00	10.00	99.90
1,000,000	1	2.00	10.00	99.99
10,000,000	1	2.00	10.00	100.00



Scalabilitatea Legii lui Amdahl – v2

- Capacitatea algoritmului paralel de a obține speedup e **proporțional cu numărul de procesoare** și **dimensiunea problemei**
- Când se aplică Legea lui Amdahl? $V_p = \frac{1}{f_{seq} + f_{par}/p}$
  - Când **dimensiunea problemei este fixa**
  - **Scalare puternică** ($p \to \infty$, $V_p \to 1 / fseq$)
- Speeup-ul este limitat de **partea seriala nu de numarul de procesoare**!
- Eficiența perfectă (strong) este greu de realizat!



112

Legea lui Gustafson-Barsis – v1

- Speedup pentru o problema scalata – perspectiva unui DataCenter
  - Probleme mari & utilizarea eficienta a resurselor
  - Problema poate fi scalata (modificarea dimensiunii!)
- Cât de mare poate fi o problema? (HPL/MD/QED)
- Care este contrangerea problemei care se executa in parallel?

Presupunem că timpul de executie este constant

- $\mathbf{T_p} = C = (f + (1-f)) * C = (f_{seq} + f_{par}) * C$
- $\mathbf{f_{seq}}$ este fracțiunea de $\mathbf{T_p}$ pentru execuția secvențială
- $\mathbf{f_{par}}$ este fracțiunea de $\mathbf{T_p}$ pentru execuție paralelă



112

113

Legea lui Gustafson-Barsis – v2

Scalarea pe **p** procesoare conduce la

- $T_s = f_{seq} + p*(1- f_{seq}) = f_{seq} + \mathbf{p*f_{par}}$, deoarece $f_{par} = 1- f_{seq}$
- $\mathbf{p*f_{par}}$ ne arata incarcarea centrului de date
- $T_s = 1- f_{par} + p*f_{par} = \mathbf{1 + (p-1)f_{par}}$
- $T_p = f_{seq} + (1-f_{seq})*p / p = 1$

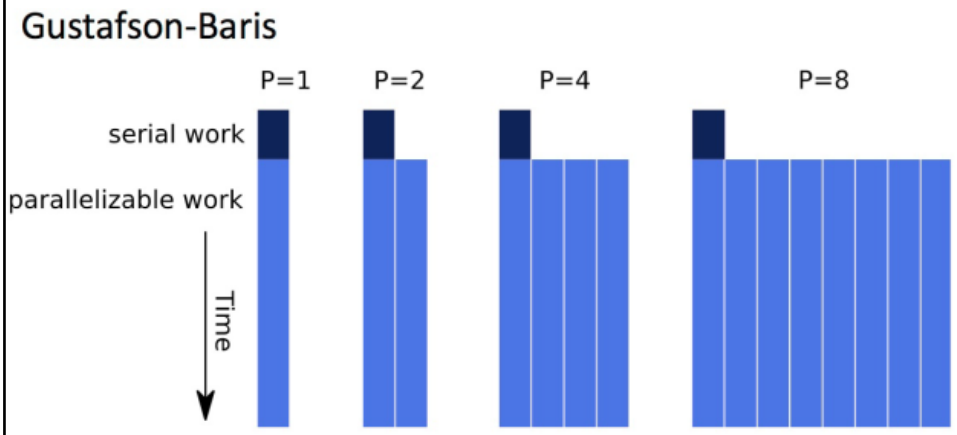
Care este speedup-ul în acest caz?

- $V_p = T_s / T_p = T_s / 1 = f_{seq} + p\,(1 - f_{seq}) = 1 + (p-1)\,f_{par}$



113

Legea lui Gustafson-Barsis – v3



Scalabilitate Gustafson-Barsis

Scalabilitate

- Capacitatea algoritmului paralel de a scala este **proporțional cu numărul de procesoare** și cu **dimensiunea problemei**
- Când se aplică Legea lui Gustafson?
  - Când dimensiunea problemei poate fi crescuta
  - Scalabilitate slabă (weak scaling) $V_p = 1 + (p-1)f_{par}$
  - Speedup-ul include si numărul de procesoare!
  - Cand se poate menține sau crește eficiența paralelă prin scalarea problemei

Comparatie Amdahl / Gustafson

- Considerand s fractiunea de cod serial f_{seq} de mai devreme si p procesoare
  - Legea lui Amdahl este Speedup = $V_p = \frac{1}{s + (1-s)/p}$
  - Legea lui Gustafson este Speedup = $V_p = \frac{1-s}{s + (1-s)/p}$
- Exemplu: daca s = 20% si regiunea de cod paralelizabil este astfel 80% cele doua legi indica:

Numar procesoare	*Speedup Legea lui Amdahl*	*Speedup Legea lui Gustafson*
2	1.6	2.5
4	2.2	4
8	2.6	5
16	2.9	6

- Gustafson prezice speedup-uri mai mari



Discutie Amdahl / Gustafson

- Care lege e mai “precisa”?
  - In practica, estimarea legii lui Gustafson este mai precisa decat legea lui Amdahl pentru ca majoritatea problemelor sunt scalabile
    - Se poate creste dimensiunea problemei pentru a folosi puterea de calcul
  - Atat Amdahl cat si Gustafson sunt limite teoretice!
- Speedup-ul real e mereu mai mic si depinde de
  - Eficienta algortimului parallel
  - Overheadul parallel
  - Dimensiunea problemei (weak scaling)
  - Dimensiunea sistemului de calcul (strong scaling)
- Legile ne ajuta sa intelegem limitarile si beneficiile sistemelor paralele de calcul



Legea lui Sun-Ni

- Ia in considerare limitarea memoriei per procesor
  - Memory-bound speedup
- Relevanta pentru sisteme HPC de mari dimensiuni unde limitarea este data de bandwidth-ul memoriei
- Formalizarea legii Sun-Ni: $T_p >= T_s / p + T_m$
  - Cu T_m timpul necesar de comunicare intre memoriile procesoarelor
  - T_s / p reprezinta speedup-ul ideal fara constrangeri de memorie
- Speedup = G(M) * p
  - G(M) = functie de utilizare a memoriei per procesor
  - p = numarul de procesoare din sistem



Control Speedup Memory-bound

- Metode de control
  - Utilizarea structurilor de date distribuite
  - Utilizarea mecanismelor de caching
  - Utilizarea compresiei de date
  - Utilizarea comunicatiilor asincrone
- Toate metodele de control presupun compromisuri
- Memoria este o resursa critica in cadrul sistemelor de calcul paralel
  - Cerintele de memorie sunt importante pentru designul algoritmilor paraleli eficienti



120

Limitari de memorie prin Sun-Ni

- Baze de date
  - Performanta accesarii BD limitata de memoria sistemului
  - Daca BD nu intra in memorie, citirea de pe disk va scadea timpul total de acces
- ML
  - Multiple modele ML necesita multa memorie (SVM, RF)
  - Antrenarea in batchuri reduce viteza de antrenare
- HPSC
  - CFD, MD necesita multa memorie pentru simulari
  - Daca datele nu intra in memorie, aplicatiile trebuie sa faca checkpointing pe disc crescand timpul de acces si reducand performanta simularii



120

121

Limita lui Worlton

- Reprezinta limita teoretica pentru speedup luand in considerare overheadul de comunicare intre procesoare
- Overheadul depinde de dimensiunea problemei
- Speedup = 1 / (1 + (C * p) / T)
  - C = overhead de comunicare pe unitatea de lucru
  - T = timpul necesar procesarii pe un singur procesor
  - p = numarul de procesoare
- Limita lui Worlton presupune ca overheadul de comunicare e proportional cu numarul de procesoare si dimensiunea datelor ce trebuiesc comunicate



121

Cresterea Vitezei de Prelucrare

- Trade-off-uri in Limita lui Worlton
  - Efectul sincronizarii poate fi redus prin micsorarea timpului de sincronizare sau prin marirea intervalului intre sincronizari
  - Efectul overhead-ului de comunicare poate fi redus prin reducerea timpului de overhead sau prin cresterea granularitatii task-urilor
  - Cresterea granularitatii task-urilor ajuta la reducerea atat a efectului sincronizarii cat si a overhead-ului.
  - Numarul pasilor de calcul poate fi redus prin
    - Cresterea numarului de procesoare
    - Luand un numar de task-uri multiplu de numarul de procesoare



Limitari Worlton

- In practica, speedup-ul e mai mic ca limita Worlton
  - Eficienta algoritmilor
  - Overheadul de comunicare al runtime-systemului paralel
- Examplu
  - Un sistem cu 16 proceoare
  - Overhead de comunicare / unitate de lucru de 0.1ms
  - Timp total de executie serial 100ms
  - Limita Worlton este:
  - Speedup <= 1 / (1 + (0.1 ms * 16) / 100 ms) = 5.88



Comparatii Amdahl / Gustafson / Sun & Ni / Worlton

124

Lege	Scop	Presupuneri	Limitari
Amdahl	Speedup maxim problema fixa (strong scaling)	Divizarea problemei in zone cunoscute (seriale / paralele)	Ignora overheadul de comunicare
Gustafson	Speedup Maxim pentru problema scalabila (weak scaling)	Dimensiunea problemei e scalabila	Ignora overheadul de comunicare
Sun-Ni	Speedup luand in considerare capacitatea memoriei	Divizarea problemei in zona memory-bound & non-memory bound	Presupune overhead de comunicare neglijabil
Worlton	Speedup luand in considerare overheadul de comunicare	Overheadul de comunicare / unitate de lucru si timpul de executie serial sunt cunoscute	Overhead de comunicare proportional cu p si memoria folosita

124

Comparatie Numerica Amdahl / Gustafson / Sun-Ni / Worlton

125

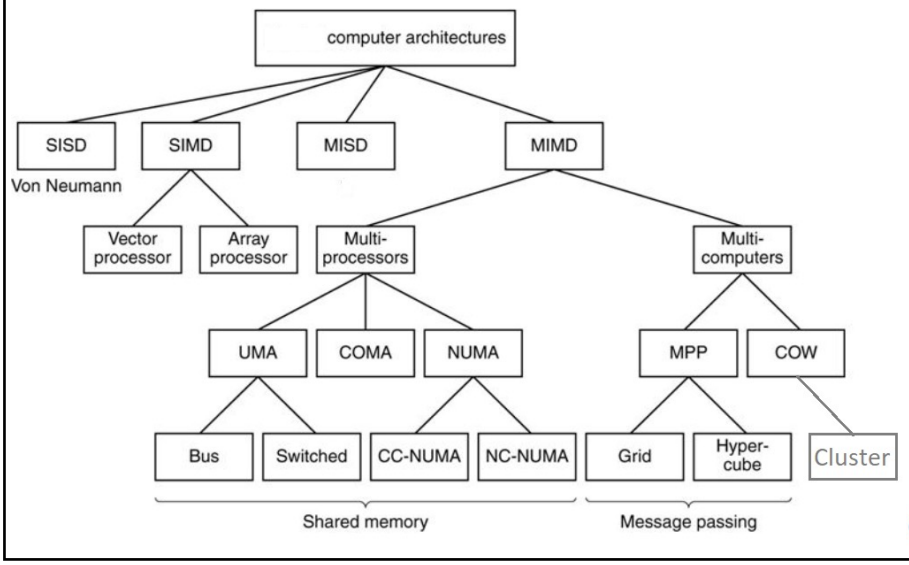
Lege	*Formula*	*Calcul*	*Limita*
Amdahl	S = 1 / (s + (1 - s) / p)	1 / (0.2 + (1 - 0.2) / 16)	**5**
Gustafson	S = (1 - s) / s + (1 - s) / p	(1 - 0.2) / 0.2 + (1 - 0.2) / 16	**6.88**
Sun-Ni	S = p * G(M)	16 * G(M)	**16 * G(M)**
Worlton	S <= 1 / (1 + (c * p) / T)	1 / (1 + (0.1 ms * 16) / 100 ms)	**5.88**

Un sistem cu: p = 16 procesoare, sectiune seriala s = 20%, timp executie T = 100ms si overhead de comunicare / unitate de lucru de c = 0.1ms



125

Arhitecturi Sisteme de Calcul



Classification

Flynn's Taxonomy (1966-now)		Nowadays
SISD *Single Instruction* *Single Data*	**SIMD** *Single Instruction* *Multiple Data*	**SPMD** *Single Program* *Multiple Data*
MISD *Multiple Instructions* *Single Data*	**MIMD** *Multiple Instructions* *Multiple Data*	**MPMD** *Multiple Program* *Multiple Data*

- Execution models impact the above programming model
- Traditional computer is SISD
- SIMD is *data parallelism* while MISD is pure *task parallelism*
- MIMD is a mixed model (harder to program)
- SPMD and MPMD are less synchronized than SIMD and MIMD
- SPMD is most used model, but MPMD is becoming popular

128

SIMD vs. SPDM

Caracteristica	SPMD	SIMD
Numar de procesoare	Multiple	Multiple
Flux Intructiuni	Unic	Unic
Flux Date	Multiplu	Multiplu
Spatiu de Memorie	Distribuit	Partajat
Comunicatie	Comunicare prin Mesaje	Broadcast / Shared
Aplicatii	HPC, ML, Data Mining	CV, DSP, Vectorial



128

129

SIMD vs. SPDM

- Ahitecturi SIMD
  - Graphics processing units (GPUs)
  - Vector processing units (VPUs)
  - SIMD units in CPUs
  - Arhitecturi mai eficiente pentru applicatii CV / DSP
- Ahitecturi SPMD
  - Beowulf clusters
  - Distributed shared memory (DSM) systems
  - Message passing interface (MPI) clusters
  - Arhitecturi mai flexibile
- Arhitecturi Hibride
  - Combinatie intre SPMD si SIMD
  - Clustere cu mix de CPU-uri + GPU-uri



129

130

Role of Parallelism?

- Goal of parallel computing
  - **Save time** - reduce wall clock time
    - Speedup - $\frac{\text{wall-clock time of serial execution}}{\text{wall-clock time of parallel execution}}$
  - **Solve larger problems** - problems that take **more memory than available to 1 CPU**



130

131

Reduce wall clock time

- Methods
  - Parallelizing serial algorithms (parallel loops)
    - Total number of operations performed changes only slightly
    - Scalability may be poor (Amdahl's law)
  - Develop parallel algorithms
    - Total number of operations may increase, but the running time decreases
- Work Complexity
  - Serialization: parallel algorithm executed sequentially Serializing parallel algorithm may lead to sub-optimal sequential complexity



131

Parallel Programming Models

- Introduction to Programming Models
  - Parallel Execution Models
    - Models for Communication
    - Models for Synchronization
    - Memory Consistency Models
    - Runtime systems
  - Productivity
  - Performance
  - Portability



Parallel Programming Models

Many languages and libraries exist for creating parallel applications.

Each presents a programming model to its users.

During this lecture, we'll discuss criteria for evaluating a parallel model and use them to explore various approaches.

OpenMP	Charm++	Linda
PThreads	UPC	MapReduce
MPI	STAPL	Matlab DCE
Cilk	X10	OpenCL
TBB	Fortress	CUDA
HPF	Chapel	…



Programming Models Evaluation

What should we consider when evaluating a parallel programming model?

- Parallel Execution Model
- Productivity
- Performance
- Portability

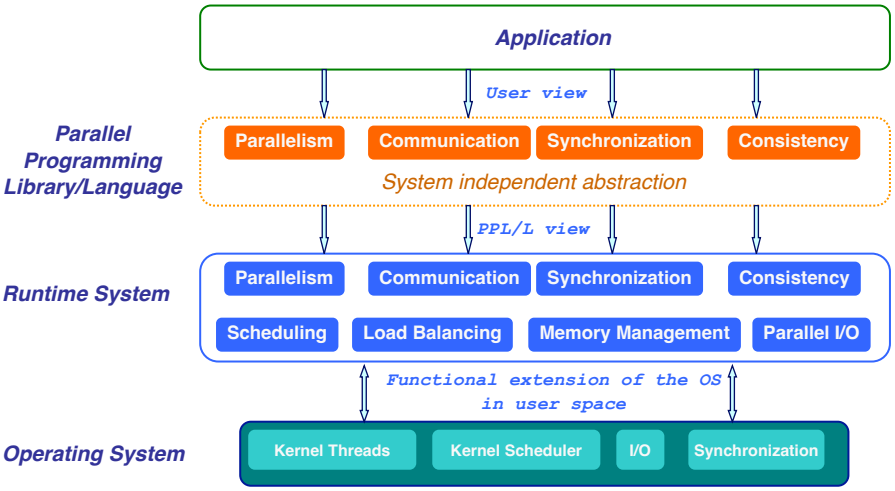


Parallel Programming Models

- Introduction to Programming Models
  - Parallel Execution Models
    - Models for Communication
    - Models for Synchronization
    - Memory Consistency Models
    - Runtime systems
  - Productivity
  - Performance
  - Portability



Parallel Execution Model



Parallel Execution Model

- Parallel Programming Model (user view)
  - Parallelism
  - Communication
  - Synchronization
  - Memory consistency
- Runtime System (RTS)
  - Introduction, definition and objectives
  - Usual services provided by the RTS
  - Portability / Abstraction

Parallel Programming Model (user view)

- Parallelism
- Communication
- Synchronization
- Memory consistency



PPM – Implicit Parallelism

Implicit parallelism (single-threaded view)

- User not required to be aware of the parallelism
  - User writes programs unaware of concurrency
    - Possible re-use previously implemented sequential algorithms
    - Often minor modifications to parallelize
  - User not required to handle synchronization or communication
    - Dramatic reduction in potential bugs
    - Straightforward debugging (with appropriate tools)
- Productivity closer to sequential programming
- Performance may suffer depending on application
- E.g. Matlab DCE, HPF, OpenMP*, Charm++*

* at various levels of implicitness



PPM – Explicit Parallelism

Explicit parallelism (multi-threaded view)

- User required to be aware of parallelism
  - User required to write parallel algorithms
    - Complexity designing parallel algorithms
    - Usually impossible to re-use sequential algorithms (except for embarrassingly parallel ones)
  - User responsible for synchronization and/or communication
    - Major source of bugs and faulty behaviors (e.g. deadlocks)
    - Hard to debug
    - Hard to even reproduce bugs
- Considered low-level
  - Productivity usually secondary
  - Best performance when properly used, but huge development cost
  - E.g. MPI, Pthreads



PPM – Mixed Parallelism

Mixed view

- Basic usage does not require parallelism awareness
- Optimization possible for advanced users

- Benefits from the two perspectives
  - High productivity for the general case
  - High performance possible by fine-tuning specific areas of the code
- E.g. STAPL, Chapel, Fortress



Parallel Programming Models

- Introduction to Programming Models
  - Parallel Execution Models
    - Models for Communication
    - Models for Synchronization
    - Memory Consistency Models
    - Runtime systems
  - Productivity
  - Performance
  - Portability



PPM – Implicit Communication

Implicit Communication

–Communication through shared variables

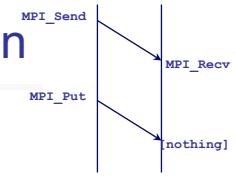
–Synchronization is primary concern
- Condition variables, blocking semaphores or monitors
- Full/Empty bit

–Producer/consumer between threads are expressed with synchronizations

–Increases productivity
- User does not manage communication
- Reduced risk of introducing bugs



PPM – Explicit Communication



Explicit Communication

- Message Passing (two-sided communication, P2P)
  - User explicitly sends/receives messages (e.g., MPI)
  - User required to match every Send operation with a Receive
  - Implicitly synchronizes the two threads
    - Often excessive synchronization (reduces concurrency)
    - Non-blocking operations to alleviate the problem (e.g., MPI_Isend/Recv)
- One-sided communication
  - User uses get/put operations to access memory (e.g., MPI-2, GASNet, Cray T3D)
  - No implicit synchronization (i.e., asynchronous communication)



PPM – Explicit Communication

Explicit Communication – Active Message, RPC, RMI

- Based on Message Passing
- Messages activate a handler function or method on the remote side
- Asynchronous
  - No return value (no `get` functions)
  - Split-phase programming model (e.g. Charm++, GASNet)
    - Caller provides a callback handler to asynchronously process “return” value
- Synchronous
  - Blocking semantic (caller stalls until acknowledgement/return is received)
  - Possibility to use `get` functions
- Mixed (can use both)
  - E.g., ARMI (STAPL)



Parallel Programming Models

- Introduction to Programming Models
  - Parallel Execution Models
    - Models for Communication
    - Models for Synchronization
    - Memory Consistency Models
    - Runtime systems
  - Productivity
  - Performance
  - Portability



PPM – Implicit Synchronization

Implicit Synchronization

- Hidden in communication operations (e.g., two-sided communication)
- Data Dependence Graph (DDG)
  - PPL synchronizes where necessary to enforce the dependences
  - E.g., STAPL
- Distributed Termination Detection
  - When implemented as background algorithm (e.g., in Charm++)
- Improved productivity
  - Less bugs from race conditions, deadlocks …
- E.g., STAPL, Charm++, MPI-1 and GASNet (to a certain extent)



PPM – Explicit Synchronization

Explicit Synchronization

- Critical section / locks
  - One thread allowed to execute the guarded code at a time
- Condition variables / blocking semaphores
  - Producer/consumer synchronization
  - Introduces order in the execution
- Monitors / counting semaphores
  - Shared resources management
- Barrier / Fence (global synchronization)
  - Threads of execution wait until all reach the same point
- E.g., Pthreads, TBB, OpenMP



Parallel Programming Models

- Introduction to Programming Models
  - Parallel Execution Models
    - Models for Communication
    - Models for Synchronization
    - Memory Consistency Models
    - Runtime systems
  - Productivity
  - Performance
  - Portability



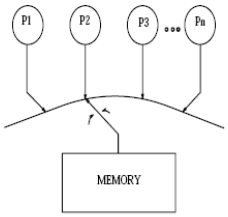
PPM – Memory Consistency

Introduction to Memory Consistency

– Specification of the effect of Read and Write operations on the memory

– Usual user assumption: Sequential Consistency

Definition: [A multiprocessor system is sequentially consistent if] the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.

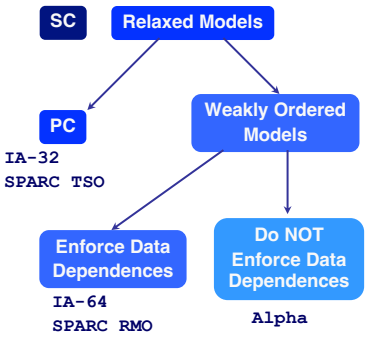


PPM – Memory Consistency

Introduction to Memory Consistency

Sequential Consistency: Don't assume it !

- Sequential Consistency (SC)
  - •MIPS/SGI
  - •HP PA-RISC
- Processor Consistency (PC)
  - •*Relax write→read dependencies*
  - •Intel x86 (IA-32)
  - •Sun TSO (Total Store Order)
- Relaxed Consistency (RC)
  - •*Relax all dependencies, but add fences*
  - •DEC Alpha
  - •IBM PowerPC
  - •Intel IPF (IA-64)
  - •Sun RMO (Relaxed Memory Order)



Material from: Hill, M. D. Revisiting "Multiprocessors Should Support Simple Memory Consistency Models", http://www.cs.wisc.edu/multifacet/papers/dagstuhl03_memory_consistency.ppt

PPM – Memory Consistency

Relaxed Memory Consistency Models

- Improve performance
  - Reduce the ordering requirements
  - Reduce the observed memory latency (hides it)
- Common practice
  - Compilers freely reorder memory accesses when there are no dependencies
  - Prefetching
  - Transparent to the user



Parallel Programming Models

- Introduction to Programming Models
  - Parallel Execution Models
    - Models for Communication
    - Models for Synchronization
    - Memory Consistency Models
    - Runtime systems
  - Productivity
  - Performance
  - Portability



Runtime System (RTS)

- Introduction
  - Definition
  - Objectives
- Usual services provided by the RTS
- Portability / Abstraction



154

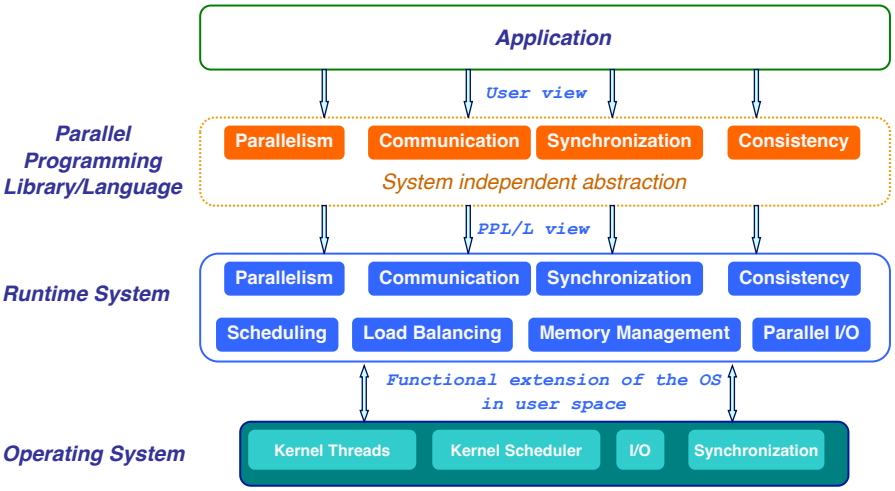
RTS – Introduction

- Software layer
  - Linked with the application
  - Executes in user space
- Provides applications with functionalities
  - Missing in the Operating System and drivers
  - More advanced/specialized than the OS counterpart



155

Parallel Execution Model



RTS – Definition*

- **Functional extension of the Operating System in user space**
  - No precise definition available
  - Fuzzy functional boundary between RTS and OS
    - Services are often a refined or extended version of the OS
    - Functional redundancy with OS services
      - Avoid entering Kernel space
      - Provide reentrancy
      - E.g., threading, synchronization, scheduling …
  - Widely variable set of provided services
    - No minimum requirements
    - No limit on the amount of functionality

*Non-formal, short definition

RTS – Objectives

- **Objectives of RTS for Parallel Programming Languages/Libraries:**
  - Enable portability
    - Decouple the PPL from the system
    - Exploit system-specific optimized features (e.g., RDMA, Coprocessor)
  - Abstract complexity of large scale heterogeneous systems to enable portable scalability
    - Provide uniform communication model
    - Manage threading, scheduling and load-balancing
    - Provide parallel I/O and system-wide event monitoring
  - Improve integration between application and system
    - Use application runtime information
      - Improve RTS services (e.g., scheduling, synchronization)
      - Adaptive selection of specialized code



158

RTS – Provided Services

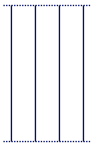
- **Common RTS provide a subset of the following** (not limited to)
  - Parallelism
    - Type of parallelism (API)
    - Threading Model (underlying implementation)
  - Communication
  - Synchronization
  - Consistency
  - Scheduling
  - Dynamic Load Balancing
  - Memory Management
  - Parallel I/O
- **Some functionalities are only provided as a thin abstraction layer on top of the OS service**



159

160

RTS – Flat Parallelism



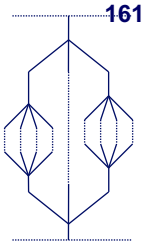
Parallelism types – Flat Parallelism

- All threads of execution have the same status
  - No parent/child relationship
- Threads are active during the whole execution
- Usually constant number of threads of execution
- Well adapted for problems with large granularity
- Difficult to achieve load-balance for non-embarrassingly parallel applications
- E.g. **MPI**



161

RTS – Nested Parallelism



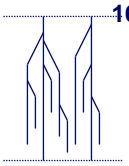
Parallelism types – Nested Parallelism

- Parallelism is hierarchal
  - Threads of execution can spawn new threads to execute their task
  - Exploits multiple levels of parallelism (e.g. nested parallel loops)
- Good affinity with heterogeneous architectures (e.g. clusters of SMPs)*
  - Allows the exploitation of different levels of granularity
- Natural fit for composed parallel data structures*
  - E.g. `p_vector< p_list< Type > >`
- E.g. **OpenMP**, Cilk, TBB

* Also for dynamic parallelism.



RTS – Dynamic Parallelism



Parallelism types – Dynamic Parallelism

- Threads of execution are dynamically created whenever new parallelism is available
  - Exploits any granularity of parallelism available
  - Necessary to achieve scalability for dynamic applications
- Improves load-balancing for dynamic applications
  - Work stealing
  - Thread migration
- Parallelism can be dynamically refined (e.g. mesh refinement*)
- E.g. STAPL, Charm++, AMPI, Chapel, **OpenMP Tasks**



* Can also be achieved by redistributing the data.

RTS – Threading Models (1:1)

- **1:1** threading model: (1 user-level thread mapped onto 1 kernel thread)
  - Default kernel scheduling
    - Possibility to give hints to scheduler (e.g., thread priority levels)
    - Reduced optimization opportunities
  - Heavy kernel threads
    - Creation, destruction and swapping are expensive
    - Scheduling requires to cross into kernel space
  - E.g., **PThreads**



RTS – Threading Models (M:1)

- **M:1** threading model: (M user-level threads mapped onto 1 kernel thread)
  - Customizable scheduling
    - Enables scheduler-based optimizations (e.g. priority scheduling, good affinity with latency hiding schemes)
  - Light user-level threads
    - Lesser threading cost
      - User-level thread scheduling requires no kernel trap
  - Problem: no effective parallelism
    - User-level threads' execution serialized on 1 kernel thread
    - Often poor integration with the OS (little or no communication)
    - E.g., GNU Portable Threads



RTS – Threading Models (M:N)

- **M:N** threading model: (M user-level threads mapped onto N kernel threads)
  - Customizable scheduling
    - Enables scheduler-based optimizations (e.g. priority scheduling, better support for relaxing the consistency model …)
  - Light user-level threads
    - Lesser threading cost
      - Can match N with the number of available hardware threads : no kernel-thread swapping, no preemption, no kernel over-scheduling …
      - User-level thread scheduling requires no kernel trap
    - Perfect and free load balancing within the node
      - User-level threads are cooperatively scheduled on the available kernel threads (they migrate freely).
  - E.g., PM2/Marcel



RTS – Communication

- Systems usually provide low-level communication primitives
  - Not practical for implementing high-level libraries
  - Complexity of development leads to mistakes
- Often based on other RTS libraries
  - Layered design conceptually based on the historic ISO/OSI stack
  - OSI layer-4 (end-to-end connections and reliability) or layer-5 (inter-host communication)
  - Communication data is not structured
  - E.g., MPI, Active Message, SHMEM
- **Objective:** Provide structured communication
  - OSI layer-6 (data representation) – data is structured (type)
  - E.g., RMI, RPC



166

RTS – Synchronization

- Systems usually provide low-level synchronization primitives (e.g., semaphores)
  - Impractical for implementing high-level libraries
  - Complexity of development leads to mistakes
- Often based on other RTS libraries
  - E.g., POSIX Threads, MPI …
- Objective: Provide appropriate synchronization primitives
  - Shared Memory synchronization
    - E.g., Critical sections, locks, monitors, barriers …
  - Distributed Memory synchronization
    - E.g., Global locks, fences, barriers …



167

RTS – Consistency

- In shared memory systems
  - Use system's consistency model
  - Difficult to improve performance in this way
- In distributed systems: relaxed consistency models
  - Processor Consistency
    - Accesses from a processor on another's memory are sequential
    - Limited increase in level of parallelism
  - Object Consistency
    - Accesses to different objects can happen out of order
    - Uncovers fine-grained parallelism
      - Accesses to different objects are concurrent
      - Potential gain in scalability



RTS – Scheduling

- Available for RTS providing some user-level threading (M:1 or M:N)
- Performance improvement
  - Threads can be cooperatively scheduled (no preemption)
  - Swapping does not require to cross into kernel space
- Automatically handled by RTS
- Provide API for user-designed scheduling



RTS – Dynamic Load Balancing

- Available for RTS providing some user-level threading (M:1 or M:N)
- User-level threads can be migrated
  - Push: the node decides to offload part of its work on another
  - Pull: when the node idles, it takes work from others (work stealing)
- For the M:N threading model
  - Perfect load balance within the node (e.g., dynamic queue scheduling of user-level threads on kernel threads)
  - Free within the node (I.e., no additional cost to simple scheduling)



RTS – Memory Management

- RTS often provide some form of memory management
  - Reentrant memory allocation/deallocation primitives
  - Memory reuse
  - Garbage collection
  - Reference counting
- In distributed memory
  - Can provide Global Address Space
    - Map every thread's virtual memory in a unique location
  - Provide for transparent usage of RDMA engines



RTS – Parallel I/O

- I/O is often the bottleneck for scientific applications processing vast amounts of data
- Parallel applications require parallel I/O support
  - Provide abstract view to file systems
  - Allow for efficient I/O operations
  - Avoid contention, especially in collective I/O
- E.g., ROMIO implementation for MPI-IO
- Archive of current Parallel I/O research:
  - http://www.cs.dartmouth.edu/pario/
- List of current projects:
  - http://www.cs.dartmouth.edu/pario/projects.html



RTS – Portability / Abstraction

- Fundamental role of runtime systems
  - Provide unique API to parallel programming libraries/languages
  - Hide discrepancies between features supported on different systems
- Additional layer of abstraction
  - Reduces complexity
  - Encapsulates usage of low-level primitives for communication and synchronization
- Improved performance
  - Executes in user space
  - Access to application information allows for optimizations



Parallel Programming Models

- Introduction to Programming Models
  - Parallel Execution Models
    - Models for Communication
    - Models for Synchronization
    - Memory Consistency Models
    - Runtime systems
  - Productivity
  - Performance
  - Portability



Productivity

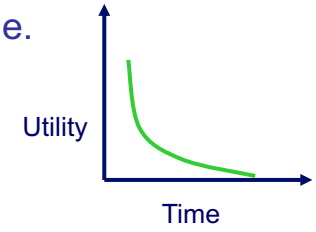
- Reduce time to solution
  - Programming time + execution time
- Reduce cost of solution
- Function of:
  - Problem solved *P*
  - System used *S*
  - Utility function *U*

$$\Psi = \Psi(P, S, U)$$

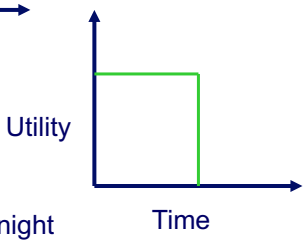


Utility Functions

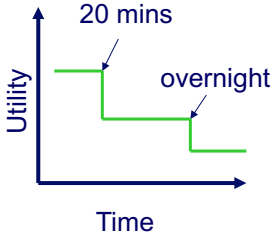
- Decreasing in time.



- Extreme example: deadline driven



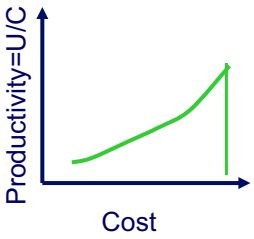
- Practical approximation: staircase





Simple Example

- Assume deadline-driven Utility and decreasing Cost



- Max productivity achieved by solving problem just fast enough to match deadline
- Need to account for uncertainty



Programming Model Impact

- Features try to reduce development time
  - Expressiveness
  - Level of abstraction
  - Component Reuse
  - Expandability
  - Base language
  - Debugging capability
  - Tuning capability
  - Machine model
  - Interoperability with other languages
- Impact on performance examined separately



Expressive

- Programming model's ability to express solution in:
  - The closest way to the original problem formulation
  - A clear, natural, intuitive, and concise way
  - In terms of other solved (sub)problems

Definition from http://lml.ls.fi.upm.es/~jjmoreno/expre.html



Level of Abstraction

- Amount of complexity exposed to developer



MATLAB

```
% a and b are matrices
c = a * b;
```

STAPL

```
// a and b are matrices
Matrix<double> c = a * b;
```

C

```
/* a and b are matrices */
double c[10][10];
int i, j, k;
for(int i=0; i<10; ++i) {
  for(int k=0; k<10; ++k) {
    for(int j=0; j<10; ++j) {
      c[i][j] += a[i][k]*b[k][j];
    }
  }
}
```



Component Reuse

- Goal: Increase reuse to reduce development time
- Programming model provides component libraries

STAPL pContainers and pAlgorithms

```
p_vector<double> x(100);
p_vector<double> y(100);

p_generate(x, rand);
p_generate(y, rand);

double result = p_inner_product(x,y);
```



Expandable

- Programming model provides a subset of components needed for a parallel application.
- Expansion enabled by:
  - Transparent components
  - Compositional construction



Component Transparency

- Opaque objects hide implementation details
  - raises level of abstraction
  - makes expansion difficult
- Transparent components
  - allow internal component reuse
  - working example in a programming model

```
int main() {
  pthread_t thread;
  pthread_attr_t attr;
  // …
}
```

```
template<class T>
class p_array : public p_container_indexed<T> {
  typedef p_container_indexed<T> base_type;
  size_t m_size;
  //…
};
```



Component Composition

Build a new component using building blocks.

```
template<typename View>
bool p_next_permutation(View& vw) {
  …
  reverse_view<View> rvw(vw);
  iter1 = p_adjacent_find(rvw);
  …
  iter2 = p_find_if(rvw, std::bind1st(pred,*iter1));
  …
  p_reverse(rvw);
  return true;
}
```



Programming Language

- Programming model language options:
  - Provide a new language
  - Extend an existing language
  - Provide directives for an existing language
  - Use an existing language

Fortress

```
component HelloWorld
  export Executable

  run()=do
    print "Hello, world!\n"
  end
end
```

Cilk

```
cilk void hello() {
  printf("Hello, world!\n");
}

int main() {
  spawn hello();
  sync;
}
```



Providing a new language

- Advantage
  - Complete control of level of abstraction
  - Parallel constructs embedded in language
- Disadvantage
  - Compiler required for every target platform
  - Developers must learn language

Fortress

```
component HelloWorld
  export Executable

  run()=do
    print "Hello, world!\n"
  end
end
```



Extending a language

- Advantage
  - Developers have less to learn
  - Complete control of level of abstraction
  - Parallel constructs embedded in syntax
- Disadvantage
  - Compiler required for every target system
  - Limited by constraints of base language

```
cilk void hello() {
  printf("Hello, world!\n");
}
int main() {
  spawn hello();
  sync;
}
```



Directives for a language

- Advantage
  - Developers have less to learn
  - Parallel constructs easily expressed in directives
  - Use available compilers if needed (no parallelization)
  - Specialized not necessarily needed on system
- Disadvantage
  - Compiler required for every target system
  - Higher levels of abstraction can't be achieved
  - Limited by constraints of base language
  - No (or limited) composition

```
#pragma omp parallel for
for(int i=0; i<N; ++i) {
  C[i] = A[i]*B[i];
}
```



188

Library for a language

- Advantage
  - Developers learn only new API
  - Compilers available on more systems
- Disadvantage
  - Limited by constraints of base language

```
void* hello(void*) {
  printf("Hello, world!\n");
  pthread_exit(NULL);
}
```

```
int main() {
  pthread_t thread;
  pthread_attr_t attr;
  pthread_attr_init(&attr);

  pthread_create(&thread, &attr,
   hello, NULL);
}
```



189

Debuggable

Programming environments provide many options for debugging parallel applications.

Built-in	Provides proprietary tools that utilize extra runtime information	Charm++
Tracing	Provides hooks for tools to log state during execution	MPI, Charm++
Interoperability with standard tools	Leverage standard tools available on platform (e.g., gdb, totalview)	STAPL, TBB, Pthreads, MPI, OpenMP



Defect Management

- Reduce Defect Potential
  - Programming style reduces likelihood of errors
  - Use of container methods reduces out-of-bounds accesses

```
class tbb_work_function {
  void operator()(const blocked_range<size_t>& r) {
    for(size_t i = r.begin(); i != r.end(); ++i)
      C[i] = A[i]*B[i];
  }
};
```

- Provide Defect Detection
  - Components support options to detect errors at runtime
  - E.g., PTHREAD_MUTEX_ERRORCHECK enables detection of double-locking and unnecessary unlocking



Tunability

Programming environments support application optimization on a platform using:

- Performance Monitoring
  - Support measuring application metrics
- Implementation Refinement
  - Support for adaptive/automatic modification of application
  - Manual mechanisms provided to allow developer to implement refinement



Performance Monitoring

- Built-in support
  - Environment's components instrumented
  - Output of monitors enabled/disabled by developer
  - Components written by developer can use same instrumentation interfaces
- Interoperable with performance monitoring tools
  - Performance tools on a platform instrument binaries



Implementation Refinement

- Adjust implementation to improve performance
  - Distribution of data in a container
  - Scheduling of iterations to processors
- Adaptive/Automatic
  - Monitors performance and improves performance without developer intervention
  - Example: Algorithm selection in STAPL
- Manual mechanisms
  - Model provides methods to allow developer adjustment to improve performance
  - Example: Grain size specification to TBB algorithms



Machine Model

- Programming models differ in the amount and type of machine information available to user
  - TBB, Cilk, OpenMP: user unaware of number of threads
  - MPI: user required to write code as a function of the machine in order to manage data mapping
- Programming as a function of the machine
  - Lowers level of abstraction
  - Increases programming complexity



Interoperability with other models

- Projects would like to use multiple models
  - Use best fit for each application module
  - Modules need data from one another
- Models need flexible data placement requirements
  - Avoid copying data between modules
  - Copying is correct, but expensive
- Models need generic interfaces
  - Components can interact if interfaces meet requirements
  - Avoids inheriting complex hierarchy when designing new components



Parallel Programming Models

- Introduction to Programming Models
  - Parallel Execution Models
    - Models for Communication
    - Models for Synchronization
    - Memory Consistency Models
    - Runtime systems
  - Productivity
  - Performance
  - Portability



Performance

- Latency Management
- Load Balancing
- Creating a High Degree of Parallelism



198

Performance - Memory Wall

Complex memory hierarchies greatly affect parallel execution.
Processing elements may share some components (e.g., L1/L2 caches, RAM), but usually not all.

Parallelism exacerbates the effects of memory latency.

- **Contention** from centralized components.
- **Non uniform latency** caused by distributed components.

Desktop Core2Duo
Private L1 Cache
Shared L2 Cache
Shared Centralized UMA

SGI Origin
Private L1 Cache
Private L2 Cache
Shared, Distributed NUMA

Linux Cluster
Private L1 Cache
Private L2 Cache
Private, Distributed NUMA



199

Performance - Memory Contention

The extent to which processes access the same location at the same time

- Types of contention and mitigation approaches
  - False sharing of cache lines
    - Memory padding to cache block size
  - 'Hot' memory banks
    - Better interleaving of data structures on banks
  - True Sharing
    - Replication of data structure
    - Locked refinement (i.e., distribution) for aggregate types
- Most models do not directly address contention



Performance - Managing Latency

There are two approaches to managing latency:

- Hiding: tolerate latency by overlapping a memory accesses with other computation
  - User Level
  - Runtime System
- Reducing: minimize latency by having data near the computation that uses it



Hiding Latency - User Level

Model has programming constructs that allow user to make asynchronous remote requests

Split-Phase Execution **(Charm++)**

Remote requests contain address of return handler:

```
class A {
  foo() {
    B b;
    b.xyz(&A::bar());
  }
  bar(int x) { … }
};
```

```
class B {
  xyz(Return ret) {
    …
    ret(3);
  }
};
```

- Futures

Remote requests create a handle that is later queried:

```
future<double> v(foo());   //thread spawned to execute foo()
…                          //do other unrelated work
double result = v.wait();  //get result of foo()
```



Hiding Latency - Runtime System

Runtime system uses extra parallelism made available to transparently hide latency

e.g., Multithreading **(STAPL / ARMI):**

- *pRange can recursively divide work - based on user defined dependence graph - to increase degree of parallelism*
- *ARMI splits and schedules work into multiple concurrent threads*

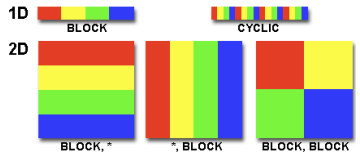


Performance - Latency Reduction

Data placement (HPF, STAPL, Chapel)

Use knowledge of algorithm access pattern to place all data for a computation near executing processor.

```
INTEGER, DIMENSION(1:16):: A,B
!HPF$ DISTRIBUTE(BLOCK) :: A
!HPF$ ALIGN WITH A :: B
```



Work placement (STAPL, Charm++)

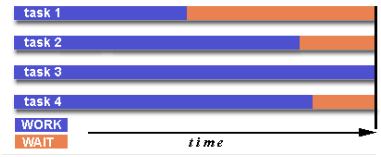
Migrate computation to processor near data and return final result. Natural in RMI based communication models.



204

Load Balancing

Keep all CPUs doing equal work.
Relies on good *work scheduling*.



- Static **(MPI)**
  Decide before execution how to distribute work.

- Dynamic **(Cilk, TBB)**
  Adjust work distribution during execution.
  - **Requires finer work granularity** (> 1 task per CPU)
    Some models change granularity as needed (minimize overhead).
  - Work Stealing
    Allow idle processors to 'steal' queued work from busier processors.



205

Enabling a High Degree of Parallelism

Parallel models must strive for a high degree of parallelism for maximum performance:

Makes transparent latency hiding easy

Enables finer granularity needed for load balancing



206

Parallel Programming Models

- Introduction to Programming Models
  - Parallel Execution Models
    - Models for Communication
    - Models for Synchronization
    - Memory Consistency Models
    - Runtime systems
  - Productivity
  - Performance
  - Portability



207

Portability

- Language versus Library
- Runtime System
  - Interchangeable
  - Virtualization
  - Load balancing
  - Reliance on specific machine features
- Effects of exposed machine model on portability
- I/O Support



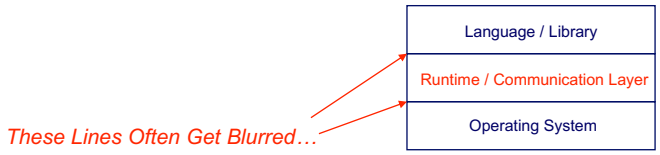
Language versus Library

- Models with specialized language require a compiler to be ported and sometimes additional runtime support.
  - Cray's **Chapel**, **Titanium**, Sun's **Fortress**.
- Library approaches leverage standard toolchains, and often rely on widely available standardized components.
  - **STAPL** requires C++, Boost, and a communication subsystem (MPI, OpenMP, Pthreads).
  - **MPI** requires communication layer interfaceand command wrappers (mpirun) to use portable versions (MPICH or LamMPI). Incremental customization can improve performance.



Runtime System

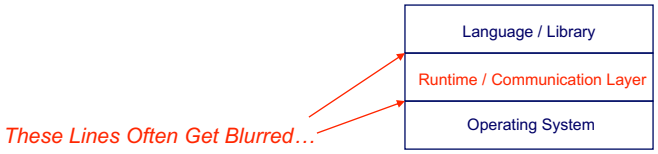
- **Interchangeable**
  Runtime system (e.g., threading and communication management) specific to model or is it modular?
- **Processor Virtualization**
  How are logical processes mapped to processors?
  Is it a 1:1 mapping or multiple processes per processor?





Runtime System

- **Load Balancing**
  Support for managing processor work imbalance?
  How is it implemented?
- **Reliance on Machine Features**
  Runtime system require specific hardware support?
  Can it optionally leverage hardware features?





Effects of Parallel Model

What effect does the model's level of abstraction have in mapping/porting to a new machine?

- Does it hide the hardware's model (e.g., memory consistency) or inherit some characteristics? Portability implications?
- Is there interface of machine characteristics for programmers? Optional use (i.e., performance tuning) or fundamental to code development?



Support for I/O

Some parallel models specifically address I/O, providing mechanisms that provide an abstract view to various disk subsystems.

ROMIO - *portable I/O extension included with MPI (Message Passing Interface).*



Shared Memory Programming

- Smaller scale parallelism (100's of CPUs/cores)
- Single system image
- Thread-based
- Threads have access to entire shared memory
  - Threads may also have private memory

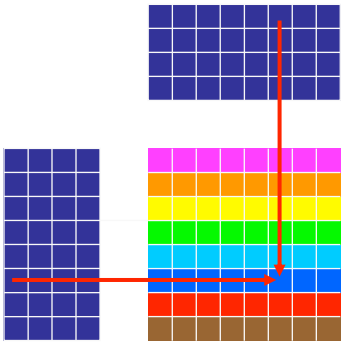


Shared Memory Programming

- No explicit communication
  - Threads write/read shared data
  - Mutual exclusion used to ensure data consistency
- Explicit Synchronization
  - Ensure correct access order
  - E.g., don't read data until it has been written



Example - Matrix Multiply



```
for(int i=0; i<M; ++i) {
  for(int j=0; j<N; ++j) {
    for(int k=0; k<L; ++k) {
      C[i][j] +=
          A[i][k]*B[k][j];
    }
  }
}
```

One way to parallelize is to compute each row Independently.



OpenMP

- Allows explicit parallelization of loops
  - Directives for Fortran and C/C++
  - Support for task parallelism

```
#pragma omp parallel for
for(int i=0; i<N; ++i) {
  C[i] = A[i] + B[i];
}
```

- Vendor standard
  - ANSI X3H5 standard in 1994 not adopted
  - OpenMP standard effort started in 1997
  - KAI first to implement new standard

Materials from http://www.llnl.gov/computing/tutorials/openMP/ & ©Intel



The OpenMP Model

Execution Model

- Explicitly parallel
- Single-threaded view
- SPMD
- Implicit data distribution
- Nested parallelism support
- Relaxed consistency within parallel sections



The OpenMP Model

Productivity

- Provides directives for existing languages
- Low level of abstraction
- User level tunability
- Composability supported with nesting of critical sections and parallel loops

Performance

- Load balancing
  - Optional selection of runtime scheduling policy
- Scalable parallelism
  - Parallelism proportional to data size



The OpenMP Model

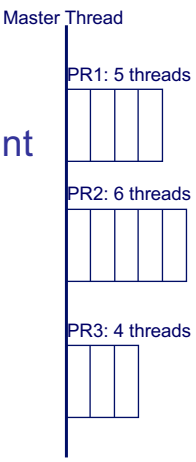
Portability

- Directives allow use of available compilers
  - Application compiles and runs, but no parallelization
- Supports processor virtualization
  - N:1 mapping of logical processes to processors
- Load balancing
  - Optional selection of runtime scheduling policy
- No reliance on system features
  - Can utilize specialized hardware to implement Atomic update



OpenMP Thread Management

- Fork-Join execution model
- User or developer can specify thread count
  - Developer's specification has priority
  - Variable for each parallel region
  - Runtime system has default value
- Runtime system manages threads
  - User/developer specify thread count only
  - Threads "go away" at end of parallel region





OpenMP Thread Management

- Determining number of threads
  - omp_set_num_threads(int) function
  - OMP_NUM_THREADS environment variable
  - Runtime library default
- Threads created only for parallel sections



Creating Parallel Sections

- Parallel for

```
#pragma omp parallel for \
  shared(a,b,c,chunk) \
  private(i) \
  schedule(dynamic,chunk)
for (i=0; i < n; i++)
  c[i] = a[i] + b[i];
```

- Options
  - Scheduling Policy
  - Data Scope Attributes

- Parallel region

```
#pragma omp parallel
{
  // Code to execute
}
```

- Options
  - Data Scope Attributes



Data Scope Attributes

Private	Variables are private to each thread
First Private	Variables are private and initialized with value of original object before parallel region
Last Private	Variables are private and value from last loop iteration or section is copied to original object
Shared	Variables shared by all threads in team
Default	Specifies default scope for all variables in parallel region
Reduction	Reduction performed on variable at end of parallel region
Copy in	Assigns same value to variables declared as thread private



OpenMP Synchronization

- Mutual exclusion by critical sections

```
#pragma omp parallel
{
  // …
  #pragma omp critical
  sum += local_sum

}
```

- Named critical sections
  - Unnamed sections **treated as one**
- Critical section is scoped
- **Always use named critical sections**

- Atomic update

```
#pragma omp parallel
{
  // …
  #pragma omp atomic
  sum += local_sum
}
```

- Specialized critical section
- May enable fast HW implementation
- Applies to following statement
- **Use atomic update whenever you're having just one statement**



OpenMP Synchronization

- Barrier directive
  - Thread waits until all others reach this point
  - Implicit barrier at end of each parallel region

```
#pragma omp parallel
{
  // … some code
  #pragma omp barrier
  // … some other code
}
```



OpenMP Scheduling

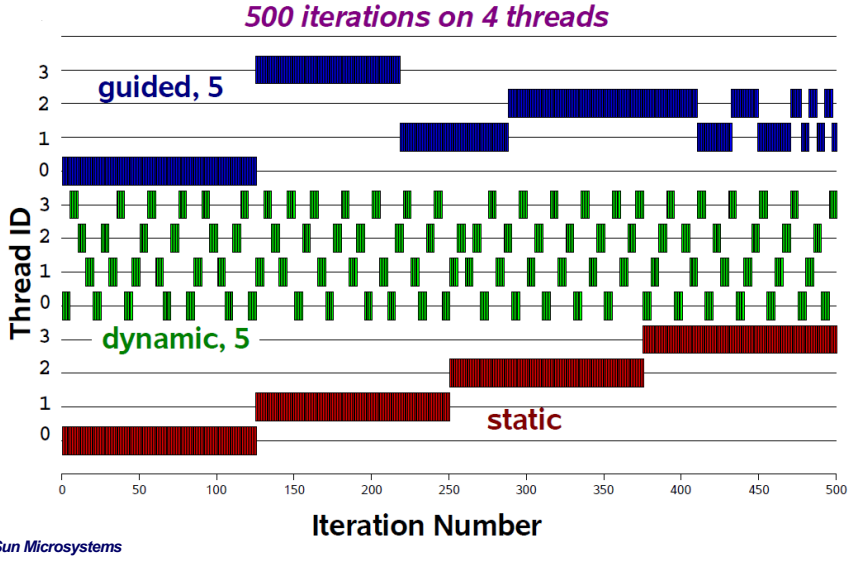
- Load balancing handled by runtime scheduler
- Scheduling policy can be set for each parallel loop

Scheduling Policies

Static	Create blocks of size *chunk* and assign to threads before loop begins execution. Default chunk creates equally-sized blocks.
Dynamic	Create blocks of size *chunk* and assign to threads during loop execution. Threads request a new block when finished processing a block. Default chunk is 1.
Guided	Block size is proportional to number of unassigned iterations divided by number of threads. Minimum block size can be set.
Runtime	No block size specified. Runtime system determines iteration assignment during loop execution.



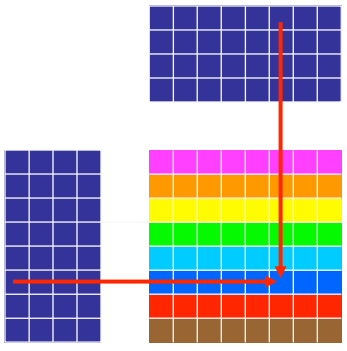
OpenMP Scheduling



©Sun Microsystems



OpenMP Matrix Multiply

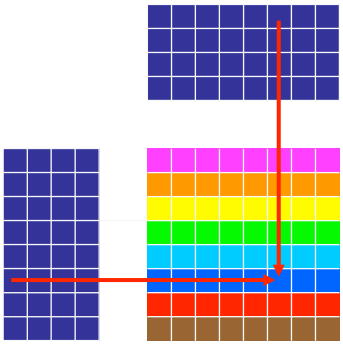


```
#pragma omp parallel for
for(int i=0; i<M; ++i) {
  for(int j=0; j<N; ++j) {
    for(int k=0; k<L; ++k) {
      C[i][j] +=
          A[i][k]*B[k][j];
    }
  }
}
```



OpenMP Matrix Multiply

- Parallelizing two loops
  - Uses nested parallelism support
  - Each element of result matrix computed independently

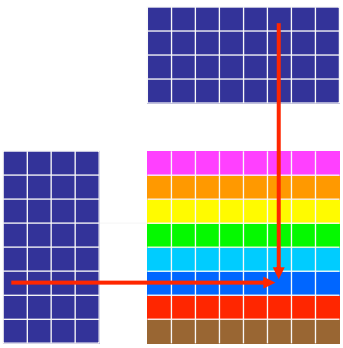


```
#pragma omp parallel for
for(int i=0; i<M; ++i) {
  #pragma omp parallel for
  for(int j=0; j<N; ++j) {
     for(int k=0; k<L; ++k) {
      C[i][j] +=
          A[i][k]*B[k][j];
    }
  }
}
```



OpenMP Matrix Multiply

- Parallelizing inner loop
  - Inner loop parallelized instead of outer loop
    - Minimizes work in each parallel loop – for illustration purposes only
  - Multiple threads contribute to each element in result matrix
  - Critical section ensures only one thread updates at a time

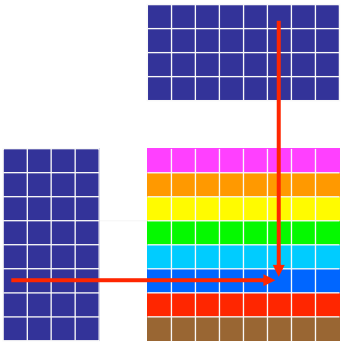


```
for(int i=0; i<M; ++i) {
  for(int j=0; j<N; ++j) {
    #pragma omp parallel for
    for(int k=0; k<L; ++k) {
      #pragma omp critical
      C[i][j] +=
          A[i][k]*B[k][j];
    }
  }
}
```



OpenMP Matrix Multiply

- Use dynamic scheduling of iterations



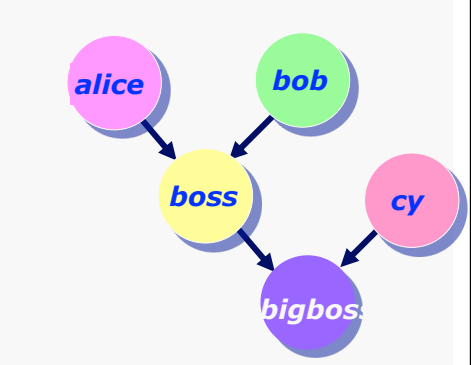
```
#pragma omp parallel for \
schedule(dynamic)
for(int i=0; i<M; ++i) {
  for(int j=0; j<N; ++j) {
    for(int k=0; k<L; ++k) {
      C[i][j] +=
          A[i][k]*B[k][j];
    }
  }
}
```



OpenMP 3.0 – Task Decomposition

```
a = alice();
b = bob();
s = boss(a, b);
c = cy();
printf ("%6.2f\n",
  bigboss(s,c));
```

alice, bob, and cy can be computed in parallel





omp sections

- **#pragma omp sections**
- Must be inside a parallel region
- Precedes a code block containing of *N* blocks of code that may be executed concurrently by N threads
- Encompasses each omp section

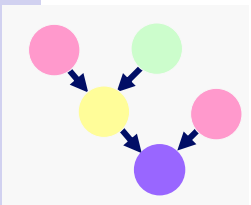
- **#pragma omp section**
- Precedes each block of code within the encompassing block described above
- May be omitted for first parallel section after the parallel sections pragma
- Enclosed program segments are distributed for parallel execution among available threads



Functional Level Parallelism w sections

```
#pragma omp parallel sections
{
#pragma omp section  /* Optional */
   a = alice();
#pragma omp section
   b = bob();
#pragma omp section
   c = cy();
}

s = boss(a, b);
printf ("%6.2f\n",
          bigboss(s,c));
```

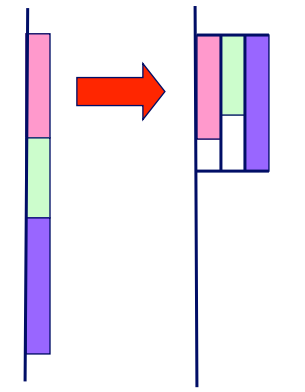




Advantage of Parallel Sections

- Independent sections of code can execute concurrently – reduce execution time

```
#pragma omp parallel sections
{
    #pragma omp section
    phase1();
    #pragma omp section
    phase2();
    #pragma omp section
    phase3();
}
```





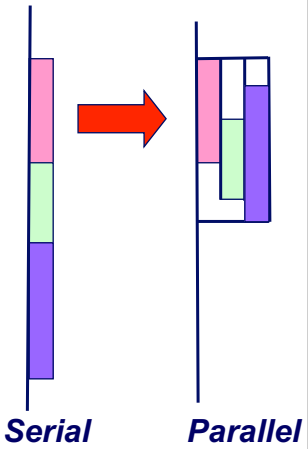
New Additions to OpenMP 3+

- Tasks – Main changes from OpenMP 3.0 (now 5.x)
- Allows parallelization of irregular problems
  - Unbounded loops
  - Recursive algorithms
  - Producer/consumer



What are tasks?

- Tasks are independent units of work
- Threads are assigned to perform the work of each task
  - Tasks may be deferred
- Tasks may be executed immediately
- The runtime system decides which of the above
  - Tasks are composed of:
    - **Code** to execute
    - **Data** environment
    - **Internal control variables** (ICV)





Simple Task Example

```
#pragma omp parallel
// assume 8 threads
{
  #pragma omp single private(p)
  {
    …
    while (p) {
    #pragma omp task
      {
        processwork(p);
      }
      p = p->next;
    }
  }
}
```

A pool of 8 threads is created here

One thread gets to execute the while loop

The single "while loop" thread creates a task for each instance of processwork()



Task Construct – Explicit Task View

- A team of threads is created at the omp parallel construct
- A single thread is chosen to execute the while loop – lets call this thread "L"
- Thread L operates the while loop, creates tasks, and fetches next pointers
- Each time L crosses the omp task construct it generates a new task and has a thread assigned to it
- Each task runs in its own thread
- All tasks complete at the barrier at the end of the parallel region's single construct

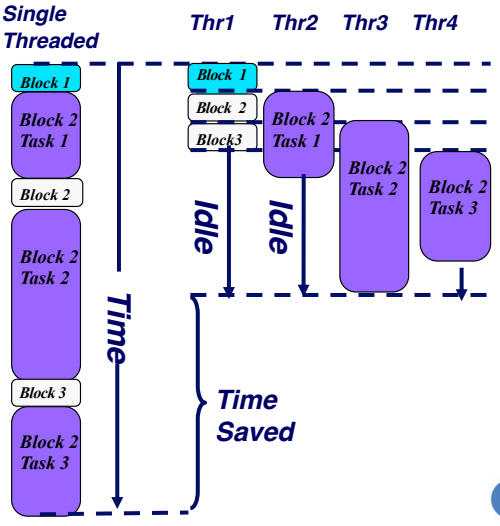
```
#pragma omp parallel
{
   #pragma omp single
   {  // block 1
      node * p = head;
      while (p) {  //block 2
      #pragma omp task
        private_process(p);
      p = p->next; //block 3
      }
   }
}
```



Why are tasks useful?

Have potential to parallelize irregular patterns and recursive function calls

```
#pragma omp parallel
{
   #pragma omp single
   {  // block 1
      node * p = head;
      while (p) {
//block 2
      #pragma omp task
         process(p);
//block 3
      p = p->next; }
   }
}
```





Linked Lists using Tasks

- **Objective:** Modify the linked list pointer chasing code to implement tasks to parallelize the application

```
while(p != NULL){
  do_work(p->data);
  p = p->next;
}
```



When are tasks guaranteed to be complete?

- ***Tasks are guaranteed to be complete:***
- ***At thread or task barriers***
  - ***At the directive:*** `#pragma omp barrier`
  - ***At the directive:*** `#pragma omp taskwait`



Task Completion Example

```
#pragma omp parallel
{
   #pragma omp task
   foo();
   #pragma omp barrier
   #pragma omp single
   {
      #pragma omp task
      bar();
   }
}
```

Multiple foo tasks created here – one for each thread

All foo tasks guaranteed to be completed here

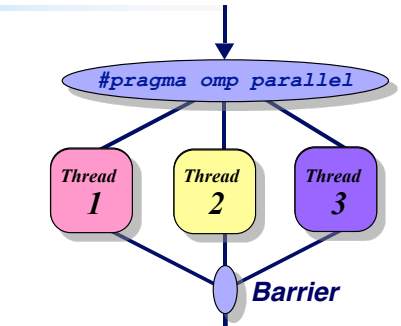
One bar task created here

bar task guaranteed to be completed here



Parallel Construct – Implicit Task View

- Tasks are created in OpenMP even without an explicit task directive.
- Lets look at how tasks are created implicitly for the code snippet below
  - Thread encountering parallel construct packages-up a set of *implicit* tasks
  - Team of threads is created
  - Each thread in team is assigned to one of the tasks (and *tied* to it)
  - Barrier holds original master thread until all implicit tasks are finished




```
#pragma omp parallel
   {
      int mydata;
      code…
   }
```



Task Construct

```
#pragma omp task [clause[[,]clause] ...]
        structured-block
```

***where** clause can be one of:*

```
if (expression)
untied
shared (list)
private (list)
firstprivate (list)
default( shared | none  )
```



Tied & Untied Tasks

- Tied Tasks:
  - A tied task gets a thread assigned to it at its first execution and the same thread services the task for its lifetime
  - A thread executing a tied task, can be suspended, and sent of to execute some other task, but eventually, the same thread will return to resume execution of its original tied task
  - Tasks are tied unless explicitly declared untied
- Untied Tasks:
  - An united task has no long term association with any given thread. Any thread not otherwise occupied is free to execute an untied task. The thread assigned to execute an untied task may only change at a "task scheduling point".
  - An untied task is created by appending “untied” to the task clause
  - Example: #pragma omp task untied



Task switching

- **task switching** The act of a *thread* switching from the execution of one *task* to another *task*
- The purpose of task switching is distribute threads among the unassigned tasks in the team to avoid piling up long queues of unassigned tasks
- Task switching, for tied tasks, can only occur at task scheduling points located within the following constructs:
  - Encountered **task** *constructs*
  - Encountered **taskwait** *constructs*
  - Encountered **barrier** *directives*
  - Implicit **barrier** *regions*
  - At the end of the *tied task region*
- Untied tasks have implementation dependent scheduling points



Task switching example

- ***The thread executing the "for loop" , AKA the generating task, generates many tasks in a short time so...***
- ***The SINGLE generating task will have to suspend for a while when "task pool" fills up***
  - ***Task switching is invoked to start draining the "pool"***
  - ***When the "pool" is sufficiently drained – then the single task can begin generating more tasks again***

```
#pragma omp single
{
    for (i=0; i<ONEZILLION; i++)
     #pragma omp task
         process(item[i]);
    }
}
```



Pthreads

- Specification part of larger IEEE POSIX standard
  - POSIX is the **P**ortable **O**perating **S**ystem **I**nterface
  - Standard C API for threading libraries
    - IBM provides Fortran API
  - Introduced in 1995
- Explicit threading of application
  - User calls functions to create/destroy threads

Materials from http://www.llnl.gov/computing/tutorials/pthreads/



The Pthreads Model

- Execution Model
  - Explicit parallelism
  - Explicit synchronization
- Productivity
  - Not a primary objective
  - Library for existing language
  - Low level of abstraction
  - Uses opaque objects – prevents expansion



The Pthreads Model

- Performance
  - No attempts to manage latency
  - Load balancing left to OS
  - Developer responsible for creating high degree of parallelism by spawning threads
- Portability
  - Library widely available



Pthreads Thread Management

- User creates/terminates threads
- Thread creation
  - `pthread_create`
  - Accepts a single argument (void *)
- Thread termination
  - `pthread_exit`
  - Called from within terminating thread



Pthreads Synchronization

Mutual Exclusion Variables (mutexes)

- Mutexes must be initialized before use
- Attribute object can be initialized to enable error checking

```
pthread_mutex_t mutexsum;
void *dot_product(void *arg) {
  …
  pthread_mutex_lock (&mutexsum);
  sum += mysum;
  pthread_mutex_unlock (&mutexsum);
  …
}
int main() {
  pthread_mutex_init(&mutexsum, NULL);
  …
  pthread_mutex_destroy(&mutexsum);
}
```



Pthreads Synchronization

Condition Variables

- Allows threads to synchronize based on value of data
- Threads avoid continuous polling to check condition
- Always used in conjunction with a mutex
  - Waiting thread(s) obtain mutex then wait
    - pthread_cond_wait() function unlocks mutex
    - mutex locked for thread when it is awakened by signal
  - Signaling thread obtains lock then issues signal
    - pthread_cond_signal() releases mutex



Condition Variable Example

Two threads update a counter
Third thread waits until counter reaches a threshold

```
pthread_mutex_t mtx;
pthread_cond_t cv;

int main() {
…
pthread_mutex_init(&mtx, NULL);
pthread_cond_init (&cv, NULL);
…
pthread_create(&threads[0], &attr,
               inc_count, (void *)&thread_ids[0]);
pthread_create(&threads[1], &attr,
               inc_count, (void *)&thread_ids[1]);
pthread_create(&threads[2], &attr,
               watch_count, (void *)&thread_ids[2]);
…
}
```



Condition Variable Example

Incrementing Threads

```
void *inc_count(void *idp) {
…
for (i=0; i<TCOUNT; ++i) {
  pthread_mutex_lock(&mtx);
  ++count;
  if (count == LIMIT)
    pthread_cond_signal(&cv);
  pthread_mutex_unlock(&mtx);
  …
}
…
}
```

Waiting Thread

```
void *watch_count(void *idp) {
…
pthread_mutex_lock(&mtx);
while (count < COUNT_LIMIT) {
  pthread_cond_wait(&cv, &mtx);
}
pthread_mutex_unlock(&mtx);
…
}
```

pthread_cond_broadcast() used if multiple threads waiting on signal



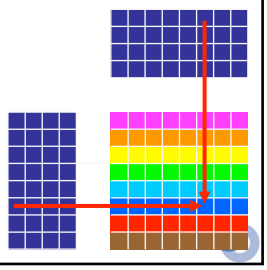
PThreads Matrix Multiply

```
int tids[M];
pthread_t threads[M];
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_attr_setdetachstate(
  &attr,
  PTHREAD_CREATE_JOINABLE);

for (i=0; i<M; ++i) {
  tids[i] = i;
  pthread_create(&threads[i],
   &attr, work, (void *) &tids[i]);
}

for (i=0; i<M; ++i) {
  pthread_join(threads[i], NULL);
}
```

```
void* work(void* tid) {
  for(int j=0; j<N; ++j) {
    for(int k=0; k<L; ++k) {
      C[tid][j] +=
        A[tid][k]*B[k][j];
    }
  }
  pthread_exit(NULL);
}
```



References

OpenMP

http://www.openmp.org

https://hpc-tutorials.llnl.gov/openmp/

PThreads

https://hpc-tutorials.llnl.gov/posix/

"Pthreads Programming". B. Nichols et al. O'Reilly and Associates.

"Programming With POSIX Threads". D. Butenhof. Addison Wesley

Message Passing Model

- Large scale parallelism (up to 100k+ CPUs)
- Multiple (possibly heterogeneous) system images
- Distributed memory
  - Nodes == servers can only access local data
  - Application (User) responsible for:
    - Distributing data
    - Redistributing data (when necessary)
    - Maintaining memory coherent



Message Passing Model

- Explicit communication
  - Two-sided P2P:
    - Communication initiated on one side requires matching action on the remote side
    - E.g. MPI_Send – MPI_Recv
  - One-sided P2P:
    - Communication is initiated on one side and no action is required on the other
    - E.g. MPI_Get/Put, gasnet_get/put ...
- Implicit synchronization with two-sided communication
  - The matching of communication operations from both sides ensures synchronization



Message Passing Model

- Objectives of the model
  - Enabling parallelization on highly scalable hardware
  - Support for heterogeneous systems
  - Often coarse-grained parallelism
- Main issues
  - Communication
  - Synchronization
  - Load balancing



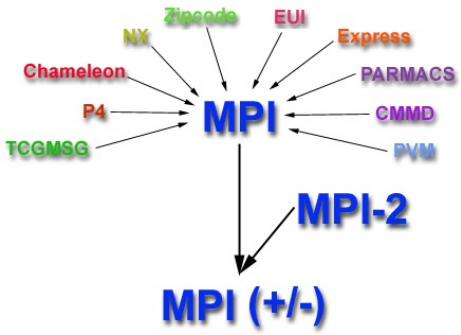
Projects of Interest

- **Message Passing Interface (MPI)**
  - De facto standard for this model
  - Deemed low level and difficult to program
  - Two-sided and one-sided communication
- Charm++ (PP@Master)
  - Asynchronous Remote Method Invocation (RMI) communication
  - Split-phase programming model
    - No synchronous communication
    - Caller provides a callback handler to asynchronously process "return" value



Message Passing Interface (MPI)

- 1980s – early 1990s
  - Distributed memory, parallel computing develops
  - Many incompatible software tools
  - Usually tradeoffs between portability, performance, functionality and price
- Recognition of the need for a standard arose.



Material from: https://hpc-tutorials.llnl.gov/mpi/



Message Passing Interface (MPI)

- Standard based on the consensus of the MPI Forum
  - Not sanctioned by any major standards body
  - Wide practical acceptance
  - No effective alternative to date
- First draft of the MPI-1 standard presented at Supercomputing 1993
- Current standard MPI-2 developed between 1995 and 1997
- Standardization committee open to all members of the HPC community

Further reading and standard documents: http://www.mpi-forum.org/



Message Passing Interface (MPI)

- Objectives
  - High performance and scalability
  - Portability
  - Productivity was not an objective (it became one)
- Used as communication layer for higher-level libraries
  - Often for more productivity-oriented libraries
  - ISO/OSI layer-5 interface
    - Communication is reliable and sessions are managed internally
    - Data is not structured



MPI: Specification, not Implementation

- Language Independent Specification (LIS)
- Library implementations of MPI vary in:
  - Performance
    - Target or rely on specific hardware (RDMA, PIM, Coprocessors …)
    - Provide load-balancing and processor virtualization (e.g., AMPI)
  - Functionality
    - Support for parallel I/O
    - Support for multithreading within MPI processes
- Standard provides language bindings for Fortran, C and C++
- Implementations usually provide APIs for C, C++ and Fortran
- Project implementations for Python, OCaml, and Java



MPI – Programming Model

Execution Model

- Explicitly parallel
  - Programmer responsible for correctly identifying parallelism and for implementing parallel algorithms using MPI constructs
  - Multi-threaded view
- SPMD
- Explicit data distribution
- Flat parallelism
  - Number of tasks dedicated to run a parallel program is static
- Processor Consistency (one-sided communication)



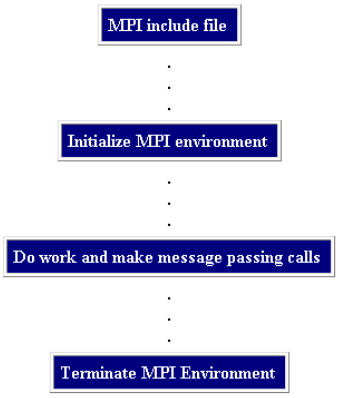
MPI – Programming Model

- Productivity
  - Not a principal objective
    - Low level of abstraction
    - Communication is not structured (marshalling done by the user)
- Performance
  - Vendor implementations exploit native hardware features to optimize performance
- Portability
  - Most vendors provide an implementation
    - E.g., Specialized open source versions of MPICH, LAM or OpenMPI
  - ”Standard” ensures compatibility



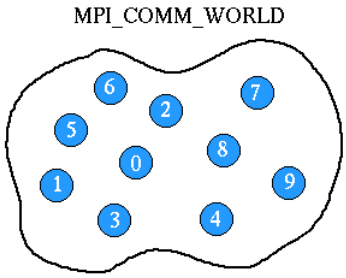
MPI – Program Structure

General program structure



Communicators and groups

- Collection of processes that may communicate
- Unique rank (processor ID) within communicator
- Default communicator: MPI_COMM_WORLD

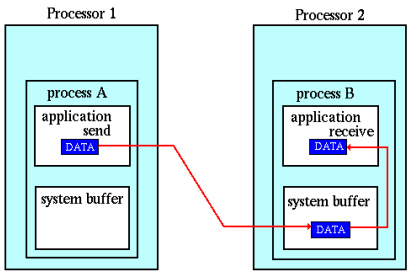




MPI – Point to Point Communication

Buffering when tasks are out of sync

- If a receive operation is not ready, sent data is buffered
  - On receiving side, sending side or both
- User can manage buffering memory on sending side



Path of a message buffered at the receiving process



MPI – Point to Point Communication

Types of Point-to-Point Operations:

- Message passing between two, and only two, different MPI tasks
  - One task performs a send operation
  - The other task matches with a receive operation
- Different types of send/receive routines used for different purposes
  - Synchronous send
  - Blocking send / blocking receive
  - Non-blocking send / non-blocking receive
  - Buffered send
  - Combined send/receive
  - "Ready" send
- Any type of send can be paired with any type of receive
- Test and Probe routines to check the status of pending operations



MPI – Point to Point Communication

Blocking vs. Non-blocking

- Most routines can be used in either blocking or non-blocking mode
- Blocking communication routines
  - Blocking send routines only return when it is safe to reuse send buffer
    - Modifications to send buffer will not affect data received on the remote side
      - Data already sent
      - Data buffered in a system buffer
  - Blocking send calls can be synchronous
    - Handshaking with the receiver
  - Blocking send calls can be asynchronous
    - System buffer used to hold the data for eventual delivery to the receiver
  - Blocking receive calls only return after the data has arrived and is ready for use by the program



MPI – Point to Point Communication

Blocking communication example

```
#include "mpi.h"
#include <stdio.h>

int main(int argc,char *argv[]) {
  int numtasks, rank, dest, source, rc, count, tag=1;
  char inmsg, outmsg='x';
  MPI_Status Stat;

  MPI_Init(&argc,&argv);
  MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  if (rank == 0) {
    dest = 1;
    source = 1;
    rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
    rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
  }
  else if (rank == 1) {
    dest = 0;
    source = 0;
    rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
    rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
  }

  rc = MPI_Get_count(&Stat, MPI_CHAR, &count);
  printf("Task %d: Received %d char(s) from task %d with tag %d \n",
         rank, count, Stat.MPI_SOURCE, Stat.MPI_TAG);

  MPI_Finalize();
}
```



MPI – Point to Point Communication

Blocking vs. Non-blocking

- Non-blocking communication routines
  - Send and receive routines behave similarly
    - Return almost immediately
    - Do not wait for any communication events to complete
      - Message copying from user memory to system buffer space
      - Actual arrival of message
  - Operations "request" the MPI library to perform an operation
    - Operation is performed when its requirements are met (e.g., message arrives)
    - User cannot predict when that will happen
  - Unsafe to modify the application buffer until completion of operation
    - Wait and Test routines used to determine completion
- Non-blocking communications primarily used to overlap computation with communication and exploit possible performance gains



MPI – Point to Point Communication

Non-blocking communication example

```
MPI_Request reqs[4];
MPI_Status stats[4];

prev = rank-1;
next = rank+1;
if (rank == 0) prev = numtasks - 1;
if (rank == (numtasks - 1)) next = 0;

MPI_Irecv(&buf[0], 1, MPI_INT, prev, tag1, MPI_COMM_WORLD, &reqs[0]);
MPI_Irecv(&buf[1], 1, MPI_INT, next, tag2, MPI_COMM_WORLD, &reqs[1]);

MPI_Isend(&rank, 1, MPI_INT, prev, tag2, MPI_COMM_WORLD, &reqs[2]);
MPI_Isend(&rank, 1, MPI_INT, next, tag1, MPI_COMM_WORLD, &reqs[3]);

  {
    // do some work
    // work will overlap with previous communication
  }

MPI_Waitall(4, reqs, stats);
```



MPI – Point to Point Communication

Order and Fairness

- Message Ordering
  - Messages do not overtake each other
    - If a sender sends two messages (Message 1 and Message 2) in succession to the same destination, and both match the same receive, the receive operation will receive Message 1 before Message 2.
    - If a receiver posts two receives (Receive 1 and Receive 2), in succession, and both match the same message, Receive 1 will receive the message before Receive 2.
  - Ordering is not thread-safe
    - If multiple threads participate in the communication, no order is guaranteed
- Fairness of Message Delivery
  - No fairness guarantee
    - Programmer responsible for preventing operation starvation
  - Example: task 0 sends a message to task 2. However, task 1 sends a competing message that matches task 2's receive. Only one of the sends will complete.



MPI – Collective Communication

- All or None
  - Must involve **all** processes in the scope of the used communicator
  - User responsible to ensure all processes within a communicator participate in any collective operation
- Types of Collective Operations
  - Synchronization (barrier)
    - Processes wait until all members of the group reach the synchronization point
  - Data Movement
    - Broadcast, scatter/gather, all to all
  - Collective Computation (reductions)
    - One member of the group collects data from the other members and performs an operation (e.g., min, max, add, multiply, etc.) on that data



MPI – Collective Communication

Programming Considerations and Restrictions

- Collective operations are blocking
- Collective communication routines do not take message tag arguments
- Collective operations within subsets of processes
  - Partition the subsets into new groups
  - Attach the new groups to new communicators
- Can only be used with MPI predefined data types
  - Not with MPI Derived Data Types



MPI – Matrix Multiply (master task)

- Initialization

Common to both master and worker processes

```
#define NRA 15     // Number of rows in matrix A
#define NCA 25     // Number of columns in A
#define NCB 10     // Number of columns in B
#define TAG 0      // MPI communication tag
// Data structures
double A[NRA][NCA];// matrix A to be multiplied
double B[NCA][NCB];// matrix B to be multiplied
double C[NRA][NCB];// result matrix C
```

- Distribute data to workers

```
avgNumRows     = NRA/numWorkers;
remainingRows = NRA%numWorkers;
offset         = 0;
for (dest = 1; dest <= numWorkers; ++dest) {
  rows = (dest <= remainingRows) ? avgNumRows + 1 : avgNumRows;
  MPI_Send(&offset, 1, MPI_INT, dest, TAG, MPI_COMM_WORLD);
  MPI_Send(&rows, 1, MPI_INT, dest, TAG, MPI_COMM_WORLD);
  count = rows * NCA;
  // Send horizontal slice of A
  MPI_Send(&A[offset][0], count, MPI_DOUBLE, dest, TAG, MPI_COMM_WORLD);
  // Send matrix B
  count = NCA * NCB;
  MPI_Send(&B, count, MPI_DOUBLE, dest, TAG, MPI_COMM_WORLD);
  offset += rows;
}
```

- Wait for results from workers

```
for (i = 1; i <= numworkers; ++i)       {
  source = i;
  MPI_Recv(&offset, 1, MPI_INT, source, TAG, MPI_COMM_WORLD, &status);
  MPI_Recv(&rows, 1, MPI_INT, source, TAG, MPI_COMM_WORLD, &status);
  count = rows * NCB;
  MPI_Recv(&C[offset][0], count, MPI_DOUBLE, source, TAG,MPI_COMM_WORLD,&status);
}
```

280

MPI – Matrix Multiply (worker task)

- Receive data from master

```
source = 0;
MPI_Recv(&offset, 1, MPI_INT, source, TAG, MPI_COMM_WORLD, &status);
MPI_Recv(&rows,   1, MPI_INT, source, TAG, MPI_COMM_WORLD, &status);
// Receive horizontal slice of A
count = rows * NCA;
MPI_Recv(&A, count, MPI_DOUBLE, source, TAG, MPI_COMM_WORLD, &status);
// Receive matrix B
count = NCA * NCB;
MPI_Recv(&B, count, MPI_DOUBLE, source, TAG, MPI_COMM_WORLD, &status);
```

- Process data

```
// Compute the usual matrix multiplication on the slice of matrix A and matrix B
for (k = 0; k < NCB; ++k) {
  for (i = 0; i < rows; ++i) {
    C[i][k] = 0.0;
    for (j = 0; j < NCA; ++j) {
      C[i][k] += A[i][j] * B[j][k];
    }
  }
}
```

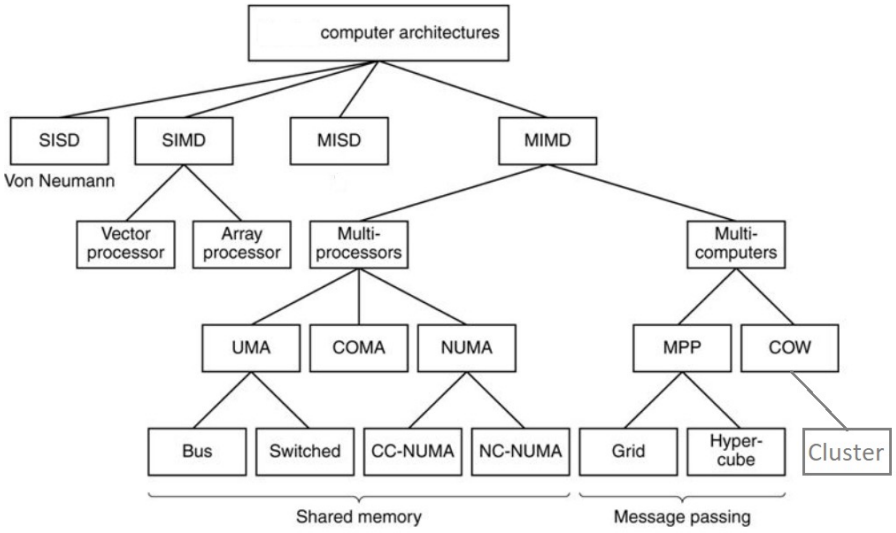
- Send results back to master

```
destination = 0;
MPI_Send(&offset, 1, MPI_INT, destination, TAG, MPI_COMM_WORLD);
MPI_Send(&rows, 1, MPI_INT, destination, TAG, MPI_COMM_WORLD);
count = rows * NCB;
// Send horizontal slice of result matrix C computed on this node
MPI_Send(&C, count, MPI_DOUBLE, destination, TAG, MPI_COMM_WORLD);
```



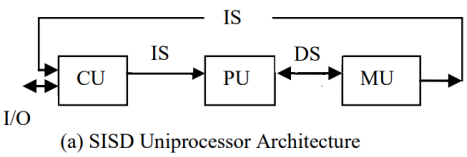
281

Arhitecturi Sisteme de Calcul

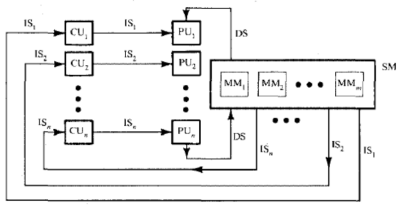


Taxonomii – Flynn

(c) MISD Architecture (the Systolic Array)

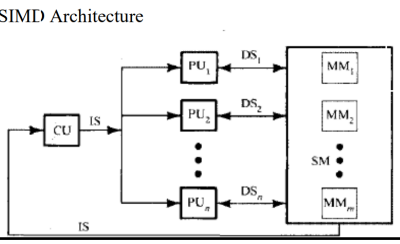


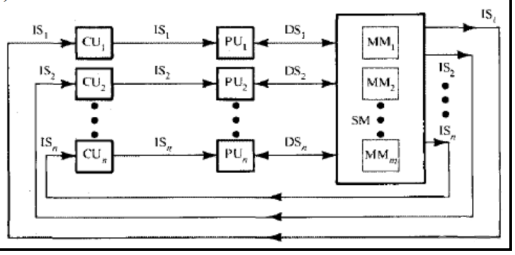
(a) SISD Uniprocessor Architecture



(d)MIMD architecture

(b) SIMD Architecture





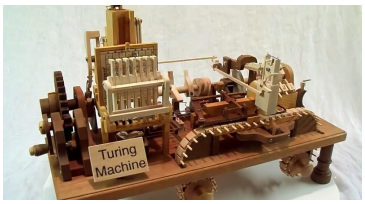
Modele ale calculului paralel

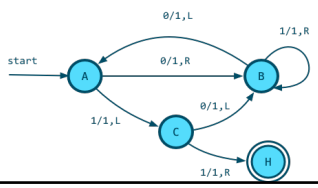
- Model RAM (Random Acces Machine)
- Modelul Pipeline
- Procesoare vectoriala
- Multiprocesoare cu memorie partajata
- Multiprocesoare cu transfer prin mesaje
- Modele PRAM (Parallel Random Acces Machine)
- Procesare Sistolica
- Procesare Data Flow

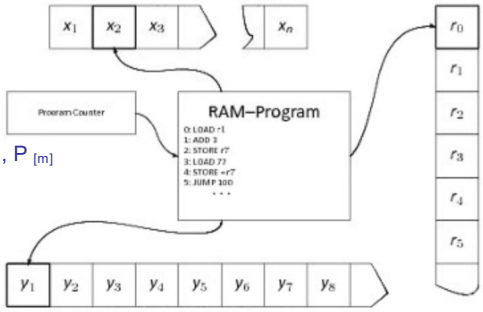


Model RAM (Random Acces Machine)

- O mașină cu acces aleatoriu (RAM):
  - Bandă intrare infinită X cu citire i ∈ N,
  - Bandă ieșire infinită Y cu scriere o ∈ N,
  - Registru contor de program PC
  - Program cu număr finit instrucțiuni P [1], …, P [m]
  - Memorie cu registrii r_0, r_1, r_2







- RAM = (X ,Y, S, s0, f, Z)
- S multimea finita a starilor automatului
- s_0 starea initiala
- X alfabetul finit al benzii de intrare
- Y alfabetul finit al benzii de iesire
- f functia de tranzitie, eventual partial definite

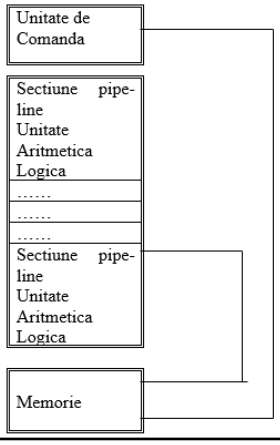
  $f : S \times X \rightarrow S \times Y$
- Z inclus in S – multime a starilor finale



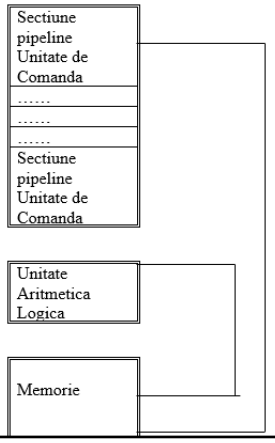
Modelul Pipeline

- Conceptul de pipeline consta in impartirea unui task T in subtask-uri T_1 , T_2 , ...,T_k si de a le atribui unor elemente de procesare.
- Paralelismul se poate realiza fie la nivel de:

prelucrare aritmetica



citire, interpretare, executie instructiuni

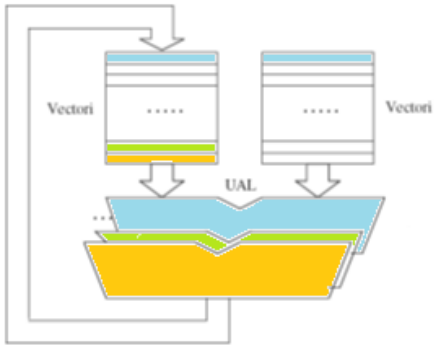




286

Procesoare vectoriala

- Set de instructiuni ce opereaza pe vectori
- Elemente de procesare vectoriala
- Disponibil pe sisteme monoprocesor ca extensii de procesare pe vectori
  - SSE, AVX, AltiVec, etc.



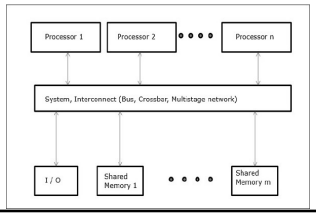


287

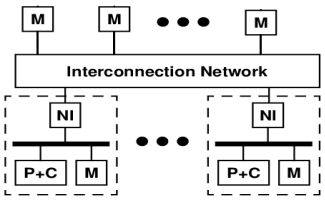
Multiprocesoare cu memorie partajata

- Fiecare procesor contine
  - Propria unitate de prelucrare
  - Propria memorie
  - Propria unitate de comanda
  - Comunica prin intermediul unei retele de comutare
- Fiecare procesor executa propriul set de instructiuni din memoria locala sau cea partajata
- Reteaua de comutare permite schimbul de informatii intre procesoare sau intre procesoare si memoria partajata

UMA



NUMA

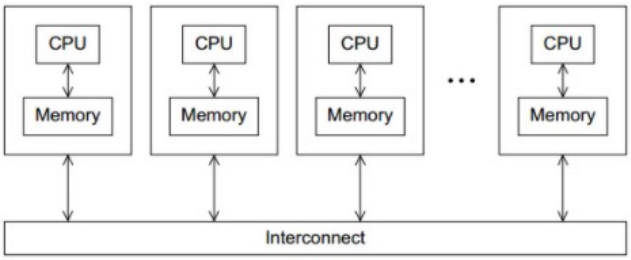




288

Multiprocesoare cu transfer prin mesaje

- Memoria este distribuita fiecarui procesor pentru cod si date
- Comunicarea datelor
  - Se face prin schimburi de mesaje
  - Printr-o retea de comutatie
  - Nu prin DMA/comutare de circuite

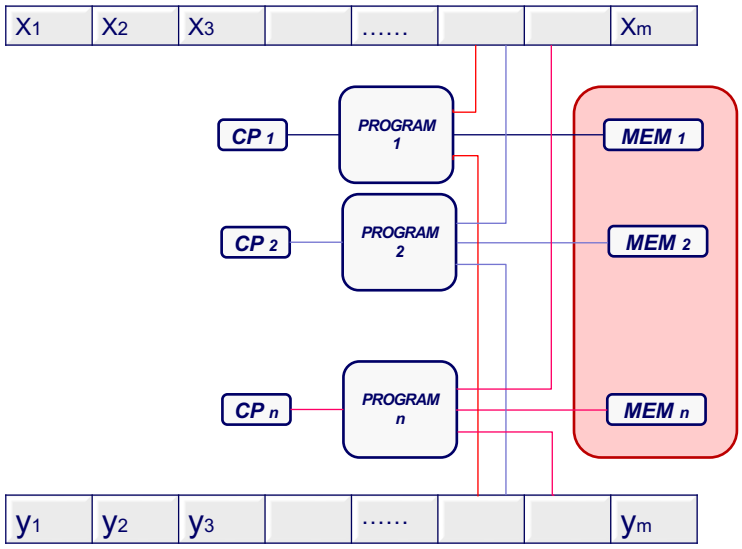




289

Modele PRAM – Parallel Random Acces Machine

290





290

Modele PRAM (2)

291

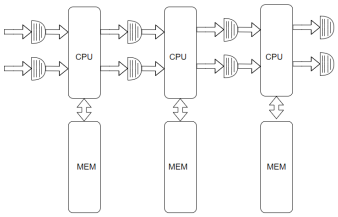
- Modele abstracte folosite pentru designul si analiza algoritmilor paraleli
- Set de procesoare ce au acces la o memorie partajata
- Procesoarele pot opera simultan si independent
- Procesoarele se pot coordona prin R/W la memorie
- Utile pentru designul algoritmilor paraleli deoarece permit un nivel crescut de paralelism
- Algoritmii PRAM sunt insa greu de mapat pe sisteme reale
- Tipuri de arhitecturi PRAM:
  - Exclusive Read Exclusive Write (EREW) – fiecare procesor poate doar citi sau scrie la un moment de timp – fara data-race-uri
  - Concurrent Read Exclusive Write (CREW) – mai multe procesoare citesc dintr-o memorie, doar un procesor scrie intr-o memorie
  - Priority PRAMs – procesoarele au prioritati la R/W

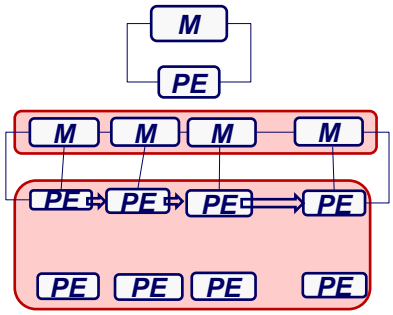


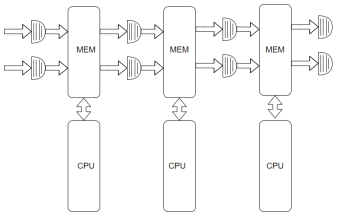
291

Procesare Sistolica

- Elemente de procesare identice aranjate intr-o structura pipeline
- Interconexiuni simple si regulate
  - Permite integrarea VLSI









292

Caracteristici ale Sistemelor Sistolice

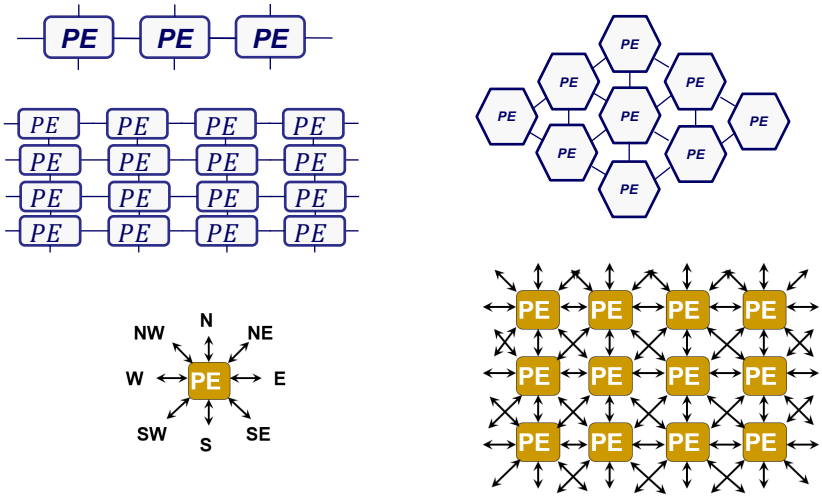
- Regularitate:
  - PE-uri identice ce permit scalabilitate crescuta
- Pipeline:
  - Calculele sunt asezate in banda de asamblare
  - Paralelismul crescut poate fi implementat eficient
- Localitate:
  - Datele sunt mutate doar adiacent
  - Comunicarea se face doar cu vecinii – overhead minimal
- Scalabilitate:
  - Sistemele de acest tip scaleaza usor prin adaugarea de noi PE-uri.
  - Se potrivesc pentru aplicatii ce necesita multa putere de calcul



293

294

Modalitati de conectare Sistolica



294

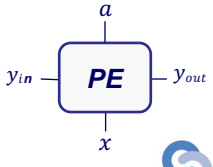
295

Exemplu de utilizare

- Fie
- A=$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{n1} & an_2 & \cdots & a_{nn} \end{bmatrix}$ Y=$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ X=$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ Y_0=$\begin{bmatrix} y_{01} \\ y_{02} \\ \vdots \\ y_{0n} \end{bmatrix}$
- Dorim sa calculam
- Y=A*X+Y0
- Presupunem ca avem un element de procesare sistólica

Y	A					X	Y_0
y_1	a_{11}	a_{12}	a_{13}		a_{1n}	x_1	y_{01}
y_2	a_{21}	a_{22}	a_{23}		a_{2n}	x_2	y_{02}
y_3	a_{31}	a_{32}	a_{33}		a_{3n}	x_3	y_{03}
y_n	a_{n1}	a_{n2}	a_{n3}		a_{nn}	x_n	y_{0n}

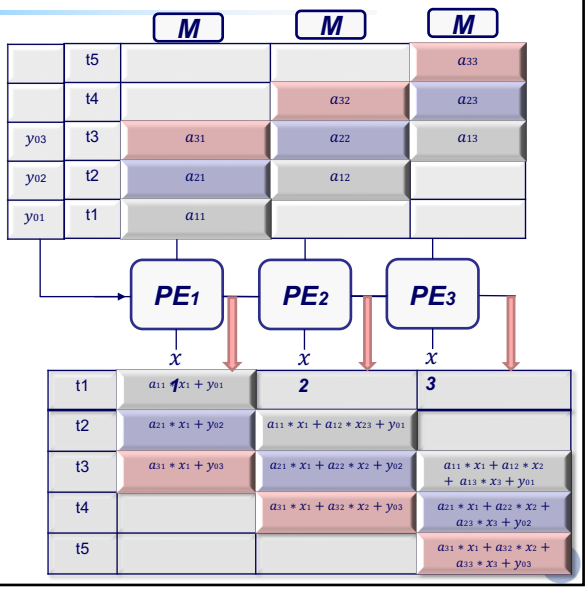
$y_{out} = a * x + yin$



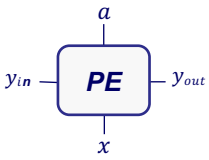
295

Exemplu unidimensional cu n=3

- $y_i = \sum_{j=1}^{n} a_{ij} * xj + y_{0i}$
- $y_1 = a_{11} * x_1 + a_{12} * x_2 + a_{13} * x_3 + y_{01}$
- $y_2 = a_{21} * x_1 + a_{22} * x_2 + a_{23} * x_3 + y_{02}$
- $y_3 = a_{31} * x_1 + a_{32} * x_2 + a_{33} * x_3 + y_{03}$



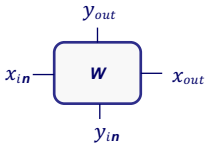
$y_{out} = a * x + yin$

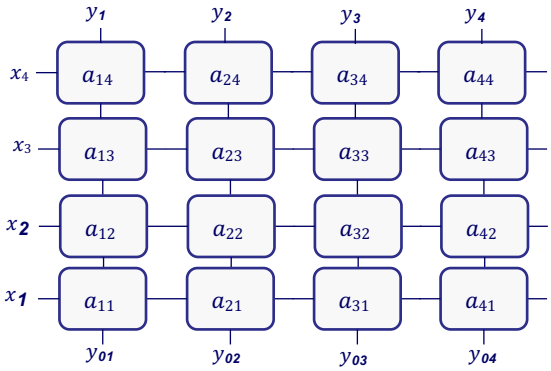


296

Exemplu bidimensional cu n=4

- $y_i = \sum_{j=1}^{n} a_{ij} * xj + y_{0i}$
- $y_1 = a_{11} * x_1 + a_{12} * x_2 + a_{13} * x_3 + a_{14} * x_4 + y_{01}$
- $y_2 = a_{21} * x_1 + a_{22} * x_2 + a_{23} * x_3 + a_{24} * x_4 + y_{02}$
- $y_3 = a_{31} * x_1 + a_{32} * x_2 + a_{33} * x_3 + a_{34} * x_4 + y_{03}$
- $y_4 = a_{41} * x_1 + a_{42} * x_2 + a_{43} * x_3 + a_{44} * x_4 + y_{04}$





$y_{out} = w * xin + yin$

$x_{out} = xin$

297

298

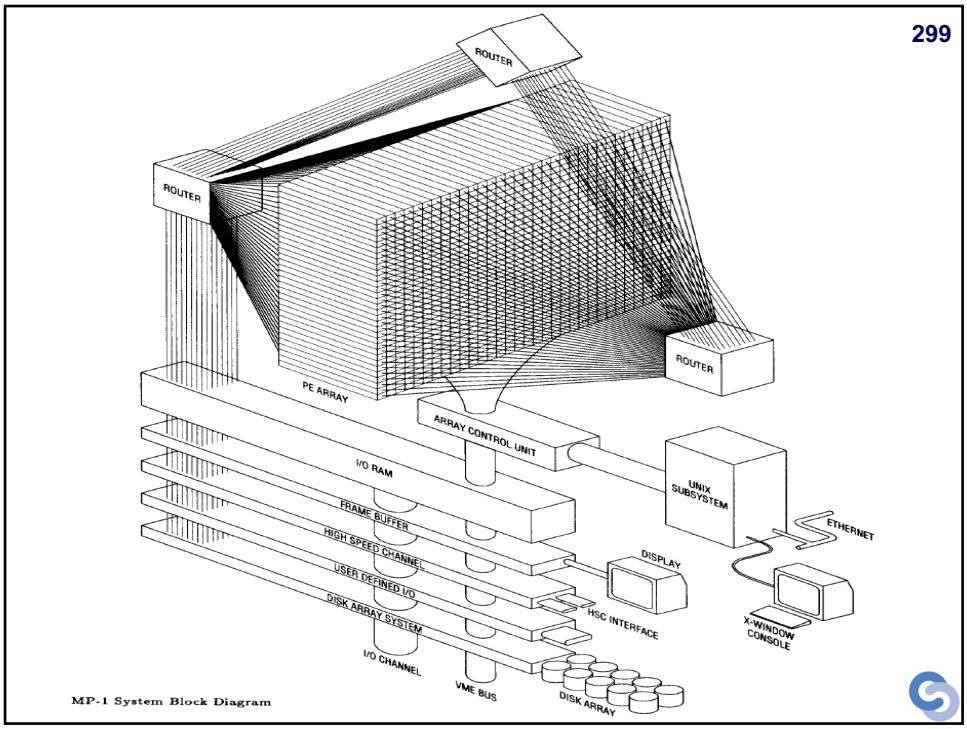
Algoritmi potriviti pentru Sisteme Sistolice

- Embarrassingly Parallel Algorithms: “trivial” paraleli
  - Fiecare PE actioneaza independent
  - Exemple:
    - Calcularea valorii de prag
    - Apartenenta unui element la un domeniu
- Near Embarrassingly Parallel Algorithms
  - Un PE are nevoie de informatii pe care le primeste de la alte PE-uri
  - Exemple:
    - Tratarea imaginilor
    - Detectia contururilor
    - Procesarea de semnale



298

299



299

300

Procesare Data Flow

- Este un mode bazat pe fluxul de date (Data Flow)
  - Execuția unei instructiuni de prelucarea este efectuata doar daca este disponibil operandul
  - Nu dispune de PC – activarea se face daca sunt disponibili toti operanzii
  - O instrucțiune este declanșată / executată când devine activă
  - Efectul declanșării unei instrucțiuni este prelucrarea datelor sale de intrare (operanzi) & generarea datelor de ieșire (rezultate)
  - Este o reprezentare grafica a modului in care circula informatia in sistem
  - Sunt utilizate pentru designul si analiza sistemelor de procesare a informatiilor



300

Caracteristici ale Sistemelor Data Flow

301

- Procese
  - Unitatile fundamentale de lucru intr-un system Data Flow
  - Reprezinta activitatile care transforma date dintr-o forma in alta
- Stocarea de date
  - Locul unde se stocheaza datele intre procese
- Data Flows
  - Reprezinta miscarea datelor intre procese si stocare
- Sistemele de Control
  - Sunt folosite pentru a determina flow-ul datelor in system
- Entitati externe
  - Reprezinta sursele si destinatiile datelor din sistem

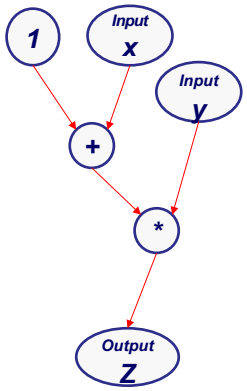


301

Exemplul de cod 1

- Un program Data Flow este compilat într-un graf Data Flow
  - Graf direcționat format din noduri
  - Reprezintă instrucțiuni și arce
  - Reprezintă dependențe de date între instrucțiuni
- Graficul Data Flow e similar cu un graf de dependență utilizat în reprezentările intermediare ale compilatoarelor
- În executarea programului datele se propagă de-a lungul arcelor în pachete de date, numite token-uri
- Acest flux de token-uri permite ca unele dintre noduri (instrucțiuni) sa inceapa executia și le declanșează

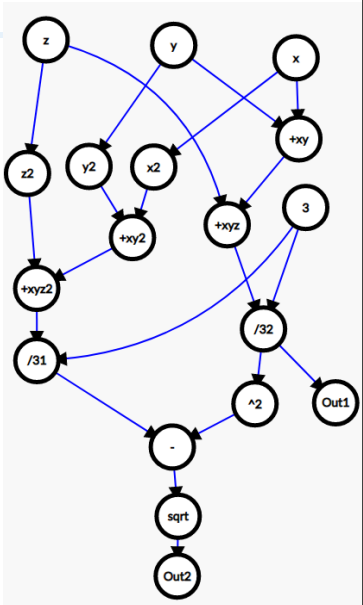
Z = (x+1) * y



Exemplul de cod 2

- Function Med_Dev calculeaza media si deviatia standard a trei variabile

```
function Med_Dev(x,y,z: real
  returns real,real);
 let
  Out1 := (x + y + z)/3;
  Out2 := SQRT((x**2 + y**2 +
            z**2)/3 - Out1**2);
 in
  Out1, Out2
 endlet
endfun
```



Caracteristici ale grafului asociat fluxului de date

- Funcționalitate:
  - Evaluarea unui graf de flux de date este echivalentă cu evaluarea funcției matematice corespunzătoare
- Compozibilitate:
  - Grafurile fluxului de date pot fi combinate pentru a forma noi grafuri
- Tipuri de structuri Data Flow
  - Static: dependentele de date sunt stabilite la compilare
  - Dinamic: dependentele de date sunt stabilite la rulare

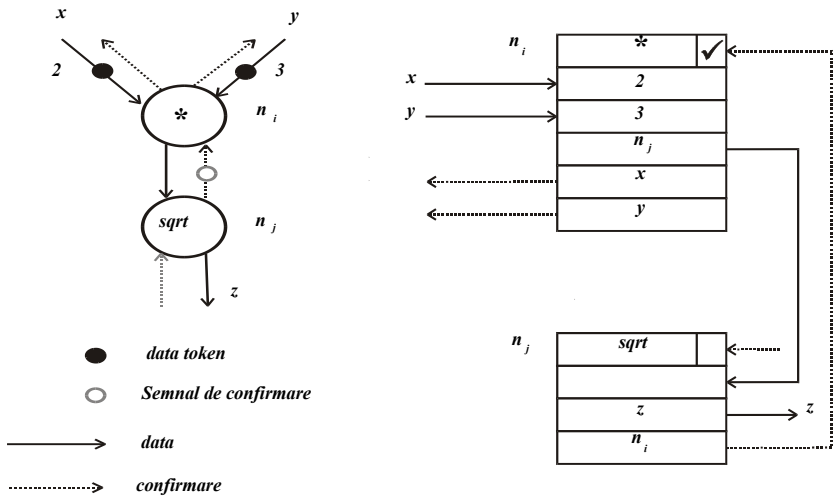


Data Flow Static

- Un graf Data Flow (DF) este reprezentat ca o colecție de șabloane de activitate conținând:
  - Codul instrucțiunii reprezentate **op**: `LHS = op (RHS1, RHS2, …)`
  - Operanzii instructiunii (**RHS**s)
  - Destinația operatiei (**LHS**) – sloturile din șabloanele de activitate din secvente care primesc valoarea rezultatului operatiei
- Deficiente Data Flow static:
  - Iterațiile consecutive ale unei bucle pot fi executate doar sub forma de pipeline
  - Datorită token-urilor de confirmare, traficul acestora este dublat
  - Lipsa suportului de programare pentru limbaje moderne
  - Lipsa apelurilor de procedură sau a recursivitatii
- Avantaj Data Flow static:
  - Este un model simplu de implementat



Data Flow – Exemplu de Activitate



Data Flow Dinamic

- Fiecare iterație de buclă sau invocare de subprogram se poate executa în paralel ca o instanță separată a unui subgraf reentrant
- Replicarea este doar conceptuală
- Fiecare **token** are un tag cu
  - Adresa instrucțiunii pentru care este destinată valoarea datelor
  - Informații de context
- Fiecare arc poate fi văzut ca o multime care poate conține un număr arbitrar de token-uri cu tag-uri diferite
- Regula de activare și declanșare este:
  - Un nod este activat și declanșat de îndată ce tokenurile cu etichete identice sunt prezente pe toate arcele de intrare
- Restrictiile structurale sunt ignorate!

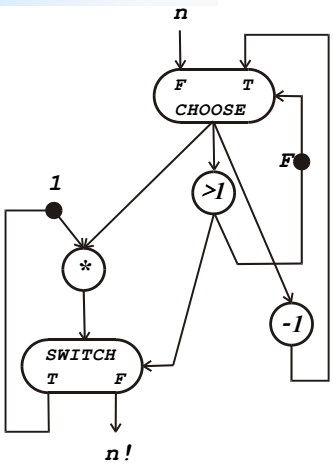
Data Flow Static vs. Dinamic

Caracteristica	Static Data Flow	Dynamic Data Flow
Dependenta date	Determinat la compilare	Determinat la runtime
Flexibilitate	Putin flexibil	Mai flexibil
Complexitate	Putin complex	Mai complex



Exemplu – Calculul Factorial == n!

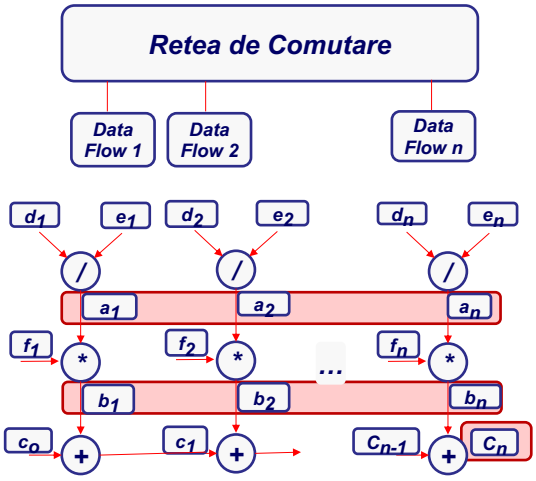
```
initial j = n; k = 1
 while j > 1 do
   j = j - 1;
  k = k * j;
return k
```





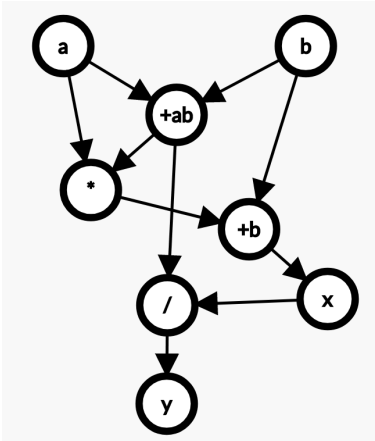
Exemplu de prelucrare vectori

```
Input d,e,f
//a,b,c,d,e,f
//vectori de date
 C0=0
For i=1 to n
    ai=di / ei
    bi=ai*fi
    ci=bi+ci-1
End for
Output a,b,cn
```



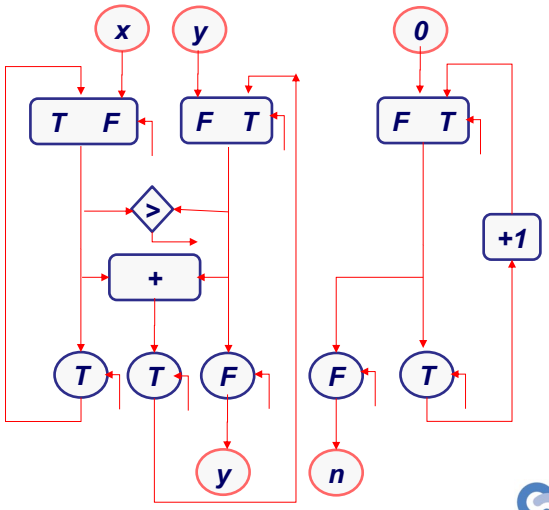
Ordinea instructiunilor este irelevanta

```
input a,b
 y = (a+b)/x
 x = (a*(a+b))+b
output y,x
```



Exemplu de bucla

```
input y,x
 n = 0
 while y<x do
    y =  y + x
    n =  n + 1
 end
output y,n
```



Caracteristici de baza

- Executia instructiunilor este bazata pe disponibilitatea datelor
- Datele sunt pastrate in cadrul instructiunii
- Disponibilitatea datelor este verificata de o unitate specializata
- Tokenul associat datelor este verificat de o unitate specializata
- Executia instructiunilor este asincrona

314

Avantaje si Dezavantaje

Avantaje

- Potential ridicat de parallelism
- Viteza crescuta de executie
- Usor de integrat comunicatia si sincronizarea

Dezavantaje

- Overhead mare pentru control
- Manipularea dificila a structurilor de date



314

315

Unde se foloseste Data Flow in Procesoarele Actuale?

- Vector Processing Units (VPUs) – Procesare SIMD
  - VPU-urile sunt procesoare specializate pentru procesarea vectoriala
  - Domenii aplicative: ML, procesare multimedia
  - Utilizeaza DF pentru executia eficienta a operatiilor vectoriale
  - Tehnologii: Intel Movidius Myriad X VPU – NNs in Edge Computing Apps
- Graphics Processing Units (GPUs)
  - GPU-urile sunt procesoare specializare pentru taskuri de procesare grafica
  - Domenii aplicative: optimizarea deplasarii de date si a executiei in SM-uri
  - Tehnologii: GPU-urile NVIDIA folosesc DF pentru rezolvarea dependentelor de date si pentru schedulingul de instructiuni
- Domain-Specific Accelerators (DSAs)
  - DSA-urile sunt procesoare specializate pentru anumite domenii aplicative
  - Domenii aplicative: procesare de semnal, criptografie
  - Utilizeaza DF pentru performante crescute si cresterea eficientei energetice
  - Tehnologii: Google TPU folosesc DF pentru executia eficienta a operatiilor de tip tensor



315

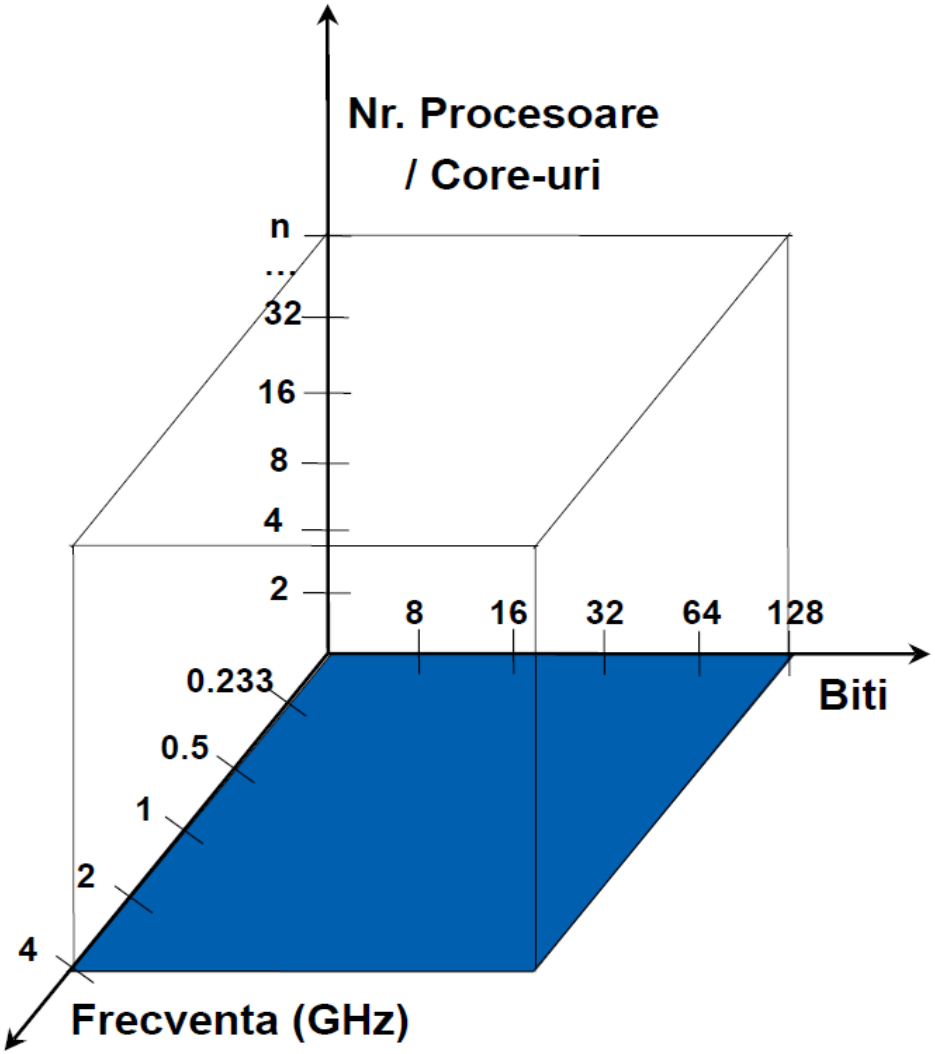
Unde se foloseste Data Flow in Procesoarele Actuale?

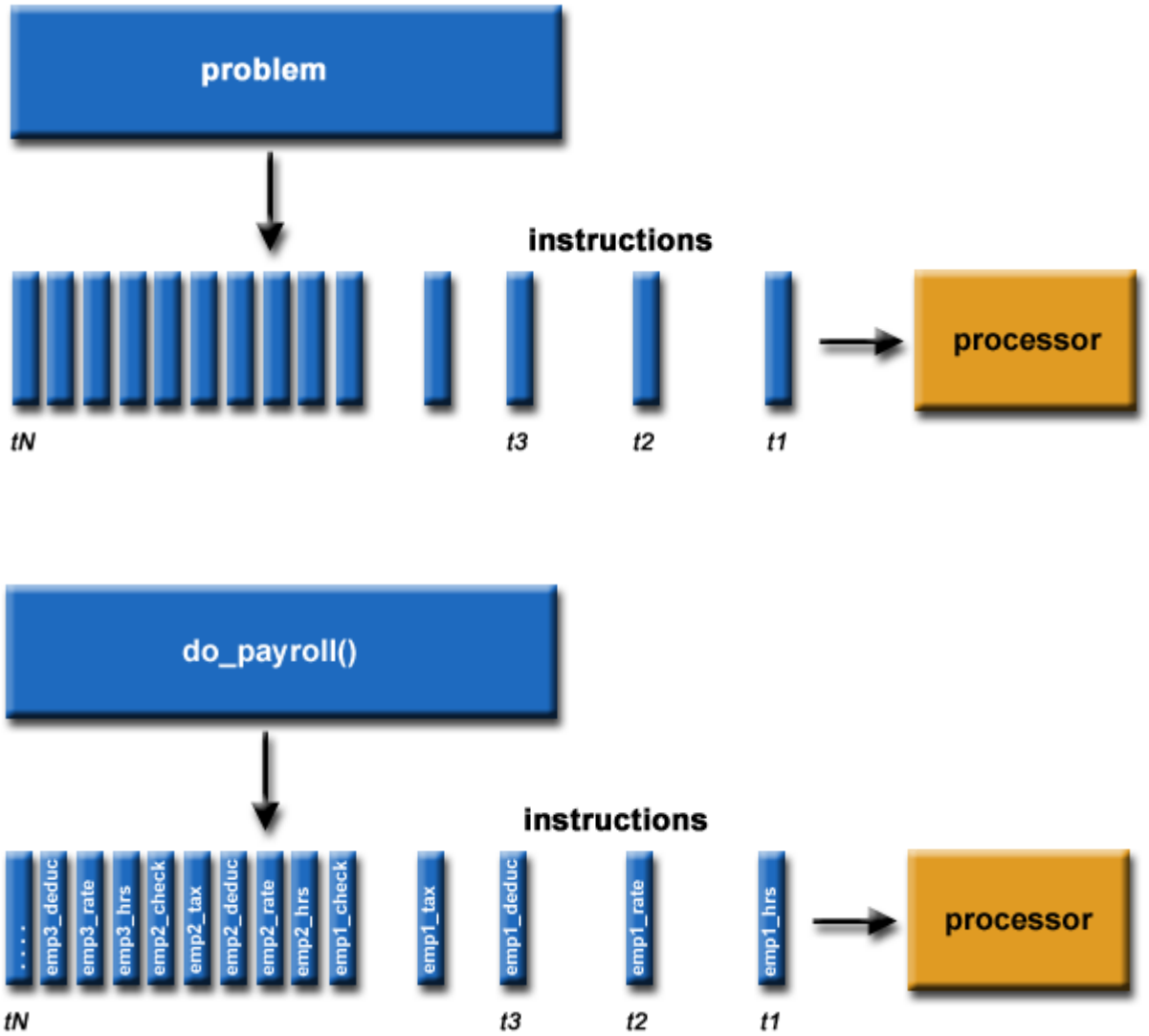
- Neural Processing Units (NPUs)
  - NPU-urile = procesoare specializate pentru accelerare workloaduri AI
  - Domenii aplicative: NNs (CNN, DNN, GANS, etc.)
  - Utilizeaza DF la optimizarea deplasarii de date in arhitecturi paralele
  - Tehnologii: Graphcore Colossus NPU folosesc abordari inspirate de DF pentru a gestiona dependentele de date si pentru a imbunatati performantele NNs
- Application-Specific Integrated Circuits (ASICs)
  - ASIC-urile sunt cipuri dedicate unor anumite tipuri de aplicatii ce ofera performanta si eficienta energetica
  - Domenii aplicative: DSP, NN, AI, ML
  - Tehnologii: Amazon Inferentia NPU sunt dezvoltate pentru a realiza inferenta in cadrul unor sisteme ML.
  - Utilizeaza DF la optimizarea traficului de date si al executiei fluxului de instructiuni

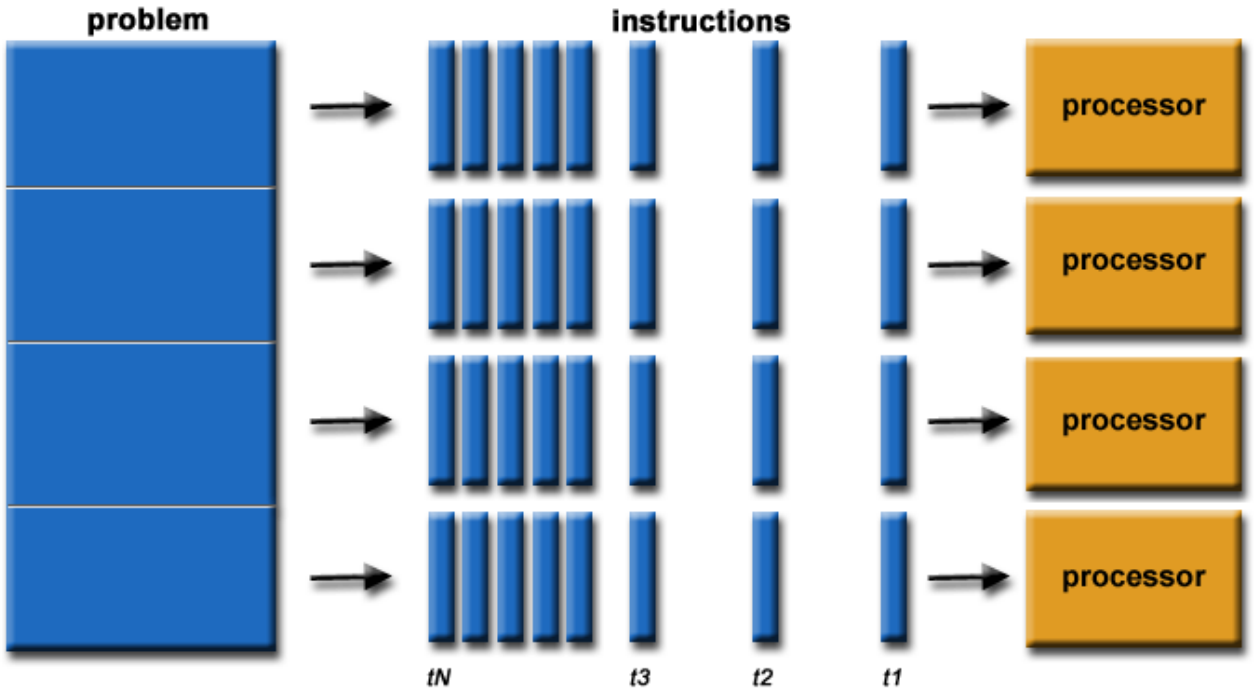


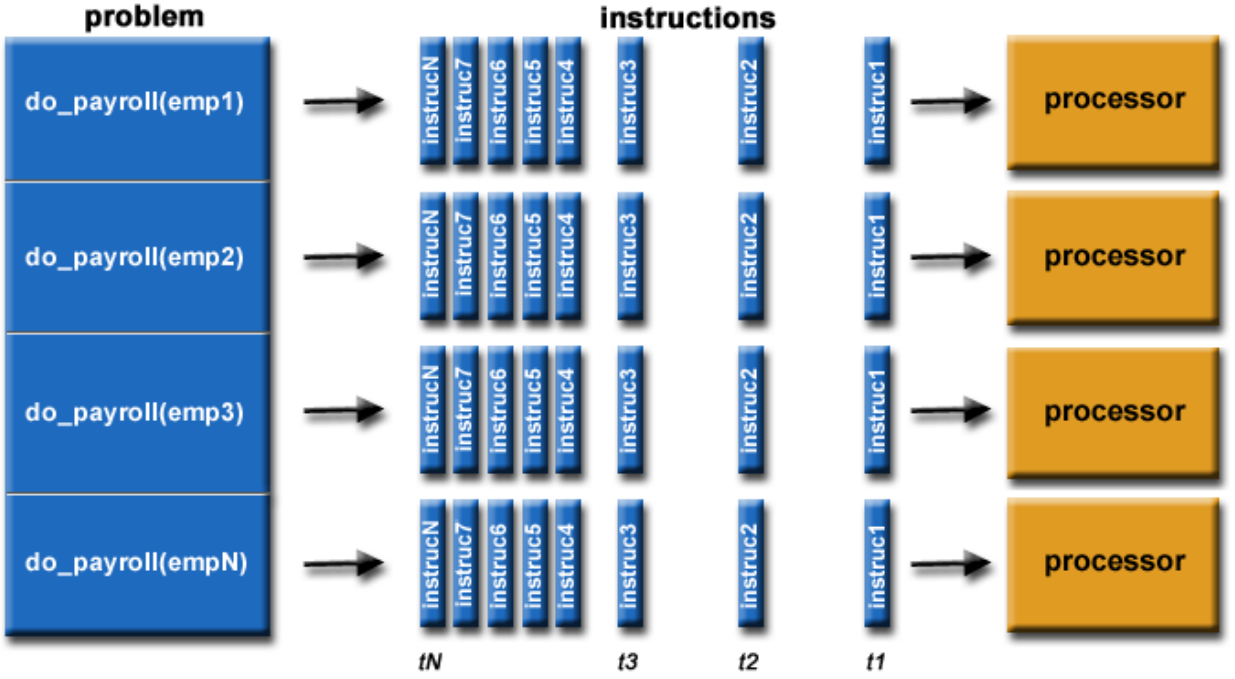
CONTROLUL PROCESELOR – SARCINILOR CONCURENTE

Evaluare putere de calcul primara sistem multiprocessor




problem
instructions
processor
tN
t3
t2
t1
do_payroll()
instructions
processor
. . . .
emp3_deduc
emp3_rate
emp3_hrs
emp2_check
emp2_tax
emp2_deduc
emp2_rate
emp2_hrs
emp1_check
emp1_tax
emp1_deduc
emp1_rate
emp1_hrs
tN
t3
t2
t1


problem
instructions
processor
processor
processor
processor
tN
t3
t2
t1


problem
instructions
do_payroll(emp1)
do_payroll(emp2)
do_payroll(emp3)
do_payroll(empN)
instrucN
instruc7
instruc6
instruc5
instruc4
instruc3
instruc2
instruc1
processor
processor
processor
processor
tN
t3
t2
t1

*Obiective

- Acest capitol este dedicat tratării separate a celor mai reprezentative aspecte legate de controlul şi gestiunea sarcinilor concurente.
- Se pune un accent deosebit pe prezentarea
  - unor algoritmi de excludere mutuala
    - context centralizat
    - context distribuit
  - analiza situaţiilor de blocare
  - Sincronizarea sarcinilor.
- Problemele tratate în acest capitol asigură baza teoretică pentru analiza sau dezvoltarea unor sisteme software care sa lucreze
  - în regim de multiprogramare
  - multiprelucrare
- atât pentru sisteme monoprocesor, cât şi pentru sisteme cu prelucrare paralelă

Definiţii şi concepte de bază

- Activităţile paralele se pot desfăşura atât în sisteme de tip MIMD, SIMD cât şi în cadrul sistemelor convenţionale, de tip SISD.
- Un exemplu de activităţi paralele într-un sistem de calcul conventional de tip SISD, Von Neumann îl constituie efectuarea simultană a operaţiilor din UCP, cu cele din subsistemul de intrări/ieşiri prin DMA sau canal de I/E.
- De asemenea, activităţi paralele se întâlnesc la nivelul microoperaţiilor care constituie o microinstrucţiune în cadrul unei unităţi de comandă microprogramată.

În general pentru creşterea performanţelor unui sistem de calcul se prevăd mai multe procesoare de diferite tipuri, cum ar fi:

- procesoare centrale de prelucrare (multicore);
- procesoare de intrări/ieşiri;
- procesoare specializate (coprocesoare matematice, coprocesoare neurale, procesoare de prelucrare grafica etc).

- Pe de altă parte mai multe programe sau părţi ale unui aceluiaşi program pot fi executate în paralel.
- Comunicaţia între acestea poate fi asigurată printr-un schimb de mesaje.
- Chiar dacă există un singur procesor (nu o structură MIMD), care este partajat între mai multe programe, vom admite că, din punct de vedere logic aceste programe se execută în paralel disputându-şi accesul la diferite resurse fizice ale sistemului.

Sarcina -proces secvenţial

Sarcina - proces secvenţial este o activitate care constă din execuţia, pe un procesor, a unui grup de instructiuni (de obicei indivizibil) asociat unui set de date specificat.

- Deşi pare că fiecare sarcina are procesorul şi datele sale, în realitate mai multe sarcini pot utiliza în comun
  - un procesor
  - o secvenţă de cod
  - o structură de date.

O *sarcină* poate fi specificată prin relaţia sa cu exteriorul:

- intrările necesare;
- funcţia executată;
- ieşirile generate;
- starea la un moment dat;
- timpul de execuţie.

- Comportarea intrinsecă, internă, nu va fi specificată decât în situaţii excepţionale, pentru a facilita înţelegerea interacţiunii cu celelalte sarcini.
- Unei sarcini i se asociază două evenimente:
  - SI - iniţiere sarcină (SI- Sarcină Initiată);
  - SF - terminare sarcină (SF – Sarcina Finalizată).
- Dacă notăm cu t(·) timpul de apariţie a unui eveniment, vom considera că:

  $t(SF) - t(SI) \neq 0$ şi finit

cu condiţia ca toate resursele necesare execuţiei lui S să fie disponibile în momentul t(SI)

Vom considera sarcinile neinterpretate, ceea ce este echivalent cu faptul că pentru o mulţime dată de sarcini să ne intereseze secvenţele evenimentelor de iniţiere şi terminare fără să ne preocupe particularităţile de execuţie ale sarcinilor.

În ceea ce priveşte granularitatea unei sarcini, aceasta poate fi:

- program -JOB;
- modul de program - task;
- procedură - sarcina;
- secventa - thread
- instrucţiune;
- microinstrucţiune;
- microoperaţie.

Sistemul de calcul în care se execută sarcinile constă dintr-o mulţime de resurse:

- procesoare;
- dispozitive de I/E;
- biblioteci de programe;
- proceduri;
- variabile;
- fişiere de date, etc.

şi este caracterizat de o mulţime Σ de stări.

În funcţie de granularitatea sarcinilor, procesorul poate fi:

- calculator universal;
- UCP într-un sistem multiprocesor (core – in sisteme multicore);
- UAL;
- unitate de comandă;
- microsecvenţiator într-o unitate de comandă.

Atât evoluția sistemului, cât şi funcţiile sarcinilor date sunt reprezentate prin tranziţii permise în

Σ : $\sigma_0\sigma_1\sigma_2$..... σ_n–spatiul starilor,

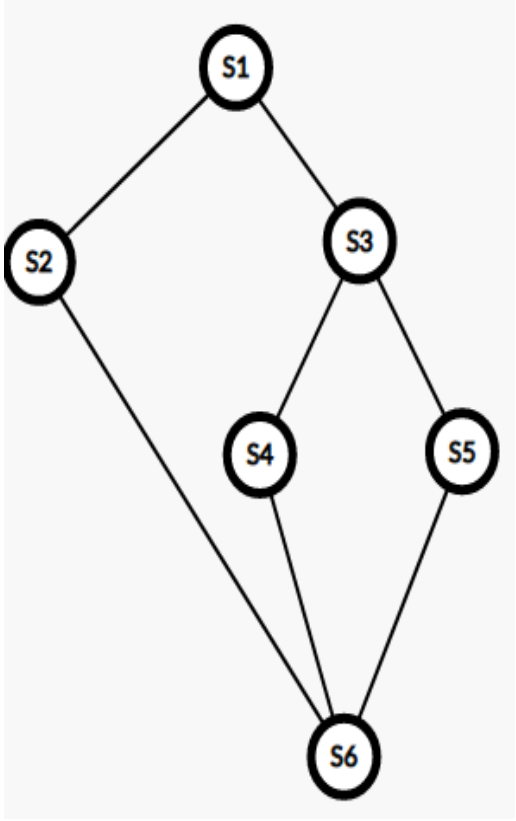
de forma $\sigma_i \rightarrow \sigma_j$, (σ_i, σ_j $\in$ Σ) definite pentru iniţierea şi terminarea sarcinilor.

Iniţierea unui sarcina corespunde unei tranziţii de stări care reflectă:

- preluarea resurselor necesare;
- iniţializarea stării sistemului (a resurselor);
- citirea parametrilor de intrare.

Terminarea unei sarcini corespunde unei tranziţii de stări care reflectă:

- eliberarea sau suspendarea temporară a utilizării resurselor;
- scrierea parametrilor de ieşire;
- dacă valori ale parametrilor sau stări interne sunt asociate cu resursele, terminarea unei sarcina trebuie însoţită de salvarea stării resurselor.



Fie $\boldsymbol{S} = \{ S_1, S_2, ..., S_n \}$ o mulţime de sarcini iar **<** o relaţie de ordine parţială, nereflexivă, pe ***S*** (denumită şi relaţie de precedenţă pe mulţimea de sarcini ***S***).
Perechea **C** = (***S*** , **<**) este denumită **sistem de sarcini.**

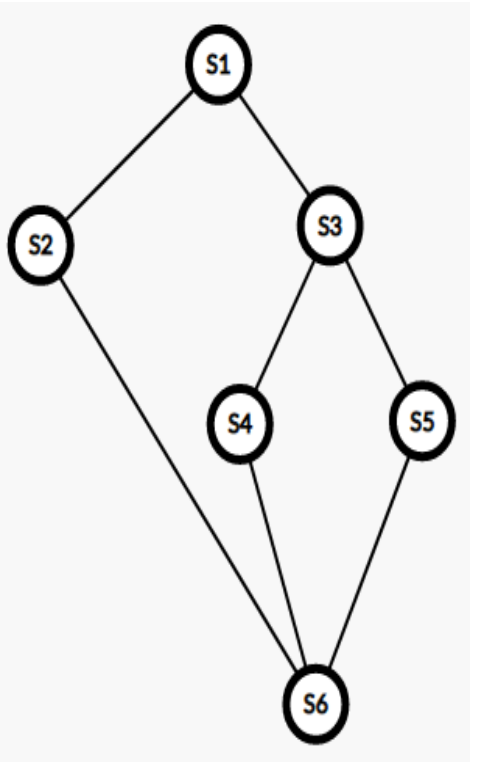
- Relaţia **<** reprezintă precedenţele în execuţia sarcinilor.

Astfel, $S_i < S_j$

- înseamnă că S_i trebuie terminată înainte de iniţierea lui S_j adică $SF_i < SI_j$

Sarcinile din ***S*** sunt independente dacă pentru **C** = (***S*** , **<**)
avem **<** = Φ; unde Φ este mulţimea vidă.

- O reprezentare grafică convenabilă a unui sistem de sarcini se obţine utilizând grafuri aciclice direcţionate, *GAD*.
- Sarcinile sunt reprezentate prin noduri iar relaţia de precedenţă prin mulţimea arcelor.
- Relaţia de precedenţă dată de mulţimea arcelor poate fi definită ca fiind cea mai mică relaţie în ***S*** a cărui închidere prin tranzitivitate este **<**.
- Deoarece **<** reprezintă o ordonare în timp a sarcinilor, vom spune că două sarcini S_i şi S_j, pot fi concurente dacă şi numai dacă ele sunt independente, adică între ele nu există o relaţie de precedenţă.

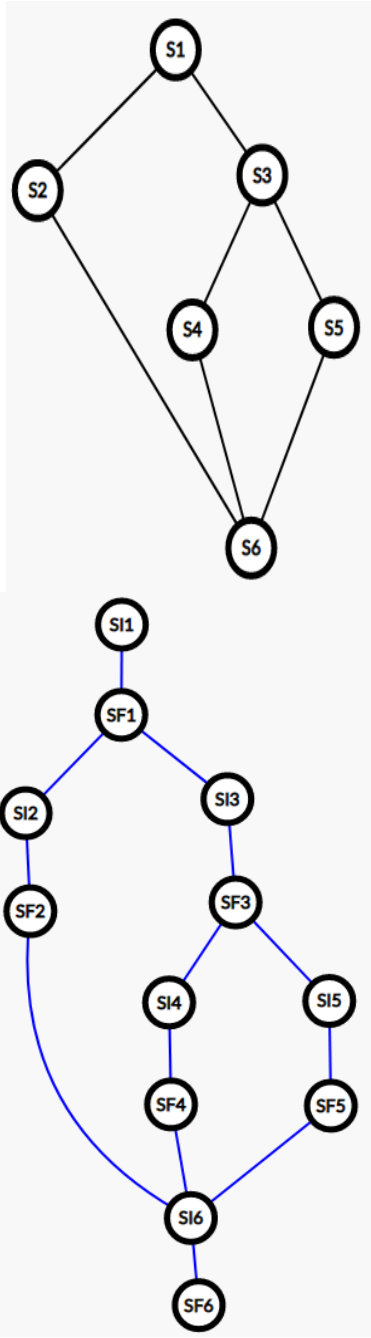


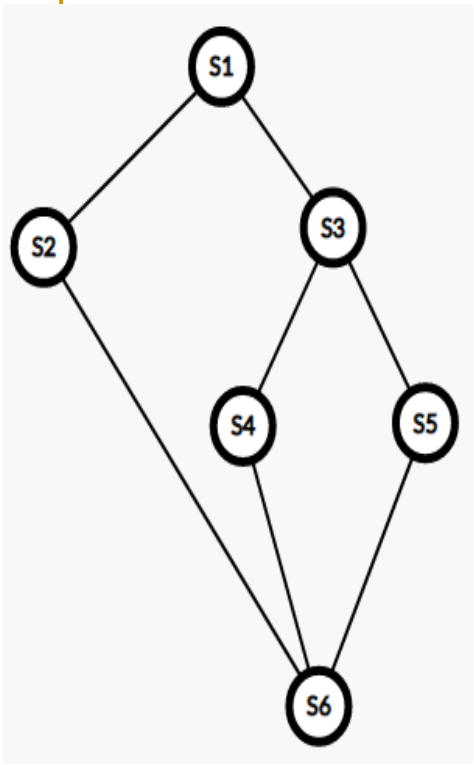
*Exemplu:

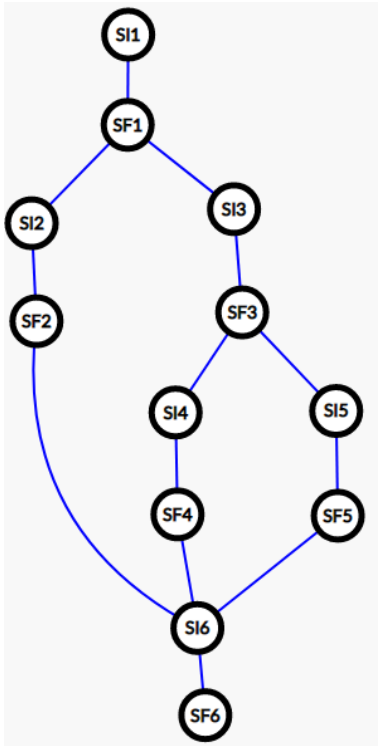
- Două sarcini S_i si S_j sunt independente dacă S_i nu este nici predecesor, nici succesor al lui S_j.
- O sarcină S din sistemul de sarcini **C** este pe nivelul k, sau are nivelul k, dacă cea mai lungă cale de la S la un nod terminal are lungimea k.
- Un nod terminal are nivelul 1.

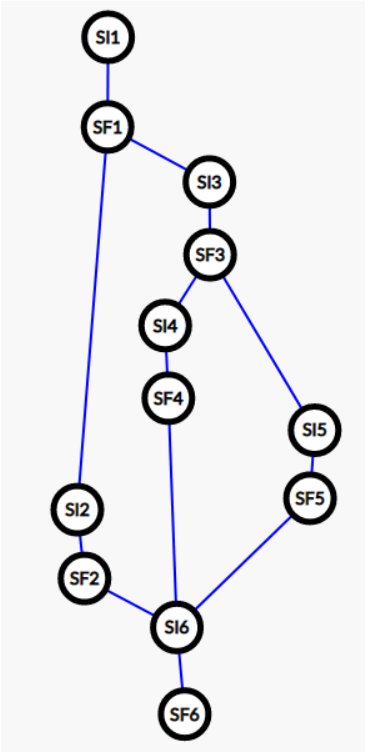
O secvenţă de execuţie a unui sistem de n sarcini, **C** = (***S***, **<**) este orice şir $\alpha = a_1\ a_2 \ldots\ldots a_{2n}$ de evenimente de iniţiere şi terminare a sarcinilor cu respectarea relaţiilor de precedenţă impuse de **<**.

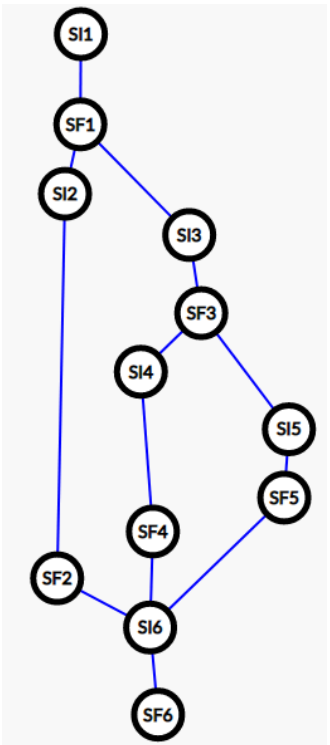
- Altfel spus:
  - pentru orice S Є ***S***, SI – S initiat şi SF - S terminat apar o singură dată în α ;
  - dacă a_i= SI şi a_j= SF, atunci i < j ;
  - dacă a_i= SF_k şi a_j= SI_l ,cu S_k **<** S_l atunci i < j ;











Sistem de sarcini reprezentat prin graf

Exemple de secvenţe de execuţie pentru sistemul de sarcini

- C=(S, **<**)
- S={S1,S2, S3,S4,S5,S6}
- reprezentat prin graful
- $\alpha_1 = SI_1\ SF_1\ SI_3\ SF_3\ SI_4\ SF_4\ SI_5\ SF_5\ SI_2\ SF_2\ SI_6\ SF_6$
- $\alpha_2 = SI_1\ SF_1\ SI_2\ SI_3\ SF_3\ SI_4\ SI_5\ SF_5\ SF_4\ SF_2\ SI_6\ SF_6$
- $\alpha_3 =$

secvență de execuție parțială

O secvenţă de execuţie parţială este orice prefix al unei secvenţe de execuţie.

Mulţimea secvenţelor de execuţie reprezintă mulţimea tuturor secvenţelor de evenimente (de iniţiere şi terminare) care conduc la finalizarea lui **C** respectându-se relaţia **<**.

Un sarcină S este activă după o secvenţă de execuţie parţială

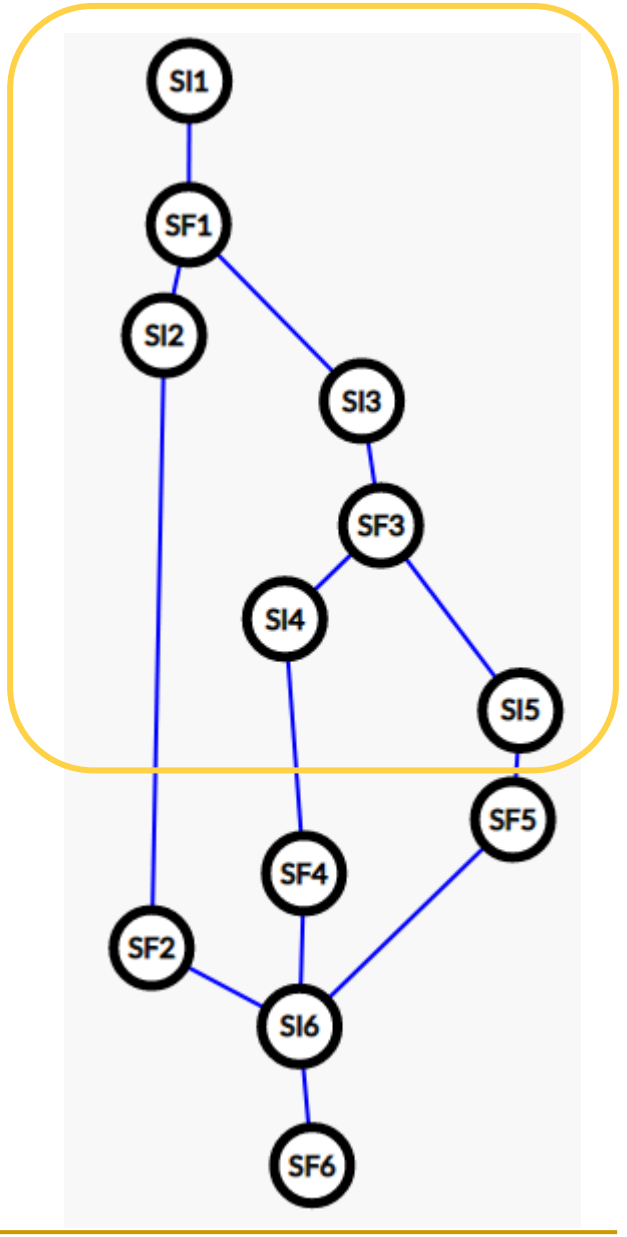
$\alpha_p = a_1 a_2 \ldots\ldots a_k$, dacă există $i \leq k$ astfel că

$a_i = SI$, dar pentru orice $j \leq k$, $a_j \neq SF$

Ex

$\alpha_p = SI_1\ SF_1\ SI_2\ SI_3\ SF_3\ SI_4\ SI_5$

- $\alpha_2 = SI_1\ SF_1\ SI_2\ SI_3\ SF_3\ SI_4\ SI_5\ SF_5\ SF_4\ SF_2\ SI_6\ SF_6$



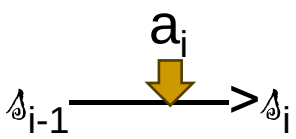
Fie Σ spaţiul stărilor pentru un sistem de sarcini.

- Secvenţa stărilor corespunzătoare secvenţei de execuţie

  $$\alpha = a_1\ a_2 \ldots\ldots a_{2n}$$

  este dată prin:

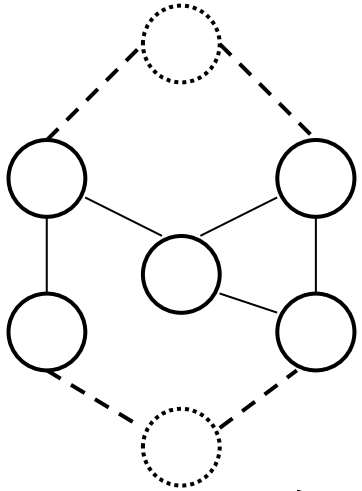
  $$\sigma = \sigma_0\sigma_1\sigma_2 \ldots\ldots\ldots \sigma_{2n}, \quad \sigma_j \in \Sigma,\ 0 \le j \le 2n\ ,$$

  iar σ_0 este starea iniţială dată.
- Tranziţia stării definită de evenimentul a_i este reprezentată de:
- 

- O tranziţie pentru o pereche de stări-eveniment (σ,a) va fi definită numai dacă evenimentul a poate să apară când sistemul este în starea σ.

Un sistem de sarcini reprezentat printr-un graf cu un sigur nod iniţial şi un singur nod terminal se numeşte închis.

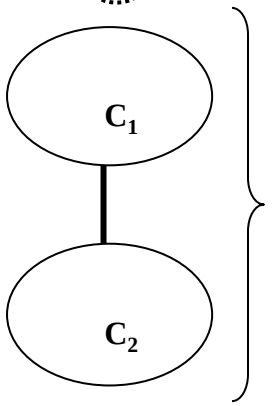
- Dacă sistemul de n sarcini nu este închis se pot prevedea sarcinile S_0 şi S_{n+1} inoperante pentru a obţine un sistem închis.



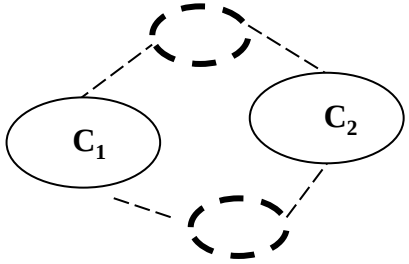
- Două sisteme de sarcini închise $\mathbf{C}_1$ şi $\mathbf{C}_2$ pot fi concatenate ($\mathbf{C}_1.\mathbf{C}_2$), graful asociat obţinându-se prin introducerea unui arc de la nodul terminal al lui $\mathbf{C}_1$ la nodul iniţial al lui $\mathbf{C}_2$.
- O secvenţă de execuţie a lui ($\mathbf{C}_1.\mathbf{C}_2$) este orice şir

$$\alpha = \alpha_1.\alpha_2$$

de secvenţe de execuţie α_1 a lui $\mathbf{C}_1$, α_2 a lui $\mathbf{C}_2$.



Combinarea paralelă a două sisteme de sarcini $\mathbf{C}_1$ şi $\mathbf{C}_2$, $\mathbf{C}_1 \mid \mathbf{C}_2$, care nu au sarcini în comun,($\forall S_i \in \mathbf{C}_1$ şi $\forall S_j \in \mathbf{C}_2$, $S_i \neq S_j$) constă în simpla lor reuniune.



- Graful lui $\mathbf{C}_1 \mid \mathbf{C}_2$ are ca subgrafuri disjuncte grafurile lui $\mathbf{C}_1$ şi $\mathbf{C}_2$.

Orice sarcină din $\mathbf{C_1}$ este independentă de orice sarcina din $\mathbf{C_2}$.
Dacă $a = a_1\ a_2.....a_{2m}$ şi $b = b_1\ b_2.....b_{2n}$ sunt secvenţe de execuţie pentru $\mathbf{C_1}$ respectiv pentru $\mathbf{C_2}$, o secvenţă de execuţie a lui $C = \mathbf{C_1} \mid \mathbf{C_2}$ se formează astfel:

$$c = c_1c_2.....c_{2(m+n)}$$

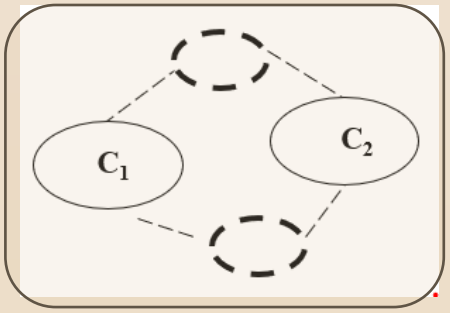
unde :

$c_1 = a_1 \sim b_1$ (fie a_1 fie b_1)

dacă $a_1\ a_2.....a_i$ şi $b_1\ b_2.....b_j$ sunt elemente ale lui $c_1\ c_2.....c_{i+j}$ cu $i+j < 2(m+n)$

atunci :

$$c_{i+j+1} = a_{i+1} \sim b_{j+1} \quad (\text{ fie } a_{i+1} \text{ fie } b_{j+1})$$



Combinarea paralelă a secvenţelor de execuţie apare frecvent în proiectarea sistemelor, în special a sistemelor cu prelucrare paralelă, uneori sub o formă diferită de cea prezentată anterior în sensul că unele sisteme implică secvenţe repetitive.

- Sistemele care conţin secvenţe repetitive pot fi modelate fie prin grafuri aciclice fie prin grafuri ciclice.
- Astfel, se pot modela k cicli ai unui sistem **C** prin concatenarea

$$\mathbf{C}^k = \mathbf{C}_1 \cdot \mathbf{C}_2 \cdot \ldots\ldots \cdot \mathbf{C}_k$$

- O secvenţă de execuţie α a lui $\mathbf{C}^k$ este de forma:

$$\alpha = \alpha_1 \alpha_2 \ldots\ldots \alpha_k$$

unde α_i este o secvenţă de execuţie a lui **C** pentru iteraţia i.

Un sistem de sarcini se numeşte ciclic dacă este de forma $\mathbf{C}^k$ pentru $k > 1$, sau dacă este o combinare paralelă de sisteme închise în care cel puţin unul este ciclic.

Proprietatea de determinare a sistemelor de sarcini

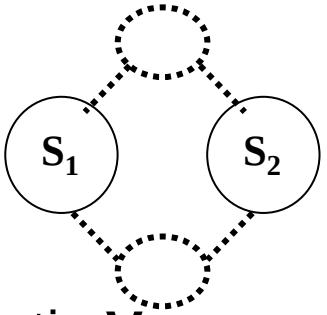
Definiţie:
Sistem determinat (sistem de sarcini functional) este un sistem format din sarcini care se execută în paralel şi cooperează pentru realizarea unor operaţii logice şi de calcul, care produce acelaşi rezultat indiferent de durata de execuţie a fiecărei sarcini **independente**, sau de ordinea în care acestea se executa.

Definiţie:
Un sistem de sarcini este nedeterminat dacă rezultatele produse de sarcini independente depind de ordinea în care acestea se executa.

- Exemplu de sistem nedeterminat:

S_1: $R_1 \leftarrow$ BUSFN (M; DCD (ADR))

S_2: M * DCD (ADR) $\leftarrow$ **V**



Memoria contine la adresa ADR informatia **M**

S_1 - citeşte din locaţia de memorie de la adresa ADR;

S_2 - scrie în locaţia de memorie de la adresa ADR informatia **V**.

$\alpha_1 = SI_1\, SF_1\, SI_2\, SF_2$

se va citi in R_1 informatia **M** si

in celula de memorie cu adresa ADR se va inscrie informatia **V**

$\alpha_2 = SI_2\, SF_2\, SI_1\, SF_1$

in celula de memorie se va inscrie valoarea **V** si

se va **citi in R1** informatia **M**

Produc rezultate diferite.

Informatia furnizata de S_1depinde de secventa de executie efectuata.

Nedeterminarea se poate rezolva introducând o relaţie de precedenţă adecvată între sarcinile care erau independente.

- Pentru a stabili condiţiile necesare şi suficiente ca un sistem să fie determinat, vom considera un model simplificat în care sistemul fizic este privit ca o mulţime ordonată de locaţii de memorie:

$$\mathbf{M} = (M_1, M_2, ..., M_m)$$

ce conţin orice valoare dintr-o mulţime de valori **V**.

- Stările sistemului vor fi definite de valorile care se găsesc în memorie la un moment dat:

De exemplu dacă:

$$\alpha = a_1 a_2 ..... a_{2n}$$ şi

$$\sigma = s_0 s_1 s_2 ..... s_{2n}$$

reprezintă o secvenţă de execuţie, respectiv secvenţa de stări corespunzătoare,

iar $M_i(k)$reprezintă valoarea din celula M_i imediat după evenimentul a_k

starea sistemului va fi:

$$s_k = [M_1(k), M_2(k), ..., M_m(k)]$$

Mulţimea tuturor stărilor poate fi definită astfel:

$\mathbf{\Sigma} = \mathbf{V}^m$ $\quad\quad$ $\mathbf{V}^m = [\ \mathbf{V} \times \mathbf{V} \times \ldots\ldots \times \mathbf{V}\ ]$, produs cartezian.

- Pentru a formaliza efectul execuţiei unei sarcini asupra celulelor de memorie vom considera că fiecărei sarcini S i se asociază funcţia:

$$f_S : V^d \longrightarrow V^r$$

unde:

d = $|D_S|$ cardinalul domeniului valorilor de intrare, D_S;

r = $|R_S|$ cardinalul domeniului valorilor de ieşire, R_S.

- Pentru o stare iniţială σ_0 şi o secvenţă de execuţie α, secvenţa corespunzătoare de stări: $\sigma = \sigma_0\sigma_1 \ldots\ldots \sigma_{2n}$ este definită în felul următor :

Fie $D_S = (M_{x1}, M_{x2}, \ldots\ldots, M_{xd})$ domeniul valorilor de intrare;

$R_S = (M_{y1}, M_{y2}, \ldots\ldots, M_{yr})$ domeniul valorilor de ieşire;

dacă a_{k+1} = SI, atunci $M_i(k+1) = M_i(k)$, $1 \le i \le m$;
dacă a_{k+1} = SF, şi a_l = SI , $l \le k$, atunci stările locaţiilor de memorie din domeniul de valori al lui S la momentul k+1 sunt:

$$[M_{y1}(k+1), M_{y2}(k+1), \ldots, M_{yr}(k+1)] = f_S\,(M_{x1}(l), M_{x2}(l), \ldots, M_{xd}(l)) \qquad \mathbf{M_j \in R_S}$$

şi

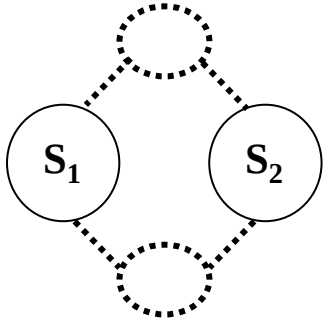
$$M_i(k+1) = M_i(k) \qquad \text{pentru } (\forall)\ M_i \notin R_S$$

- Altfel spus, dacă a_{k+1} este un eveniment de iniţiere nu apare o schimbare a stării, adică $\sigma_{k+1} = \sigma_k$.
- Dacă însă a_{k+1} este un eveniment de terminare a lui S, $\sigma_{k+1} \neq \sigma_k$ doar în domeniul R**s**, noile valori din R**s** fiind determinate de f_S pe baza valorilor din domeniul de definiţie din momentul imediat precedent iniţierii lui S .
- Secvenţa de stări $\sigma = \sigma_0\sigma_1 \ldots\ldots \sigma_{2n}$ ce rezultă dintr-o secvenţă de execuţie, poate fi reprezentată sub forma unui tablou de dimensiuni m * (2n+1) având
- pe linii celulele de memorie M, iar
- pe coloane stările.

Sistem de sarcini nedeterminat

*Exemplu:

Să considerăm un sistem format din două sarcini, iar relaţia de precedenţă fiind mulţimea vidă



$\mathbf{C} = (\{S_1, S_2\}, \Phi)$

$\mathbf{M} = (M_1, M_2)$

$D_{S1} = D_{S2} = R_{S1} = R_{S2} = \mathbf{M}$

Sistemului i se atribuie două interpretări:

Sarcini	Interpretare 1	Interpretare 2
S_1	$(M_1,M_2) \leftarrow (1,M_2)$	$(M_1,M_2) \leftarrow (M_1+M_2,M_2)$
S_2	$(M_1,M_2) \leftarrow (2,M_2)$	$(M_1,M_2) \leftarrow (M_1,M_1+M_2)$

Valorile iniţiale corespunzătoare stării iniţiale s_0: $M \leftarrow (1,2)$

Fie două secvenţe de execuţie α_1 şi α_2

$\alpha_1 = SI_1\, SF_1\, SI_2\, SF_2$ şi

$\alpha_2 = SI_2\, SF_2\, SI_1\, SF_1$

Comportarea celor două secvențe de execuție

Sarcini	Interpretare 1	Interpretare 2
S_1	$(M_1,M_2) \leftarrow (1,M_2)$	$(M_1,M_2) \leftarrow (M_1+M_2,M_2)$
S_2	$(M_1,M_2) \leftarrow (2,M_2)$	$(M_1,M_2) \leftarrow (M_1,M_1+M_2)$

Să analizăm comportarea celor două secvenţe de execuţie conform cu cele două interpretări ale sistemului:

	$\alpha_1 = SI_1\ SF_1\ SI_2\ SF_2$	$\alpha_2 = SI_2\ SF_2\ SI_1\ SF_1$
	$\sigma = \delta_0\ \delta_1\ \delta_2\ \delta_3\ \delta_4$	$\sigma = \delta_0\ \delta_1\ \delta_2\ \delta_3\ \delta_4$
Interpretare 1	M_1 1 1 1 1 2	M_1 1 1 2 2 1
	M_2 2 2 2 2 2	M_2 2 2 2 2 2
Interpretare 2	M_1 1 1 3 3 3	M_1 1 1 1 1 4
	M_2 2 2 2 2 5	M_2 2 2 3 3 3

Se observă că sistemul de sarcini C nu este determinat pentru nici una din cele două interpretări .

Reprezentare prin secventa de valori

- O reprezentare mai convenabilă constă din secvenţa de valori pe care un sistem de sarcini dat C o înscrie într-o celulă de memorie Mi, în timpul unei secvenţe de execuţie α .
- Această reprezentare se notează sub formă vectorială:

$V(M_i,\alpha) = (V_1,V_2,.....,V_p)$

Considerând

$\alpha = a_1a_2.....a_{2n}$ secvenţa de execuţie

$\sigma = \sigma_0\sigma_1\ldots\ldots \sigma_{2n}$, secvenţa de stări corespunzătoare,

ε= Φ şirul vid, secvenţa de valori corespunzătoare se defineşte astfel:

$V(M_i,\varepsilon) = M_i(\Phi)$ în lipsa unei secvenţe de execuţie, iar

$V(M_i, a_1a_2 \ldots a_k)=$ <u>dacă</u> a_k= SF şi $M_i \in R_S$ (domeniul unde S produce valori)

<u>Atunci</u> = $(V(M_i, a_1a_2 \ldots a_{k-1}), M_i(k))$

<u>Altfel</u> nu se modifică deci este $V(M_i, a_1 \ldots a_{k-1})$

nu se mai adaugă o nouă valoare

- Se iau în considerare numai valorile înscrise în celulele de memorie din domeniul de valori şi numai la terminarea S.

Obs. $V(M_i, \alpha)$= ..... $V(M_j, \alpha)$= ..... pot avea dimensiuni diferite

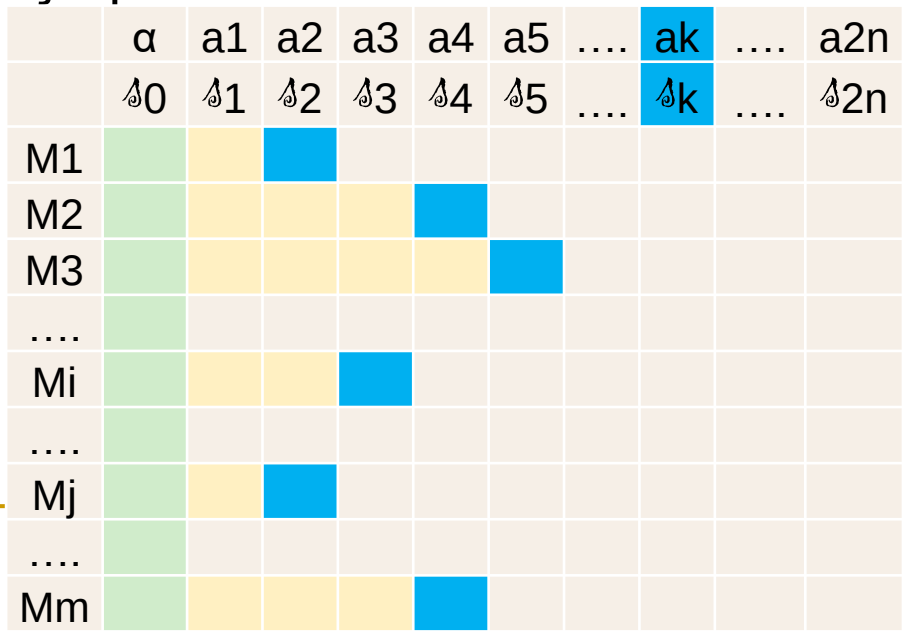
- De notat că $V(M_i,\alpha)$ şi $V(M_j,\alpha)$ nu trebuie să fie de aceeaşi lungime deoarece M_i poate fi în Rs pentru anumite sarcini
- iar M_j poate fi în Rs pentru altele (în număr diferit).

Pentru secvenţa de valori: $V(M_i,\alpha) = (v_1,v_2,.....,v_p)$ se defineşte *valoarea finală* $F(M_i,\alpha) = v_p$.

Rezultă că pentru secvenţa de execuţie partiala

$\alpha = a_1a_2...a_k$

$\beta_k = [F(M_1,\alpha),F(M_2,\alpha),...,F(M_m,\alpha)]$



Exemplu: Să reconsiderăm exemplul precedent:

* Secvenţa de stări poate fi reprezentată astfel printr-un tablou de dimensiuni mai mici de cel mult m(n+1):

	$\alpha_1 = SI_1\ SF_1\ SI_2\ SF_2$			$\alpha_2 = SI_2\ SF_2\ SI_1\ SF_1$		
	V(M,ε)	V(M,a_1a_2)	V(M,$a_1..a_4$)	V(M,ε)	V(M,a_1a_2)	V(M,$a_1..a_4$)
Interpretare 1	M_1 1	1	2	M_1 1	2	1
	M_2 2	2	2	M_2 2	2	2
Interpretare 2	M_1 1	3	3	M_1 1	1	4
	M_2 2	2	5	M_2 2	3	3

Interpretarea 1 este nedeterminată deoarece S_1 şi S_2 sunt în dispută pentru a scrie în M_1, iar în interpretarea 2 o sarcina scrie într-o celulă citită de alta sarcina. In implementarea anterioara dimensiunea memoriei era de m(2n+1)

Sarcini	Interpretare 1	Interpretare 2
S_1	$(M_1,M_2) \leftarrow (1,M_2)$	$(M_1,M_2) \leftarrow (M_1+M_2,M_2)$
S_2	$(M_1,M_2) \leftarrow (2,M_2)$	$(M_1,M_2) \leftarrow (M_1,M_1+M_2)$

	$\alpha_1 = SI_1\ SF_1\ SI_2\ SF_2$	$\alpha_2 = SI_2\ SF_2\ SI_1\ SF_1$
	$\sigma = \aleph_0\ \aleph_1\ \aleph_2\ \aleph_3\ \aleph_4$	$\sigma = \aleph_0\ \aleph_1\ \aleph_2\ \aleph_3\ \aleph_4$
Interpretare 1	M_1 1 1 1 1 2	M_1 1 1 2 2 1
	M_2 2 2 2 2 2	M_2 2 2 2 2 2
Interpretare 2	M_1 1 1 3 3 3	M_1 1 1 1 1 4
	M_2 2 2 2 2 5	M_2 2 2 3 3 3

Definiţie:
Sistem determinat (sistem de sarcini functional) este un sistem format din sarcini care se execută în paralel şi cooperează pentru realizarea unor operaţii logice şi de calcul, care produce acelaşi rezultat indiferent de durata de execuţie a fiecărei sarcini **independente**, sau de ordinea în care acestea se executa.

- Un sistem de sarcini **C** = (***S***, **<**) este neinterpretat dacă se cunosc doar relatia de precedenta **<** , domeniile de definitie Ds şi de valori Rs pentru fiecare sarcina S.
- O interpretare pentru **C** constă din specificarea funcţiei f_S pentru fiecare S_i Є ***S*** .

În continuare vom considera sisteme de sarcini **neinterpretate.**

Definiţie:
Un sistem de sarcini ***C*** *este determinat* dacă pentru o stare iniţială $\mathcal{A}_0$ dată, **$V(M_i,\alpha) = V(M_i,\alpha')$,** $1 \leq i \leq m$, pentru toate secvenţele de execuţie α ,….α' din **C**.

(Secvenţele de valori depind numai de valorile iniţiale în $\mathcal{A}_0$.)

Definiţie:
Sarcinile Si şi Sj sunt neinterferente dacă:

Si este succesor sau predecesor a lui Sj, sau

$R_{Si} \cap R_{Sj} = R_{Si} \cap D_{Sj} = D_{Si} \cap R_{Sj} = \Phi$

- Mulţimea ***S*** = { $S_1, S_2, ... S_n$ } se spune că este formată din sarcini mutual neinterferente dacă
  - pentru $\forall$ i , j (i $\neq$ j) S_i si S_j sunt neinterferente.
- Pentru a arăta că un sistem de sarcini mutual neinterferente este determinat vom prezenta *teorema de suficienţă şi necesitate*.

Teorema de suficienţă:

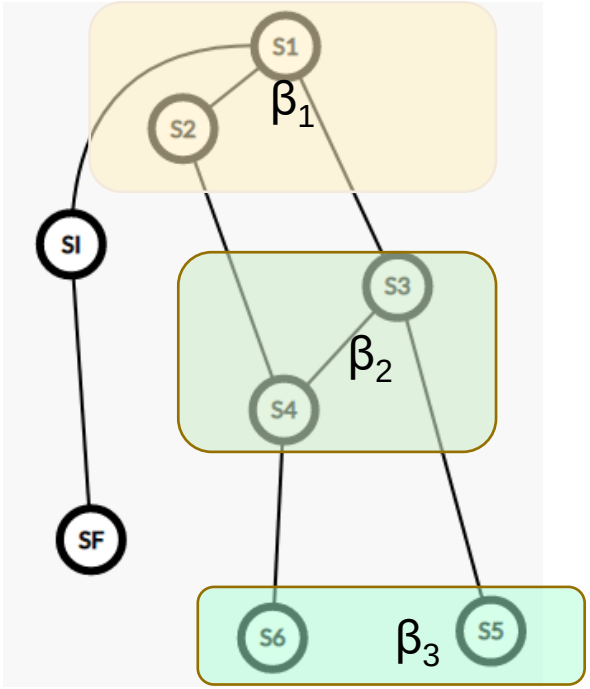
Sistemele de sarcini formate din sarcini mutual neinterferente sunt determinate.

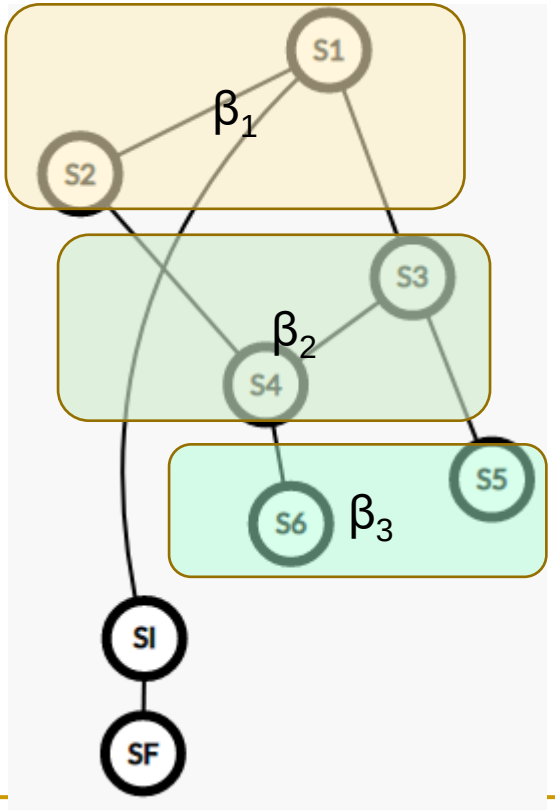
Teorema de necesitate:

Fie **C** un sistem de sarcini astfel că pentru fiecare S$\in$***S,*** f_S nu este specificată, dar D_S şi $R_S \neq \Phi$ sunt date.

C este determinat pentru orice interpretare a sarcinilor sale numai dacă acestea sunt mutual neinterferente.

- α = β1 SI β2 SF β3 α’ = β1 β2 β3 SI SF





Vom stabili o lemă utilizată în cadrul justificarii teoremelor de necesitate şi suficienţă.

Lemă:

Fie **C** = (**<** , α) un sistem de n sarcini mutual neinterferente ($R_S \cap R_{S'} = R_S \cap D_{S'} = D_S \cap R_{S'} = \Phi$) unde S ∈ **C** si oricare S' ∈ **C** iar S o sarcina terminala a lui **C**.

Dacă α = β_1 SI β_2 SF β_3 este o secvenţă de execuţie validă a lui **C**, atunci

α' = β_1 β_2 β_3 SI SF este de asemenea o secvenţă de execuţie a lui **C** pentru $\mathcal{S}_0$ dată,

$$V(M_i,\alpha) = V(M_i,\alpha'),\ \forall\ 1 \le i \le m$$

Justificare :

- S nu are succesori în **C** conform cu (**<** , α') satisface relaţiile de precedenţă din **C** şi deci trebuie să fie o secvenţă de execuţie validă.
- S scrie numai în celulele ce aparţin lui R_S şi deoarece pentru orice S' iniţiată în β_3, după terminarea lui S în α, $R_S \cap D_{S'} = \Phi$ fiind neinterferente, orice astfel de sarcini S' găsesc aceleaşi valori în $D_{S'}$ atât pentru α cât şi pentru α'.

- Deci pentru $M_i \notin R_S$ rezultă $V(M_i,\alpha)=V(M_i,\alpha')$

- Pentru orice $M_j \in D_S$, $V(M_j,\beta_1)=V(M_j , \beta_1\beta_2\beta_3)$ deoarece nici o sarcina S' din $\beta_1\beta_2\beta_3$ nu scrie în D_S deoarece $R_{S'} \cap D_S = \Phi$ fiind mutual neinterferente.
- Deci $F(M_j ,\beta_1) = F(M_j ,\beta_1\beta_2\beta_3)$ pentru $\forall M_j \in D_S$, iar S scrie aceeaşi valoare în $\forall\ M_i \in R_S$ atât pentru α cât şi pentru α'.
- Considerând că v este valoarea înscrisă de S în $M_i \in R_S$ pentru α, rezultă că:

$V(M_i,\alpha) = V(M_i , \beta_1\ SI\ \beta_2\ SF\ \beta_3)$

$V(M_i,\alpha) = V(M_i , \beta_1\ SI\ \beta_2\ SF)$ nu există $S' \in \beta_3$ care scrie în R_S

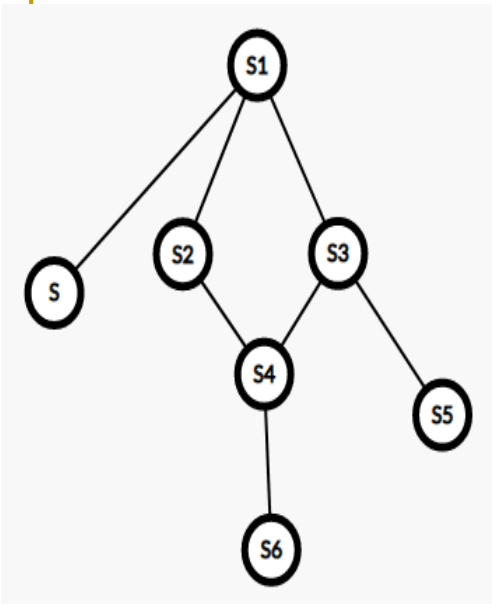
$V(M_i,\alpha) = (V(M_i , \beta_1\ SI\ \beta_2) ,v)$ S scrie valoarea v în

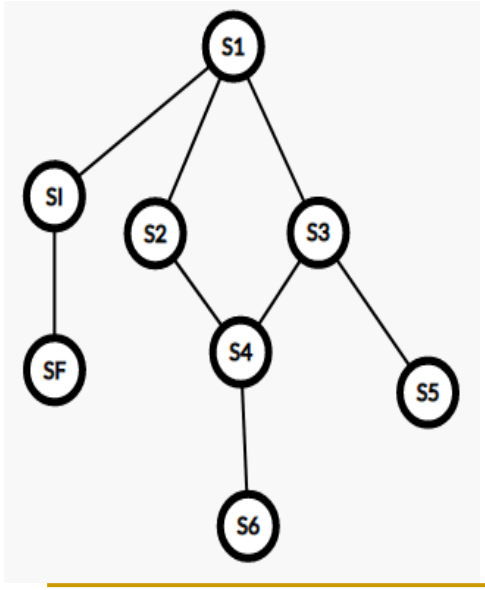
$V(M_i,\alpha) = (V(M_i , \beta_1) , v)$ nu există $S' \in \beta_2$ care scrie în R_S

$V(M_i,\alpha) = (V(M_i , \beta_1\ \beta_2\ \beta_3),v)$ nu există $S' \in \beta_2\beta_3$ care scrie în R_S

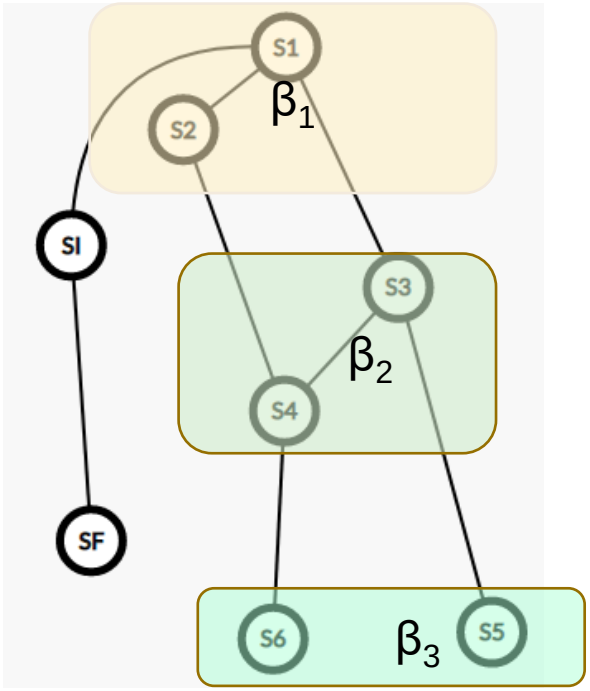
$V(M_i,\alpha) = (V(M_i , \beta_1\ \beta_2\ \beta_3\ SI\ SF)$ S scrie v în M_i

$V(M_i,\alpha) = V(M_i , \alpha')$

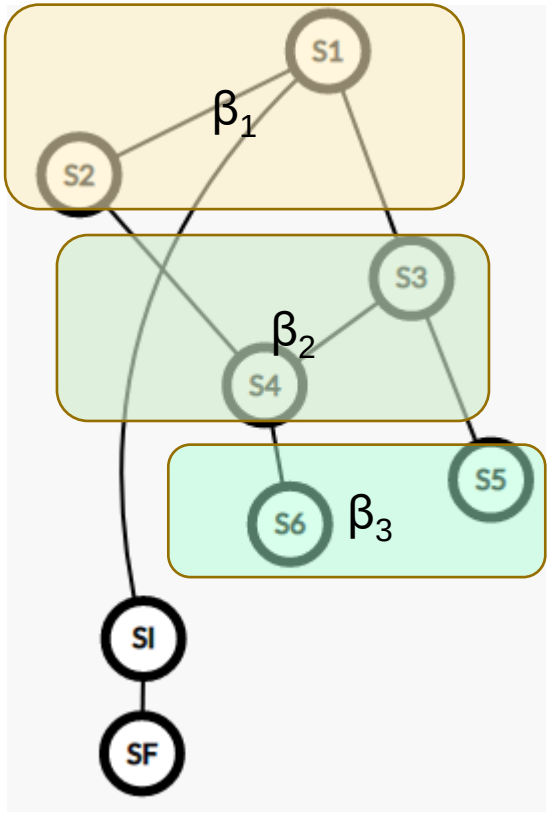




$\alpha = \beta_1$ SI β_2 SF β_3



$\alpha' = \beta_1 \ \beta_2 \beta_3$ SI SF



Lema

Teorema de suficienţă:
Sistemele de sarcini formate din sarcini mutual neinterferente sunt determinate.

Justificare (prin inducţie)

- Pentru un sistem de sarcini format dintr-o singura sarcina este evident **determinat.**
- Presupunem că **afirmaţia este adevărată** pentru sisteme cu n-1 sarcini, (α'_1 şi α'_2 sunt secvenţe de execuţie pentru un sistem cu n-1 sarcini)
- Presupunem sistemul **C** = (***S***, **<**) format din n sarcini.

- Dacă sistemul **C** are o singură secvenţă de execuţie validă, el este determinat.
- Presupunem 2 secvenţe de execuţie α_1, α_2 în **C** şi S sarcina terminala în ***S***.

Conform lemei putem forma 2 secvenţe de execuţie α'_1 şi α'_2 astfel:

$\alpha_1 = \alpha'_1$ SI SF; $\alpha_2 = \alpha'_2$ SI SF,

în condiţiile în care

$$V(M_i, \alpha_1) = V(M_i, \alpha'_1), \quad 1 \le i \le m$$

$$V(M_i, \alpha_2) = V(M_i, \alpha'_2), \quad 1 \le i \le m$$

α'_1 şi α'_2 sunt secvenţe de execuţie pentru un sistem cu n-1 sarcini: **C'** = (***S*** \ S, **<'**), relaţia de precedenţă **<'** fiind obţinută din **<** eliminând relaţia de precedenţă ce implică pe S.

$V(M_i, \alpha'_1) = V(M_i, \alpha'_2)$ din ipoteza de inducţie.

- Deci se poate presupune că valorile din D_S sunt aceleaşi, atât pentru α'_1 cât şi pentru α'_2,
- respectiv $F(M_i, \alpha'_1) = F(M_i, \alpha'_2)$ pentru $\mathbf{M_i \in D_S}$.
- Rezultă astfel că pentru α'_1 şi α'_2, sarcina S scrie aceeaşi valoare v pentru orice celulă $M_i \in D_S$.

Pentru $\mathbf{M_i \notin R_S}$:

$\mathbf{V(M_i, \alpha_1)}$	$= V(Mi\ ,\ \alpha'_1$ SI SF)	Lemă
	$= V(M_i\ ,\ \alpha'_1)$	ip. de inducţie
	$= V(M_i\ ,\ \alpha'_2)$	
	$= V(M_i\ ,\ \alpha'_2$ SI SF)	$M_i \notin R_S$ si Lemă
	$\mathbf{= V(M_i\ ,\ \alpha_2)}$	

Pentru $M_i \notin R_S$ $\mathbf{V(M_i, \alpha_1) = V(M_i, \alpha_2)}$

Pentru $\mathbf{M_i \in R_S}$:

$\mathbf{V(M_i, \alpha_1)}$	$= V(Mi\ ,\ \alpha'1$ SI SF)	Lemă
	$= (V(M_i\ ,\ \alpha'_1),v)$	
	$= (V(M_i\ ,\ \alpha'_2),v)$	ip. de inducţie
	$= V(M_i\ ,\ \alpha'_2$ SI SF)	Lemă
	$\mathbf{= V(M_i\ ,\ \alpha_2)}$	

Pentru Mi $\in$ RS **V(Mi, α1) = V(Mi, α2)**

- *Sistemul **C** este determinat dacă sarcinile sunt mutual neinterferente.*

Teorema de necesitate:
Fie **C** un sistem de sarcini astfel că pentru fiecare $S \in \boldsymbol{S}$, f_S nu este specificată, dar D_S şi $R_S \neq \Phi$ sunt date.
C este determinat pentru orice interpretare a sarcinilor sale numai dacă acestea sunt mutual neinterferente.

Justificare (reducere la absurd**)**

- Presupunem că S, S' sunt interferente, independente (adică nu există nici o relaţie de ordine între ele).
- Atunci există două'secvenţe de execuţie valide:

  α = β1 SI SF SI' SF' β2

  α' = β1 SI' SF' SI SF β2
- Presupunem că există o celulă de memorie $M_i \in R_S \cap R_{S'}$ (sunt considerate interferente) şi putem alege f_S şi $f_{S'}$ (două interpretări, pentru S şi S'), astfel încât pentru:

  f_S: S să scrie în M_i valoarea u;

  $f_{S'}$: S' să scrie în M_i valoarea v; $u \neq v$

În ceea ce priveşte valorile din R_S şi $R_{S'}$ pentru secvenţele α şi α’ putem spune că:

$$V(M_i, \alpha) = V(M_i, \beta 1 \text{ SI SF SI' SF'}) = (V(M_i, \beta_1), u, v);$$

$$u \neq v$$

$$V(M_i, \alpha') = V(M_i, \beta_1 \text{ SI' SF' SI SF}) = (V(M_i, \beta_1), v, u);$$

Deci în celula M_i, secvenţele α si α' înscriu secvenţe de valori diferite

$$(V(M_i, \beta_1), u, v); \text{ respectiv } (V(M_i, \beta_1), v, u);$$

$$V(M_i, \alpha) \neq V(M_i, \alpha')$$

ceea ce înseamnă că sistemul de sarcini **C** nu este determinat.

Deci este necesar ca $R_S \cap R_{S'} = \Phi$ pentru că altfel rezultă că **C** este nedeterminat.

$\alpha = \beta 1$ SI SF SI' SF' $\beta 2$

$\alpha' = \beta 1$ SI' SF' SI SF $\beta 2$

- Presupunem că există o celulă $M_j \in D_S \cap R_{S'}$.
- Presupunem că există o celulă $M_i \in R_S$.
- Atunci, putem alege o interpretare a lui S', o funcţie $f_{S'}$ astfel încât $F(M_j, \beta_1) \neq F(M_j, \beta_1$ SI' SF').
- Rezultă în acest caz că S citeşte valori diferite pentru α şi α' (având în vedere că $M_j \in D_S \cap R_{S'}$).

- Putem alege în acest caz o funcţie f_S astfel încât sarcina S să scrie în M_i, valoarea u pentru secvenţa α şi valoarea v pentru secvenţe α', cu $u \neq v$.

În acest caz putem spune că:

$V(M_i, \alpha) = V(M_i, \beta_1$ SI SF SI' SF')

$= V(M_i, \beta_1$ SI SF) $\quad R_S \cap R_{S'} = \Phi$

$= (V(M_i, \beta_1), u)$ $\quad$ S scrie u pentru α

$V(M_i, \alpha') = V(M_i, \beta_1\ SI'\ SF'\ SI\ SF)$

$= (V(M_i, \beta_1\ SI'\ SF'), v)$

$= (V(M_i, \beta_1), v)$ S scrie v pentru α'

deci $V(M_i, \alpha) \neq V(M_i, \alpha')$ deci **C** nu este determinat.

Rezultă că trebuie ca : $D_S \cap R_{S'} = \Phi$

Similar se poate arăta că trebuie ca : $D_{S'} \cap R_S = \Phi$

O aplicaţie foarte importantă a teoremelor arătate anterior se referă la paralelismul maxim în sisteme de sarcini în care constrângerile de precedenţă sunt impuse numai de cerinţele de determinare.

- Din definiţia dată pentru determinare rezultă că fiecare sarcina produce o singură secvenţă de valori pentru o celulă de memorie M_i în condiţiile unei stări iniţiale date.

- Două sisteme cu aceeaşi mulţime de sarcini S sunt echivalente dacă sunt determinate şi dacă pentru aceeaşi stare iniţială produc aceeaşi secvenţă de valori.

Un sistem de sarcini **C** şi graful asociat G (definit de relatia de precedenta **<**) implică un paralelism maxim dacă **C** este determinat şi dacă **eliminarea** unui arc oarecare (S, S') din G, face ca sarcinile S şi S' să devină **interferente**.

- Astfel dacă (S , S') este un arc într-un graf cu maximum de paralelism atunci:

$$(R_S \cap R_{S'}) \cup (R_S \cap D_{S'}) \cup (D_S \cap R_{S'}) \neq \Phi$$

- Având un sistem de sarcini **C**, determinat, este util să construim sistemul echivalent **C'** cu maximum de paralelism.

Sistem de sarcini cu maximum de paralelism

Teoremă :
Avand un sistem de sarcini **C**= (***S***, **<**) se poate construi un sistem
C' = (***S***, **<'**) unde
relaţia <' este închiderea prin tranzitivitate a relaţiei:
$Z=\{(S,S') \in < \mid (R_S \cap R_{S'}) \cup (R_S \cap D_{S'}) \cup (D_S \cap R_{S'}) \neq \Phi\}$
C' este unicul sistem echivalent cu **C** care implică
maximum de paralelism.

Exemplu :
Fie sistemul de sarcini C = (S , **<**) unde
S = {S1,S2,S3,S4,S5,S6,S7,S8 }
Să se construiască sistemul de sarcini C'=(S, **<'**) echivalent, care realizează maxim de paralelism tinand seama de cerinţele de determinare (care are la baza conflictul de resurse)

- Sistemul este caracterizat prin:
- $M = \{ M_1, M_2, M_3, M_4, M_5 \}$

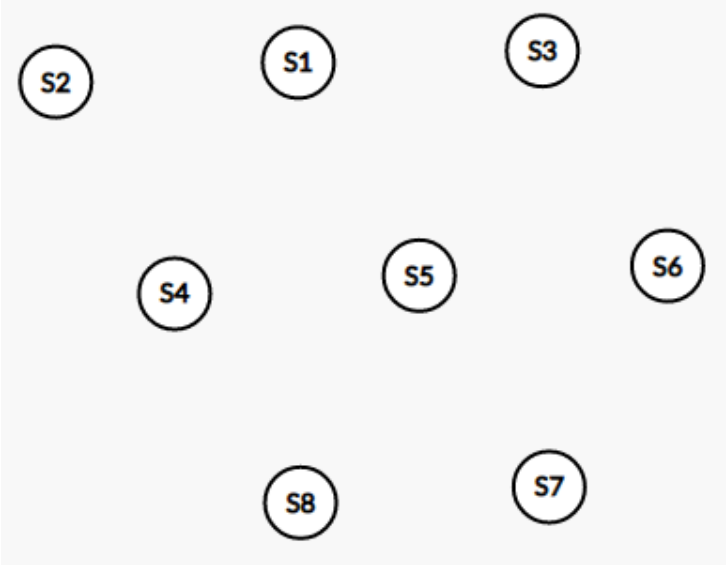
iar domeniile D_S şi R_S sunt specificate prin tabelul :

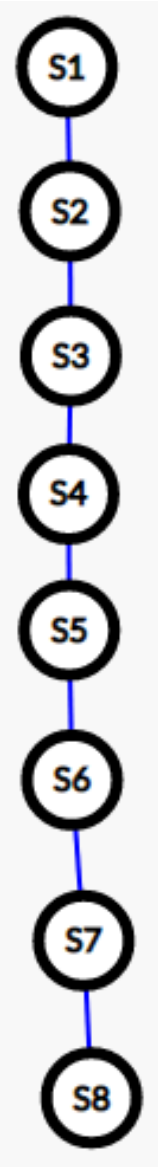
M	Domeniu de definiţie D_S	Domeniu de valori R_S
M1	S1,S2,S7,S8	S3
M2	S1,S7	S5
M3	S3,S4,S8	S1
M4	S3,S4,S5,S7	S2,7
M5	S6	S4,S6,S8

Posibillitati de executie a sistemului de sarcini



maxim de paralelism tinand seama de cerinţele de determinare bazate pe eliminarea conflictului de resurse





$$Z=\{(S,S') \in < \mid (R_S \cap R_{S'}) \cup (R_S \cap D_{S'}) \cup (D_S \cap R_{S'}) \neq \Phi\}$$

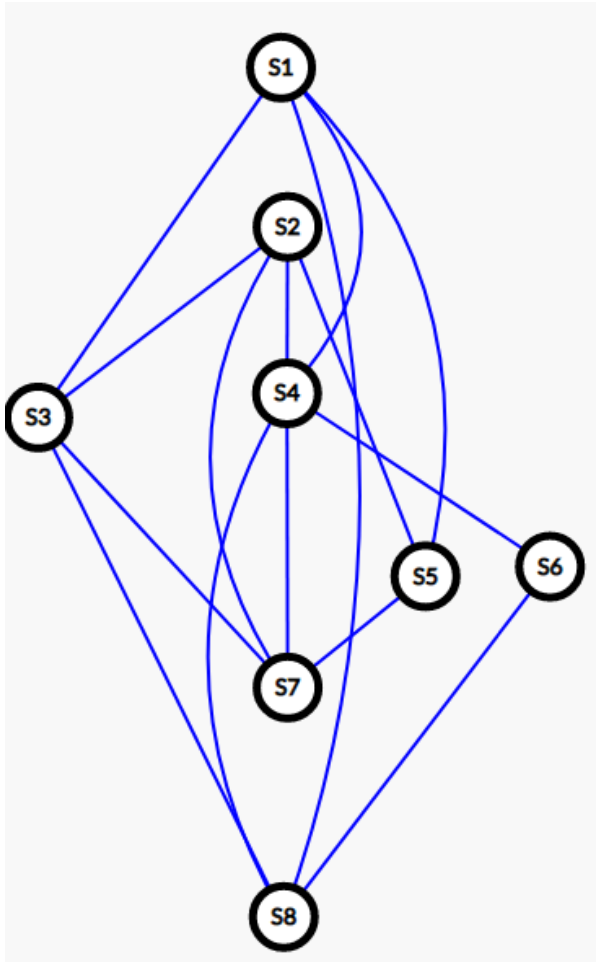
Relaţia Z, conform definiţiei de mai sus va fi dată de mulţimea arcelor:

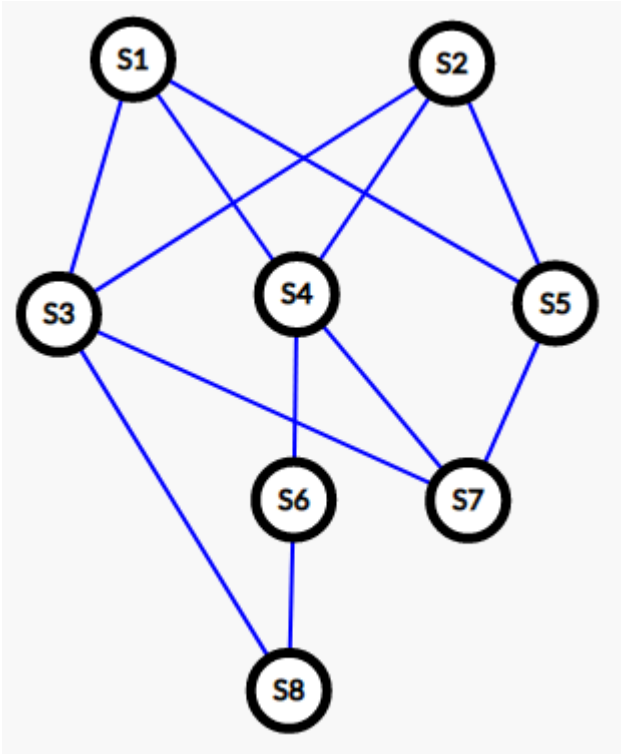
Z : { (S1,S3) (S1,S4) (S1,S5) (S1,S8)
(S2,S3) (S2,S4) (S2,S5) (S2,S7)
(S3,S7) (S3,S8)
(S4,S6) (S4,S7) (S4,S8)
(S5,S7)
(S6,S8)
}

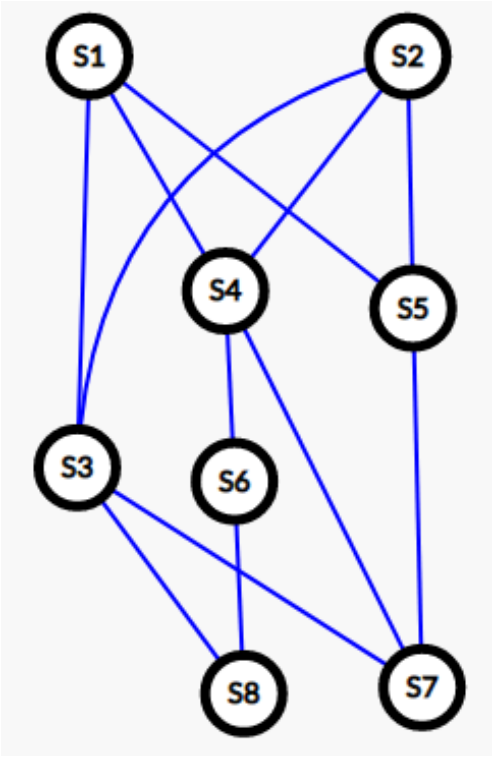
M	Domeniu de definiţie D_S	Domeniu de valori R_S
M1	S1,S2,S7,S8	S3
M2	S1,S7	S5
M3	S3,S4,S8	S1
M4	S3,S4,S5,S7	S2,7
M5	S6	S4,S6,S8

Rezultă graful asociat G', care este organizat pe niveluri eliminând arcele excluse prin tranzitivitate

graful asociat







Sistemul C' echivalent se execută în 4 perioade de timp considerând că avem un număr suficient de procesoare (3 procesoare) sau in 4 perioade avand la dispozitie numai 2 procesoare

Analiza situațiilor de blocare

- Un sistem care este format din sisteme de sarcini combinate paralel

$$\mathbf{C}=\mathbf{C}_1 \mid \mathbf{C}_2 \mid \ldots \mid \mathbf{C}_n$$

este *blocat* dacă evoluţia sa este suspendată indefinit din cauză că mai multe sisteme de sarcini $\mathbf{C}_i$ sunt active simultan, fiecare utilizând, fără a putea fi întrerupte, resurse necesare altor sisteme din **C**.

- O resursă deţinută de o sarcina neinteruptibila poate fi eliberată numai de sistemul de sarcini care o utilizează la un moment dat.

- Pentru studiul situaţiilor de blocare vom privi sistemul fizic ca o colecţie de resurse R de tip:

  $R = R_1\ R_2 ... R_m$, fiecare tip conţinând:

  $W = W_1\ W_2 ... W_m$ exemplare

  Vectorul $W=(W_1,W_2,...,W_m)$ reprezintă capacitatea sistemului.

Exemplu de resurse R_i:

- unităţi centrale de prelucrare;
- coprocesoare matematice;
- pagini de memorie;
- magistrale de interconectare;
- înregistrări fizice (sectoare, piste, cilindri) la discuri magnetice;
- fişiere de date;
- zone de mesaje;
- tabele de descriere a diferitelor sarcini;
- variabile globale, etc.

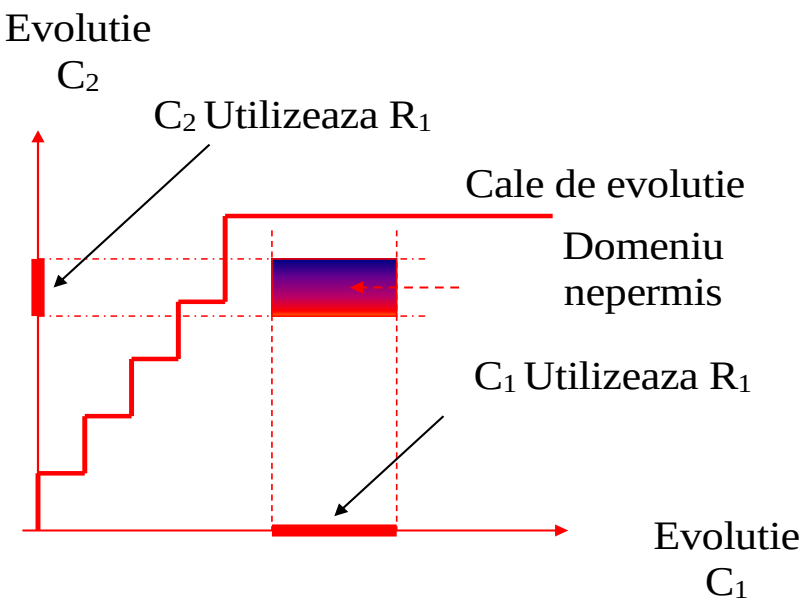
- Să considerăm un exemplu simplu de blocare:

  Fie două sisteme $\mathbf{C}_1$ şi $\mathbf{C}_2$ şi câte o resursă de tip R_1 şi R_2.

Evoluţia întregului sistem poate fi reprezentată în spaţiul cu două dimensiuni definit spaţiul de evoluţie.

Evoluţia sistemului $\mathbf{C}=\mathbf{C}_1 \mid \mathbf{C}_2$ este reprezentată de succesiunea de puncte $(x_1y_1)\ (x_2y_2)...(x_ky_k)...$ în acest spaţiu

Presupunem că originea este în punctul (0,0).
Evoluţia este ireversibilă deci $x_k \leq x_{k+1}$, $y_k \leq y_{k+1}$ şi
Nu orice cale de evoluţie, este realizabilă.
Evoluţia în care $\mathbf{C}_1$ şi $\mathbf{C}_2$ asftel sa utilizeaze simultan aceeaşi resursă R_1 este nepermisă
Zona hasurată reprezintă un domeniu nepermis, care conduce la blocarea sistemului.



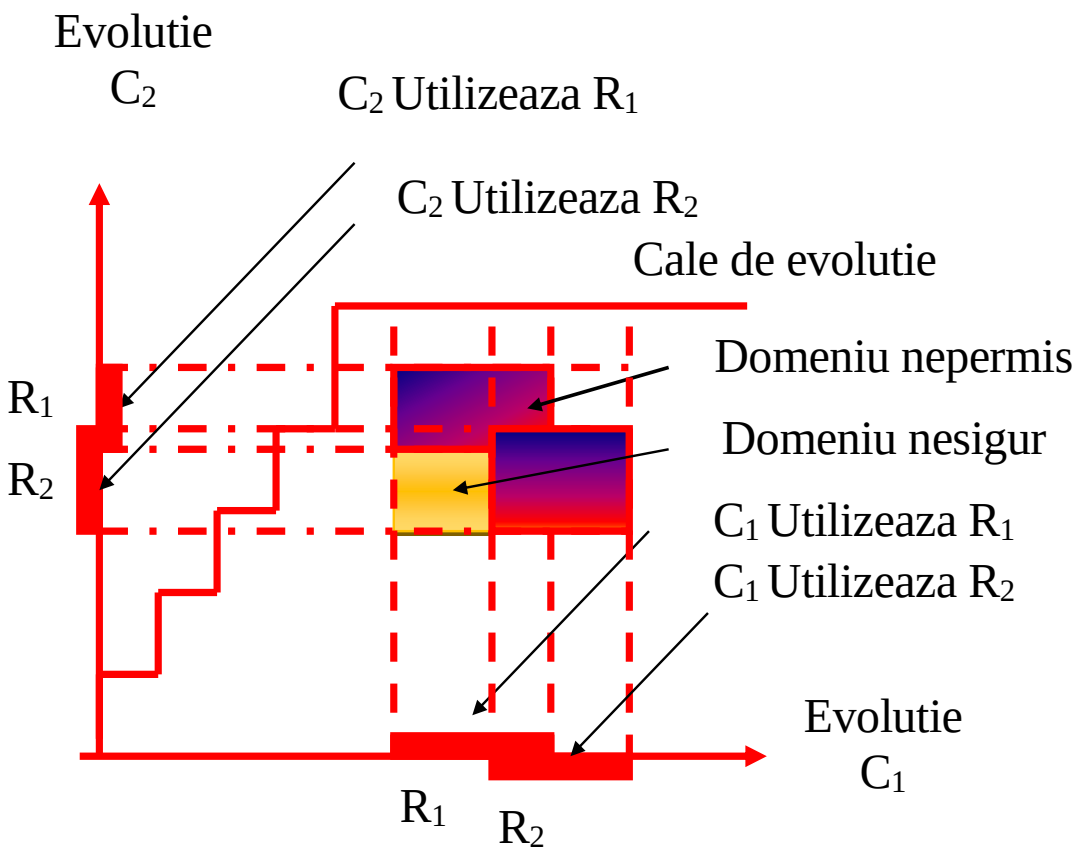
Exemplu de utilizare a doua resurse

Fie utilizarea resurselor R1 şi R2 de către C1 şi C2.

Zona haşurată reprezintă un domeniu nepermis, care conduce la blocarea sistemului.

Zona portocalie reprezintă un domeniu nesigur, care permite în continuare evoluţia sistemului, însă pentru un timp limitat.

Datorită ireversibilităţii procesului de evoluţie se va ajunge în mod sigur la blocarea sistemului.



- Blocarea este definită ca o stare de *aşteptare* circulară nerezolvabilă din cauza condiţiilor de excludere mutuală şi neintreruptibilitate.
- Pentru a exista o cale de evoluţie care conduce la blocare sunt necesare următoarele condiţii:

- **Utilizare exclusivă**: Fiecare sistem de sarcini doreşte
- controlul exclusiv asupra resursei pe care o utilizează;
- **Neîntreruptibilitate**: O resursă nu este eliberată până la terminarea completă a utilizării acesteia;
- **Aşteptare circulară**: Fiecare sistem de sarcini ţine ocupate resurse în timp ce aşteaptă ca altele să elibereze resurse pe care să le preia.

- Relativ la blocare se pun trei probleme:

1. Prevenirea

Una sau mai multe din condiţiile care conduc la blocare sunt înlăturate printr-o proiectare adecvată sau prin restricţii în utillizarea resurselor.

2. Detectarea şi înlăturarea

Sistemul de operare si planificatorul supraveghează evoluţia tuturor sistemelor de sarcini luând măsuri speciale de corecţie a evolutiei la detectarea situaţiilor de blocare.

3. Evitarea

Pe baza unor informaţii preliminare despre cerinţele de resurse, sistemul de operare / planificatorul dirijează evoluţia pentru evitarea zonelor nesigure.

Considerând că fiecare tip de resursă conţine un singur element $w_1=w_2=...=w_n=1$, putem alege o reprezentare sub forma de grafuri, fiecare nod reprezentând o resursă iar arcele reprezinta cererea de resurse.

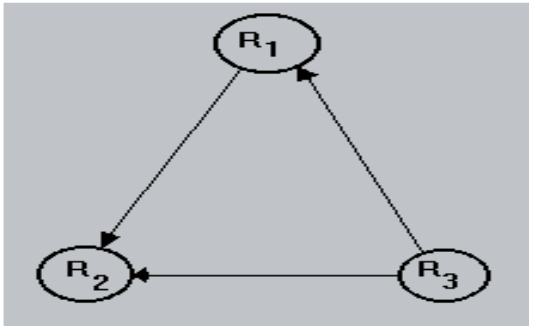
Arcele pot fi etichetate cu necesarul de resurse.

- Arcele sunt orientate şi sunt definite astfel:

  dacă la un moment dat, după evenimentul a_k, sistemul **C** ţine ocupată resursa R_i în timp ce aşteaptă preluarea lui R_j, graful conţine un arc orientat de la nodul R_i la nodul R_j.
- În exemplul urmator numărul de tipuri de resurse este n=3;

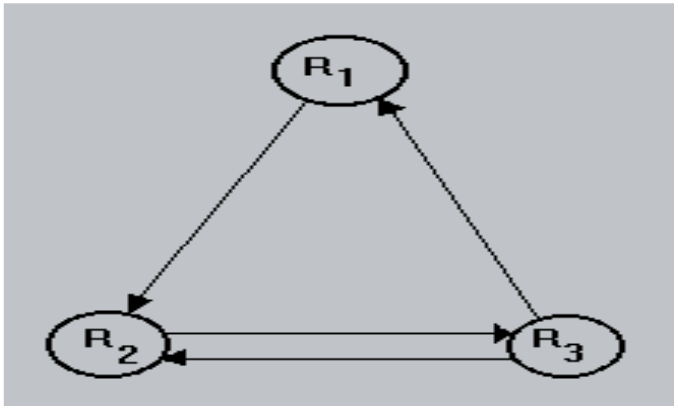
  O sarcina utilizează R_1 şi o cere pe R_2 ,

  Alta sarcina utilizează pe R_3 şi cere pe R_1 şi R_2.



Graf de cereri fara bucle

Dacă cele trei condiţii de blocare sunt posibile (operative) condiţia necesară şi suficientă pentru blocare este existenţa unei bucle (circuit, ciclu) în graful de cereri.
Exemplu , de graf de cereri în care se realizează condiţia de aşteptare circulară cu resurse ocupate.



Graf de cereri cu bucle

- Pentru cazul general în care există mai multe resurse de un anumit tip, fiecare nod reprezintă un tip de resursă.
- Cererile pentru resurse R_i trebuie să nu depăşească, ca număr, w_i.
- Un arc de la R_i la R_j arată că există cel puţin o sarcina care cere una sau mai multe resurse de tip R_j şi deţine una sau mai multe resurse de tip R_i.
- În cazul acesta o buclă în graful de cereri reprezintă o condiţie necesară dar nu şi suficientă pentru existenţa situaţiei de blocare.

Prevenirea blocării

- Pentru a proiecta un sistem în care posibilitatea blocării este exclusă trebuie să existe certitudinea că în orice moment de timp cel puţin una din condiţiile necesare nu este îndeplinită.
- Prima din cele trei condiţii (***excluderea mutuală***) este uneori impusă de restricţii fizice privind accesul la resurse nepartajabile şi deci nu poate fi eliminată pentru orice fel de resurse.

Pentru celelalte condiţii vom enumera câteva posibilităţi:

- Dacă o sarcina care utilizează o resursă, cere o altă resursă *trebuie* să o elibereze pe prima şi dacă este necesară în continuare să o ceară din nou împreună cu resursele adiţionale de care are nevoie (***interuptibilitate***).
- Fiecare sarcina să ceară toate resursele de care are nevoie, putând continua numai după ce le are pe toate (***aşteptare cu resurse***).

- ***Ordonarea liniară*** a tipurilor de resurse pentru toate sarcinile. Dacă o sarcina utilizează resurse de tip R_i, la un moment dat, ea poate cere numai resurse din tipurile următoare lui R_i în acea ordonare,

  exemplu: R_j cu j>i. În acest caz, graful de cereri nu are circuite.

*Observaţie:

- **Prima soluţie** se pretează numai pentru resurse interuptibile, a căror stare poate fi uşor salvată şi restaurată. De exemplu UCP.
- **A doua soluţie** poate deveni costisitoare dacă unele resurse sunt alocate unor sarcini şi sunt neutilizate o perioadă lungă de timp.
- **A treia soluţie** poate fi utilizată numai pentru anumite componente, de exemplu lansarea job-urilor într-un sistem cu multiprogramare prin alocarea memoriei şi a dispozitivelor de I/E într-o ordine fixă.

Detectarea şi înlăturarea blocării

- Pentru cazul în care există o singură resursă de fiecare tip, mecanismul de detectare a blocării constă:
  - dintr-o *procedură* care actualizează graful de cereri ori de câte ori apare o cerere, o preluare sau eliberare de resurse şi
  - o *procedură* de detectare a circuitelor (buclelor) în acest graf.
- Pentru cazul general, la fiecare moment de timp t, corespunzător secvenţei de execuţie $\alpha = a_1 a_2 ... a_k$, se definesc
- $x_{ij}[k]$ reprezintă numărul de resurse de tip R_j alocate, respectiv şi
- $y_{ij}[k]$ cerute de $\mathbf{C}_i$, după evenimentul a_k din α, unde:

  $\mathbf{C}=\mathbf{C}_1 \mid \mathbf{C}_2 \mid ... \mid \mathbf{C}_n$, $1 \le i \le n$ iar $1 \le j \le m$,

  m fiind numărul de tipuri de resurse

Se obţin astfel:

$$X[k] = \begin{bmatrix} x_1(k) \\ x_2(k) \\ . \\ . \\ x_n(k) \end{bmatrix} \text{ matrice de alocare}$$

cu

$$Y[k] = \begin{bmatrix} y_1(k) \\ y_2(k) \\ . \\ . \\ y_n(k) \end{bmatrix} \text{ matrice de cereri}$$

$X_i[k] = (x_{i1}[k], x_{i2}[k],..., x_{im}[k])$

$Y_i[k] = (y_{i1}[k], y_{i2}[k],..., y_{im}[k]),$ $1 \leq i \leq n$

- Starea s_k corespunzătoare secvenţei α după evenimentul a_k este dată de perechea [X[k], Y[k]].

Să notăm cu:

$V[k]=(v_1[k],v_2[k],...,v_m[k])$ vectorul resurselor disponibile.

Elementul $v_j[k] \le w_j$, $1 \le j \le m$, numărul de resurse de tip R_j disponibile la un moment dat după evenimentul a_k este:

$$v_j(k) = w_j - \sum_{i=1}^{n} x_{ij}[k], \quad 1 \le j \le m$$

- Pentru doi vectori X şi Y, relaţia $X \le Y$ este adevarată, dacă şi numai dacă relaţia este adevărată pentru fiecare pereche de elemente corespondente din X şi Y.
- Utilizând informaţia conţinută în starea s_k dată de perechea de matrice [X[k], Y[k]] vom elabora un algoritm de detectare a blocării.

Definiţie:

Fie $\mathbf{C} = \mathbf{C}_1 \mid \mathbf{C}_2 \mid ... \mid \mathbf{C}_n$ unde $\mathbf{C}_i$, $1 \le i \le n$, este de forma

$$\mathbf{C}_i = S_i[1]\ S_i[2].....S_i[l_i], \qquad l_i \ge 1.$$

Fie

$\alpha = a_1 a_2 ... a_k$ o secvenţă de execuţie parţială a lui **C** iar

$\sigma = s_0\ s_1 ... s_k$ secvenţa parţială de stări, corespunzătoare.

Dacă există o mulţime nevidă D de indici ai lui **C** astfel că pentru orice i din D nu este adevărată relaţia:

$Y_i[k] \le$	$V[k]$	$+ \sum_{j \notin D} X_j[k]$
vector de cereri	*vector de resurse disponibile*	*vector de resurse allocate catre (deţinute de) sarcini ce nu aparţin lui D*

atunci σ este blocată (sau s_k este o stare de blocare).

De asemenea putem spune că orice sistem de sarcini $\mathbf{C}_i$ cu i ε D este blocat.

Exemplu:

Fie **C_1** şi **C_2** două sisteme de sarcini:

$C_1 = S_1[1], S_1[2], S_1[3]$ şi $C_2 = S_2[1], S_2[2]$

care utilizează resursele $R=(R_1,R_2,R_3,R_4)$ cu $w = (3,3,3,3)$, în felul următor :

	C 1			C 2	
	resurse cerute	resurse eliberate		resurse cerute	resurse eliberate
$S_1[1]$	(0,1,2,0)	(0,0,2,0)	$S_2[1]$	(2,0,2,2)	(0,0,0,2)
$S_1[2]$	(1,0,0,1)	(0,0,0,1)	$S_2[2]$	(1,0,1,0)	(3,0,3,0)
$S_1[3]$	(0,0,2,0)	(1,1,2,0)			

Secvenţa de execuţie $\alpha = SI_1[1]$ $SF_1[1]$ $SI_2[1]$ $SF_2[1]$ $SI_1[2]$ $SF_1[2]$ $SI_2[2]$ .....

	X	Initial	$SI_1[1]$	$SF_1[1]$	$SI_2[1]$	$SF_2[1]$	$SI_1[2]$	$SF_1[2]$	$SI_2[2]$
		R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	
Alocare	X_1	0 0 0 0	0 1 2 0	0 1 0 0	0 1 0 0	0 1 0 0	1 1 0 1	1 1 0 0	Blocare
X[k]	X_2	0 0 0 0	0 0 0 0	0 0 0 0	2 0 2 2	2 0 2 2	2 0 2 0	2 0 2 0	
Cereri	Y_1	0 1 2 0	0 0 0 0	1 0 0 1	1 0 0 1	1 0 0 1	0 0 0 0	0 0 2 0	
Y[k]	Y_2	2 0 2 2	2 0 2 2	2 0 2 2	0 0 0 0	1 0 1 0	1 0 1 0	1 0 1 0	
V[k]		3 3 3 3	3 2 1 3	3 2 3 3	1 2 1 1	1 2 1 3	0 2 1 2	0 2 1 3	

După evenimentul S1[2], atât Y1[k] ≤ V[k] cât şi Y2[k] ≤ V[k] astfel că P1[3] şi P2[2] nu pot avea loc cu valorile obţinute pentru V[k] înaintea acestor evenimente.

Sistemul s-a blocat deoarece nici C1 nici C2 nu pot continua fără ca unul din ele să fie întrerupt pentru a elibera resursele deţinute.

Algoritm pentru detectarea blocării

Etapele algoritmului sunt:

1°. Iniţializare : $D \leftarrow \{1,2,...,n\}$; $V \leftarrow V(k)$
2°. Caută indicii i din D pentru care $Y_i(k) \leq V$ şi execută pasul 3.
Dacă nu există nici un indice, algoritmul se termină.
3°. $D \leftarrow D \setminus \{i\}$; $V \leftarrow V + X_i(k)$ şi salt la pasul 2.

- Algoritmul se bazează direct pe definiţia de mai sus şi constă din simularea execuţiei sarcinilor până când rămâne un set de sisteme de sarcini $\mathbf{C}_i$ pentru care, în final, resursele disponibile nu sunt suficiente sau mulţimea de indici D devine vidă.
- În cel mai defavorabil caz algoritmul dat analizează la pasul i, întreaga mulţime de sisteme de sarcini din D, în număr de (n-i+1).

Timpul de execuţie al algoritmului este în acest caz proporţional cu

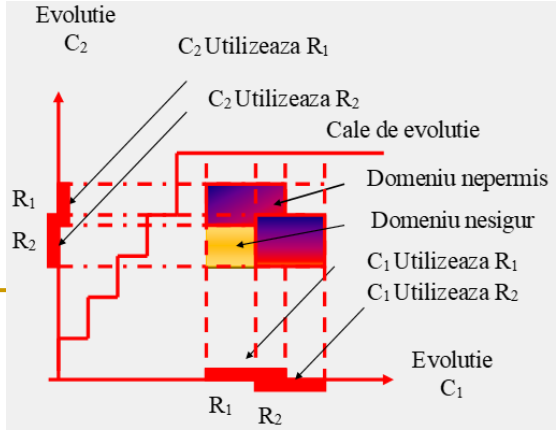
$$m \cdot \sum_{i=1}^{n} (n-i+1) = O(m.n^2)$$

- Printr-o aranjare a structurilor de date şi o reprezentare mai elaborată a stărilor resurselor se poate obţine un algoritm cu timpul de execuţie proporţional cu O(m.n).
- Complexitatea acestui algoritm face ca implementarea lui să fie justificată numai în sisteme în care apariţia unei blocări este intolerabilă.
- În multe sisteme se utilizează o soluţie simplă care se bazează pe limitarea timpului în care sistemul poate sta într-o anumită stare.
- Dacă s-a ajuns la această limită se consideră că s-a realizat o situaţie de blocare şi sistemul este scos din blocare şi trecut într-o anumită stare, predeterminată din care continuă evoluţia.
- Implementarea metodelor de detectare a situaţiilor de "time out" se face atât la nivelul maşinii de bază cât şi la nivelul sistemului de operare.

Evitarea blocării

- Pentru a analiza problema evitării blocării se presupune că o sarcină S constă dintr-o succesiune de paşi.
- Pe parcursul execuţiei unui pas utilizarea resurselor rămâne constantă şi aceasta implică achiziţionarea resurselor necesare pentru acest pas în plus faţă de cele primite la pasul precedent.
- Urmează apoi o perioadă de execuţie în care cerinţele de resurse nu se schimbă.
- În final când se termină execuţia, toate resursele care nu sunt cerute la pasul următor sunt eliberate şi trecute într-o colecţie de resurse disponibile.

- Fie $\sigma = s_0 s_1 ... s_k$ secvenţa de stări corespunzătoare secvenţei de execuţie $\alpha = a_1 ... a_k$.
- Dacă există cel puţin o secvenţa de execuţie completă, validă, care are ca prefix pe α atunci s_k este o stare sigură, altfel starea s_k este o stare nesigură cu pericol de blocare în evoluţia următoare.
- Cu alte cuvinte s_k este *sigură* dacă utilizând resursele disponibile V[k] şi cele care vor fi eliberate de paşii care se execută la momentul respectiv, este posibil să găsim o secvenţă validă de paşi încă neiniţiaţi, pentru sarcinile iniţiate dar neterminate astfel încât toate sarcinile din sistem să poată continua execuţia spre finalizare.
- Starea iniţială în care încă nu s-au alocat resurse, toate fiind disponibile, este intotdeauna o stare sigură.
- Deci există posibilitatea finalizării tuturor sarcinilor.



- Să considerăm în continuare problemele pe care trebuie să le rezolve un supervizor pentru evitarea blocării.
- Să presupunem că sistemul se găseşte în starea s_k sigură şi că există cererile de resurse date de Q[k].
- Supervizorul trebuie să determine dacă orice cerere din Q[k] poate fi satisfacută, $y_{ij}[k] \leq v_j[k]$.
- În general pot fi mai multe astfel de cereri şi deci se pot prevedea diferite *criterii de alocare* pentru a stabili care cerere să fie satisfacută mai întîi, de exemplu:

  FIFO - (First In First Out)-primul venit primul servit

  SJF - (Shortest Job First)- sarcina cea mai scurtă,prima servită;

  ROUND ROBIN - rulare prin rotaţie;

  Priorităţi atribuite sarcinilor;

  etc.

- Pentru a determina dacă o stare s_k este sigură ca urmare a satisfacerii unor cereri de resurse trebuie căutată o secvenţa adecvată de paşi.
- Pentru aceasta se presupune că resursele disponibile sunt completate cu cele alocate sarcinilor în curs de execuţie.
- Aceasta este echivalentă cu presupunerea că primul pas într-o secvenţă analizată nu începe până când toţi paşii în curs de execuţie au toate condiţiile pentru a se finaliza.

Excluderea mutuală
Definiții. Concepte de bază.

- Într-un sistem de sarcini concurente, acestea pot interacţiona în două moduri:
  - indirect, prin disputa pentru aceleaşi resurse;
  - direct, prin transmitere / recepţie de mesaje sau informaţii de stare.

- Când se execută sarcini independente evoluţia acestora poate fi controlată de către supervizor aşa cum am văzut la analiza situaţiilor de blocare.

- Totusi când sarcinile interactionează în mod direct mecanismul de control al evoluţiei lor trebuie să apară în mod explicit în programele care le definesc.

- Interacţiunea directă se efectuează prin mecanisme specifice contextului în care evoluează sistemul de sarcini concurente.

Contextul în care evoluează un sistem de sarcini concurente poate fi:

- centralizat, dacă există o memorie comună accesibilă pentru citire/scriere tuturor sarcinilor, sau
- distribuit dacă nu există memorie comună, ci doar memorie locală accesibilă unei sarcini şi inaccesibilă celorlalte sarcini.

- Excluderea mutuală se ocupă de problemele legate de controlul resurselor reutilizabile (acele resurse a căror stare internă este modificată în timpul utilizării dar care poate fi initializată la atribuirea lor altor sarcini) astfel ca la un moment dat să fie utilizate de o singură sarcină.

- Deoarece sarcinile modifică starea resursei în timpul utilizării, este necesar ca o resursă să fie alocată unei singure sarcini la un moment dat pentru a asigura o execuţie corectă a sarcinilor.
- Pe de altă parte, deoarece o sarcină iniţializează starea unei resurse înainte de a o utiliza, ordinea în care mai multe sarcini o utilizează este indiferentă.
- Astfel orice soluţie pentru excluderea mutuală care implică a priori constrîngeri privind ordinea de execuţie a sarcinilor este neadecvată.
- O soluţie corectă trebuie să se aplice unui sistem de sarcini independente.
- Dacă utilizarea resurselor este sub controlul unui sistem de operare centralizat, la construirea unui sistem de sarcini trebuie să se ţină seama numai de convenţiile prin care se comunică sistemului de operare cererile şi eliberările de resurse.

- Vom considera că excluderea mutuală trebuie rezolvată de însuşi sistemul de sarcini, utilizând *variabile globale* sau variabile de stare şi mesaje pentru comunicaţia între sarcini în funcţie de context: centralizat sau distribuit.

- Sarcinile care trebuie să se execute mutual exclusiv constituie secţiuni critice.

- O soluţie corectă pentru excluderea mutuală trebuie să elimine apariţia unor rezultate diferite sau blocari în tranziţiile sistemului în condiţiile unor timpi de execuţie a sarcinilor diferiţi, pentru diferite procesoare.
- De exemplu: Să considerăm posibilitatea ca două sarcini S_1 şi S_2 care se execută pe procesoare diferite să utilizeze în comun o resursă R.

- S_1 şi S_2 utilizează o variabilă globală care specifică disponibilitatea resursei R.
- S_1 testează variabila, găseşte R disponibilă şi înainte ca S_1 să schimbe valoarea variabilei pentru a arăta că R este ocupată, S_2 testează şi ea variabilă şi găseşte R disponibilă.
- Mai concret S_1 şi S_2 care evoluează într-un context centralizat au primit aceeaşi interpretare, citirea variabilei Semafor, incrementarea valorii citite şi scrierea noii valori.

Fie, de exemplu, secvenţa $SI_1\ SI_2\ SF_1\ SF_2$, deci

1°. S_1 citeşte variabila Semafor;

2°. S_2 citeşte variabila Semafor;

3°. S_1 incrementează şi scrie noua valoare;

4°. S_2 incrementează şi scrie noua valoare.

În urma acestei secvenţe de execuţie, Semafor primeşte valoarea Semafor + 1 în loc de Semafor + 2.

- Rezolvarea cererilor de acces la memorie pentru UCP şi DMA reprezintă un exemplu de excludere mutuală la nivel hardware.
- Scrierea unor texte la o imprimantă comună într-o reţea locală de calculatoare constituie un exemplu de excludere mutuală, la nivel de aplicaţii.
- Accesul la o înregistrare dintr-un fişier pentru a fi citită şi modificată de mai multe sarcini constituie deasemenea un exemplu de excludere mutuala în context distribuit.

- Pentru simplitate vom considera în continuare o singură resursă R care poate fi utilizată de o singură sarcină la un moment dat.

Definiţie:
Fie $\alpha = a_1,a_2,...,a_k$ o secvenţă de execuţie parţială a sistemului de sarcini **C**=(***S***,<). Sarcinile S şi S' sunt mutual exclusive în α dacă şi numai dacă cel mult una din ele este activă după orice prefix a lui α.

Pentru a implementa excluderea mutuală vom considera că o soluţie este corectă dacă cel mult o sarcină este în secţiunea critică la un moment dat şi sunt satisfacute următoarele condiţii adiţionale:

- Nu se face nici o presupunere relativă la activitatea şi numărul de procesoare pe care se execută sistemul de sarcini.

  Desigur, aşa cum s-a arătat şi mai sus, se consideră ca citirea şi scrierea unei variabile de memorie sunt operaţii atomice, indivizibile. Lansarea simultană a unor astfel de operaţii va avea ca efect execuţia lor secvenţială, într-o ordine oarecare, necunoscută apriori.

- Soluţia implementată nu trebuie să fie afectată de vitezele relative ale sarcinilor care se exclud mutual, considerând că aceste viteze sunt diferite de zero.
- O sarcină care operează în afara secţiunii critice nu poate bloca altă sarcină de la intrarea în secţiunea critică.
- Două sarcini care doresc să intre în secţiunea critică nu pot fi într-o buclă de aşteptare reciprocă.

  Deci dacă se eliberează secţiunea critică şi există sarcini în aşteptare, este exclusă posibilitatea ca nici un proces să nu intre în secţiunea critică. În plus, o sarcină care doreşte să intre în secţiunea critică nu poate fi în stare de aşteptare un timp nedefinit.

- Toate sarcinile trebuie tratate la fel, fără priorităţi sau alte privilegii.

Algoritmi de excludere mutuală în mediu centralizat

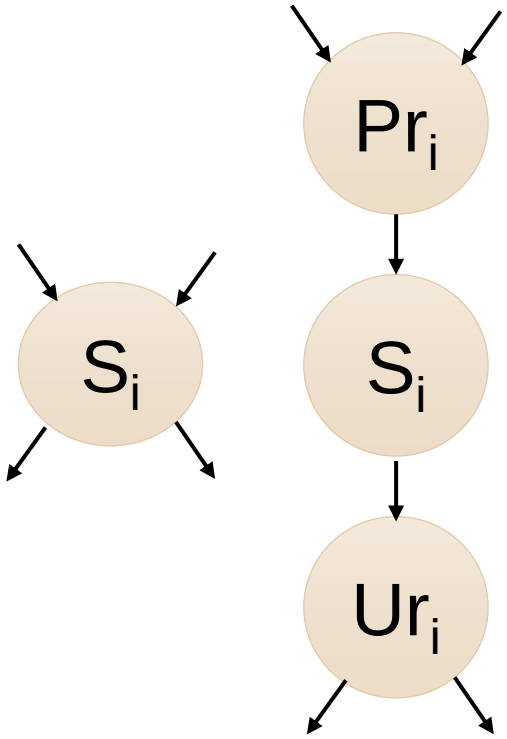
- Complexitatea soluţiei pentru excluderea mutuală depinde de tipurile de operaţii indivizibile care se pot efectua asupra variabilelor globale.
- Astfel dacă se lasă pe seama proiectantului hardware responsabilitatea implementării excluderii mutuale prin operaţii indivizibile mai elaborate, se obţine o soluţie mai simplă pentru excluderea mutuală decât dacă se consideră ca operaţii indivizibile simple citiri şi scrieri în memorie.
- Astfel, sunt calculatoare care au instrucţiuni de tipul Test and Set (Read Modify Write), sau chiar primitive de sincronizare indivizibile, implementate ca operaţii atomice.

- Dacă cel puţin două sarcini independente ale unui sistem **C** trebuie să utilizeze mutual exclusiv resursa R vom modifica **C** pentru a asigura această cerinţă.
- Pentru fiecare S_i care utilizează resursa R vom introduce sarcinile Pr_i şi Ur_i astfel că:

  Pr_i precede imediat sarcina S_i, iar

  Ur_i urmează imediat sarcinii S_i.
- Vom nota cu **C'** sistemul de sarcini extins. Cu toate că S_i rămâne neinterpretat Pr_i şi Ur_i vor fi complet specificate.
- Pr_i va fi astfel proiectat încât să poată fi terminat şi deci să permită iniţierea lui S_i dacă şi numai dacă pentru $\forall\ j \neq i$, Pr_j nu poate fi terminat.
- Funcţia principală a lui Ur_i este de a comunica celorlalte sarcini că S_i a terminat utilizarea lui R.

```
do{
  Entry Section
  Critical Section
  Exit Section
  } while (TRUE);
```



- Pentru a proiecta pe Pr_i şi Ur_i astfel ca să nu existe subsecvenţe de execuţie de forma:

$PrF_i \; PrF_j \, UrF_i$

vom considera ca variabile globale:

p - o variabilă de tip întreg.

Q - (Q[0],Q[1],...,Q[n]) - un vector de n+1 variabile de tip întreg.

Iniţial p=0 şi Q[i]=0, $0 \le i \le n$.

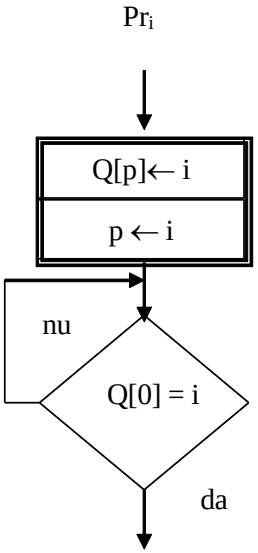
- Funcţia vectorului Q este de a memora într-o ordine FIFO lista sarcinilor ce aşteaptă utilizarea lui R. Acest lucru se realizează prin memorarea indicelui sarcinei care utilizează pe R în Q.
- Astfel, coada de aşteptare la resursa R este organizată ca o simplă listă înlănţuită, memorată în Q.
- *Variabila p* conţine indicele ultimului element din lista (ultimul element intrat în coada Q).
- Condiţia de coadă vidă, considerată ca şi condiţie iniţială este dată de p= 0 şi Q[0] = 0.

Definirea sarcinilor auxiliare Pr si Ur

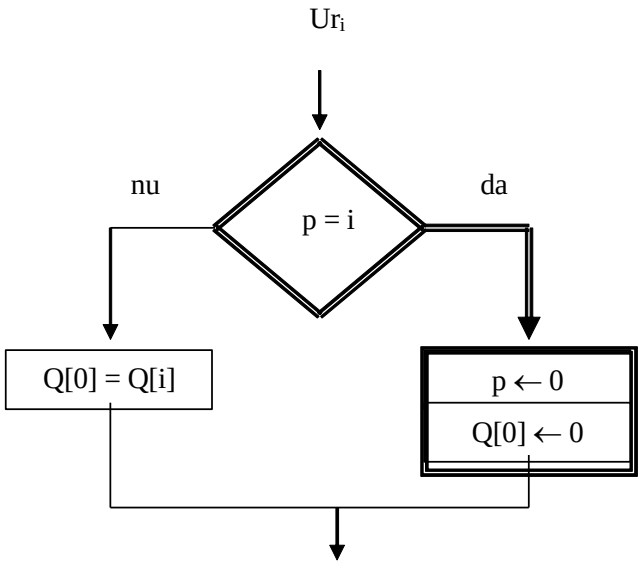
Procesele auxiliare Pr_i şi Ur_i pot fi specificate prin organigramele urmatoare

Sarcina Pr_i arată intenţia lui S_i de a utiliza resursa R, introduce pe S_i în coada Q şi aşteaptă într-o buclă până când S_i ajunge în capul listei Q[0].

Sarcina Ur_i înlătură pe S_i din coadă şi trece următoarea sarcină pentru execuţie în capul listei.
Dacă S_i a fost ultima sarcină introdusă în coadă
(p=i când se execută Ur_i)
atunci coada este vidă şi se trec în zero p şi Q[0].



a) Sarcina Pr_i



b) Sarcina Ur_i

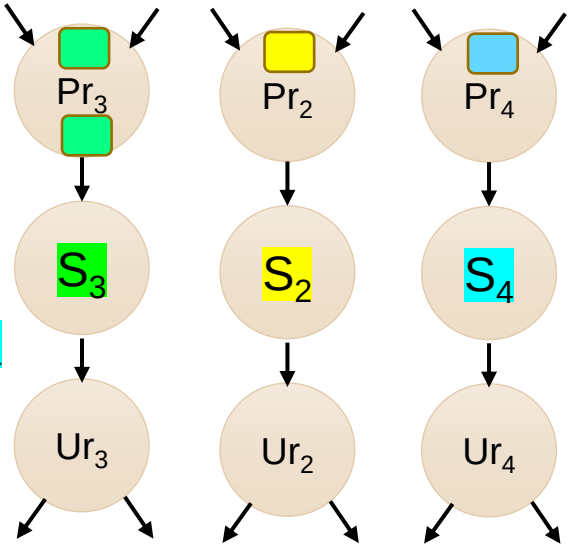
Pentru a putea elimina posibilitatea interferenţelor între Pr_i,Pr_j sau Pr_i,Ur_j ce se execută concurent trebuie ca anumite operaţii asupra variabilelor globale să **fie indivizibile**, cele în chenar dublu.

Fie un sistem de sarcini C=(S,<)

$S=\{S_1,S_2,S_3,S_4,S_5, \ldots S_n\}$

- Se arată modul de reprezentare în coada Q considerând solicitările de a intra în secţiunea critică în ordinea $S_3S_2S_4$.

α= .. Pr_3 Pr_2 Pr_4 PrF_3 SI_3 SF_3 Ur_3 PrF_2 SI_2 SF_2 Ur_2 PrF_4 SI_4 SF_4 Ur_4 ..



p	0				
Q[0]	0				
Q[1]					
Q[2]					
Q[3]					
Q[4]					

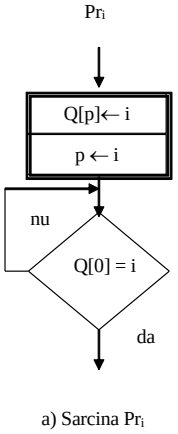
p	0	3			
Q[0]	0	3			
Q[1]					
Q[2]					
Q[3]					
Q[4]					

p	0	3	2		
Q[0]	0	3			
Q[1]					
Q[2]					
Q[3]	2				
Q[4]					

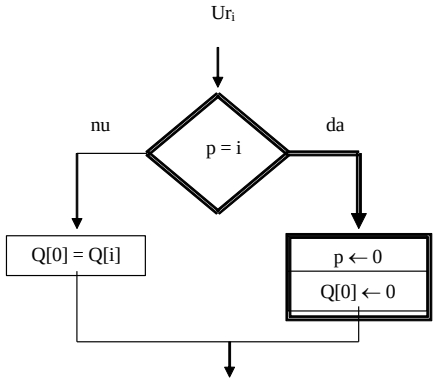
p	0	3	2	4	
Q[0]	0	3			
Q[1]					
Q[2]	4				
Q[3]	2				
Q[4]					

p	0	3	2	4	
Q[0]	0	3 S_3	2		
Q[1]					
Q[2]	4				
Q[3]	2				
Q[4]					

p	0	3	2	4	0
Q[0]	0	3	2 S_2	4	0
Q[1]					
Q[2]	4				
Q[3]	2				
Q[4]					



a) Sarcina Pr_i



b) Sarcina Ur_i

Initial p = 0 şi Q[0] = 0.

Exemplu de interferenţă care conduce la o funcţionare incorectă:

Să presupunem că p=Q[0]=0 şi că Pr_j începe execuţia.

- **Situația 1.** După ce Pr_j se execută Q[0]←j şi înainte de a executa p←j, o sarcină Pr_k cu viteza mai mare în execuţie (pe un procesor mai rapid) execută ambele operaţii **Q[p] ←k ; p ← k**

S_k se va executa imediat, dar cererea pentru Sj s-a pierdut deoarece Q[0] a fost rescris de Pr_k.

Considerând *cele două operaţii indivizibile* **Q[p] ←i ; p ← i**

se elimină astfel de interferenţe.

- **Situația 2.** Să presupunem că p a fost testat de Ur_i şi a fost găsit egal cu i (p=i).

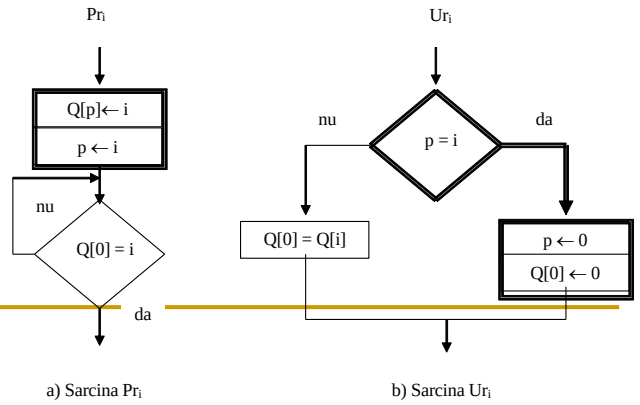
Inainte ca Ur_i să şteargă Q[0] şi p **Q[0] ←0 ; p ← 0** se execută sarcina Pr_k.

Înainte de a se testa egalitatea Q[0]=k în cadrul sarcinii Pr_k, Ur_i se execută mai departe trecând pe Q[0] şi p în zero.

Este evient că Pr_k nu se va executa niciodată,

deci operaţiile implicate de Ur_i

trebuie să fie indivizibile.



- În procedura de realizare a excluderii mutuale arătate anterior **primitiva de aşteptare** Pr_i ocupă procesorul, ceea ce duce la utilizarea ineficientă a acestuia în sisteme în care un număr mare de sarcini intră în competiţie pentru resursa R.
- În acelaşi timp apare o puternică interferenţă la memorie prin accesul la variabilele globale; operaţiile indivizibile fiind destul de lungi.
- Soluţia de excludere mutuală prezentată are marele dezavantaj că necesită execuţia unor operaţii indivizibile complicate, ceea ce de cele mai multe ori nu este posibil prin arhitectura procesoarelor sau chiar dacă este posibil, duce la scăderea performanţelor sistemului.

În continuare vom prezenta o suită de algoritmi, în ordinea dezvoltării lor, care presupun că singurele operaţii indivizibile sunt citirea şi scrierea unei locaţii de memorie.

- Dacă există mai multe cereri de citire sau scriere, ele se vor executa serial, într-o ordine oarecare, nespecificată.
- Prezentarea acestor algoritmi se va face, pe cât posibil, în forma în care au fost publicaţi, desprinderea sarcinilor Pr_i şi Ur_i fiind banală.

Algoritmi de excludere mutuală în mediu centralizat

- **Algoritmul lui Dekker**
- Algoritmul lui Dijkstra
- Algoritmul incorect al lui Hyman
- Algoritmul lui Knuth
- Algoritmul lui De Brujin
- Algoritmul lui Eisenberg şi Mac Quire
- **Algoritmul lui Peterson**
- Algoritmul lui Burns

Algoritmul lui Dekker

- Algoritmul lui Dekker se aplică pentru 2 sarcini S_0 şi S_1 care partajază variabilele:

```
wants_to_enter : array of 2 booleans
turn : integer
wants_to_enter[0] ← false
wants_to_enter[1] ← false
turn ← 0 // or 1
```

Semnificaţia acestor variabile este următoarea:

`wants_to_enter[] ← false` S_i nu doreşte să intre în secţiunea critică.

`wants_to_enter[] ← true` S_i doreşte să intre în secţiunea critică.

`turn = i;` Este rândul lui S_i să intre în secţiunea critică.

- Prin variabilele `wants_to_enter` sarcinile îşi dispută accesul la secţiunea critică.
- Dacă ele doresc simultan să intre în secţiunea critică, vor utiliza variabila comună turn pentru a arbitra această situaţie de simultaneitate.

```
variables
wants_to_enter : array of 2 booleans
turn : integer
wants_to_enter[0] ← false
wants_to_enter[1] ← false
turn ← 0 // or 1
```

S0:

```
wants_to_enter[0] ← true
while wants_to_enter[1] {
  if turn ≠ 0 {
     wants_to_enter[0] ← false
     while turn ≠ 0 {
       // busy wait
     }
     wants_to_enter[0] ← true
   }
}

// critical section
...
turn ← 1
wants_to_enter[0] ← false
// remainder section
```

S1:

```
wants_to_enter[1] ← true
while wants_to_enter[0] {
  if turn ≠ 1 {
     wants_to_enter[1] ← false
     while turn ≠ 1 {
       // busy wait
     }
     wants_to_enter[1] ← true
   }
}

// critical section
...
turn ← 0
wants_to_enter[1] ← false
// remainder section
```

https://en.wikipedia.org/wiki/Dekker%27s_algorithm

Comentarii

- Sarcinile indică intenția de a intra în secțiunea critică care este testată de bucla exterioară while.
- Dacă celălalt proces nu a marcat intenția, secțiunea critică poate fi introdusă în siguranță, indiferent de tura curentă.
- Excluderea reciprocă va fi în continuare garantată, deoarece niciuna dintre sarcini nu poate intra in zona critică înainte de a-și seta indicatorul (ceea ce implică cel puțin o sarcina va intra în bucla while).
- Alternativ, dacă variabila celuilalt proces a fost setată, bucla while este activă și variabila turn va stabili cine are voie să devină critic.
- Sarcinile fără prioritate își vor retrage intenția de a intra în secțiunea critică până când li se va acorda din nou prioritate (bucla interioară while).
- Sarcinile cu prioritate vor intra în secțiunea lor critică.

```
S0:
wants_to_enter[0] ← true
while wants_to_enter[1] {
  if turn ≠ 0 {
     wants_to_enter[0] ← false
     while turn ≠ 0 {
       // busy wait
     }
     wants_to_enter[0] ← true
   }
}

// critical section
...
turn ← 1
wants_to_enter[0] ← false
// remainder section
```

Comentariu

- Un avantaj al acestui algoritm este că nu necesită instrucțiuni speciale de testare și setare (citire / modificare / scriere atomică) și, prin urmare, este extrem de portabil pe arhitecturi de mașini.
- Un dezavantaj este că este limitat la două sarcini și folosește așteptarea ocupată în loc de suspendarea procesului. (Utilizarea așteptării ocupate sugerează că sarcinile ar trebui să petreacă o cantitate minimă de timp în secțiunea critică.)
- Sistemele de operare moderne oferă primitive de excludere reciprocă, care sunt mai generale și mai flexibile decât algoritmul lui Dekker.
- Cu toate acestea, în absența unei dispute reale între cele două sarcini, intrarea și ieșirea din secțiunea critică sunt extrem de eficiente atunci când este utilizat algoritmul lui Dekker.
- Multe procesoare moderne își execută instrucțiunile ”out of order” Acest algoritm nu va funcționa pe mașinile cu procesoare ”out of order”

Algoritmul lui Peterson

- Algoritmul lui Peterson oferă o descriere algoritmică bună a rezolvării problemei secțiunilor critice și ilustrează unele dintre complexitățile implicate în proiectarea software-ului care răspunde cerințelor de excludere reciprocă, progres și așteptare limitată.
- Intr-un articol cu titlu destul de sugestiv pentru ce o să urmeze, "Miths about the Mutual Exclusion Problem" Peterson a elaborat un algoritm extrem de simplu şi elegant.
- Sunt utilizate doua variabile globale,

  flag - care arată poziţia fiecărui sarcini faţă de secţiunea critică şi turn - care rezolvă situaţiile de simultaneitate.

Algoritmul lui Peterson este:

```
bool flag[2] = {false, false};
int turn;
```

```
S0:      flag[0] = true;
S0_gate: turn = 1;
    while (flag[1] == true && turn == 1)
        {
            // busy wait
        }
        // critical section
        ...
        // end of critical section
        flag[0] = false;
```

```
S1:      flag[1] = true;
S1_gate: turn = 0;
while (flag[0] == true && turn == 0)
        {
            // busy wait
        }
        // critical section
        ...
        // end of critical section
        flag[1] = false;
```

https://en.wikipedia.org/wiki/Peterson%27s_algorithm

Comentariu

- Algoritmul îndeplinește cele trei criterii esențiale pentru a rezolva problema secțiunii critice, cu condiția ca modificările variabilelor turn, flag [0] și flag [1] să se propage imediat și atomic.
- Cele trei criterii sunt excluderea reciprocă, progresul și așteptarea limitată.

Excludere mutuala

- S0 și S1 nu pot fi niciodată în secțiunea critică în același timp:
- Dacă S0 se află în secțiunea critică, atunci flag [0] este adevărat.
  - În plus, fie flag [1] este fals (adică S1 și-a părăsit secțiunea critică),

    fie turn=0 (adică S1 încearcă acum să intre în secțiunea critică, dar așteaptă),
- Dacă ambele sarcini sunt în secțiunile lor critice, atunci starea trebuie să îndeplinească flag [0] și flag [1] și turn = 0 și turn = 1.
- Nici o stare nu poate satisface atât turn = 0, cât și turn = 1, deci nu poate exista o stare în care ambele sarcini se află în secțiunile lor critice.
- Deoarece turn poate lua una din cele două valori, poate fi înlocuit cu un singur bit, ceea ce înseamnă că algoritmul necesită doar trei biți de memorie pentru implementare sicronizare.

Progresul: o sarcină nu poate intra imediat din nou în secțiunea critică dacă celălaltă sarcină și-a setat flag-ul pentru a spune că ar dori să intre în secțiunea sa critică.

Așteptarea limitată : în algoritmul lui Peterson, o sarcină nu va aștepta niciodată mai mult de o tură pentru intrarea în secțiunea critică.

Algoritmul lui Peterson pentru mai multe sarcini

- Algoritmul de filtrare generalizează algoritmul lui Peterson la N> 2 procese.
- În loc de un flag boolean, necesită o variabilă întreagă pe sarcină, stocată într-un singur registru atomic care este scrisă de o singura sarcină și citită de mai multe sarcini (SWMR) și N-1 variabile suplimentare în registre similare.
- Registrele pot fi reprezentate în pseudocod sub formă de tablouri:
- level : array of N integers
- last_to_enter : array of N−1 integers
- Variabilele de nivel iau valori de până la N − 1, fiecare reprezentând o „variabilă de așteptare” distinctă înainte de secțiunea critică.
- Sarcinile avansează de la o variabilă la alta, terminând cu variabila în celula N − 1, care este secțiunea critică. Mai exact, pentru a obține o blocare, sarcina Si va executa

```
i ← Si
for ℓ from 0 to N−1 exclusive
    level[i] ← ℓ
    last_to_enter[ℓ] ← i
    while last_to_enter[ℓ] = i and there exists k ≠ i, such that level[k] ≥ ℓ
        wait
```

Pentru a elibera blocarea la ieșirea din secțiunea critică, sarcina Si stabilește nivelul [i] la -1.

Soluţii hardware de excludere mutuală în mediu centralizat pentru sisteme monoprocesor

- În vederea realizării excluderii mutuale se pot prevedea diferite instrucţiuni sau primitive implementate la nivelul arhitecturii maşinii.
- Există o mare diversitate de astfel de soluţii, iar în continuare vor fi considerate doar câteva exemple mai sugestive.
- Într-un sistem monoprocesor pot exista sarcini concurente care îsi dispută accesul la resurse.
- Apare şi un paralelism efectiv între Unitatea Centrală de prelucrare şi subsistemul de I/E care poate antrena şi alte subansamble, cum ar fi:
  - DMA, canale de I/E, module de memorie,
- Relaţia între UCP şi celelalte subansamble este de tip "master-salve".

- În special datorită sistemului de întreruperi, într-un sistem monoprocesor apare o întreţesere a execuţiei sarcinilor, iar anumite secţiuni ale acestora pot fi regiuni critice care trebuie executate în mod exclusiv, fără a fi întrerupte.
- Pentru a realiza aceasta trebuie ca sistemul de intrerupere SIntr să poată fi dezactivat / activat sau anumite nivele să fie mascate / demascate.

  Astfel avem:

<început sarcina>;	<început sarcina>;
dezactivare_SIntr;	*mascare_într;*
<secţiune critică>;	<sectiune critică>;
activare_SIntr;	*demascare_într;*
<rest sarcina>;	<rest sarcina>;

- Desigur că ar exista o soluţie pentru excluderea mutuală prin evitarea acesteia, prin eliminarea concurenţei, dar aceasta nu este acceptabilă datorită degradării performanţelor.

Soluţii hardware de excludere mutuală în mediu centralizat pentru sisteme multiprocesor

Pentru sistemele multiprocesor realizarea excluderii mutuale depinde şi mai mult de arhitectura sistemului. În principiu, trebuie realizată execuţia indivizibilă a unor secvenţe de acces la memoria comună.

Dintre soluţiile posibile amintim:

- instrucţiuni atomice de tip XCHG care schimbă între ele două locaţii de memorie sau un registru general şi o locaţie de memorie;
- instrucţiuni atomice de tip TAS (Test And Set) sau RMW (Read Modify Write) care citesc o celulă de memorie, o modifică şi actualizează noua valoare;
- instrucţiuni atomice de tip Lock, Unlock;
- instrucţiuni atomice de incrementare, decrementare;
- instrucţiuni atomice de înlocuire şi adunare.

Excludere mutuală cu instrucțiuni XCHG

- Instrucţiunea XCHG(r,m) interschimbă, ca o operaţie atomică, conţinutul registrului r cu celula de memorie m (semafor).
  - deci r_i este o variabilă locală a sarcinii S_i şi este iniţializată cu 0, iar
  - m este o variabilă globală, comună tuturor sarcinilor care este iniţializată cu 1.
- Ambele variabile pot lua valorile 0, 1.

Algoritmul pentru S_i va fi:

<început sarcina>;

repetă

XCHG(r_i,m);

până_când r_i;

<secţiune critică>;

XCHG(r_i,m);

<rest sarcina>;

- Numai procesul care găseşte m=1 va intra în secţiunea critică şi va trece în zero. La ieşire din secţiunea critică va returna pe m la valoarea 1 pentru a permite celorlalte sarcini să intre în secţiunea critică. (m=1 este o conventie, in unele sisteme se considera m=0)
- Când o sarcina Si intră în secţiunea critică, r_i=1.
- Deci pentru a realiza excluderea mutuală trebuie ca în orice moment să se verifice relaţia:

$$m + \sum_{i=0}^{n-1} r_i = 1,$$

- Microprocesorul x86 include în setul de instrucţiuni o astfel de instrucţiune care prefixată cu LOCK realizează funcţia necesară pentru implementarea excluderii mutuale.

LOCK_Si

```
AST:    MOV     AL,0
        LOCK
        XCHG    AL,m
        TEST    AL,AL
        JZ      AST
```

se intra in executia sarcinii , zona critica

UNLOCK_Si

```
        MOV     m,1
```

LOCK_Sj

```
AST:    MOV     AL,0
        LOCK
        XCHG    AL,m
        TEST    AL,AL
        JZ      AST
```

se intra in executia sarcinii , zona critica

UNLOCK_Sj

```
        MOV     m,1
```

Excluderea mutuală cu instrucțiuni TAS

- Instrucţiunea TAS(m) testează conţinutul lui m şi
  - dacă este 0, m ← 1 şi reîntoarce rezultat adevărat.
  - dacă m este 1, lasă conţinutul nemodificat şi întoarce rezultat fals.
- Variabila globală m este iniţializată cu 0.

Pentru S_i, algoritmul este:

<început sarcina>;

repetă

până_când TAS(m);

<secţiune critică>;

m ← 0;

<rest sarcina>

- Având în vedere că instrucţiunea TAS(m) se execută strict serial, numai sarcina care găseşte m=0 va intra în secţiunea critică, şi va face atribuirea m ← 1.
- La ieşirea din secţiunea critică va face m ← 0 pentru a permite celorlalte sarcini să intre în secţiunea critică.

- Microprocesorul x68000 include în setul de instrucţiuni o instrucţiune TAS indivizibilă, pentru excludere mutuală în structuri multiprocesor.

Excludere mutuală cu instrucțiuni LOCK, UNLOCK

Primitiva LOCK(m)

este echivalentă cu:

{

repetă nimic *cât_timp* m=0;

m ← 0;

}

Primitiva UNLOCK(m)

este echivalentă cu:

{

m ← 1;

}

Pentru familia x86

LOCK_Si

```
AST:    MOV     AL,0
        LOCK
        XCHG    AL,m
        TEST    AL,AL
        JZ      AST
```

se intra in executia sarcinii , zona critica

UNLOCK_Si

```
        MOV     m,1
```

- Iniţializând variabila m cu 1, algoritmul de excludere mutuală este:

<început sarcina>

LOCK(m);

<secţiune critică>;

UNLOCK(m);

<rest sarcini>;

```
LOCK_Si
AST:    MOV     AL,0
        LOCK
        XCHG    AL,m
        TEST    AL,AL
        JZ      AST
```

se intra in executia sarcinii , zona critica

```
UNLOCK_Si
        MOV     m,1
```

- La unele procesoare, această funcţie este realizată de instrucţiuni denumite TSB (Test and Switch Branch).

MICROPROGRAMARE

Noțiuni şi concepte de bază

Organizarea memoriei de control

Suportul fizic pentru păstrarea microprogramelor

Organizarea logică a instrucțiunilor

Codificare verticalã

Codificarea orizontalã

Codificarea minimalã

Codificarea cu control rezidual

Codificarea cu control prin adrese

Codificare mixtã

Implementarea microinstrucțiunilor

NOTIUNI SI CONCEPTE DE BAZĂ



Din punct de vedere structural unităţile de comandă sunt de două tipuri:
-convenţionale în sensul propus de Von Newmann ;
-microprogramate conform conceptului introdus de M.Wilkes.

Microprogramarea este o tehnică de proiectare şi implementare a funcțiilor de control a sistemelor de prelucrare a datelor numerice, ca o secvență de semnale de control ce interpretează static sau dinamic funcțiile de prelucrare a datelor. Semnalele de comandă, necesare la un moment dat pentru controlul primitivelor funcționale sunt organizate într-un cuvânt de control, memorat într-o memorie PROM sau RAM. Structura cuvântului de control este influențată de semnificația atribuită noțiunii de microoperație .

Microoperația (μO) este o primitivă a funcțiilor de prelucrare a datelor, care reprezintă o operație elementară asupra unei primitive funcționale (transfer, acțiune de înscriere sau de incrementare, activare pe magistrală, etc.), ce se desfaşoară de obicei într-o perioadă de timp (perioada de tact sau de ceas).

Microoperația implică existența unui operator, care corespunde unei unități funcționale şi a operanzilor asupra cărora acționează.

Cuvântul de control este pus în corespondență cu noțiunea de microinstrucțiune.

Microinstrucțiunea (μl) este un set de microoperații independente de date, fără conflict de resurse, care se pot executa simultan (pe perioada unei perioade de sincronizare).

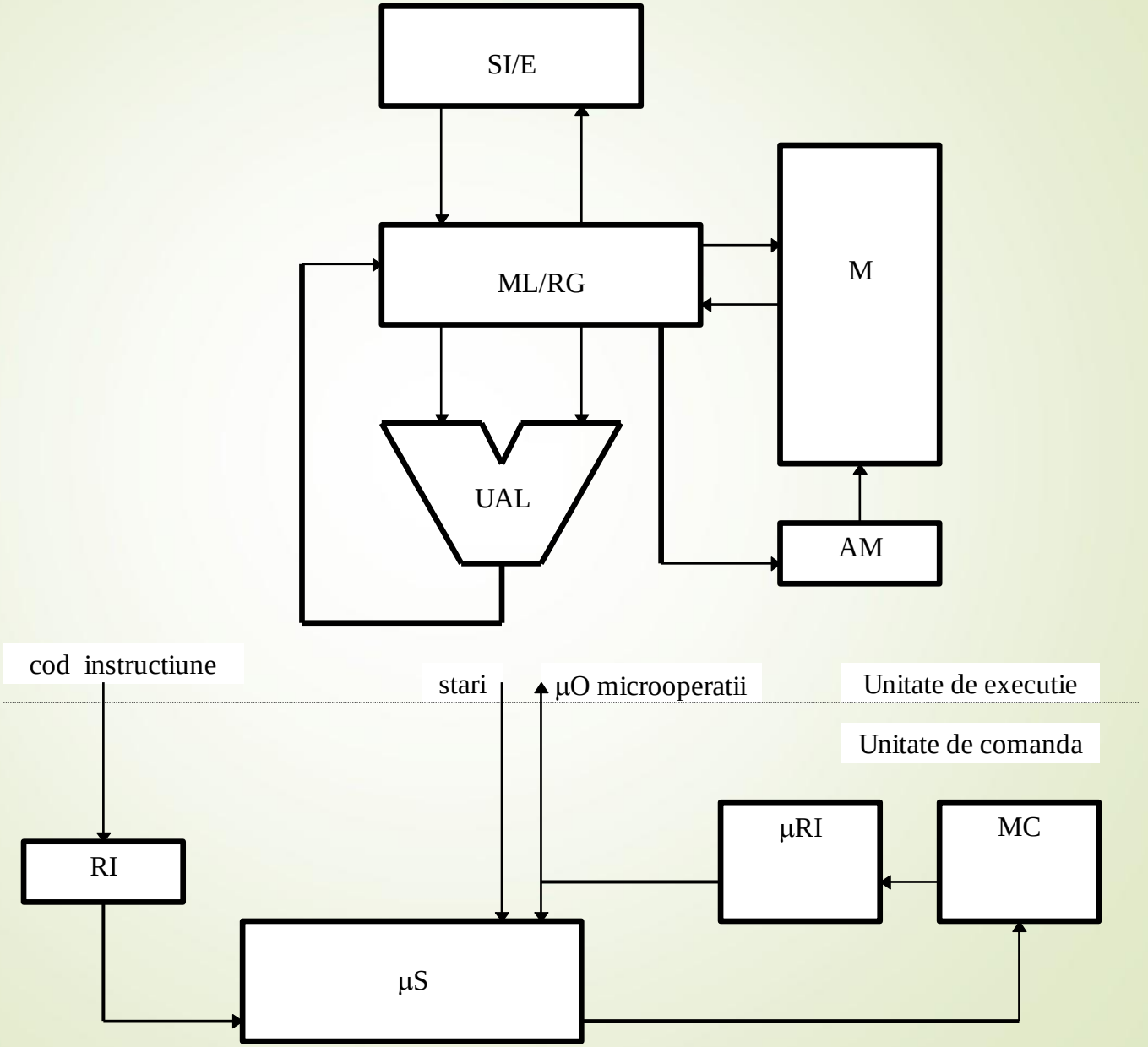
TIPURI DE MICROUNSTRUCTIUNI

- microinstrucțiuni operaționale care controlează primitivele funcționale ale unității de execuție a sistemului numeric, asigurând fluxul de informație şi acțiunile asupra resurselor;
- microinstrucțiune de ramificație (de salt) care inspectează starea primitivelor funcționale şi asigură ramificația în algoritmul de control, constituind suportul pentru implementarea deciziilor.

Prin microprogram se înțelege o secvență de microinstrucțiuni ce implementează un algoritm care descrie :

- citirea interpretarea şi execuția unui set de instrucțiuni maşină;
- primitive ale sistemului de operare;
- primitive ale limbajelor de programare;
- etc.

Structura generală a unui sistem de calcul cu unitate de comandă microprogramată



PRIMITIVE FUNCTIONALE

- **M** - memoria principală a sistemului, în care se păstrează programele ca secvență de instrucțiuni maşină şi datele care se prelucrează ;
- **ML** - memoria locală a sistemului, reprezentată de registrele generale de lucru accesibile sau nu utilizatorului;
- **UAL** - unitatea aritmetică logică ;
- **S I/E** - subsistemul de intrări / ieşiri ;
- **MC** - memoria de control în care se păstrează microprogramul ca secvență de microinstrucțiuni ;
- **μRI** - registrul de microinstrucțiuni, care păstrează microinstrucțiunea curentă ce se execută. Conținutul său specifică toate microoperațiile care se execută în acel moment în unitatea de execuție ;
- **μS** - microsecvențiatorul, unitatea de comandă convențională, elementară, care asigură citirea interpretarea şi execuția microinstrucținilor din memoria de control precum şi înlănțuirea acestora pe baza registrului de instrucțiuni maşină RI şi a stării primitivelor funcționale.



O structură microprogramată se caracterizează prin:

- organizarea memoriei de control ;
- suportul fizic pentru păstrarea microprogramelor
- organizarea logică a microinstrucţiunilor ;
- implementarea microinstrucţiunilor





ORGANIZAREA MEMORIEI DE CONTROL

Dupa criteriu de analiză relația MC față de M:

-*memorie de control separată de memoria principală*, atât din punct de vedere fizic cât şi din punct de vedere al adresării logice,

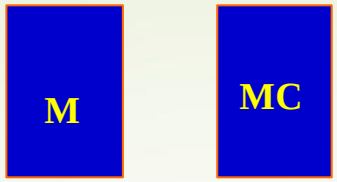
-Raportul dintre viteza de lucru a memoriei de control şi cea a memoriei principale este de circa 10 în favoarea memoriei de control, Vmc >> Vm

- memoria de control este implementată în acelaşi spațiu fizic şi de adresare cu cel al memoriei principale. Ambele au acelaşi ciclu de memorie şi sunt adresate prin aceeaşi logică. Divizarea în memorie de control şi memorie principală se poate face la nivel fizic sau la nivel logic.

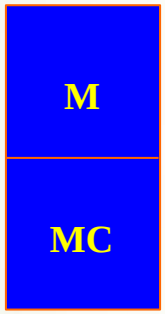
Acest tip de organizare cere ca memoria să fie suficient de rapidă pentru a fi folosită ca MC şi destul de ieftină pentru a fi folosită ca memorie principală.

–memoria de control este implementată separat de memoria principală însă este încărcată din aceasta. Se utilizează un sistem de memorie ierarhică

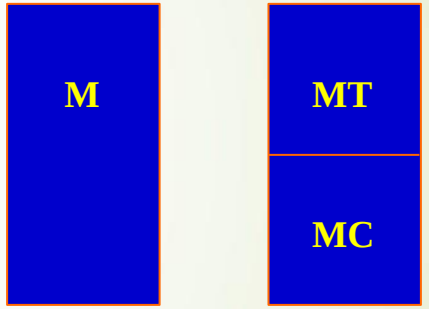
Memoria MC este încărcată din memoria principală prin intermediul memoriei tampon MT.



Organizarea memoriei de control



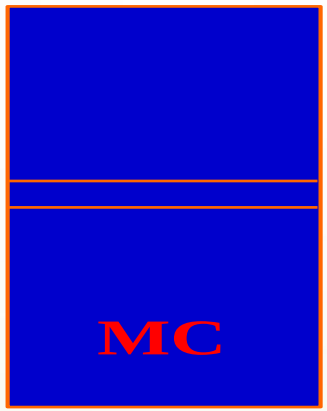
Organizarea MC în cadrul MP



Organizarea MC separată de MP dar incarcabilă din aceasta

CLASIFICAREA MEMORIEI DE CONTROL MC

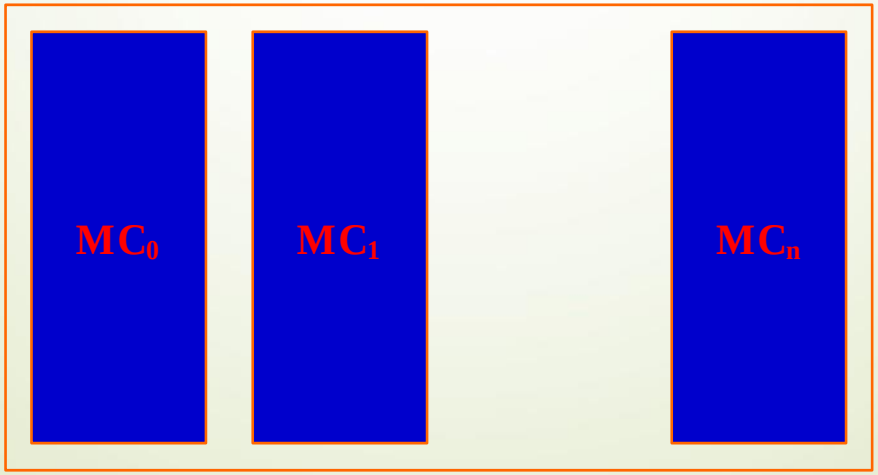
1. ***memorie de control cu o microinstrucțiune pe cuvânt,*** în care fiecărui cuvânt de memorie îi corespunde o singură microinstrucțiune. Citirea unei microinstrucțiuni presupune un singur acces la MC.



2. *memorie de control cu organizare pe pagini*

Unei adrese din memoria de control i se asociază mai multe microinstrucţiuni, din pagini diferite. în acest fel se asigură la nivel de microprogram execuţia unei microinstrucţiuni de tip CASE, care introduce facilitatea de decizii multiple.

O adresă de MC va adresa aceeaşi locaţie în toate paginile mémoriei de control iar vectorul de condiţii de test va activa pagina ce specifică microinstrucţiunea următoare care se va executa.



3. *memorie de control cu organizare pe blocuri*

Pentru acest tip de organizare există două feluri de adrese:

-adrese de microinstrucţiuni din acelasi bloc cu microinstrucţiunea curentă ;

-adrese de blocuri.

Împărţirea memoriei de microprograme în blocuri se face ţinând seama atât de structura microprogramului cât şi de resursele fizice disponibile. O organizare de acest fel conduce, în general, la micşorarea lungimii microinstrucţiunii însă introduce timp suplimentar cu comutarea adreselor de blocuri.



4. *memorie de control divizată*

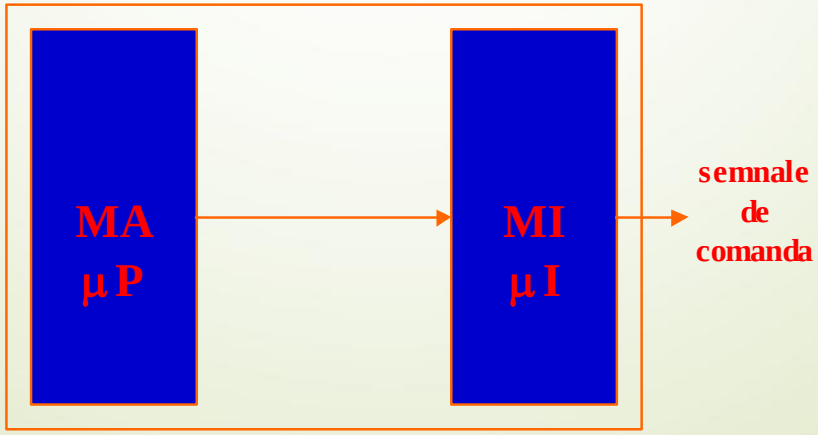
Memoria de microprograme divizată este alcătuită din două unităţi de memorie distincte :

MI - memorie de microinstrucţiuni, care păstrează toate microinstruciunile distincte posibile necesare pentru controlul resurselor fizice ;

MA - memorie de adrese de microinstrucţiuni, care păstrează programul specificat, nu prin microinstrucţiuni ce controlează resursele fizice, ci prin adrese de microinstrucţiuni (adrese pentru memoria MI).

În general numărul de tipuri de microinstrucţiuni distincte este mult mai redus decât microprogramul în sine, ceea ce implică ca numărul de biţi necesari pentru adresarea memoriei MI să fie şi el redus. In acest fel, lungimea cuvântului din memoria MA este mult mai mic decât al memoriei MI, ceea ce conduce la o reducere substanţială a memoriei de control.

În schimb, pentru a executa o microinstrucţiune trebuie făcute două adresări, una la memoria MA şi una la memoria MI ceea ce conduce la un ciclu mai mare de microinstrucţiune.



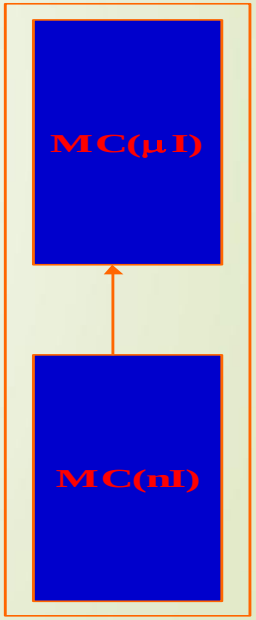
5. *memorie de control structurată pe două niveluri*

La o astfel de organizare mecanismul citirii interpretării şi execuţiei unei instrucţiuni maşină este următorul :

- o instrucţiune maşină este interpretată de un set de microinstrucţiuni rezident în memoria de control MC (μI) ;
- la rândul ei, o microinstrucţiune este interpretată de o secvenţă de nanoinstrucţiuni rezidentă în memoria MC (nI) .

Această tehnică a nanoprogramării este conceptual echivalentă cu microprogramarea, structurarea pe două niveluri oferă o flexibilitate mai mare şi posibiltatea implementării unor structuri de control foarte complexe.

O astfel de organizare întâlnim la calculatorul NANODATA QM1.





SUPORTUL FIZIC PENTRU PASTRAREA MICROPROGRAMELOR

Memoria de microprograme MC este o unitate de memorie de mare viteză care păstrează microprogramele ce se execută.

În ceea ce priveşte suportul fizic, pentru păstrarea microprogramelor, acesta poate fi realizat cu memorii PROM sau cu memorii RAM.

Realizarea memoriei de control :

-cu componente de tip RAM conferă o caracteristică statică

- componentelor de tip RAM oferă o caracteristică dinamică ce permite utilizatorului ca printr-un ansamblu de mijloace software să aibă acces la microprogramul din memoria de control..

Este foarte important în a face deosebire între :

- maşini microprogramate, şi
- maşini microprogramabile

Prima categorie se referă la modalitatea de implementare a unităţii de comandă în sensul conceptului introdus de Wilkes fără a oferi resursele hardware şi suportul de programe pentru accesul utilizatorului la nivelul microprogramului.

Cea de a două categorie oferă atât resursele hardware cât şi facilităţile software pentru accesul utilizatorului la nivelul microinstrucţiunilor.



ORGANIZAREA LOGICA A INSTRUCTIUNILOR

Un cuvânt din memoria de control specifică un set de microoperații ce

constituie componentele primitive ale controlului resurselor sistemului. Organizarea logică a microinstrucțiunilor este influențată de :

- gradul de paralelism între microoperații, ce se doreşte realizat;
- structura maşinii de bază ;
- gradul de codificare sau de flexibilitate dorit ;
- gradul de optimizare al lungimii cuvântului de control .

Presupunând că microprogramele sunt specificate ca o secvență de

seturi disjuncte de microoperații, se pot distinge mai multe modalități de codificare a acestora şi anume:

- codificare verticală sau maximală ;
- codificare orizontală sau cu control direct ;
- codificare minimală ;
- codificare cu control rezidual ;
- codificare cu control prin adrese ;
- codificare mixtă.



CODIFICARE VERTICALA

Fiecare microinstrucțiune operațională specifică o singură microoperație.

Setul de microoperații μ {μO} necesar pentru controlul primitivelor funcționale se codifică în [$\log_2$ | μ {μO}|] biți, ce constituie lungimea cuvântului din MC.

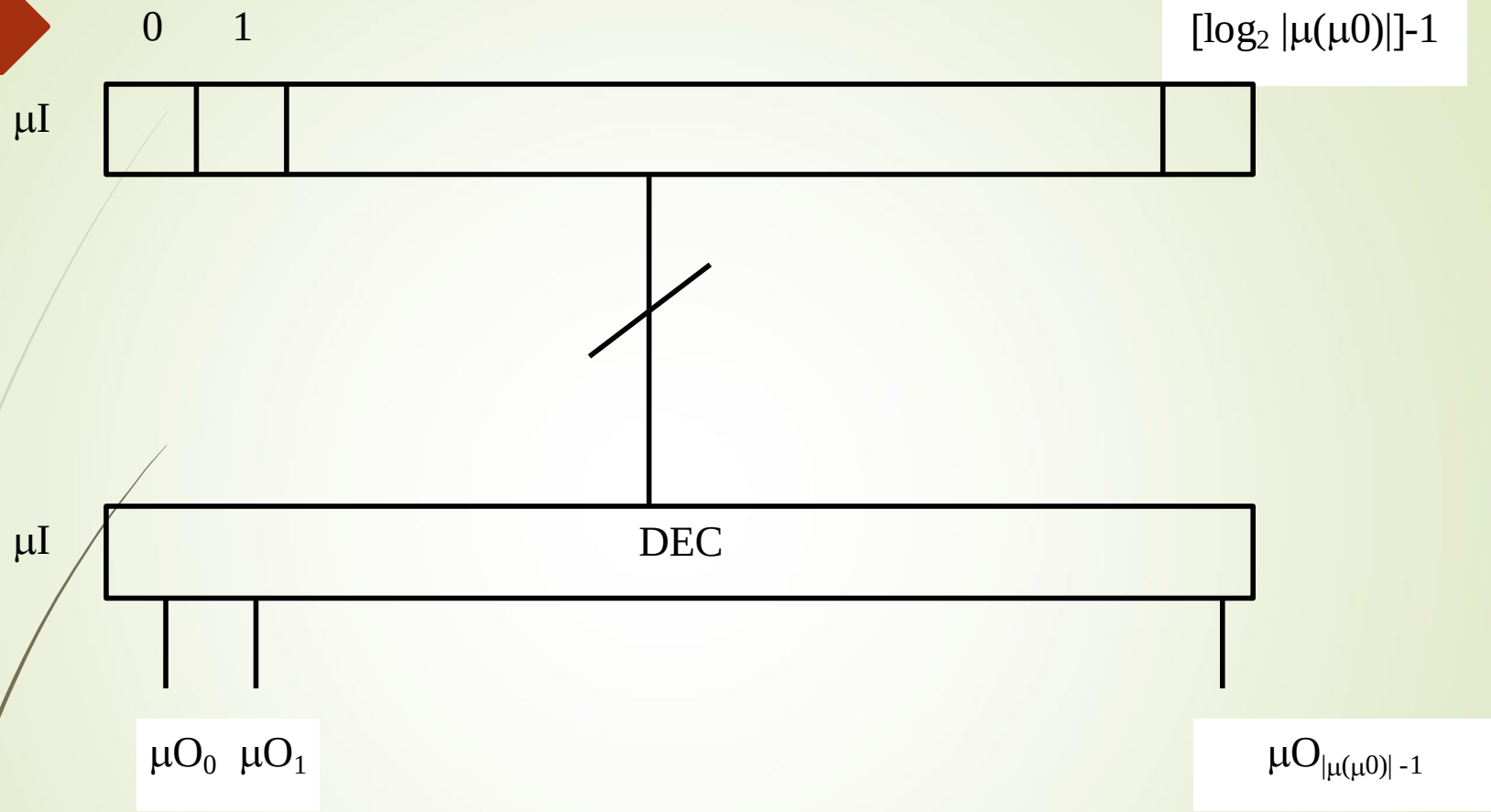
Această codificare reprezintă un caz extrem, deoarece elimină orice posibilitate de desfăşurare paralelă a operațiilor elementare.

Din punctul de vedere al minimizării cuvântului de control, codificarea verticală implică numărul cel mai mic de biți. Dimensiunea mare a decodificatorului face ca realizarea fizică a acestuia să aibe loc pe mai multe niveluri, ceea ce conduce la introducerea de întârzieri

Un dezavantaj major al codificării maximale îl reprezintă eliminarea controlului paralel asupra resurselor precum şi inflexibilitatea dezvoltării sau completării sistemului în ceea ce priveşte introducerea de noi microoperații.

Este aplicabilă numai în sisteme dedicate care au o structură specifică.

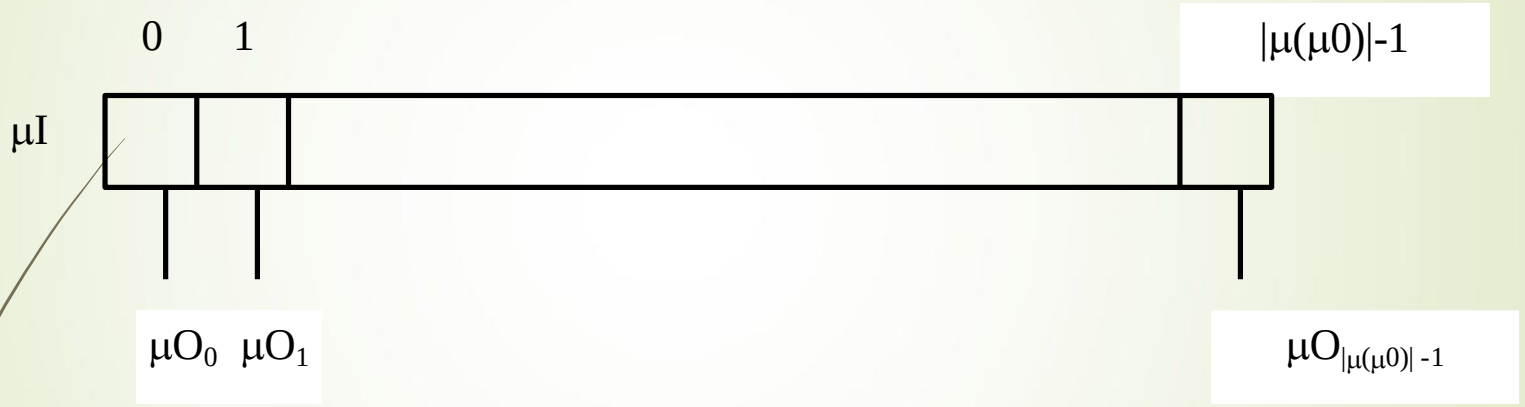
`



Codificarea verticală a microoperațiilor

CODIFICAREA ORIZONTALA

În cadrul acestei codificări, fiecare microoperație din setul (MO) este pusă în corespondență cu un bit din cadrul cuvântului de control. Controlul microoperațiilor se face în mod direct.



Codificarea orizontala

Codificarea orizontală realizează controlul tuturor microoperațiilor paralele, posibile, ce se pot desfăşura în sistem.

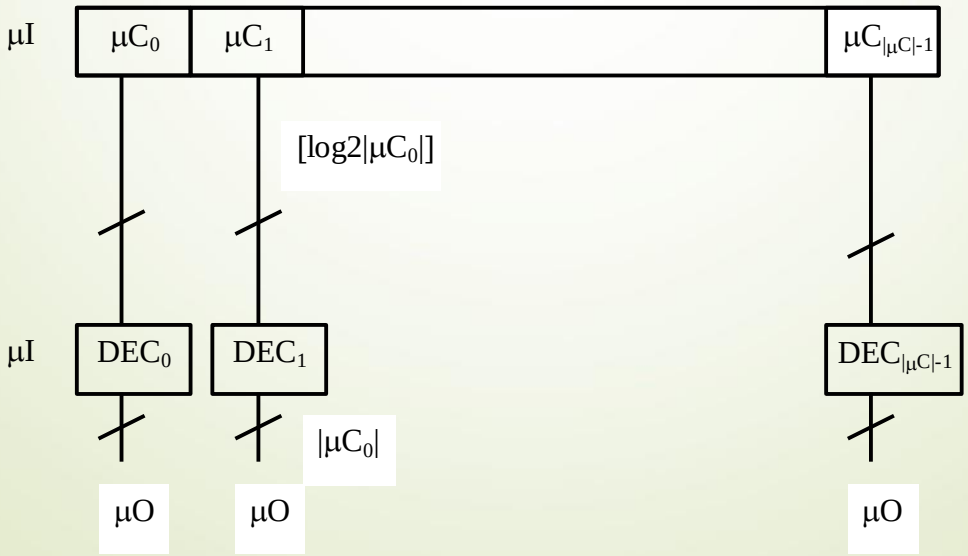
Deşi oferă o flexibilitate mare şi asigură paralelismul maxim, utilizarea acestei codificări este un caz extrem datorită folosirii ineficiente a memoriei de control.

CODIFICAREA MINIMALA

Combină flexibilitatea şi paralelismul potențial oferite de codificarea orizontală cu eficiența codificării verticale.

Ideea de bază este de a grupa în clase de compatibilitate setul de microoperații care se exclud reciproc (μO dintr-o clasă de compatibilitate nu se vor efectua niciodată simultan).

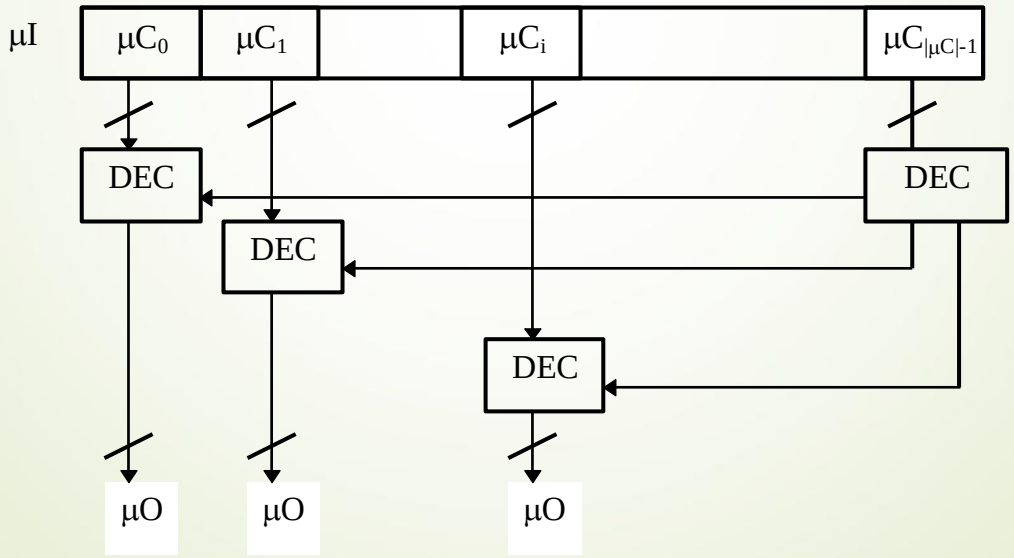
Microinstrucțiunea este împărțită în câmpuri. Un câmp corespunde unei clase de compatibilitate. La nivel de câmpuri se realizează o codificare orizontală iar în cadrul câmpurilor se realizează o codificare verticală.



Pentru un câmp μC_j care codifică $|\mu C_j|$ microoperații sunt necesari $|\log_2(|\mu C_j|+1)|$ biți, deoarece trebuie să se prevadă şi posibilitatea de a nu specifica nici o microoperație din cadrul câmpului.

O variantă a acestei codificări o reprezintă codificarea pe două niveluri sau indirectă.

În codificarea pe două niveluri, validarea unor câmpuri depinde de valoarea altui câmp de control din microinstrucțiune.



Codificare minimală pe două niveluri

CODIFICAREA CU CONTROL REZIDUAL

Această metodă de codificare foloseşte pentru controlul primitivelor funcționale registre de control rezidual. Cuvântul de control nu controlează direct resursele ci prin intermediul registrelor de control rezidual încărcate sub acțiunea microinstrucțiunilor

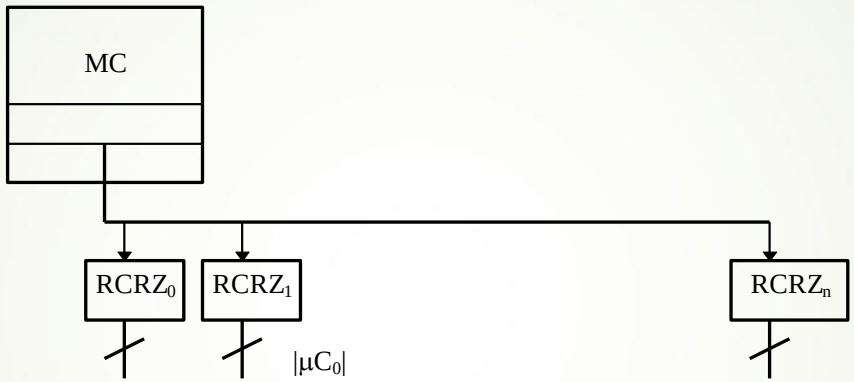


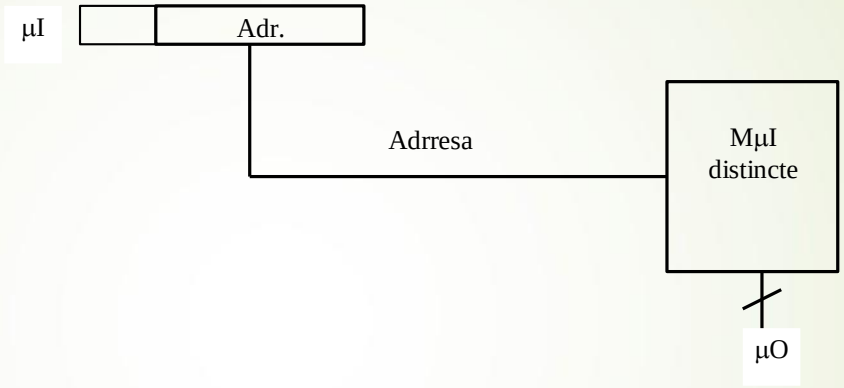
Figura 3.1 Codificare cu control rezidual

Microinstrucțiunile pot inlocui sau modifica valoarea unuia sau mai multor registre de control. Astfel, se asigură o economie de memorie de control atunci când unele primitive funcționale realizează aceeaşi operație în mod repetat sau când un set de microoperații este activ o perioadă mare de timp, iar alte seturi de microoperații se modifică.

Registrele de control rezidual $RCRZ_j$, care specifică microoperațiile de control al resurselor hardware pot fi manevrate cu ajutorul unor microinstrucțiuni de dimensiuni reduse

CODIFICAREA CU CONTROL PRIN ADRESE

O modalitate de implementare a microinstrucțiunilor operaționale este aceea în care se specifică o adresă în cadrul unei memorii, unde sunt memorate toate microinstrucțiunile distincte posibile ce controlează sistemul.



Codificare cu control prin adrese

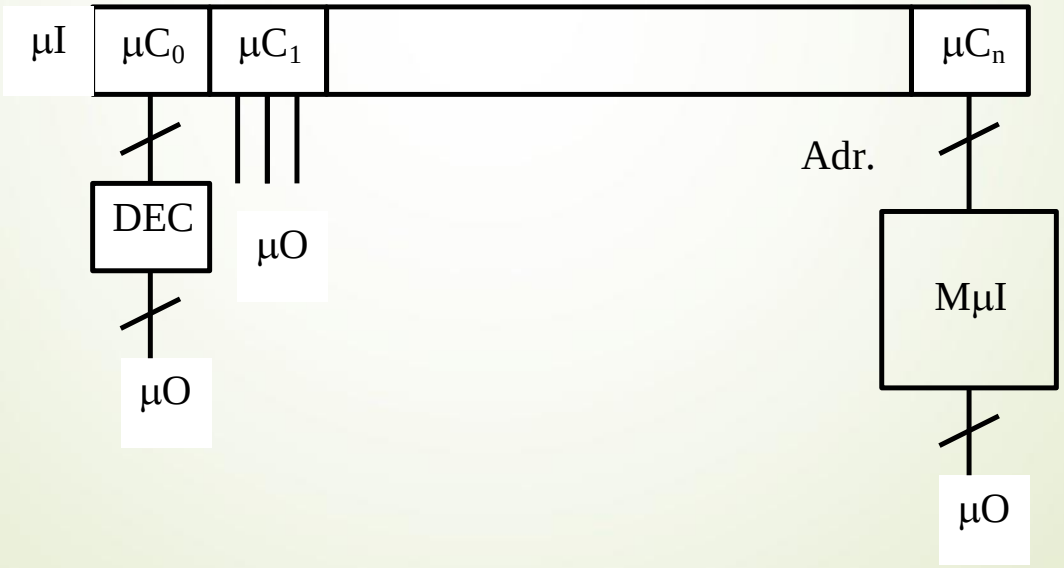
Numărul de microinstrucțiuni distincte nu depinde de numărul de resurse controlate ci de numărul de μO disticte, de mărimea μP şi de numărul de variabile de stare testate.

Memoria care păstrează microinstrucțiunile distincte va avea lungimea cuvântului suficient de mare pentru a controla toate microoperațiile care se pot efectua simultan. Trebuie notat că fiecare μl este memorată o singură dată.

O astfel de implementare, face ca microprogramul să fie format dintr-o secvență de adrese care apelează μl păstrate în memoria de μl.

CODIFICAREA MIXTA

O variantă utilizată mult în practică este aceea în care microinstrucțiunea este împărțită în câmpuri. Unele câmpuri controlează direct microoperațiile (sub formă codificată sau directă) iar altele specifică adrese de memorie ce conține un subset de microinstrucțiuni distincte.





IMPLEMENTAREA INSTRUCTIUNILOR

Microinstrucțiunile sunt citite, interpretate şi executate de către microsecventiator (μS) în acelaşi fel în care o unitate de comandă convențională execută instrucțiuni maşină.

Se poate defini o caracteristică serie-paralel care măsoară cantitatea de suprapunere între faza de execuție a μl curente şi citirea, interpretarea μl următoare.

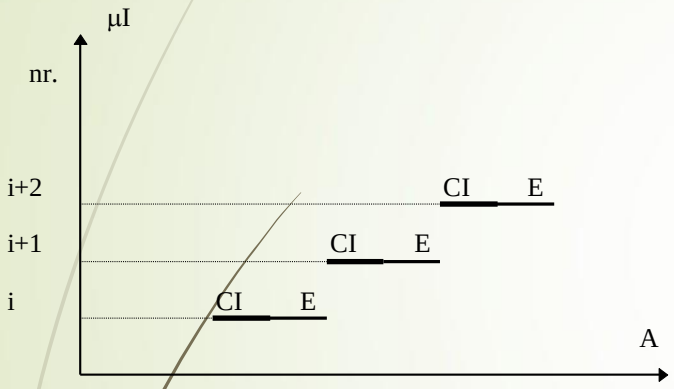
Din punctul de vedere al caracteristicii serie-paralel distingem trei tipuri de implementări :

- implementare serie;
- implementare serie-paralelă ;
- implementare paralelă.

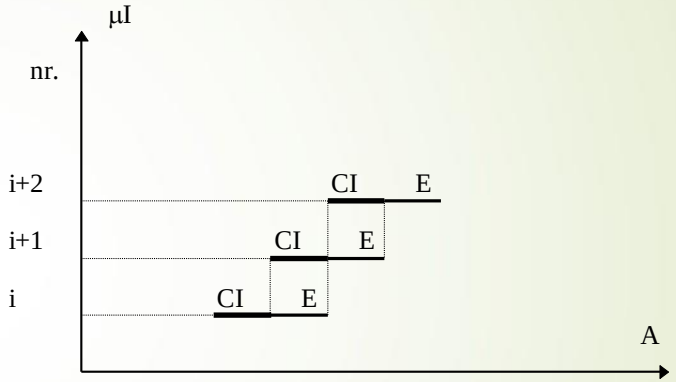
Fie CI-faza de citire interpretare şi E-faza de execuție a unei μl.

În implementarea serie, citirea microinstrucțiunii următoare nu începe decât după ce s-a terminat execuția microinstrucțiunii curente.

În implementarea paralelă, Fig. 9.17, faza de citire a μl următoare se desfăşoară în acelaşi timp cu execuția μl curente.

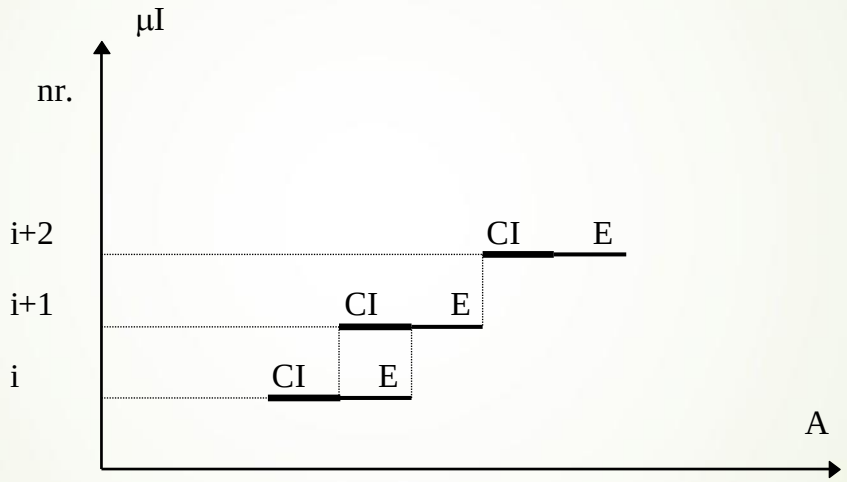


Implementare serie



Implementare paralel

O îmbinare a performanțelor implementării paralele şi a costului redus al implementării serie este realizată de implementarea serie-paralelă a microinstrucțiunilor din figura urmatoare.



Implementare serie-paralel

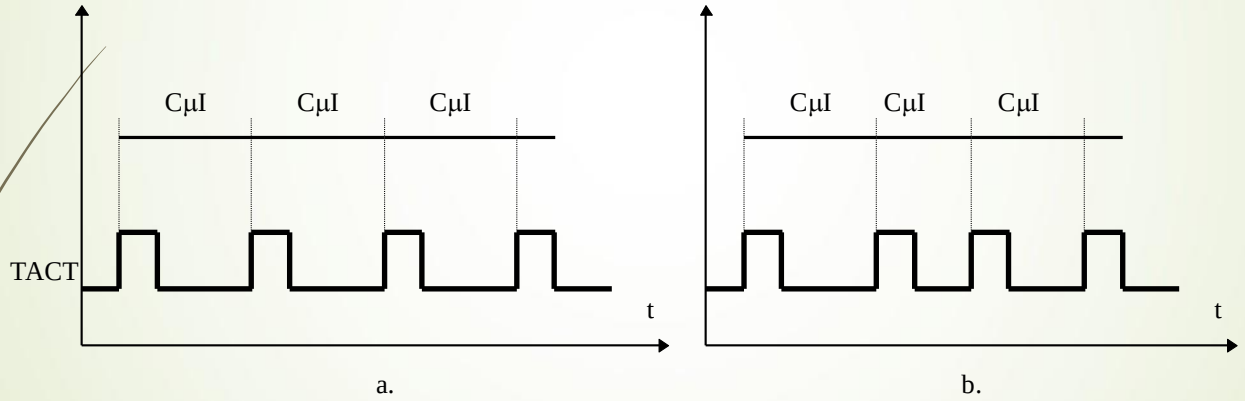
Faza de citire a μl următoare se desfăşoară fie în timpul execuției μl curente, fie după terminarea ei, în funcție de tipul microinstrucțiunii curente.

Succesiunea de microinstrucțini operaționale se desfăşoară prin suprapunerea fazelor de citire şi execuție, iar cele imediat următoare unor ramificații se citesc după terminarea execuției μl curente (de ramificație).

Implementarea serie-paralelă este caracteristică arhitecturilor "pipe line", şi este cea mai folosită.

Referitor la implementarea microinstrucțiunilor se poate defini şi o caracteristică monofază-polifază care se referă la numărul de faze utilizate într-un ciclu de microinstrucțiune.

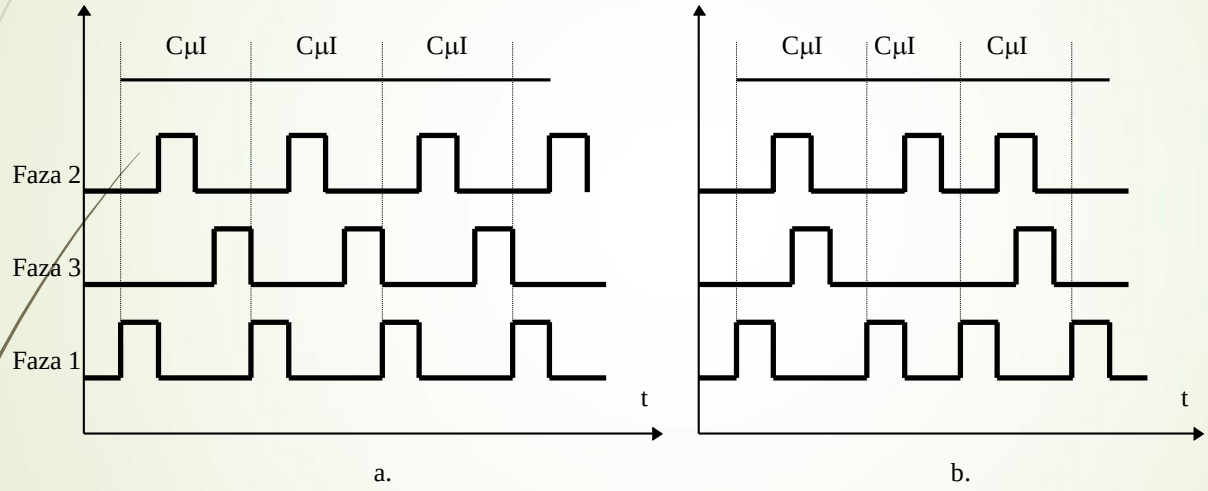
Într-o implementare monofază, microoperațiile sunt generate simultan, toate semnalele de control specificate de microinstrucțiune fiind active în acelaşi timp (pentru cele de tip impuls se ține seama de front).



Implementare monofază

a. ciclul macroinstrucțiunii CμI constant

b. ciclul macroinstrucțiunii CμI variabil



Implementare polifază

a. ciclul microinstructiunii constant

b. ciclul microinstructiunii variabil

Determinarea paralelismului între operații elementare în cadrul structurilor numerice.

- La nivelul unității de comandă a structurii numerice, e necesar să se implementeze controlul concurent al op. elementare.
- Obiectivul problemei de determinare a microoperațiilor paralele constă în a stabili, pe baza:
  - grafului de dependențe de date şi
  - a cererilor de resurse,

  seturile de microoperații ce pot controla simultan resurse diferite ale sistemului.
- Obs: Conceptele si algoritmii prezentati pot fi aplicati şi la execuția sarcinilor sau a task-urilor.

Definitii

- **Def.1** Se numeşte microsubbloc **µSB** un subset maxim al unui set de microoperații ce constituie o secvență indivizibilă cu un singur punct de intrare şi un singur punct de ieşire şi conține maximum o microoperație de salt, ca ultimă operație din secvență.
- µSB(µo)=(µB(µo), <) unde
  - µB(µo)= {μo_1, μo_2, μo_3, … , $\mu o_{|\mu B|}$ }
  - < relatie de ordine nereflexiva
- **Def.2 µOd** – microoperatie disponibila- acea microoperatie pentru care toate microoperatiile de care ea era dependenta de date au fost asignate unor microinstrucțiuni.

$$\mu O_d = \{\mu O_j / \forall \mu O_i \angle \mu O_j \Rightarrow \mu O_i \in \mu I\}$$

- **Def. 3** Prin nivel într-un subbloc **μON_k** înțelegem toate microoperatiile independente situate la aceeaşi distanță (adâncime) față de nodul inițial.

Def.4 Setul de operatii disponibile la nivelul k, μOD_k, este construit din multimea de microoperatii disponibile din cadrul asociat microoperatiilor la nivelul k de generare de seturi paralele de microoperatii.

PROP.1. Microoperatiile din seturile de microoperatii disponibile sunt independente.

Presupunem ca nu ar fi independente. Daca avem

$$\mu O_i \in \mu OD_k \qquad \mu O_j \in \mu OD_k$$

si nu sunt independente, atunci fie $\mu O_i < \mu O_j$ sau $\mu O_j < \mu O_i$.

Conform Def 2 $\forall \mu O_l < \mu O_j \qquad \mu O_j \in \mu OD_k$

rezulta ca μO_l a fost asignata (atribuita) unei microinstructiuni.

Rezulta ca μO_l nu poate face parte din μOD_k si singura solutie este ca ele sa fie independente.

$|\mu ON|$ - numarul de niveluri dintr-un microsubloc egale cu timpul minim de executie pentru microsublocul respectiv

Fiecare nivel va specifica toate microoperatiile paralele care se desfasoara la un anumit moment de timp.

nmμl- numarul minim de microinstructiuni ce ar putea fi generate intr-un microsubloc

Evaluare estimativa:

- $\boldsymbol{nm\mu I} = \frac{|\mu oB(\mu O)|}{nR}$ - multimea de microoperatii din bloc
- unde:

$$nR - numarul\ de\ resurse\ ale\ sistemului$$

$\mu CR = \emptyset$ conflictul intre resurse

nMμI – numarul maxim de microinstructiuni

nMμI = $|\ \mu oB(\mu O)|$ - toate microoperatiile se executa secvential

nmμI, nMμI sunt cazuri extreme.

nOμI- numarul optim de microinstructiuni

$|\boldsymbol{\mu ON}|$= cardinalul nivelelor pe care le stabilim

$$\forall \mu ON_j \in \{\mu ON\} \Rightarrow \mu CR(\mu ON_j) = \emptyset$$

μPT_I – partitia cea mai timpurie de executie a unei μoperatii.

Aceasta este constituita din multimea de niveluri care contin toate μoperatiile independente situate la aceeasi distanta fata de un nod initial in graful de dependente de date.

- $\boldsymbol{\mu PT_l = \{\mu ON \big| \forall \mu O_i, \mu O_j \in \mu ON \ avem \ DI(\mu O_i) = DI(\mu O_j)\}}$

unde prin DI am notat adancimea fata de noul initial in graful de dependente de date.

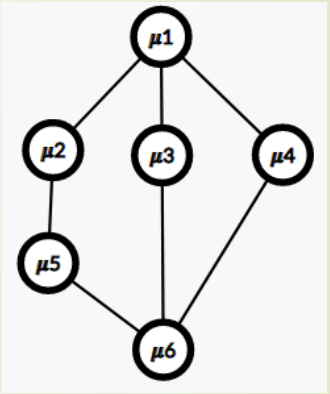
μPT_L – partitia cea mai tarzie de executie a unei μoperatii

- $\boldsymbol{\mu PT_L = \{\mu ON \big| \forall \mu O_i, \mu O_j \in \mu ON \ avem \ DF(\mu O_i) = DF(\mu O_j)\}}$

unde prin DF am notat adancimea fata de nivelul final (nodul final)

EX:

$$\mu PT_l = \{< \mu O_1 >; < \boldsymbol{\mu O_2}, \mu O_3, \mu O_4 >; < \boldsymbol{\mu O_5} >; \quad < \mu O_6 >\}$$
$$\mu PT_L = \{< \mu O_1 >; < \boldsymbol{\mu O_2} >; \quad < \mu O_3, \mu O_4, \boldsymbol{\mu O_5} >; < \mu O_6 >\}$$



Plecand de la cele doua definitii , se numeste

microoperatie critica o microoperatie care apartine aceluiasi subnivel in ambele partitii (μPT_I, μPT_L).

Ex: in cazul anterior avem μO_2 si μO_5

Orice modificare (mutare pe alt nivel) pentru o microoperatie critica conduce la un numar suplimentar de momente de timp pentru executie. Pentru celelalte (necritice), le putem pune oriunde intre nivelele date din partitia cea mai timpurie si cea mai tarzie, fara ca pentru executie sa avem necesar un timp suplimentar.

PARTITIA OPERATIILOR ELEMENTARE PE NIVELURI DE EXECUTIE

Partitiile stabilite numai pe baza dependentei de date nu sunt optime, intrucat intervine si conflict de resurse. Pentru a pune in evidenta acest conflict, in cadrul partitiilor se face o grupare dupa tipul de resurse utilizate de fiecare microoperatie.

O micropartitie este egala cu multimea de niveluri.

$$\mu PT = \{\mu ON_1, \mu ON_2, \ldots, \mu ON_{|\mu PT|}\}$$

$$care\ reprezinta\ multimea\ de\ niveluri$$

$in\ care$

$$\mu ON_j = \{\mu ON_{j1}, \ldots, \mu ON_{j|nR|}\}$$

$$N_{ji} = \{\mu O_h \,|\, \forall \mu O_h \in \mu ON_j \rightarrow \mu O_h\ utilizeaza\ aceeasi\ RES_i\};\ 1 \leq i \leq NR$$

RES_i – *resursa i*

μON_j- reprezinta nivelul j in cadrul partitiei, care este format din subniveluri care utilizeaza aceeasi resursa

Este bine ca un nivel sa-l impartim in subniveluri .

Pentru cele care folosesc aceeasi resursa, intre ele avem conflicte.

Trebuie executate secvential.

- **DEF** Cererea de resurse QRES este o aplicatie definita pe μoB si multimea de resurse din bloc: $\mu oB \times RES \rightarrow N$

  unde $RES = RESS \cup RESD \cup RESB \cup RESULC \cup RESSH.$

  RESS – resursa de tip sursa

  RESD – resursa de tip destinatie

  RESB – resursa de tip bus

  ULC – unitati logice combinationale

  SH – shuffler

  $DRES_i$ – resurse de tipul i disponibile

  $QRES_j$ – resurse de tipul j cerute

  $DRES = \{DRES_1, DRES_2, \ldots, DRES_{nR}\}$

  $QRESj \quad \{QRES_{j1}, QRES_{j2}, \ldots, QRES_{j,nR}\}$

  $-$ *vectorul de cereri de resurse necesare microoperatiilor*

  care apartin nivelului j (μON_j)

  $$QRES_{ji} = \sum_{\mu O_h \in \mu ON_j} \mu O_h \qquad sau \qquad QRES_{ji} = \sum QRES_{i,\mu O_h}$$

  unde $\mu O_h \in \mu ON_j$ care utilizeaza resursa i

- Daca cererea de resurse este mai mare decat disponibilul de resurse, atunci pe nivelul j care foloseste resursa i apare conflict de resurse.
- $QRES_{ji} > DRES_i \Rightarrow \boldsymbol{\mu CR(\mu ONji)=1}$

Se numeste microinstructiune completa μIC acea microinstructiune la care nu se poate adauga nici o operatie din setul de microoperatii disponibile, fara a se crea conflict de resurse.

$$\mu IC: \forall \mu I(\forall \mu O_i \in \mu OD, \mu O_i \notin \mu I \Rightarrow \mu CR(\mu O_i, \mu I) = 1$$

In cazul in care cererea de resurse:

$$QRES_{ji} > DRES_i \Rightarrow \mu IC \subset \mu ON_j; \qquad \mu ON_j \backslash \mu IC \neq \emptyset$$

Acest lucru inseamna ca exista microoperatii din nivelul j care vor fi trecute pe nivelele urmatoare datorita conflictului de resurse.

DEFINITII

Intarzierea in ceea ce priveste numarul de niveluri cu care se va muta o microoperatie din nodul curent j spre nodul final din graful de precedenta datorita conflictului de resurse i:

$$nN_{ji} = \begin{cases} 0 & , \ daca \ QRES_{ji} \leq DRES_I \\ \left[\frac{QRES_{ji} - DRES_i}{DRES_i}\right] & , \ daca \ QRES_{ji} > DRES_i \end{cases}$$

Intarzierea maxima pe nivelul j:

$$nN_j = max(nN_{ji}) \ , \ unde \ i = 1,2,\ldots nR$$

Stabilirea unei metodologii de determinare a partitionarilor unui microsubloc in microinstructiuni complete (care sunt actiunile pe care le fac la un moment dat).

Stabilirea efectului setului de microoperatii disponibile asupra generarii partitiilor.

Pentru a genera o singura microinstructiune completa din μOD,

ar trebui sa existe un criteriu care sa fie dependent numai de celelalte microoperatii din setul disponibil.

Aceasta alegere ar trebui sa fie independenta de microoperatiile ce vor deveni disponibile dupa asignarea microoperatiei respective.

PROPOZITIE: Pentru a alege o microoperatie care apartine setului de microoperatii disponibile k, pentru a o introduce intr-o instructiune μIC (completa, astfel incat numai o partitie, si anume μPT optima, sa fie generata), trebuie sa existe un criteriu de selectie dependent numai de microoperatiile care apartine setului disponibil dat, si independent de orice microoperatie din setul ce va deveni disponibil. *Aceasta implica existenta unui criteriu de ordonare a microoperatiilor din setul disponibil*.

JUSTIFICARE: Presupunem ca $\mu CR(\mu OD_k) = 1$, atunci avem conflict de resurse. Nu toate microoperatiile pot fi plasate in aceeasi microinstructiune.

Fie μIC generata de μOD_k.

Faptul ca generam aceasta microinstructiune inseamna ca toate operatiile dependente de aceasta devin disponibile. Vom genera un nou set de microoperatii disponibile:

- $\mu OD_{k+1} = \{\{\mu OD_k \backslash \mu IC\} \cup \{\mu O_d\}_k\}$
  - microoperatii care devin disponibile datorita asignarii acelei instructiuni complete
- $\{\mu O_d\}_k = \{\mu O | \forall \mu O_i \in \mu IC \Rightarrow \mu O_i < \mu O\}$

Fara examinarea efectului unor microoinstructiuni suplimentare asupra dinamicii lui μOD_{k+1} , alegerea unei microoperatii trebuie sa fie independenta de setul $\{\mu O_d\}_k$ rezultat. Trebuie sa existe acest criteriu de ordonare (care nici nu exista).

Orice metoda de ordonare a microoperatiilor din setul μOD_k , bazata pe resursele folosite de fiecare microoperatie si independenta de resursele folosite de celelalte microoperatiil produce o ordonare arbitrara.

Masurarea efectului se poate face pe baza urmatoarelor informatii:

- resursele solicitate:
  - de microoperatia respectiva
  - de microoperatiile dependente de microoperatia analizata
- gradul de dependenta a celorlalte microoperatii fata de cea analizata

Microoperatiile din setul disponibil sunt independente, deci nu avem acces la cele 3 aspecte (influenta uneia asupra celeilalte).

Orice metoda de ordonare va conduce la o alegere arbitrara.

Fiecare microoinstructiune generata produce un singur set de microoperatii disponibile.

Daca doua microinstructiuni complete genereaza acelasi set de microoperatii disponibile, atunci cele doua microinstructiuni complete sunt identice.

- Fie microinstructiunile μIC_i, μIC_j generate din setul disponibil la nivelul k (μOD_k).

  Presupunem ca $\mu IC_i \neq \mu IC_j$.

Fie μOD_{k+1}^i , μOD_{k+1}^j seturile de microoperatii disponibile rezultate la momentul urmator.

$$\mu OD_{k+1}^i = (\mu OD_k \backslash \mu IC_i) \cup \{\mu O_d\}_{ki}$$
$$\mu OD_{k+1}^j = (\mu OD_k \backslash \mu IC_j) \cup \{\mu O_d\}_{kj}$$
$$\mu IC_i \backslash (\mu IC_i \cap \mu IC_j) \subset \mu OD_{k+1}^i$$
$$\mu IC_j \backslash (\mu IC_i \cap \mu IC_j) \subset \mu OD_{k+1}^j$$

- Daca $\mu OD_{k+1}^i \equiv \mu OD_{k+1}^j$ ar trebui ca:
- $\mu IC_j \backslash (\mu IC_i \cap \mu IC_j) \equiv \mu IC_i \backslash (\mu IC_i \cap \mu IC_j) \Rightarrow \mu IC_i \equiv \mu IC_j$

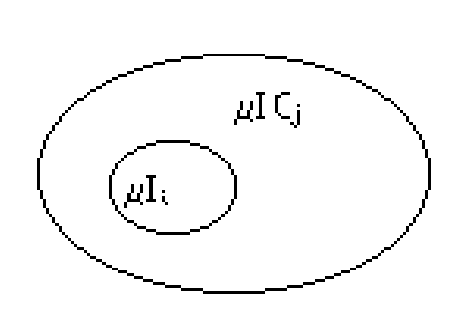
PROPOZITIE: Partitia unui microsubloc se obtine numai prin generarea de μIC (nu pot lua combinatii mai reduse de microoperatii).

Presupunem ca din setul μOD_k putem genera μI_i si μIC_j .

Alegerea μI_i va produce un set de operatii disponibile

$$\mu OD_{k+1}^{i}$$

.



$$\mu IC_j \setminus \mu I_i \neq \emptyset$$

$$\mu OD_{k+1}^{i} = \mu OD_{k+1}^{j} \cup (\mu IC_j \setminus \mu I_i)$$

$\mu I_i \subset \mu IC_j$ atunci $\mu IC_j \setminus \mu I_i \neq \emptyset$.

$$\mu OD_{k+1}^{i} \subset \mu OD_{k+1}^{j} ,$$

- avem eventual un numar mai mare de subniveluri (nu conduce la solutia optima).

Pe baza observatiei rezulta 2 categorii de algoritmi.

Partitie minima: - prin generarea din setul de microoperatii μOD_k a tuturor μIC posibile si pentru fiecare dintre ele sa se analizeze influenta ce o are asupra generarii setului de microoperatii disponibile urmator, μOD_{k+1}. **Acest algoritm are complexitate NP complete.**

Partitie euristica: - bazat pe ordonare a microoperatiilor din setul disponibil μOD_k in functie de succesorii in graful de dependente de date. Aceasta metoda nu va genera o partitie optima, insa va fi mult mai rapida in situatii acceptabile.

Calculul limitei inferioare pentru nivelurile de alocare

Presupunem că avem μSB =(μB(μo) , ∠) care are graful asociat G

μB(μo)={ μo_1, $\mu o_{2,...,}$ $\mu o_{|\mu B|}$ } microoperatiile din subblock

P= {P_1, P_2, …..$P_{|P|}$} resursele, elemente de executie

În procesul de calcul al limitei inferioare apar 4 situaţii care trebuie analizate.

CAZUL 1 . Prima valoare a limitei inferioare a **înălţimea h a grafului G**, unde h = calea cea mai lungă de la un nod rădăcină la un nod terminal.

Valoarea h =valoarea minimă a limitei inferioare (nu s-a ţinut seama de numarul de resurse si de conflictul de resurse).

CAZUL al 2-lea – estimare mai corectă – se analizează şi distribuţia microoperatiilor pe setul de resurse pe care le controleaza.

Fie ρ – această valoare limită.

Pentru calculul lui ρ, graful G va fi împărţit într-un subsistem de grafuri $G_i, 1 \leq i \leq |P|$

Resursele pot fi omogene sau nu.

Împărţirea se face în stabilirea – unor subseturi de sarcini alocate pe resurse, definite astfel:

G_i : $\mu B(\mu o)_{|Pi|}$ = {μo | μo controleaza resursa i}

Subgraf maximal
Microoperatie terminală

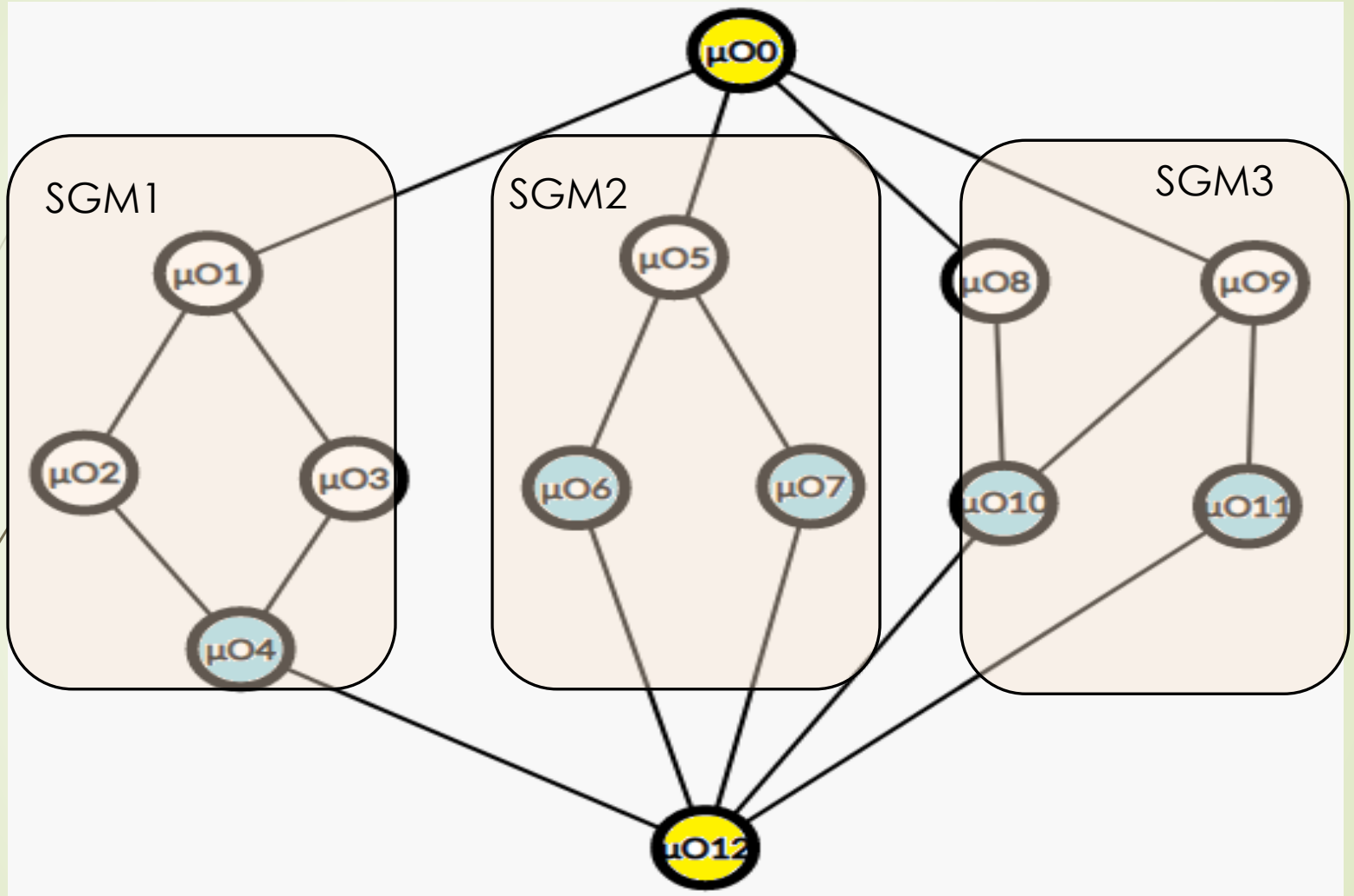
Def. O componentă a unui graf orientat aciclic G_i este un subgraf maximal $SGM_{ij} \subset G_i$ aî pentru orice microperatie μo_k care aparţine lui SGM_{ij}
($\mu o_k \in SGM_{ij}$) există $\mu o_m \in SGM_{ij}$ aî
$\mu o_k \angle \mu o_m$ sau $\mu o_m \angle \mu o_k$
(deci, trebuie să existe o legătură între cele două noduri)

Obs. G_i e considerat ca o combinare paralelă a subgrafurilor maximale.,

$$G_i = || SGM_{ij}$$

Def. Microoperatie terminală. O microoperatie μo a unui graf orientat aciclic e o microoperatie terminală μo_T dacă nu mai există nici o sarcină μo_j aî $\mu o_T < \mu o_j$
Nivelul unei microoperatii μo_i într-un graf G_i orientat e dat de nr. de arce obţinute într-o parcurgere pe drumul maxim între μo_i şi μo_T

Exemplu



Cosiderand ca avem trei resurse P= {P1, P2, P3 }
Sarcini terminale: µo 4, µo 6 , µo 7 , µo 10, µo 11
Nivelul µo1= 2 (sarcinile terminale au nivelul 0)

Calcul ρ

Pentru fiecare microoperatie terminală din SGM_{ij} obţinută prin împărţirea pe resurse a microoperatiilor, vom calcula nivelul din cadrul lui G şi alegem microoperatia terminală cu nivel maxim.

Fie μo_{Tj} aceste microoperatii, iar cu $\|\mu o_{Tj}\|$ notăm nivelul unei astfel de micro-operatii în, graful G.

Pentru fiecare subgraf G_i , i=1, ..., $|P|$ calculăm limita inferioară.

$\rho i = \max(\rho ij\}$ unde $1 <= i <= |P|, \quad 1 <= j <= |Gi|$

$\rho ij = |SGMij| + \|\mu o_{Tj}\|$

$\|\mu o_{Tj}\| = \min_{k}\{\|\mu o_{Tjk}\|\}$ cu $1 <= k <= N\mu o_T$

$|SMG_{ij}|$= nr. de microoperatii din cadrul componentei j a subgrafului G_i

$\|\mu o_{Tj}\|$ - nivelul minim al micro-operatiei terminale din cadrul componentei j a subgrafului G_i calculat faţă de poziţia în G asociat sistemului de microoperatii asociat µSB.

$|G_i|$=nr. de componente ale subgrafului G_i

$$\rho = \max_i \left\{ \max_j \left\{ |SGM_{ij}| + \min_k \{ \|\mu o_{Tjk}\| \} \right\} \right\} unde, \quad \begin{matrix} 1 \le k \le N_{\mu o_T} \\ 1 \le i \le |P| \\ 1 \le j \le |G_i| \end{matrix}$$

Exemplu

$\mu B(\mu o) = (\mu o_1, \mu o_2, \mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7, \mu o_8, \mu o_9, \mu o_{10}, \mu o_{11}, \mu o_{12})$

$P=\{P_1, P_2, P_3\}$

Presupunem că avem următoarea repartiţie pe resurse.

$\mu o_{P1}=\{ \mu o_1, \mu o_4, \mu o_5, \mu o_7, \mu o_9\}$ G1

$\mu o_{P2}= \{\mu o_2, \mu o_3, \mu o_6, \mu o_8, \mu o_{10}\}$ G2

$\mu o_{P3}=\{ \mu o_{11}, \mu o_{12}\}$ G3

G1:

$SGM_{11} = \{\mu o1, \mu o4, \mu o5, \mu o7, \mu o9\}$

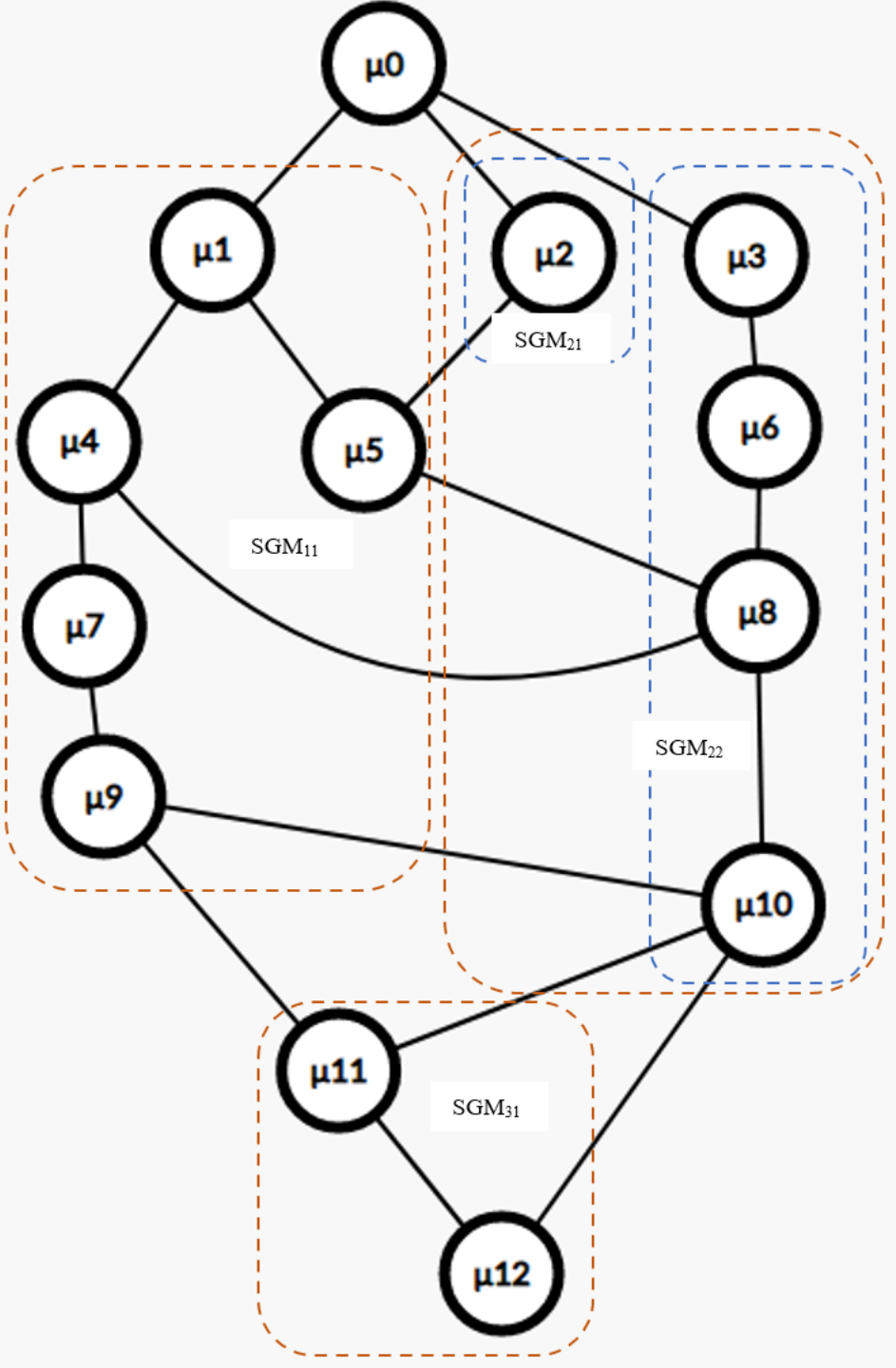
G2:

$SGM_{21} = \{ \mu o2\}$;

$SGM_{22} = \{ \mu o3, \mu o6, \mu o8, \mu o10\}$

G3:

$SGM_{31} = \{ \mu o11, \mu o12\}$



G_1: 2 sarcini terminale μo5, μo9, nivelul lui G_5, calculat în G, pe calea cea mai lungă

$$\rho = \max_i \left\{ \max_j \left\{ |SGM_{ij}| + \min_k \{ \|\mu o_{Tjk}\| \} \right\} \right\}$$

$\| \mu o5\|=4$

$\| \mu o9\|=3$

$|SGM_{11}|=5$

$\rho 1 = |SGM_{11}| + \min \{\|\mu o_5\|, \| \mu o_9\|\} = 5 +3=8$

G_2: pentru SGM_{21}, o singură sarcină care este şi terminală

$$\|\mu o_2\| = 5$$

$$\rho_{21} = \left\{ \left| \underbrace{SGM_{21}}_{1} \right| + min \|\mu o_2\| \right\} = = 1 + 5 = 6$$

pentru SGM_{22},

$$\rho_{22} = \left\{ \left| \underbrace{SGM_{22}}_{4} \right| + min \|\mu o_{10}\| \right\} = = 4 + 2 = 6$$

$$\Rightarrow \rho_2 = max(\rho_{21}, \rho_{22}) = max(6,6) = 6$$

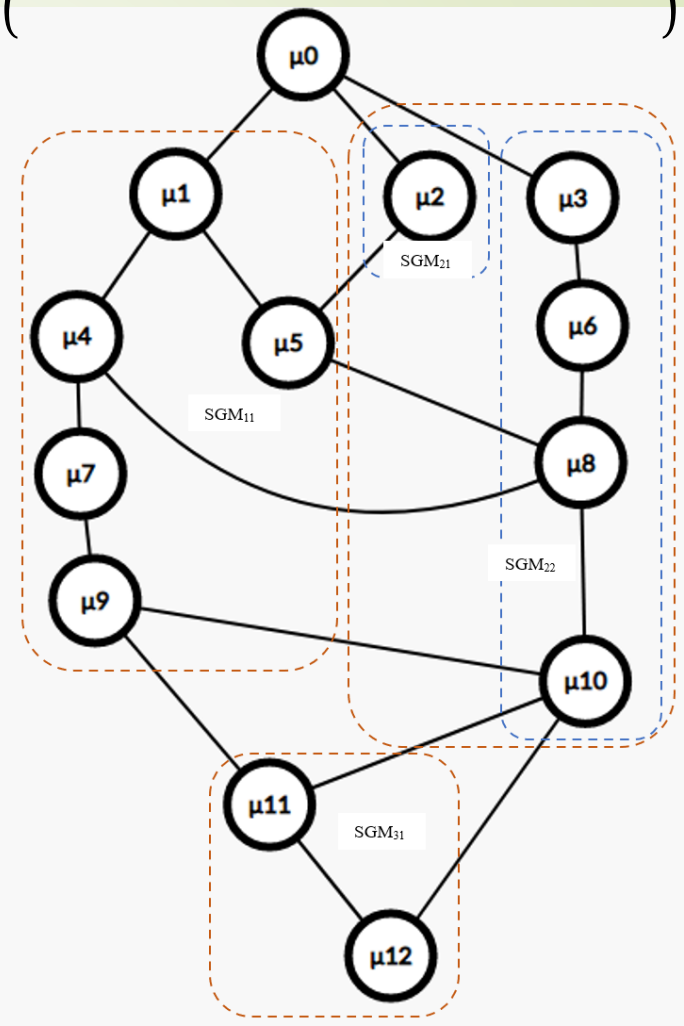
pentru SGM_{31} si $\mu o_T=12$

$|SGM_{31}|=2$

$|\mu o_{12}|=0$

$\rho 3= 2+0=2$

Am obţinut 2, 6, 8 $\Rightarrow \rho = 8$



Pe acelaşi exemplu, h=6 (înălţimea maximă a grafului).
Deci, le luăm pe cele cu 8 niveluri sau mai mult.
Justificarea alegerii, în cadrul componentei SGM_{ij}, a sarcinii terminale cu nivel minim în G e aceea că orice parcurgere a unei componente trebuie să se termine într-un astfel de nod.

Nu s-a ţinut cont de conflictul de resurse.

În general, resursele nu sunt distincte, ρ va fi mai mare decât în realitate.

Vom defini pentru fiecare resursa un ρi

$$\rho_i = \max_j \left\{ max \left[\left(\left\lceil |SGM_{ij}| / |P_i| \right\rceil \right), \left(h(SGM_{ij}) + \left\| \mu o_{T_j} \right\| \right) \right] \right\}$$

unde $|Pi=|$ număr de resurse de tip i

$|SGMij|=$nr. de microoperatii din cadrul componentei j,

nr. minim de momente de timp.

Trebuie respectată dependenţa de date.

$h(SGMij)+||\mu oTj||$ dat de dependenţa de date

$\left[\frac{|SGM_{ij}|}{|P_i|}\right]$ - e dat de nr. de resurse pe care le avem

$h(SGM_{ij})$ - nr. de momente de timp necesare pentru execuţia sarcinilor, având în vedere dependenţa de date

CAZUL 3

Limita inferioară $= \mu$, limita inferioară pentru situaţia în care resursele sunt distincte.

- $\mu = \max_i\{|\mu o P_i|\} 1 \leq i \leq |P|$

Fiind o singur resursa, se va fi controlata secvenţial, deci avem $|\mu o P_i|$, fiind $|\mu o P_i|$ microoperatii.

Când resursele nu ar fi fost diferite, am putea considera că

- $\mu = \max_i\{[|\mu o P_i|/|P_i|]\}$

În calculul acestei limite, s-a presupus că nu există dependenţă de date între microoperatiile care controleaza diferite resurse.

CAZUL 4

Se consideră că o microoperatie nu se desfăşoară numai într-o perioadă de timp ca în cazurile anterioare, ci pe durata a mai mulţi cicli.

Notăm cu μMc setul de microoperatii care se desfăşoară pe n_c cicli.

$\Rightarrow$ limita inferioară e influenţată de nr. de cicli şi o posibilitate de calcul ar fi

$z = |\mu M_c| \cdot n_c$ - e o variantă foarte simplistă, numai pentru punerea problemei.

În general, ajungem la $LI = max\{h, \rho, \mu, z\}$

$C^K_{|AD|} \Rightarrow Q$; în funcţie de LI, eliminăm f. mult.

Algoritmul de partitie va merge până la limita inferioară.

Nu se vor gasi soluţii cu valori mai mici decat LI.

Dacă am găsit o soluţie egală cu LI, rezultă că am găsit o soluţie.

ALGORITM CARE CONDUCE LA PARTITIE MINIMA:

Algoritm de partiționare pe niveluri a setului de micro-operații dintr-un micro-subbloc

$\mu SB = (\mu B(\mu o), <)$ - sistemul de microoperatii pe care trebuie sa-l partitionam

μPT – partitia ce se va genera

μPTA – partitia de microinstructiunea anterioara, cosiderata cea mai buna pana in momentul respectiv

μoD – setul de microoperatii disponibile

μIC_i – microinstructiunea completa generata din setul de microoperatii disponibil curent

nmμl – numarul minim de microinstructiuni ce se poate obtine prin codificarea microoperatiilor din subloc

$\{\mu o_d\}_i$ – multimea de microoperatii ce devin disponibile prin generarea microinstructiunii complete curente

Algoritmul genereaza o partitie optima de μIC, tinand cont de conflictul de resurse si dependenta de date impusa de relatia impusa de ordine partiala.

Algoritm stabilire partitie microinstructiuni complete μPT

P1. Initializare

- Se initializeaza partitia $\mu PT = \Phi$, $\mu PTA = \mu B(\mu o)$ - tot microsublocul (deci, daca le luam si le facem secvential)
- $\mu oD = \{\mu o_j | \forall \mu o_j \in \mu B(\mu o)\ nu\ exista\ \mu o_i < \mu o_j\}$ - deci pentru care nu exista predecesori
- $\mu oND = \{\mu o_i | \mu o_i \in \mu B(\mu o) \backslash \mu oD\}$ - operatii nedisponibile

P2. Daca $\mu oND = \Phi\ si\ |\mu oD| \leq 3$ atunci salt la pas 5

P3. Se genereaza {μIC} – multimea de microinstructiuni complete

- $\{\mu IC\} = \{\mu IC_j, \ldots, \mu IC_k\}$ - din μoD curent
  daca j ≠ k (s-au generat mai multe μIC)
  atunci salvare μPT_j = μPT
  μoD_j = setul curent μoD
  $\{\mu IC\}_j = \{\mu IC\} \setminus \mu IC_j$

P4. $\mu PT = \mu PT \cup \mu IC_j$

$$\mu oD = \mu oD \backslash \mu IC_j \cup \{\mu o_d\}_j$$

$$\mu oND = \mu oND \backslash \{\mu o_d\}_j$$

- daca $|\mu oD| \neq 0\ si\ |\mu PT| \leq |\mu PT_j| - 1$ atunci salt pas 2

P5. daca $\mu OD = \Phi$ atunci salt pas 7

P6. Se genereaza μIC din μoD curent.

μPT = μPT U μIC;

μoD = μoD \ μIC;

daca μoD ≠ Φ si |μPT|<|μPTA|-1 atunci pas 4

altfel pas 2

P7. daca | μPT| < | μPTA| atunci μPTA = μPT

altfel daca |μPTA| ≤ nmμI atunci μPTA este optima GATA

.

P8. daca $\{\mu IC\}_j \neq 0$ (cel care a fost salvat la pasul 3)

atunci reface $\mu PT = \mu PT_j$;

$\mu oD = \mu oD_j$;

$j \leftarrow j + 1$; (trec la urmatorul, selecteaza urmatoarea μIC din setul generat)

$\mu IC = \mu IC_j$

$\mu oND = \{\mu o_j | \mu o_j \in \mu B(\mu o) \backslash (\mu PT \cup \mu oD)\}$

salt pas 4

altfel μPTA este optima. GATA.

- Obs nmμI – Limita inferioara

Algoritm euristic pentru partitionarea unui microsubbloc

Pentru a evita generarea tuturor partitiilor de microinstructiuni complete, folosim un criteriu de ordonare al operatiilor din setul de microoperatii disponibile pa baza numarului de succesori.

Nu duce la solutia optima, dar duce la o solutie acceptabila.

$$psucc(\mu O_i) = \{\mu O_j | \forall \mu O_i, \mu O_j \in \mu B(\mu O),\ cu\ i \neq j,\ avem\ \mu O_i < \mu O_j\}$$

$$psucc(\mu I_i) = \sum_{j=1}^{n} psucc(\mu O_{ij})$$

n = numarul de microoperatii din μI_i

Etapele algoritmului

Pas 1

Se initializeaza partitia $\mu PT = \Phi$, $\mu PTA = \mu B(\mu o)$

$\mu oD = \{\mu o_j | \forall \mu o_j \in \mu B(\mu o)\ nu\ exista\ \mu o_i < \mu o_j\}$ – deci pentru care nu exista predecesori

$\mu oND = \{\mu o_i | \mu o_i \in \mu B(\mu o) \backslash \mu oD\}$ – operatii nedisponibile

pas 2

facem o noua partitie, numai in anumite conditii

daca μoND=Φ (am avansat cu acest test pana la sfarsit) atunci **pas 4**

pas 3.

genereaza μIC = {μIC_j , …, μIC_k } micoinstructiuni complete, ordonam acest set in functie de succesori; ordonam astfel incat

psucc(μIC_j) ≥ psucc (μIC_{j+1}) ≥ … ≥ psucc(μIC_k)

Dintre doua microinstructiuni , pastram pe nivel pe cea cu cei mai multi succesori, intrucat mutarea ei pe nivelul urmator ar implica mutarea pe nivelul urmator a tuturor succesorilor ei. Deci, ea ar apartine micropartitiei curente.

$\mu PT = \mu PT \cup \mu IC_j$

Reactualizam operatiile disponibile, nedisponibile la momentul respectiv.

$\mu oD = \mu oD \setminus \{\mu o \backslash \mu o \in \mu IC_j\} \cup \{\mu o_d\}_j$

$\mu oND = \mu oND \setminus \{\mu\ o_d\}$

transfer **pas 2**

Pas 4

spatiul disponibil nu exista; gasesc ultima structura de microinstructiuni complete.

Genereaza $\mu IC = \{\mu IC_h, \ldots, \mu IC_l\}$.

Ultima data: iau toate μIC care se genereaza aici (intrucat aici nu mai pot merge pe o cale sau alta).

$\mu PT = \mu PT \cup \mu IC_h$

$\mu oD = \mu oD \setminus \{\mu o \mid \mu o \in \mu IC_h\}$ - nu mai adaug nimic, nemaiavand succesorii (e ultima data).

h = h+1 – le iau pe toate, dar din cele generate din spatiul disponibil.

Continuam acest pas pana ce nu mai am nici o μIC.

daca $\mu oD \neq \Phi$ atunci **pas 4**

altfel μPT – partitia generata STOP

Specificarea paralelismului operatiilor elementare

Codificarea minimă a unui set de microinstrucțiuni

Punerea problemei

- În proiectarea unităților de comandă microprogramate, după ce s-a determinat setul de microinstrucțiuni complete ce realizează execuția unui microsubbloc în minimum de paşi prin controlul tuturor operațiilor paralele ce se pot executa în sistem, se pune **problema codificării minime a acestui set de microinstrucțiuni**.
- Problema optimizării numărului de biți **constă în a stabili lungimea cuvântului memoriei de control ce specifică microinstrucțiunile, astfel încât aceasta să fie minimă**, având în vedere asigurarea controlului tuturor microoperațiilor paralele specificate de secvența de microinstrucțiuni ce descrie unitatea de comandă.
- Problema generală a **optimizării numărului de biți este o problemă din clasa "NP complete".** Apartenență la clasa "NP complete" a fost demonstrată de Robertson

μo **compatibile**
μo **incompatibile**

- Fie
  - $\mu B(\mu o)=\{\mu o_1, \mu o_2, ..., \mu o_{[\mu B]}\}$ setul de microoperații distincte din cadrul microsubblocului.
  - $\mu PT=\{\mu IC_1, \mu IC_2, ..., \mu IC_{[\mu PT]}\}$ partiția unui microsubbloc în microinstrucțiuni complete şi
- **Def. 1**
- Două microoperații μo_i şi μo_j sunt **compatibile** dacă pentru orice k, $1 <= k <= [\mu PT]$, dacă $\mu o_i \in \mu IC_k$ atunci $\mu o_j \notin \mu IC_k$.
- **Def. 2**
- Două microoperații $\mu o_i \in \mu B(\mu o)$, $\mu o_j \in \mu B(\mu o)$ sunt **incompatibile** dacă există cel puțin o microinstrucțiune completă μIC astfel încât $\mu o_i \in \mu IC_k$ şi $\mu o_j \in \mu IC_k$.

clasă de compatibilitate

- **Def. 3**
- O **clasă de compatibilitate CC(μo)** este un set (subset) al mulțimii μB(μo) în care oricare două microoperații sunt compatibile între ele.
- $CC(\mu o)=\{\mu o \mid$ pt orice $\mu o_i, \mu o_j \in CC(\mu o)$ avem μo_i compatibilă cu $\mu o_j\}$

- **Def. 4**
- O **clasă de compatibilitate maximă MCC(μo)** este acea clasa de compatibilitate la care nu mai poate fi adăugată nici o microinstrucțiune fără a se pierde compatibilitatea.
- $MCC(\mu o)=\{\mu o_j \mid$ pt orice $\mu o_i \notin MCC(\mu o)$, există $\mu o_j \in MCC(\mu o)$ astfel încât $\mu o_i \in \mu IC_k$ şi $\mu o_j \in \mu IC_k\}$.

În mod analog se defineşte clasa de incompatibilitate maximă MIC(μo).

Costul de implementare a unei clase de compatibilitate

- **Def. 5**
- **Costul de implementare a unei clase de compatibilitate** (măsurat în numărul de biți necesari pentru codificare) este dat de implementarea codificării verticale a microoperațiilor ce compun clasa.
- Cost $CC_i=[\log_2(|CC_i|+1)]$

iar costul total de implementare al cuvântului de control

- $$\text{Cost CC} = \sum_{i=1}^{k} [\log_2(|CC_i|+1)]$$

unde k este numărul de clase de compatibilitate.

- **Obs**
- **O clasă de compatibilitate corespunde unui câmp din cadrul microinstrucțiunii.**

Estimarea costului minim

- Problema codificării minime presupune două faze distincte:
  - enumerarea tuturor claselor de compatibilitate maximă MCC;
  - determinarea unui subset de MCC care să implementeze costul minim pentru codificare.

Costul implementării depinde de

- numărul de MCC necesar pentru acoperirea întregului set de microoperații şi
- numărul de microoperații din fiecare clasă de compatibilitate maximă.

- De notat faptul că un cuvânt din memoria de control, adică o microinstrucțiune completă, este un set de clase incompatibile între microoperațiile componente.
- O clasă de compatibilitate specifică cel mult o microoperație din cadrul unui cuvânt al memoriei de control.

Clase de compatibilitate maximă asociate (AMCC)

- **Def. 6**
- Pentru orice clasă de incompatibilitate IC, clasele de compatibilitate maximă ce conțin un element din IC se numesc **clase de compatibilitate maximă asociate (AMCC) asociate** clasei de incompatibilitate.
- **Prop. 1**
- ***Pentru orice clasă de incompatibilitate maximă MIC reuniunea claselor de compatibilitate maximă asociate acoperă întreg setul de microoperații.***
- **Justificare**
- Fie $MIC(\mu o)=\{\mu o_1, ..., \mu o_{|MIC|}\}$ formată din $|MIC|$ microoperații, unde $|MIC| <= |\mu B|$. Presupunem contrariul şi anume că reuniunea claselor de compatibilitate maximă asociate nu acoperă întreg setul de microoperații. Fie μo_j una din aceste microoperații presupuse neacoperite cu proprietatea că:

$$\mu o_j \in \mu B(\mu o) \text{ dar } \mu o_j \notin MIC(\mu o) \text{ şi } \mu o_j \notin \bigcup_{MIC} AMCC.$$

O clasa de incompatibilitate maximă MIC nu poate acoperi μB(μo).

- **Prop. 2**
- Pentru o clasă de incompatibilitate maximă MIC, **reuniunea oricăror k clase de compatibilitate, k < |MIC|**, nu poate acoperi setul de microoperații μB(μo).

- **Justificare**
- Considerăm MIC(μo)={μo_1, μo_2, ..., $\mu o_{|MIC|}$}. |MIC| este limita inferioară pentru numărul de clase de compatibilitate ce satisface acoperirea, deoarece o clasă de compatibilitate poate acoperi un singur element din MIC.
- Rezultă că orice reuniune de k clase de compatibilitate k < |MIC| nu poate acoperi setul de microoperații.

Clasele MCI si MCC

- **Clasele de incompatibilitate maximale MCI** sunt determinate pe baza grafului de dependență şi a conflictului de resurse
  - reprezintă microoperațiile specificate de microinstrucțiunile complete ce descriu microblocul.
- **Clasele MCC sunt determinate din matricea de microoperații** ce indică incompatibilitatea.
  - Aplicarea operatorului SAU între liniile matricii va specifica microoperațiile ce fac parte din clasa de compatibilitate maximă.
- **Generarea MCC este o binecunoscută problemă în teoria automatelor finite**, existând numeroase metode pentru aceasta soluție.
- AHO arată că problema calculării MCC este o problema "NP complete".

Prop. 3 partiționare în q câmpuri

- ***Dacă un set de microoperații μB(μo) este partiționat în q câmpuri ale unei microinstrucțiuni, costul minim se va realiza atunci când (q-1) câmpuri specifică câte o microoperație (au un singur bit), iar cel de al q-lea câmp codifică restul de |μB(μo)|-q+1 microoperații.***

Justificare

- Considerăm o partiție arbitrară a LMI biți în q câmpuri. Fie câmpul cu b_{max} biți, câmpul cu lungimea cea mai mare şi fie un oricare alt câmp care are b_i biți.
- Numărul de microoperații ce poate fi codificat de cele două câmpuri este:
- **$N\mu o = (2^{bmax} - 1) + (2^{bi} - 1)$**
- Facem o modificare în organizarea logică a microinstrucțiunii şi mutăm un bit din câmpul cu b_i biți în câmpul cu b_{max} biți. în acest caz numărul de microoperații ce se pot codifica este:
- **$N\mu o' = (2^{(bmax+1)} - 1) + (2^{(bi-1)} - 1)$**
- **$N\mu o' - N\mu o = 2^{bmax} - 2^{bi} >= 0$ deoarece $b_{max} > b_i$.**
- Deci se pot codifica mai multe operații (microoperații) după modificarea structurii microinstrucțiunii.
- Repetând procesul de mutare a unui bit dintr-un câmp oarecare în câmpul de lungime maximă, numărul de microoperații ce poate fi codificat creşte.
- Numărul maxim de microoperații ce poate fi codificat rezultă a fi egal cu $(q+2^{(LMI-q+1)}-2)$. Transformând totul în cost se obtine costul minim:
- ***Cost = $q-1+\lceil \log_2(|\mu B(\mu o)|-q+2) \rceil$ când (q-1) câmpuri specifică fiecare câte o microoperație, iar ultimul câmp ($|\mu B(\mu o)|-q+1$) microoperații.***

Partiționare a setului μB(μo) microoperații în (q+h) câmpuri

Prop. 4

- ***Nu este posibilă o soluție de partiționare a setului μB(μo) microoperații în (q+h) câmpuri astfel încât costul să fie mai mic decât partiția în q câmpuri.***

Justificare

Fie

- $C_q = q-1+[\log_2(|\mu B(\mu o)|-q+2)]$ costul minim obținut prin partiția în q câmpuri şi
- $C_{q+h} = q+h-1+[\log_2(|\mu B(\mu o)|-q-h+2)]$ costul minim obținut prin partiția în q+h câmpuri.
- Deosebim 2 cazuri şi anume:

  1) $h=1$ şi $|\mu B(\mu o)|-q-1=2^k$

  2) $h \neq 1$ sau $|\mu B(\mu o)|-q+1 \neq 2^k$

- în primul caz:

  $C_q=q-1+k+1=q+k$

  $C_{(q+h)}=q+k$

  deci $C_q=C_{(q+h)}$

- în al doilea caz, deoarece :

  $[\log_2(|\mu B(\mu o)|-q+2)]-[\log_2(|\mu B(\mu o)|-q-h+2)] < h$

  rezultă $C_q < C_{(q+h)}$

Obsservatie

Costul minim pentru codificarea setului de microoperații μ B(μo) poate să fie mai mare decât cel dat de propoziția 3 adică:

$$C \geq q-1+[\log_2(|\mu B(\mu o)|-q+2)] \text{ când } |MCC|_{max} \leq |\mu B(\mu o)|-q.$$

într-adevăr costul minim corespunde când:

(q-1) câmpuri specifică fiecare câte o microoperație;

cel de-al q-lea câmp codifică $|\mu B(\mu o)|-q+1$ microoperații.

Dacă $|MCC|_{max} \leq |\mu B(\mu o)|-q$, rezultă că $|MCC|_{max} < |\mu B(\mu o)|-q+1$,

ceea ce ar face ca în ultimul câmp să nu fie toate microoperațiile compatibile.

În acest caz trebuie renunțat la organizarea microinstrucțiunii în (q-1) câmpuri de 1 bit, dar prin aceasta se măreşte şi costul de implementare.

Metodă de minimizare a numărului de biți necesari pentru codificarea microinstrucțiunilor complete

- **1. Se alege clasa de incompatibilitate maximă care are cardinalitatea maximă MIC.**

Fie **$MIC_m \in MIC$** astfel încât $|MIC_m| >= |MIC_j| <$ pentru $j \# m$, $1 <= j <= |MIC|$

Se generează clasele de compatibilitate maximă asociate AMCC clasei MIC.

$AMCC_m$={MCC | pt orice μo $\in$ MIC_m $\exists$ MCC astfel încât μo $\in$ MCC}

- **2. Se formează tabela de acoperire modificată prin considerarea numai a claselor de compatibilitate maximă ce aparțin AMCC.**

TAM : $AMCC_m$ x μB(μo) --> B

Se caută multimea de clase de compatibilitate maximă esențiale {MCC_e} inclus în $AMCC_m$ astfel încât MCC_e unic pentru care : $\mu o_i \in MCC_e$ avem TAM (MCC_e, μo_i)=1.

Se elimină coloanele corespunzătoare microoperațiilor ce sunt acoperite de clasele esențiale şi cele corespunzătoare microoperațiilor componente ale clasei MIC_m, obținându-se o **tabelă de acoperire redusă** :

TAR : $AMCC_m$ x (μB(μo) \ MIC_m) \ {MCC_e} --> B

Cont

- **3. Se generează setul soluțiilor de acoperire a microoperațiilor $\{MCC_{ap}\}$: $\{\mu B(\mu o) \setminus MIC_m) \setminus \{MCC_e\}$**

  Fie $SOL=\{SOL_1, SOL_2, ..SOL_p\}$ soluțiile de acoperire în care

  $SOL_j = \wedge / (\{MCC_e\} \cup \{MCC_{ap}\})$

  Se consideră soluția parțială cu cardinalitatea minimă SOL_j.

  Se generează soluția de acoperire a microoperațiilor ce aparțin clasei MIC_m, încă neacoperite: $\{MCC_{am}\}$.

- **4. Se generează setul soluțiilor de acoperire completă:**

  $SOLC_j=SOL_j \wedge \{MCC_{am}\}$.

  Se calculează costul soluției de acoperire completă cu cardinalitatea minimă.

Exemplu

- Considerăm setul de microinstrucțiuni din cadrul microsubblocului :

$$\mu B=\{\mu o_1,\ \mu o_2,\ \mu o_3,\ \mu o_4,\ \mu o_5,\ \mu o_6,\ \mu o_7,\ \mu o_8,\ \mu o_9,\ \mu o_{10}\}$$

- Presupunem că pe baza dependenței de date şi a conflictului de resurse între microoperații a rezultat următoarea partiție a microsubblocului :
- $\mu IC_1=\{\mu o_1, \mu o_2, \mu o_4\}$
- $\mu IC_2=\{\mu o_1, \mu o_3, \mu o_5\}$
- $\mu IC_3=\{\mu o_2, \mu o_6, \mu o_8, \mu o_9\}$
- $\mu IC_4=\{\mu o_4, \mu o_5, \mu o_7, \mu o_8\}$
- $\mu IC_5=\{\mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7\}$
- $\mu IC_6=\{\mu o_6, \mu o_9, \mu o_{10}\}$
- $\mu IC_7=\{\mu o_7, \mu o_{10}\}$

$\mu IC_1=\{\mu o_1, \mu o_2, \mu o_4\}$
$\mu IC_2=\{\mu o_1, \mu o_3, \mu o_5\}$
$\mu IC_3=\{\mu o_2, \mu o_6, \mu o_8, \mu o_9\}$
$\mu IC_4=\{\mu o_4, \mu o_5, \mu o_7, \mu o_8\}$
$\mu IC_5=\{\mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7\}$
$\mu IC_6=\{\mu o_6, \mu o_9, \mu o_{10}\}$
$\mu IC_7=\{\mu o_7, \mu o_{10}\}$

$\mu o \backslash \mu o$	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
μo_1										
μo_2										
μo_3										
μo_4										
μo_5										
μo_6										
μo_7										
μo_8										
μo_9										
μo_{10}										

Tabela de incompatibilitate

- $\mu IC_1=\{\mu o_1, \mu o_2, \mu o_4\}$
- $\mu IC_2=\{\mu o_1, \mu o_3, \mu o_5\}$
- $\mu IC_3=\{\mu o_2, \mu o_6, \mu o_8, \mu o_9\}$
- $\mu IC_4=\{\mu o_4, \mu o_5, \mu o_7, \mu o_8\}$
- $\mu IC_5=\{\mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7\}$
- $\mu IC_6=\{\mu o_6, \mu o_9, \mu o_{10}\}$
- $\mu IC_7=\{\mu o_7, \mu o_{10}\}$

$\mu o \backslash \mu o$	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
μo_1	x	x	x	x	x					
μo_2	x	x		x		x		x	x	
μo_3	x		x	x	x	x	x			
μo_4	x	x	x	x	x	x	x	x		
μo_5	x		x	x	x	x	x	x		
μo_6		x	x	x	x	x	x	x	x	x
μo_7			x	x	x	x	x	x		x
μo_8		x		x	x	x	x	x	x	
μo_9		x				x		x	x	x
μo_{10}						x	x		x	x

Clasele maximale de incompatibilitate

- $MIC_1=\{\mu o_1, \mu o_2, \mu o_4\}$
- $MIC_2=\{\mu o_1, \mu o_3, \mu o_4, \mu o_5\}$
- $MIC_3=\{\mu o_2, \mu o_4, \mu o_6, \mu o_8\}$
- $MIC_4=\{\mu o_2, \mu o_6, \mu o_8, \mu o_9\}$
- **$MIC_5=\{\mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7\}$**
- **$MIC_6=\{\mu o_4, \mu o_5, \mu o_6, \mu o_7, \mu o_8\}$**
- $MIC_7=\{\mu o_6, \mu o_7, \mu o_{10}\}$
- $MIC_8=\{\mu o_6, \mu o_9, \mu o_{10}\}$

- **MIC_{max} = MIC_5 sau MIC_6**

$\mu o \backslash \mu o$	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
μo_1	x	x	x	x	x					
μo_2	x	x		x		x		x	x	
μo_3	x		x	x	x	x	x			
μo_4	x	x	x	x	x	x	x	x		
μo_5	x		x	x	x	x	x	x		
μo_6		x	x	x	x	x	x	x	x	x
μo_7			x	x	x	x	x	x		x
μo_8		x		x	x	x	x	x	x	
μo_9		x				x		x	x	x
μo_{10}						x	x		x	x

Clasele maximale de compatibilitate

- $MCC_1=\{\mu o_1, \mu o_6\}$
- $MCC_2=\{\mu o_1, \mu o_7, \mu o_9\}$
- $MCC_3=\{\mu o_1, \mu o_8, \mu o_{10}\}$
- $MCC_4=\{\mu o_2, \mu o_3, \mu o_{10}\}$
- $MCC_5=\{\mu o_2, \mu o_5, \mu o_{10}\}$
- $MCC_6=\{\mu o_2, \mu o_7\}$
- $MCC_7=\{\mu o_3, \mu o_8, \mu o_{10}\}$
- $MCC_8=\{\mu o_3, \mu o_9\}$
- $MCC_9=\{\mu o_4, \mu o_9\}$
- $MCC_{10}=\{\mu o_4, \mu o_{10}\}$
- $MCC_{11}=\{\mu o_5, \mu o_9\}$

$\mu o \backslash \mu o$	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
μo_1	x	x	x	x	x					
μo_2	x	x		x		x		x	x	
μo_3	x		x	x	x	x	x			
μo_4	x	x	x	x	x	x	x	x		
μo_5	x		x	x	x	x	x	x		
μo_6		x	x	x	x	x	x	x	x	x
μo_7			x	x	x	x	x	x		x
μo_8		x		x	x	x	x	x	x	
μo_9		x				x		x	x	x
μo_{10}						x	x		x	x

Tabela de acoperire a microoperațiilor de către clasele maximale de compatibilitate

- $MCC_1=\{\mu o_1, \mu o_6\}$
- $MCC_2=\{\mu o_1, \mu o_7, \mu o_9\}$
- $MCC_3=\{\mu o_1, \mu o_8, \mu o_{10}\}$
- $MCC_4=\{\mu o_2, \mu o_3, \mu o_{10}\}$
- $MCC_5=\{\mu o_2, \mu o_5, \mu o_{10}\}$
- $MCC_6=\{\mu o_2, \mu o_7\}$
- $MCC_7=\{\mu o_3, \mu o_8, \mu o_{10}\}$
- $MCC_8=\{\mu o_3, \mu o_9\}$
- $MCC_9=\{\mu o_4, \mu o_9\}$
- $MCC_{10}=\{\mu o_4, \mu o_1 0\}$
- $MCC_{11}=\{\mu o_5, \mu o_9\}$

MCC\μo	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
MCC_1	x					x				
MCC_2	x						x		x	
MCC_3	x							x		x
MCC_4		x	x							x
MCC_5		x			x					x
MCC_6		x					x			
MCC_7			x					x		x
MCC_8			x						x	
MCC_9				x					x	
MCC_{10}				x						x
MCC_{11}					x				x	

Obs

- **MIC_5={μo_3, μo_4, μo_5, μo_6, μo_7}**
- **MIC_6={μo_4, μo_5, μo_6, μo_7, μo_8}**
- Se observă că există două clase maximale de incompatibilitate cu cardinalitate 5 (MIC_5 şi MIC_6).

- Costul minim absolut, conform proprietății 4 este **C = 7**.
- ***Să considerăm $AMCC_6$ - clasele maximale de compatibilitate asociate clasei maximale de incompatibilitate MIC_6 :***

  $AMCC_6$={MCC_1, MCC_2, MCC_3, MCC_5, MCC_6, MCC_7, MCC_9, MCC_{10}, MCC_{11})
- Conform propoziției 1 clasele maximale de compatibilitate din **$AMCC_6$** acoperă toate microoperațiile microsubblocului.

- MCC_1={μo_1, μo_6}
- MCC_2={μo_1, μo_7, μo_9}
- MCC_3={μo_1, μo_8, μo_{10}}
- MCC_4={μo_2, μo_3, μo_{10}}
- MCC_5={μo_2, μo_5, μo_{10}}
- MCC_6={μo_2, μo_7}
- MCC_7={μo_3, μo_8, μo_{10}}
- MCC_8={μo_3, μo_9}
- MCC_9={μo_4, μo_9}
- MCC_{10}={μo_4, μo_{10}}
- MCC_{11}={μo_5, μo_9}

Tabela de acoperire modificată

$AMCC_6$={MCC_1, MCC_2, MCC_3, MCC_5, MCC_6, MCC_7, MCC_9, MCC_{10}, MCC_{11}

- MCC_1={μo_1, μo_6}
- MCC_2={μo_1, μo_7, μo_9}
- MCC_3={μo_1, μo_8, μo_{10}}
- MCC_4={μo_2, μo_3, μo_{10}}
- MCC_5={μo_2, μo_5, μo_{10}}
- MCC_6={μo_2, μo_7}
- MCC_7={μo_3, μo_8, μo_{10}}
- MCC_8={μo_3, μo_9}
- MCC_9={μo_4, μo_9}
- MCC_{10}={μo_4, $\mu o_1 0$}
- MCC_{11}={μo_5, μo_9}

	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
MCC_1	x					**x**				
MCC_2	x						x		x	
MCC_3	x							x		x
MCC_5		x			x					x
MCC_6		x					x			
MCC_7			**x**					x		x
MCC_9				x					x	
MCC_{10}				x						x
MCC_{11}					x				x	

Se observă că MCC_1 şi MCC_7 sunt esențiale şi vor face parte din soluția finală.

Tabela de acoperire redusă

Considerăm, conform algoritmului, acoperite microoperațiile:

- incluse în clasele de compatibilitate maximă **MCC_1**=$\{\mu o_1, \mu o_6\}$ **şi MCC_7**=$\{\mu o_3, \mu o_8, \mu o_{10}\}$ esentiale şi
- cele componente ale clasei de incompatibilitate maximă **MIC_6**=$\{\mu o_4, \mu o_5, \mu o_6, \mu o_7, \mu o_8\}$

- Astfel, din tabela de acoperire se elimină microoperațiile
  - **μo_1, μo_6, μo_3, μo_8, μo_{10}, respectiv μo_4, μo_5, μo_7.**

	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
MCC_1	x					**x**				
MCC_2	x						x		x	
MCC_3	x							x		x
MCC_5		x			x					x
MCC_6		x					x			
MCC_7			**x**					x		x
MCC_9				x					x	
MCC_{10}				x						x
MCC_{11}					x				x	

MCC \µo	μo_2	μo_9
MCC_2		x
MCC_5	x	
MCC_6	x	
MCC_9		x
MCC_{11}		x

Acoperirea microoperațiilor μo_2 şi μo_9

- Se observă că tabela de acoperire s-a redus substanțial. Acoperirea microoperațiilor μo_2 şi μo_9 se poate face cu ajutorul următoarelor clase maximale de compatibilitate :
- MCC_2, MCC_5
- MCC_2, MCC_6
- MCC_5, MCC_9
- MCC_5, MCC_{11}
- MCC_6, MCC_9
- MCC_6, MCC_{11}

MCC \µo	μo_2	μo_9
MCC_2		x
MCC_5	x	
MCC_6	x	
MCC_9		x
MCC_{11}		x

Setul soluțiilor

- SOL = {SOL_1, SOL_2, SOL_3, SOL_4, SOL_5, SOL_6} astfel :
- SOL = {

  MCC_1 MCC_7 MCC_2 MCC_5 ;

  MCC_1 MCC_7 MCC_2 MCC_6 ;

  MCC_1 MCC_7 MCC_5 MCC_9 ;

  MCC_1 MCC_7 MCC_5 MCC_{11} ;

  MCC_1 MCC_7 MCC_6 MCC_9 ;

  MCC_1 MCC_7 MCC_6 MCC_{11}

  }

	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
MCC_1	x					**x**				
MCC_2	x						x		x	
MCC_3	x							x		x
MCC_5		x			x					x
MCC_6		x					x			
MCC_7			**x**					x		x
MCC_9				x					x	
MCC_{10}				x						x
MCC_{11}					x				x	

- Considerând soluția : **MCC_1 MCC_7 MCC_5 MCC_9 ∈ SOL**, se acoperă toate microoperațiile cu excepția $\mu o_7 \in MIC_6$.
- Pentru acoperirea lui μo_7 se poate lua una din clasele maximale de compatibilitate MCC_2 sau MCC_6.
- $SOLC_{31}$={MCC_1, MCC_7, MCC_5, MCC_9, MCC_2}
- $SOLC_{32}$={MCC_1, MCC_7, MCC_5, MCC_9, MCC_6}

Setul soluțiilor complete

- $SOLC_3 = \{SOLC_{31}, SOC_{32}\}$

unde

- **$SOLC_{31}=\{MCC_1, MCC_7, MCC_5, MCC_9, MCC_2\}$**
- $SOLC_{32}=\{MCC_1, MCC_7, MCC_5, MCC_9, MCC_6\}$

Ambele soluții complete au aceeaşi cardinalitate ce specifică numărul minim de câmpuri necesar pentru codificarea microinstrucțiunilor ce descriu microsubblocul.

Alegând $SOLC_{31}$, rezultă următoarea grupare a microoperațiilor:

MC2 MC7 MC5 MC9 MC1

$\{ \mu o_1 \mu o_7 \mu o_9 ; \mu o_3 \mu o_8 \mu o_{10} ; \mu o_2 \mu o_5 ; \mu o_4 ; \mu o_6 \}$

Microperatiile cuprinse intr-un camp nu mai apar in alte campuri

Sunt mai multe posibilitati

MC2 MC5 MC7 MC9 MC1

$\{ \mu o_1 \mu o_7 \mu o_9 ; \mu o_2 \mu o_5 \mu o_{10} ; \mu o_3 \mu o_8; \mu o_4 ; \mu o_6 \}$

- ce necesită 8 biți pentru codificare.

- **$MCC_1=\{\mu o_1, \mu o_6\}$**
- **$MCC_2=\{\mu o_1, \mu o_7, \mu o_9\}$**
- $MCC_3=\{\mu o_1, \mu o_8, \mu o_{10}\}$
- $MCC_4=\{\mu o_2, \mu o_3, \mu o_{10}\}$
- **$MCC_5=\{\mu o_2, \mu o_5, \mu o_{10}\}$**
- $MCC_6=\{\mu o_2, \mu o_7\}$
- **$MCC_7=\{\mu o_3, \mu o_8, \mu o_{10}\}$**
- $MCC_8=\{\mu o_3, \mu o_9\}$
- **$MCC_9=\{\mu o_4, \mu o_9\}$**
- $MCC_{10}=\{\mu o_4, \mu o_10\}$
- $MCC_{11}=\{\mu o_5, \mu o_9\}$